

Code Export

Source Code PDF Report

Generated 2026-05-04 16:51

Root directory C:\Users\LENOVO\Documents\Java_run\Smart_EmployeeManagementSystem\SmartEmployeeManagementSys

Files found 171

Total lines 19,832

Table of Contents

1. Parent: README.md (3 lines)
2. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
3. Current: pom.xml (138 lines)
4. Current: src\main\java\com\example\EmployeeManagementSystem\code\CodeToPdf.java (347 lines)
5. Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java (52 lines)
6. Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java (196 lines)
7. Current: src\main\java\com\example\EmployeeManagementSystem\config\LeaveTypeDataInitializer.java (44 lines)
8. Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java (32 lines)
9. Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java (38 lines)
10. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AdminController.java (182 lines)
11. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ApiKeyController.java (37 lines)
12. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java (184 lines)
13. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java (33 lines)
14. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeviceController.java (71 lines)
15. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java (59 lines)
16. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java (132 lines)
17. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\HolidayController.java (40 lines)
18. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java (119 lines)
19. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java (60 lines)
20. Current: src\main\java\com\example\EmployeeManagementSystem\Controller>LoginController.java (17 lines)
21. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ManagerController.java (72 lines)
22. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\NotificationController.java (60 lines)
23. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OauthController.java (52 lines)
24. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java (124 lines)
25. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java (52 lines)

26. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java (74 lines)

27. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java (96 lines)

28. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java (73 lines)

29. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java (134 lines)

30. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java (59 lines)

31. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java (18 lines)

32. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ActionDTO.java (34 lines)

33. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminActionDTO.java (34 lines)

34. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminEmployeeDTO.java (61 lines)

35. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AssignDeviceRequest.java (14 lines)

36. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java (28 lines)

37. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java (28 lines)

38. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java (91 lines)

39. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceDTO.java (45 lines)

40. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceResponseDTO.java (103 lines)

41. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.java (43 lines)

42. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDetails.java (52 lines)

43. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java (61 lines)

44. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeStatusDTO.java (71 lines)

45. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\HolidayDTO.java (50 lines)

46. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveDashboardResponse.java (122 lines)

47. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java (48 lines)

48. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java (82 lines)

49. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerDTO.java (43 lines)

50. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerEmployeeDetailsDTO.java (72 lines)

51. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java (73 lines)

52. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\PromoteRequest.java (23 lines)

53. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RepairDTO.java (34 lines)

54. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java (42 lines)

55. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java (46 lines)

56. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestDTO.java (44 lines)

57. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestResponseDTO.java (116 lines)

58. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java (103 lines)

59. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java (72 lines)

60. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java (26 lines)

61. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java (47 lines)

62. Current:
src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication.java (16 lines)

63. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ApiKey.java (79 lines)

64. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Delivery.java (93 lines)

65. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Device.java (107 lines)

66. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\DeviceAssignment.java (68 lines)

67. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Employee.java (197 lines)

68. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\FixedHoliday.java (61 lines)

69. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveEntitlement.java (104 lines)

70. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveRequest.java (105 lines)

71. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveType.java (49 lines)

72. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveWarning.java (58 lines)

73. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Notification.java (42 lines)

74. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java (62 lines)

75. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Restaurant.java (58 lines)

76. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ServiceRequest.java (141 lines)

77. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Subscription.java (105 lines)

78. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Vendor.java (124 lines)

79. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\YearEndCarryForwardLog.java (62 lines)

80. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java (8 lines)

81. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java (9 lines)

82. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java (10 lines)

83. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceType.java (15 lines)

84. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java (5 lines)

85. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java (9 lines)

86. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java (9 lines)

87. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java (8 lines)

88. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Permission.java (21 lines)

89. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Role.java (60 lines)

90. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java (7 lines)

91. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestStatus.java (9 lines)

92. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestType.java (10 lines)

93. Current: src\main\java\com\example\EmployeeManagementSystem\Enum>Status.java (8 lines)

94. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java (8 lines)

95. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java (8 lines)

96. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java (10 lines)

97. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java (8 lines)

98. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java (8 lines)

99. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java (8 lines)

100. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundException.java (8 lines)

101. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.java (8 lines)

102. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException.java (8 lines)

103. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java (8 lines)

104. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.java (7 lines)

105. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java (7 lines)

106. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java (67 lines)

107. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\JwtAuthFilter.java (83 lines)

108. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java (34 lines)

109. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java (23 lines)

110. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java (21 lines)

111. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceRepository.java (13 lines)

112. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java (26 lines)

113. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\FixedHolidayRepository.java (13 lines)

114. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveEntitlementRepository.java (31 lines)

115. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveRequestRepo.java (88 lines)

116. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java (13 lines)

117. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java (21 lines)

118. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\NotificationRepository.java (12 lines)

119. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java (19 lines)

120. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java (31 lines)

121. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.java (23 lines)

122. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java (32 lines)

123. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java (13 lines)

124. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRepository.java (18 lines)

125. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java (30 lines)

126. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveAccrualScheduler.java (54 lines)

127. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java (107 lines)

128. Current:
src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.java (85 lines)

129. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java (84 lines)

130. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ApiKeyService.java (52 lines)

131. Current:
src\main\java\com\example\EmployeeManagementSystem\Service\CarryForwardWarningService.java (126 lines)

132. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java (41 lines)

133. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CustomOAuth2UserService.java (87 lines)

134. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryService.java (178 lines)

135. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java (74 lines)

136. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeviceService.java (193 lines)

137. Current: src\main\java\com\example\EmployeeManagementSystem\Service\EmployeeService.java (214 lines)

138. Current: src\main\java\com\example\EmployeeManagementSystem\Service\HolidayService.java (39 lines)

139. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveAccrualService.java (128 lines)

140. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveDashboardService.java (215 lines)

141. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveRequestService.java (412 lines)

142. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ManagerService.java (219 lines)

143. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java (22 lines)

144. Current: src\main\java\com\example\EmployeeManagementSystem\Service\NotificationService.java (57 lines)

145. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java (41 lines)

146. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RestaurantService.java (162 lines)

147. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ServiceRequestService.java (209 lines)

148. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SickLeaveResetService.java (65 lines)

149. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SubscriptionService.java (228 lines)

150. Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java (45 lines)

151. Current:
src\main\java\com\example\EmployeeManagementSystem\Service\YearEndCarryForwardService.java (224 lines)

152. Current: src\main\java\com\example\EmployeeManagementSystem\Util\AuthUtil.java (21 lines)

153. Current: src\main\java\com\example\EmployeeManagementSystem\Util\JWTUtil.java (76 lines)

154. Current: src\main\java\com\example\EmployeeManagementSystem\Util\OAuth2SuccessHandler.java (85 lines)

155. Current: src\main\resources\application.yaml (69 lines)

156. Current: src\main\resources\static\dashboard.html (1,609 lines)

157. Current: src\main\resources\static\employee-dashboard.html (1,494 lines)

158. Current: src\main\resources\static\google.html (764 lines)

159. Current: src\main\resources\static\leave_management.html (769 lines)

160. Current: src\main\resources\static\login.html (10 lines)

161. Current: src\main\resources\static\manager-dashboard.html (1,962 lines)

162. Current: src\main\resources\static\pending.html (76 lines)

163. Current: src\main\resources\static\tech-vendor-dashboard.html (991 lines)

164. Current: src\main\resources\static\otp-setup.html (442 lines)

165. Current: src\main\resources\static\vendor-dashboard.html (529 lines)

166. Current: src\main\resources\static\vendor-login.html (396 lines)

167. Current:
src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplicationTests.java (14 lines)

168. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveAccrualServiceTest.java (112 lines)

169. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveRequestServiceTest.java (86 lines)

170. Current: src\test\java\com\example\EmployeeManagementSystem\service\SickLeaveResetServiceTest.java (93 lines)

171. Current:
src\test\java\com\example\EmployeeManagementSystem\service\YearEndCarryForwardServiceTest.java (137 lines)

1 Parent: README.md

3 lines

```
1 # SmartEmployeeManagementSystem
2 A secure and scalable Spring Boot backend for employee management, leave tracking, and meal subscription
services with JWT, OAuth2, API key authentication, and role-based access control.
3
```

2 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
1 wrapperVersion=3.3.4
2 distributionType=only-script
3 distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.14/apache-maven-
3.9.14-bin.zip
4
```

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>4.0.5</version>
9          <relativePath/> <!-- lookup parent from repository -->
10     </parent>
11     <groupId>com.example</groupId>
12     <artifactId>EmployeeLeaveManagementSystem</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name/>
15     <description/>
16     <url/>
17     <licenses>
18         <license/>
19     </licenses>
20     <developers>
21         <developer/>
22     </developers>
23     <scm>
24         <connection/>
25         <developerConnection/>
26         <tag/>
27         <url/>
28     </scm>
29     <properties>
30         <java.version>21</java.version>
31     </properties>
32     <dependencies>
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-data-jpa</artifactId>
36         </dependency>
37         <dependency>
38             <groupId>org.springframework.boot</groupId>
39             <artifactId>spring-boot-starter-webmvc</artifactId>
40         </dependency>
41         <dependency>
42             <groupId>org.springdoc</groupId>
43             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
44             <version>3.0.2</version>
45         </dependency>
46         <dependency>
47             <groupId>dev.samstevens.totp</groupId>
48             <artifactId>totp-spring-boot-starter</artifactId>
49             <version>1.7.1</version>
50         </dependency>
51         <dependency>
52             <groupId>org.springframework.session</groupId>
53             <artifactId>spring-session-core</artifactId>
54         </dependency>
55         <!-- TOTP -->
56         <dependency>
57             <groupId>dev.samstevens.totp</groupId>
58             <artifactId>totp-spring-boot-starter</artifactId>
59             <version>1.7.1</version>
60         </dependency>
61         <!-- -->
62         <dependency>
63             <groupId>org.springframework.boot</groupId>
64             <artifactId>spring-boot-devtools</artifactId>
65             <scope>runtime</scope>
66             <optional>true</optional>
67         </dependency>
68         <dependency>
69             <groupId>com.mysql</groupId>
70             <artifactId>mysql-connector-j</artifactId>
71             <scope>runtime</scope>
72     </dependencies>

```

```

73     <dependency>
74         <groupId>org.springframework.boot</groupId>
75         <artifactId>spring-boot-starter-data-jpa-test</artifactId>
76         <scope>test</scope>
77     </dependency>
78     <dependency>
79         <groupId>com.itextpdf</groupId>
80         <artifactId>layout</artifactId>
81         <version>7.2.5</version>
82     </dependency>
83     <dependency>
84         <groupId>org.springframework.boot</groupId>
85         <artifactId>spring-boot-starter-security</artifactId>
86     </dependency>
87
88     <dependency>
89         <groupId>org.springframework.boot</groupId>
90         <artifactId>spring-boot-starter-security-test</artifactId>
91         <scope>test</scope>
92     </dependency>
93     <dependency>
94         <groupId>org.springframework.boot</groupId>
95         <artifactId>spring-boot-starter-webmvc-test</artifactId>
96         <scope>test</scope>
97     </dependency>
98     <dependency>
99         <groupId>io.jsonwebtoken</groupId>
100        <artifactId>jjwt-api</artifactId>
101        <version>0.13.0</version>
102        <scope>compile</scope>
103    </dependency>
104    <dependency>
105        <groupId>io.jsonwebtoken</groupId>
106        <artifactId>jjwt-impl</artifactId>
107        <version>0.13.0</version>
108        <scope>runtime</scope>
109    </dependency>
110    <dependency>
111        <groupId>io.jsonwebtoken</groupId>
112        <artifactId>jjwt-jackson</artifactId>
113        <version>0.13.0</version>
114        <scope>runtime</scope>
115    </dependency>
116    <dependency>
117        <groupId>org.springframework.boot</groupId>
118        <artifactId>spring-boot-starter-security-oauth2-client</artifactId>
119    </dependency>
120
121    <dependency>
122        <groupId>org.springframework.boot</groupId>
123        <artifactId>spring-boot-starter-security-oauth2-client-test</artifactId>
124        <scope>test</scope>
125    </dependency>
126 </dependencies>
127
128 <build>
129     <plugins>
130         <plugin>
131             <groupId>org.springframework.boot</groupId>
132             <artifactId>spring-boot-maven-plugin</artifactId>
133         </plugin>
134     </plugins>
135 </build>
136
137 </project>
138

```

```

1 package com.example.EmployeeManagementSystem.code;
2
3 import com.itextpdf.kernel.colors.ColorConstants;
4 import com.itextpdf.kernel.colors.DeviceRgb;
5 import com.itextpdf.kernel.font.PdfFont;
6 import com.itextpdf.kernel.font.PdfFontFactory;
7 import com.itextpdf.kernel.geom.PageSize;
8 import com.itextpdf.kernel.pdf.PdfDocument;
9 import com.itextpdf.kernel.pdf.PdfWriter;
10 import com.itextpdf.layout.Document;
11 import com.itextpdf.layout.element.AreaBreak;
12 import com.itextpdf.layout.element.Cell;
13 import com.itextpdf.layout.element.Paragraph;
14 import com.itextpdf.layout.element.Table;
15 import com.itextpdf.layout.element.Text;
16 import com.itextpdf.layout.properties.AreaBreakType;
17 import com.itextpdf.layout.properties.UnitValue;
18 import com.itextpdf.io.font.constants.StandardFonts;
19
20 import java.io.File;
21 import java.io.IOException;
22 import java.nio.file.*;
23 import java.nio.file.attribute.BasicFileAttributes;
24 import java.time.LocalDateTime;
25 import java.time.format.DateTimeFormatter;
26 import java.util.*;
27
28 /**
29  * CodeToPdf - Extracts all source code files from the current, parent,
30  * and immediate child directories into a formatted PDF document.
31  *
32  * Dependencies (add to your pom.xml or build.gradle):
33  *   com.itextpdf:itext7-core:7.2.5 (or itext7 bundle)
34  *
35  * Compile:
36  *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
37  *
38  * Run:
39  *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
40  *
41  * Or with Maven:
42  *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
43  */
44 public class CodeToPdf {
45
46     // Configuration
47
48     /** Output PDF filename */
49     private static final String OUTPUT_FILE = "code_export.pdf";
50
51     /** Maximum file size to include (skip very large generated files) */
52     private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB
53
54     /** Source code file extensions to include */
55     private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
56         // JVM languages
57         ".java", ".kt", ".groovy", ".scala", ".clj",
58         // Web
59         ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
60         // Python / Ruby / PHP
61         ".py", ".rb", ".php",
62         // Systems / low-level
63         ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
64         // Data / config
65         ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
66         // Shell / scripts
67         ".sh", ".bash", ".zsh", ".bat", ".ps1",
68         // Build
69         ".gradle", ".cmake", "Makefile", "Dockerfile",
70         // Other
71         ".sql", ".md", ".txt"

```

```

72     });
73
74     /** Directories to skip entirely */
75     private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
76         ".git", ".svn", ".hg", "node_modules", "target", "build",
77         "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
78     ));
79
80     // Colors
81     private static final DeviceRgb COLOR_HEADER_BG = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
82     private static final DeviceRgb COLOR_HEADER_TEXT = new DeviceRgb(0xFF, 0xFF, 0xFF);
83     private static final DeviceRgb COLOR_FILE_TITLE = new DeviceRgb(0x18, 0x5F, 0xA5);
84     private static final DeviceRgb COLOR_CODE_BG = new DeviceRgb(0xF7, 0xF7, 0xF7);
85     private static final DeviceRgb COLOR_LINE_NUM = new DeviceRgb(0x99, 0x99, 0x99);
86     private static final DeviceRgb COLOR_TOC_HEADER = new DeviceRgb(0x18, 0x5F, 0xA5);
87
88     // Font sizes
89     private static final float FONT_TITLE = 20f;
90     private static final float FONT_SECTION = 13f;
91     private static final float FONT_FILE = 11f;
92     private static final float FONT_CODE = 7.5f;
93     private static final float FONT_TOC = 10f;
94
95     // State
96     private final List<FileEntry> fileEntries = new ArrayList<>();
97     private int totalLines = 0;
98
99     //
100
101     public static void main(String[] args) throws Exception {
102         new CodeToPdf().run();
103     }
104
105     private void run() throws Exception {
106         Path currentDir = Paths.get("").toAbsolutePath();
107         Path parentDir = currentDir.getParent();
108
109         System.out.println("=== Code to PDF Extractor ===");
110         System.out.println("Current directory : " + currentDir);
111         System.out.println("Parent directory : " + (parentDir != null ? parentDir : "(none)"));
112         System.out.println();
113
114         // 1. Collect files
115
116         // Parent directory (top-level files only, not recursive)
117         if (parentDir != null && Files.exists(parentDir)) {
118             collectFiles(parentDir, false, "Parent");
119         }
120
121         // Current directory (recursive – includes all child subdirectories)
122         collectFiles(currentDir, true, "Current");
123
124         System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
125             fileEntries.size(), totalLines);
126
127         if (fileEntries.isEmpty()) {
128             System.out.println("No code files found. Exiting.");
129             return;
130         }
131
132         // 2. Generate PDF
133         generatePdf(currentDir);
134
135         System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
136     }
137
138     // File Collection
139
140     private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
141         if (!Files.exists(root)) return;
142
143         Files.walkFileTree(root, new SimpleFileVisitor<>() {
144             @Override
145             public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
146                 String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
147                 // Never recurse into skip-dirs; always allow the root itself

```

```

148         if (!dir.equals(root) && SKIP_DIRS.contains(name)) {
149             return FileVisitResult.SKIP_SUBTREE;
150         }
151         // If not recursive, skip any subdirectory of root
152         if (!recursive && !dir.equals(root)) {
153             return FileVisitResult.SKIP_SUBTREE;
154         }
155         return FileVisitResult.CONTINUE;
156     }
157
158     @Override
159     public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
160         if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
161             String content = Files.readString(file);
162             int lines = content.split("\n", -1).length;
163             String relativePath = scope + ": " + root.relativize(file);
164             fileEntries.add(new FileEntry(relativePath, file, content, lines));
165             totalLines += lines;
166             System.out.println("  " + relativePath + " (" + lines + " lines)");
167         }
168         return FileVisitResult.CONTINUE;
169     }
170 });
171 }
172
173 private boolean isCodeFile(Path file) {
174     String name = file.getFileName().toString();
175     // Check for exact filename match (e.g. "Makefile", "Dockerfile")
176     if (CODE_EXTENSIONS.contains(name)) return true;
177     // Check extension
178     int dot = name.lastIndexOf('.');
179     if (dot >= 0) {
180         return CODE_EXTENSIONS.contains(name.substring(dot));
181     }
182     return false;
183 }
184
185 // PDF Generation
186
187 private void generatePdf(Path currentDir) throws IOException {
188     PdfWriter writer = new PdfWriter(OUTPUT_FILE);
189     PdfDocument pdfDoc = new PdfDocument(writer);
190     Document doc = new Document(pdfDoc, PageSize.A4);
191     doc.setMargins(40, 40, 40, 40);
192
193     PdfFont sansFont = PdfFontFactory.createFont(StandardFonts.HELVETICA);
194     PdfFont boldFont = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
195     PdfFont monoFont = PdfFontFactory.createFont(StandardFonts.COURIER);
196
197     // Cover page
198     addCoverPage(doc, sansFont, boldFont, currentDir);
199     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
200
201     // Table of contents
202     addTableOfContents(doc, sansFont, boldFont);
203     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
204
205     // Code sections
206     for (int i = 0; i < fileEntries.size(); i++) {
207         FileEntry entry = fileEntries.get(i);
208         addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
209         if (i < fileEntries.size() - 1) {
210             doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
211         }
212     }
213
214     doc.close();
215 }
216
217 private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
218     throws IOException {
219
220     doc.add(new Paragraph("\n\n\n"));
221
222     // Big title
223     doc.add(new Paragraph("Code Export"))

```

```

224         .setFont(boldFont).setFontSize(FONT_TITLE + 10)
225         .setFillColor(COLOR_FILE_TITLE)
226         .setMarginBottom(6));
227
228     doc.add(new Paragraph("Source Code PDF Report")
229         .setFont(sansFont).setFontSize(FONT_SECTION)
230         .setFillColor(ColorConstants.DARK_GRAY)
231         .setMarginBottom(24));
232
233     // Stats table
234     Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
235         .setWidth(UnitValue.createPercentValue(80));
236     addStatRow(stats, "Generated",      LocalDateTime.now()
237         .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
238     addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
239     addStatRow(stats, "Files found",   String.valueOf(fileEntries.size()), sansFont, boldFont);
240     addStatRow(stats, "Total lines",   String.format("%,d", totalLines), sansFont, boldFont);
241     doc.add(stats);
242 }
243
244 private void addStatRow(Table t, String label, String value,
245     PdfFont sans, PdfFont bold) {
246     t.addCell(new Cell().add(new Paragraph(label)
247         .setFont(bold).setFontSize(10)
248         .setFillColor(ColorConstants.DARK_GRAY))
249         .setBorder(null).setPaddingBottom(6));
250     t.addCell(new Cell().add(new Paragraph(value)
251         .setFont(sans).setFontSize(10))
252         .setBorder(null).setPaddingBottom(6));
253 }
254
255 private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
256     throws IOException {
257     doc.add(new Paragraph("Table of Contents")
258         .setFont(boldFont).setFontSize(FONT_SECTION + 2)
259         .setFillColor(COLOR_TOC_HEADER)
260         .setMarginBottom(12));
261
262     for (int i = 0; i < fileEntries.size(); i++) {
263         FileEntry entry = fileEntries.get(i);
264         String label = String.format("%d.  %s  (%,d lines)",
265             i + 1, entry.relativePath, entry.lineCount);
266         doc.add(new Paragraph(label)
267             .setFont(sansFont).setFontSize(FONT_TOC)
268             .setMarginBottom(3));
269     }
270 }
271
272 private void addCodeSection(Document doc, FileEntry entry, int index,
273     PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
274     throws IOException {
275
276     // Section header bar
277     Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
278         .setWidth(UnitValue.createPercentValue(100))
279         .setMarginBottom(8);
280
281     header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
282         .setFont(boldFont).setFontSize(FONT_FILE)
283         .setFillColor(COLOR_HEADER_TEXT))
284         .setBackgroundColor(COLOR_HEADER_BG)
285         .setBorder(null).setPadding(6));
286
287     header.addCell(new Cell().add(new Paragraph(entry.relativePath)
288         .setFont(boldFont).setFontSize(FONT_FILE)
289         .setFillColor(COLOR_HEADER_TEXT))
290         .setBackgroundColor(COLOR_HEADER_BG)
291         .setBorder(null).setPadding(6));
292
293     header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
294         .setFont(sansFont).setFontSize(FONT_FILE - 1)
295         .setFillColor(COLOR_HEADER_TEXT))
296         .setBackgroundColor(COLOR_HEADER_BG)
297         .setBorder(null).setPadding(6));
298
299     doc.add(header);

```



```

300
301     // Code block with line numbers
302     String[] lines = entry.content.split("\n", -1);
303     int padWidth = String.valueOf(lines.length).length();
304
305     for (int ln = 0; ln < lines.length; ln++) {
306         String lineNum = String.format("%" + padWidth + "d", ln + 1);
307         String codeLine = lines[ln]
308             .replace("\t", "    ") // expand tabs
309             .replace("\r", "");    // strip CR
310
311         // Truncate very long lines
312         if (codeLine.length() > 200) {
313             codeLine = codeLine.substring(0, 197) + "...";
314         }
315
316         Text numText = new Text(lineNum + " ").setFont(monoFont)
317             .setFontSize(FONT_CODE).setFontColor(COLOR_LINE_NUM);
318         Text codeText = new Text(codeLine).setFont(monoFont)
319             .setFontSize(FONT_CODE).setFontColor(ColorConstants.BLACK);
320
321         Paragraph p = new Paragraph().add(numText).add(codeText)
322             .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
323             .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
324             .setMultipliedLeading(1.3f)
325             .setFixedLeading(FONT_CODE + 2.5f);
326
327         doc.add(p);
328     }
329 }
330
331 // Data class
332
333 private static class FileEntry {
334     final String relativePath;
335     final Path absolutePath;
336     final String content;
337     final int lineCount;
338
339     FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
340         this.relativePath = relativePath;
341         this.absolutePath = absolutePath;
342         this.content = content;
343         this.lineCount = lineCount;
344     }
345 }
346 }
347

```

```

1 //package com.example.EmployeeManagementSystem.config;
2 //
3 //import org.springframework.beans.factory.annotation.Qualifier;
4 //import org.springframework.context.annotation.Bean;
5 //import org.springframework.context.annotation.Configuration;
6 //import org.springframework.core.annotation.Order;
7 //import org.springframework.http.HttpMethod;
8 //import org.springframework.security.authentication.AuthenticationManager;
9 //import org.springframework.security.authentication.ProviderManager;
10 //import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
11 //import org.springframework.security.config.Customizer;
12 //import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
13 //import org.springframework.security.config.annotation.web.builders.HttpSecurity;
14 //import org.springframework.security.core.userdetails.UserDetailsService;
15 //import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
16 //import org.springframework.security.crypto.password.PasswordEncoder;
17 //import org.springframework.security.web.SecurityFilterChain;
18 //
19 //
20 // @Configuration
21 // @EnableMethodSecurity
22 // public class BasicAuthConfig {
23 //     @Bean
24 //     @Order(3)
25 //     public SecurityFilterChain basicAuth(HttpSecurity security){
26 //         security
27 //             .csrf(csrf-> csrf.disable())
28 //             .authorizeHttpRequests(
29 //                 auth->auth
30 //                 .requestMatchers("/employee/register","/employee/register/manager").permitAll()
31 //                 .requestMatchers(HttpMethod.POST,"/vendors").permitAll()
32 //                 .anyRequest().authenticated()
33 //             )
34 //             .httpBasic(Customizer.withDefaults());
35 //         return security.build();
36 //     }
37 //
38 //     @Bean
39 //     public PasswordEncoder passwordEncoder(){
40 //         return new BCryptPasswordEncoder();
41 //     }
42 //
43 //     @Bean
44 //     public AuthenticationManager authenticationManager(@Qualifier("combinedUserDetailService")
45 // UserDetailsService userDetailsService,
46 // PasswordEncoder passwordEncoder){
47 //         DaoAuthenticationProvider employeeAuthenticationProvider=new
48 // DaoAuthenticationProvider(userDetailsService);
49 //         employeeAuthenticationProvider.setPasswordEncoder(passwordEncoder);
50 //         return new ProviderManager(employeeAuthenticationProvider);
51 //     }
52 // }

```

```

1 // src/main/java/com/example/EmployeeManagementSystem/config/JWTAuth/JWTAuthConfig.java
2 package com.example.EmployeeManagementSystem.config.JWTAuth;
3
4 import com.example.EmployeeManagementSystem.Filter.ApiKeyFilter;
5 import com.example.EmployeeManagementSystem.Filter.JwtAuthFilter;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
8 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
9 import com.example.EmployeeManagementSystem.Service.CustomOAuth2UserService;
10 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
11 import com.example.EmployeeManagementSystem.Util.JWTUtil;
12 import com.example.EmployeeManagementSystem.Util.OAuth2SuccessHandler;
13 import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationProvider;
14 import org.springframework.beans.factory.annotation.Qualifier;
15 import org.springframework.context.annotation.Bean;
16 import org.springframework.context.annotation.Configuration;
17 import org.springframework.context.annotation.Primary;
18 import org.springframework.http.HttpMethod;
19 import org.springframework.security.authentication.AuthenticationManager;
20 import org.springframework.security.authentication.ProviderManager;
21 import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
22 import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
23 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
24 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
25 import org.springframework.security.config.http.SessionCreationPolicy;
26 import org.springframework.security.core.userdetails.UserDetailsService;
27 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
28 import org.springframework.security.crypto.password.PasswordEncoder;
29 import org.springframework.security.web.SecurityFilterChain;
30 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
31 import org.springframework.security.web.context.DelegatingSecurityContextRepository;
32 import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
33 import org.springframework.security.web.context.RequestAttributeSecurityContextRepository;
34 import org.springframework.web.client.RestTemplate;
35 import org.springframework.web.cors.CorsConfiguration;
36 import org.springframework.web.cors.CorsConfigurationSource;
37 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
38
39 import java.util.List;
40
41 @Configuration
42 @EnableWebSecurity
43 @EnableMethodSecurity
44 public class JWTAuthConfig {
45
46     private final JwtAuthFilter jwtAuthFilter;
47     private final ApiKeyAuthenticationProvider apiKeyAuthProvider;
48     private final EmployeeRepo employeeRepo;
49     private final VendorRepo vendorRepo;
50     private final RefreshTokenService refreshTokenService;
51     private final RefreshTokenRepository refreshTokenRepository;
52     private final JWTUtil jwtUtil;
53
54     public JWTAuthConfig(JwtAuthFilter jwtAuthFilter,
55                         ApiKeyAuthenticationProvider apiKeyAuthProvider,
56                         EmployeeRepo employeeRepo,
57                         VendorRepo vendorRepo,
58                         RefreshTokenService refreshTokenService,
59                         RefreshTokenRepository refreshTokenRepository,
60                         JWTUtil jwtUtil) {
61         this.jwtAuthFilter = jwtAuthFilter;
62         this.apiKeyAuthProvider = apiKeyAuthProvider;
63         this.employeeRepo = employeeRepo;
64         this.vendorRepo = vendorRepo;
65         this.refreshTokenService = refreshTokenService;
66         this.refreshTokenRepository = refreshTokenRepository;
67         this.jwtUtil = jwtUtil;
68     }
69
70     @Bean
71     public RestTemplate getRestTemplate() {

```

```

72     return new RestTemplate();
73 }
74
75 @Bean
76 public ApiKeyFilter apiKeyFilter() {
77     return new ApiKeyFilter();
78 }
79
80 @Bean
81 public PasswordEncoder passwordEncoder() {
82     return new BCryptPasswordEncoder();
83 }
84
85 @Bean
86 public AuthenticationManager authenticationManager(
87     @Qualifier("combinedUserDetailsService") UserDetailsService userDetailsService,
88     PasswordEncoder passwordEncoder) {
89     DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
90     provider.setPasswordEncoder(passwordEncoder);
91     return new ProviderManager(provider);
92 }
93
94 @Bean
95 public CustomOAuth2UserService customOAuth2UserService() {
96     return new CustomOAuth2UserService(employeeRepo, vendorRepo, passwordEncoder());
97 }
98
99 @Bean
100 public OAuth2SuccessHandler oAuth2SuccessHandler() {
101     return new OAuth2SuccessHandler(jwtUtil, employeeRepo, vendorRepo,
102         refreshTokenService, refreshTokenRepository);
103 }
104
105 @Bean
106 @Qualifier("apiKeyAuthenticationManager")
107 public AuthenticationManager apiKeyAuthenticationManager() {
108     return new ProviderManager(apiKeyAuthProvider);
109 }
110
111 @Bean
112 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
113     http
114         .csrf(csrf -> csrf.disable())
115         .cors(cors -> cors.configurationSource(corsConfigurationSource()))
116         .securityContext(ctx -> ctx
117             .securityContextRepository(new DelegatingSecurityContextRepository(
118                 new HttpSessionSecurityContextRepository(),
119                 new RequestAttributeSecurityContextRepository()
120             ))
121         )
122         .sessionManagement(session -> session
123             .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
124             .maximumSessions(1)
125             .maxSessionsPreventsLogin(false)
126         )
127         .authorizeHttpRequests(auth -> auth
128             // Auth endpoints
129             .requestMatchers("/Authenticate",
130                 "/Authenticate/totp", "/Authenticate/refresh").permitAll()
131             .requestMatchers("/session/login", "/session/logout").permitAll()
132             // OAuth2 endpoints
133             .requestMatchers("/oauth2/**", "/login/oauth2/**").permitAll()
134             .requestMatchers("/auth/google/init").permitAll()
135             // API docs
136             .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
137             // Shared login page
138             .requestMatchers("/login.html", "/google.html").permitAll()
139             .requestMatchers("/error").permitAll()
140             // Dashboards – served as static HTML, actual data is protected by JWT
141             .requestMatchers("/login.html",
142                 "/google.html",
143                 "/vendor-login.html",

```

```

147         "/vendor-dashboard.html",
148         "/tech-vendor-dashboard.html",
149         "/employee-dashboard.html",
150         "/manager-dashboard.html",
151         "/dashboard.html",
152         "/pending.html",
153         "/leave_management.html",
154         "/totp-setup.html").permitAll()
155
156         // Vendor pages
157         .requestMatchers("/vendor-login.html").permitAll()
158         .requestMatchers("/vendor-dashboard.html").permitAll()
159
160         // API key generation requires authentication
161         .requestMatchers("/api-keys/generate").authenticated()
162         .requestMatchers("/manager/createAdmin").permitAll()
163         // totp
164         .requestMatchers("/totp/**").authenticated()
165         // notification
166         .requestMatchers(HttpMethod.GET, "/notifications").authenticated()
167         .requestMatchers(HttpMethod.GET, "/notifications/unread-count").authenticated()
168         .requestMatchers(HttpMethod.POST, "/notifications/mark-read").authenticated()
169         // Everything else requires a valid token
170         .anyRequest().authenticated()
171     )
172     .oauth2Login(oauth -> oauth
173         .userInfoEndpoint(userInfo -> userInfo
174             .userService(customOAuth2UserService()))
175         )
176         .successHandler(oAuth2SuccessHandler())
177     )
178     .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
179     .addFilterBefore(apiKeyFilter(), UsernamePasswordAuthenticationFilter.class);
180
181     return http.build();
182 }
183 @Bean
184 public CorsConfigurationSource corsConfigurationSource() {
185     CorsConfiguration config = new CorsConfiguration();
186     config.setAllowedOrigins(List.of("http://192.168.1.3:8080"));
187     config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
188     config.setAllowedHeaders(List.of(""));
189     config.setAllowCredentials(true);
190
191     UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
192     source.registerCorsConfiguration("/**", config);
193     return source;
194 }
195 }
196

```

```
1 package com.example.EmployeeManagementSystem.config;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveType;
4 import com.example.EmployeeManagementSystem.Enum.Gender;
5 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class LeaveTypeDataInitializer implements CommandLineRunner {
11
12     private final LeaveTypeRepository leaveTypeRepository;
13
14     public LeaveTypeDataInitializer(LeaveTypeRepository leaveTypeRepository) {
15         this.leaveTypeRepository = leaveTypeRepository;
16     }
17
18     @Override
19     public void run(String... args) {
20         ensureLeaveType("SICK", false, true, 0, false, null);
21         ensureLeaveType("CASUAL", true, false, 30, false, null);
22         ensureLeaveType("MATERNITY", true, false, 30, true, Gender.F);
23         ensureLeaveType("UNPAID", false, false, 0, false, null);
24     }
25
26     private void ensureLeaveType(String name,
27                                boolean carriesForward,
28                                boolean monthlyReset,
29                                int maxCarryForward,
30                                boolean genderRestricted,
31                                Gender genderRestriction) {
32         leaveTypeRepository.findByName(name).orElseGet(() -> {
33             LeaveType leaveType = new LeaveType();
34             leaveType.setName(name);
35             leaveType.setCarriesForward(carriesForward);
36             leaveType.setMonthlyReset(monthlyReset);
37             leaveType.setMaxCarryForward(maxCarryForward);
38             leaveType.setGenderRestricted(genderRestricted);
39             leaveType.setGenderRestriction(genderRestriction);
40             return leaveTypeRepository.save(leaveType);
41         });
42     }
43 }
44
```

8 Current:

src\main\java\com\example\EmployeeManagementSystem\config\Session\SessionAuthConfig.java

```
1 //package com.example.EmployeeManagementSystem.config.Session;
2 //
3 //import org.springframework.context.annotation.Bean;
4 //import org.springframework.context.annotation.Configuration;
5 //import org.springframework.core.annotation.Order;
6 //import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 //import org.springframework.security.web.SecurityFilterChain;
8 //import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
9 //
10 //@Configuration
11 //public class SessionAuthConfig {
12 //
13 //    @Bean
14 //    @Order(1)
15 //    public SecurityFilterChain sessionFilterChain(HttpSecurity http) throws Exception {
16 //        http
17 //            .securityMatcher("/session/**", "/web/**")
18 //            .csrf(csrf -> csrf.disable())
19 //            .authorizeHttpRequests(auth -> auth
20 //                .requestMatchers("/session/login", "/session/logout").permitAll()
21 //                .anyRequest().authenticated()
22 //            )
23 //            .sessionManagement(session -> session
24 //                .maximumSessions(1) // one session per user
25 //                .maxSessionsPreventsLogin(false) // new login kicks old session
26 //            )
27 //            .securityContext(ctx -> ctx
28 //                .securityContextRepository(new HttpSessionSecurityContextRepository())
29 //            );
30 //        return http.build();
31 //    }
32 //}
```

```
1 package com.example.EmployeeManagementSystem.config;
2
3 import io.swagger.v3.oas.models.Components;
4 import io.swagger.v3.oas.models.OpenAPI;
5 import io.swagger.v3.oas.models.security.SecurityRequirement;
6 import io.swagger.v3.oas.models.security.SecurityScheme;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 public class swaggerConfig {
12     @Bean
13     public OpenAPI openAPI() {
14         return new OpenAPI()
15             .components(new Components()
16                 .addSecuritySchemes("bearerAuth",
17                     new SecurityScheme()
18                         .type(SecurityScheme.Type.HTTP)
19                         .scheme("bearer")
20                         .bearerFormat("JWT")
21                 )
22             )
23             .addSecurityItem(new SecurityRequirement().addList("bearerAuth"));
24     }
25     // @Bean
26     // public OpenAPI openAPI() {
27     //     return new OpenAPI()
28     //         .components(new Components()
29     //             .addSecuritySchemes("basicAuth",
30     //                 new SecurityScheme()
31     //                     .type(SecurityScheme.Type.HTTP)
32     //                     .scheme("basic")
33     //             )
34     //         )
35     //         .addSecurityItem(new SecurityRequirement().addList("basicAuth"));
36     // }
37 }
38
```



```

1  package com.example.EmployeeManagementSystem.Controller;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Service.*;
6  import org.springframework.format.annotation.DateTimeFormat;
7  import org.springframework.http.HttpStatus;
8  import org.springframework.http.ResponseEntity;
9  import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.time.LocalDate;
13 import java.util.List;
14
15 @RestController
16 @RequestMapping("/admin")
17 @PreAuthorize("hasRole('ADMIN')")
18 public class AdminController {
19
20     private final EmployeeService employeeService;
21     private final DeliveryService.VendorService vendorService;
22     private final LeaveRequestService leaveRequestService;
23     private final RestaurantService restaurantService;
24     private final SubscriptionService subscriptionService;
25     private final LeaveAccrualService accrualService;
26     private final SickLeaveResetService resetService;
27
28     public AdminController(
29         EmployeeService employeeService,
30         DeliveryService.VendorService vendorService,
31         LeaveRequestService leaveRequestService,
32         RestaurantService restaurantService,
33         SubscriptionService subscriptionService,
34         LeaveAccrualService accrualService, SickLeaveResetService resetService) {
35         this.employeeService = employeeService;
36         this.vendorService = vendorService;
37         this.leaveRequestService = leaveRequestService;
38         this.restaurantService = restaurantService;
39         this.subscriptionService = subscriptionService;
40         this.accrualService = accrualService;
41         this.resetService = resetService;
42     }
43
44     //
45     // EMPLOYEE BRANCH
46     //
47
48     /** List every employee in the system. */
49     @GetMapping("/employees")
50     public ResponseEntity<?> getAllEmployees() {
51         return ResponseEntity.ok(employeeService.getAllEmployees());
52     }
53
54     /** Create a regular employee. */
55     @PostMapping("/employees")
56     public ResponseEntity<Employee> createEmployee(@RequestBody AdminEmployeeDTO dto) {
57         return ResponseEntity.status(HttpStatus.CREATED)
58             .body(employeeService.createEmployee(dto));
59     }
60
61     @PreAuthorize("hasRole('ADMIN')")
62     @GetMapping("/managers")
63     public ResponseEntity<List<ManagerDTO>> getAllManagers(){
64         return employeeService.getAllManagers();
65     }
66
67
68     /** Promote / create a manager. */
69     @PostMapping("/employees/manager")
70     public ResponseEntity<Employee> createManager(@RequestBody EmployeeDTO dto) {
71         return ResponseEntity.status(HttpStatus.CREATED)

```

```

72         .body(employeeService.createManager(dto));
73     }
74
75     /** Update any employee record. */
76     @PutMapping("/employees/{id}")
77     public ResponseEntity<Employee> updateEmployee(
78         @PathVariable long id,
79         @RequestBody EmployeeDTO dto) {
80         return ResponseEntity.ok(employeeService.updateEmployee(id, dto));
81     }
82
83     /** Hard-delete an employee. */
84     @DeleteMapping("/employees/{id}")
85     public ResponseEntity<String> deleteEmployee(@PathVariable long id) {
86         employeeService.deleteEmployee(id);
87         return ResponseEntity.ok("Employee with id: " + id + " has been deleted.");
88     }
89
90     /** Soft-deactivate an employee (sets status to INACTIVE). */
91     @PutMapping("/employees/{id}/inactive")
92     public ResponseEntity<String> deactivateEmployee(@PathVariable long id) {
93         return employeeService.inactivateUser(id);
94     }
95
96     //
97     // VENDOR BRANCH (only admin can register vendors)
98     //
99
100    /** List every vendor. */
101    @GetMapping("/vendors")
102    public ResponseEntity<List<VendorDTO>> getAllVendors() {
103        return ResponseEntity.ok(vendorService.getAllVendors());
104    }
105
106    @PostMapping("/vendors")
107    public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
108        return ResponseEntity.status(HttpStatus.CREATED)
109            .body(vendorService.registerVendor(request));
110    }
111
112    /** Remove a vendor by ID. */
113    @DeleteMapping("/vendors/{vendorId}")
114    public ResponseEntity<String> deleteVendor(@PathVariable Long vendorId) {
115        vendorService.deleteVendorById(vendorId);
116        return ResponseEntity.ok("Vendor with id: " + vendorId + " has been removed.");
117    }
118
119    //
120    // LEAVE MANAGEMENT BRANCH
121    //
122
123    /** Fetch all pending leave requests across the organisation. */
124    @GetMapping("/leaves/pending")
125    public ResponseEntity<?> getPendingLeaves() {
126        return ResponseEntity.ok(leaveRequestService.getPendingLeaves());
127    }
128
129    /** Approve a leave request. */
130    @PutMapping("/leaves/{id}/approve")
131    public ResponseEntity<?> approveLeave(@PathVariable Long id) {
132        return ResponseEntity.ok(leaveRequestService.approveLeave(id));
133    }
134
135    /** Reject a leave request. */
136    @PutMapping("/leaves/{id}/reject")
137    public ResponseEntity<?> rejectLeave(@PathVariable Long id) {
138        return ResponseEntity.ok(leaveRequestService.rejectLeave(id));
139    }
140
141    //
142    // RESTAURANT MANAGEMENT BRANCH
143    //
144
145    /** View all restaurants. */
146    @GetMapping("/restaurants")
147    public ResponseEntity<?> getAllRestaurants() {

```

```

148         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
149     }
150
151     //
152     // SUBSCRIPTION MANAGEMENT BRANCH
153     //
154
155     /** View all active subscriptions. */
156     @GetMapping("/subscriptions")
157     public ResponseEntity<?> getAllSubscriptions() {
158         return ResponseEntity.ok(subscriptionService.getAllSubscriptions());
159     }
160
161     @PostMapping("/leave/accrue")
162     @PreAuthorize("hasRole('ADMIN')")
163     public ResponseEntity<String> triggerAccrual(
164         @RequestParam(required = false)
165         @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
166     ) {
167         LocalDate accrualDate = (date != null) ? date : LocalDate.now();
168         accrualService.runMonthlyAccrual(accrualDate);
169         return ResponseEntity.ok("Accrual complete for " + accrualDate);
170     }
171
172     @PostMapping("/reset-sick")
173     @PreAuthorize("hasRole('ADMIN')")
174     public ResponseEntity<String> triggerSickReset(
175         @RequestParam(required = false)
176         @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
177     ) {
178         LocalDate resetDate = (date != null) ? date : LocalDate.now();
179         resetService.runMonthlyReset(resetDate);
180         return ResponseEntity.ok("Sick leave reset complete for " + resetDate);
181     }
182 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.ApiKey;
4 import com.example.EmployeeManagementSystem.Service.ApiKeyService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.access.prepost.PreAuthorize;
8 import org.springframework.web.bind.annotation.*;
9
10 @RestController
11 @RequestMapping("/api-keys")
12 public class ApiKeyController {
13
14     @Autowired
15     private ApiKeyService apiKeyService;
16
17     // Admin generates a key for a vendor
18     @PostMapping("/vendor/{vendorId}/generate")
19     @PreAuthorize("hasRole('ADMIN')")
20     public ResponseEntity<ApiKey> generateVendorKey(
21         @PathVariable Long vendorId,
22         @RequestParam String name,
23         @RequestParam(defaultValue = "VENDOR_WRITE") String permissions,
24         @RequestParam(defaultValue = "365") int validDays) {
25         return ResponseEntity.ok(
26             apiKeyService.generateForVendor(vendorId, name, permissions, validDays));
27     }
28
29     // Revoke
30     @DeleteMapping("/{keyId}")
31     @PreAuthorize("hasRole('ADMIN')")
32     public ResponseEntity<String> revokeKey(@PathVariable Long keyId) {
33         apiKeyService.revokeKey(keyId);
34         return ResponseEntity.ok("Key revoked");
35     }
36 }
37
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AccessTokenRequest;
4 import com.example.EmployeeManagementSystem.DTO.AuthRequest;
5 import com.example.EmployeeManagementSystem.DTO.AuthResponse;
6 import com.example.EmployeeManagementSystem.Entity.Employee;
7 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
8 import com.example.EmployeeManagementSystem.Entity.Vendor;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
11 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
12 import com.example.EmployeeManagementSystem.Service.TotpService;
13 import com.example.EmployeeManagementSystem.Util.JWTUtil;
14 import org.springframework.http.ResponseEntity;
15 import org.springframework.security.authentication.AuthenticationManager;
16 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
17 import org.springframework.security.core.Authentication;
18 import org.springframework.security.core.userdetails.UserDetails;
19 import org.springframework.web.bind.annotation.*;
20
21 import java.time.Instant;
22 import java.util.Map;
23 import java.util.UUID;
24 import java.util.concurrent.ConcurrentHashMap;
25
26 @RestController
27 @RequestMapping("/Authenticate")
28 public class AuthController {
29
30     private final AuthenticationManager authenticationManager;
31     private final JWTUtil jwtUtil;
32     private final RefreshTokenService refreshTokenService;
33     private final RefreshTokenRepository refreshTokenRepository;
34     private final TotpService totpService;
35     private final EmployeeRepo employeeRepo;
36
37     // Temporary store for pre-auth tokens (username -> pre-auth token)
38     // In production, use Redis or DB-backed store with TTL
39     private static final ConcurrentHashMap<String, PreAuthEntry> PRE_AUTH_STORE = new ConcurrentHashMap<>();
40
41     private record PreAuthEntry(String username, String role, long expiresAt) {}
42
43     public AuthController(AuthenticationManager authenticationManager,
44                           JWTUtil jwtUtil,
45                           RefreshTokenService refreshTokenService,
46                           RefreshTokenRepository refreshTokenRepository,
47                           TotpService totpService,
48                           EmployeeRepo employeeRepo) {
49         this.authenticationManager = authenticationManager;
50         this.jwtUtil = jwtUtil;
51         this.refreshTokenService = refreshTokenService;
52         this.refreshTokenRepository = refreshTokenRepository;
53         this.totpService = totpService;
54         this.employeeRepo = employeeRepo;
55     }
56
57     /**
58      * Step 1 of login. If TOTP is enabled, returns a pre-auth token instead of a JWT.
59      * The client must then call /Authenticate/totp with that pre-auth token + 6-digit code.
60      */
61     @PostMapping
62     public ResponseEntity<?> authenticateToken(@RequestBody AuthRequest authRequest) {
63         try {
64             Authentication auth = authenticationManager.authenticate(
65                 new UsernamePasswordAuthenticationToken(
66                     authRequest.getUsername(),
67                     authRequest.getPassword()
68                 )
69             );
70
71             UserDetails user = (UserDetails) auth.getPrincipal();

```

```

72
73     String role = user.getAuthorities().stream()
74         .map(a -> a.getAuthority())
75         .filter(a -> a.startsWith("ROLE_"))
76         .findFirst()
77         .map(a -> a.substring("ROLE_".length()))
78         .orElseThrow(() -> new RuntimeException("Role not found"));
79
80     // Handle Vendor login – no TOTP, issue tokens directly
81     if (user instanceof Vendor vendor) {
82         return ResponseEntity.ok(issueTokens(vendor, role));
83     }
84
85     // Handle Employee login
86     Employee employee = employeeRepo.findByEmail(authRequest.getUsername())
87         .orElseThrow(() -> new RuntimeException("Employee not found"));
88
89     if (employee.isTotpEnabled()) {
90         String preAuthToken = UUID.randomUUID().toString();
91         PRE_AUTH_STORE.put(preAuthToken, new PreAuthEntry(
92             employee.getEmail(),
93             role,
94             System.currentTimeMillis() + 5 * 60 * 1000L
95         ));
96         return ResponseEntity.ok(Map.of(
97             "requiresTotp", true,
98             "preAuthToken", preAuthToken
99         ));
100     }
101
102     return ResponseEntity.ok(issueTokens(user, role));
103
104 } catch (Exception e) {
105     throw new RuntimeException(e);
106 }
107 }
108
109 /**
110  * Step 2 of login when TOTP is enabled.
111  * Submit the pre-auth token + 6-digit TOTP code to get the real JWT.
112  */
113 @PostMapping("/totp")
114 public ResponseEntity<?> completeTotpLogin(@RequestBody Map<String, String> body) {
115     String preAuthToken = body.get("preAuthToken");
116     String code = body.get("code");
117
118     if (preAuthToken == null || code == null) {
119         return ResponseEntity.badRequest().body(Map.of("error", "preAuthToken and code are required"));
120     }
121
122     PreAuthEntry entry = PRE_AUTH_STORE.get(preAuthToken);
123     if (entry == null || entry.expiresAt() < System.currentTimeMillis()) {
124         PRE_AUTH_STORE.remove(preAuthToken);
125         return ResponseEntity.status(401).body(Map.of("error", "Pre-auth token expired or invalid.
Please login again."));
126     }
127
128     Employee employee = employeeRepo.findByEmail(entry.username())
129         .orElseThrow(() -> new RuntimeException("Employee not found"));
130
131     if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
132         return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
133     }
134
135     // TOTP verified – consume the pre-auth token and issue JWT
136     PRE_AUTH_STORE.remove(preAuthToken);
137     return ResponseEntity.ok(issueTokens(employee, entry.role()));
138 }
139
140 /**
141  * Refresh token endpoint – unchanged from original.
142  */
143 @PostMapping("/refresh")
144 public AuthResponse refresh(@RequestBody AccessTokenRequest request) {
145     RefreshToken storedToken = refreshTokenRepository.findByToken(request.getRefreshToken())
146         .orElseThrow(() -> new RuntimeException("Invalid refresh token"));

```

```

147
148     if (storedToken.getExpiryTime().isBefore(Instant.now())) {
149         refreshTokenRepository.delete(storedToken);
150         throw new RuntimeException("Refresh token expired");
151     }
152
153     String username;
154     String role;
155     if (storedToken.getEmployee() != null) {
156         username = storedToken.getEmployee().getUsername();
157         role = storedToken.getEmployee().getRole().name();
158     } else if (storedToken.getVendor() != null) {
159         username = storedToken.getVendor().getUsername();
160         role = storedToken.getVendor().getRole().name();
161     } else {
162         throw new RuntimeException("Refresh token has no associated user");
163     }
164
165     String newAccessToken = jwtUtil.generateToken(username, role);
166     return new AuthResponse(newAccessToken, storedToken.getToken());
167 }
168
169 // ---- helper ----
170
171 private AuthResponse issueTokens(UserDetails user, String role) {
172     String accessToken = jwtUtil.generateToken(user.getUsername(), role);
173     RefreshToken refreshToken;
174     if (user instanceof Employee employee) {
175         refreshToken = refreshTokenService.createForEmployee(employee);
176     } else if (user instanceof Vendor vendor) {
177         refreshToken = refreshTokenService.createForVendor(vendor);
178     } else {
179         throw new RuntimeException("Unknown user type");
180     }
181     refreshTokenRepository.save(refreshToken);
182     return new AuthResponse(accessToken, refreshToken.getToken());
183 }
184 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Delivery;
4 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
5 import com.example.EmployeeManagementSystem.Service.DeliveryService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/delivery")
13 public class DeliveryController {
14     private final DeliveryService deliveryService;
15
16     public DeliveryController(DeliveryService deliveryService) {
17         this.deliveryService = deliveryService;
18     }
19
20     @PutMapping("/{id}/status")
21     public ResponseEntity<Delivery> updateDeliveryStatus(
22         @PathVariable Long id,
23         @RequestParam DeliveryStatus status) {
24         return ResponseEntity.ok(deliveryService.updateDeliveryStatus(id, status));
25     }
26
27     @GetMapping("/subscription/{subscriptionId}")
28     public ResponseEntity<List<Delivery>> getDeliveriesBySubscription(
29         @PathVariable Long subscriptionId) {
30         return ResponseEntity.ok(deliveryService.getDeliveriesBySubscription(subscriptionId));
31     }
32 }
33
```



```

1  package com.example.EmployeeManagementSystem.Controller;
2
3  import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
4  import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
5  import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
6  import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
7  import com.example.EmployeeManagementSystem.Entity.Device;
8  import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
9  import com.example.EmployeeManagementSystem.Service.DeviceService;
10 import org.springframework.http.HttpStatus;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.security.access.prepost.PreAuthorize;
13 import org.springframework.security.core.Authentication;
14 import org.springframework.web.bind.annotation.*;
15
16 import java.util.List;
17
18 @RestController
19 @RequestMapping("/devices")
20 public class DeviceController {
21     private final DeviceService deviceService;
22
23     public DeviceController(DeviceService deviceService) {
24         this.deviceService = deviceService;
25     }
26
27     @PostMapping
28     @PreAuthorize("hasRole('TECH_VENDOR')")
29     public ResponseEntity<Device> addDevice(@RequestBody DeviceDTO deviceDTO,
30                                             Authentication authentication) {
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body(deviceService.addDevice(deviceDTO, authentication));
33     }
34
35     @GetMapping("/vendor/me")
36     @PreAuthorize("hasRole('TECH_VENDOR')")
37     public ResponseEntity<List<DeviceResponseDTO>> getMyVendorDevices(Authentication authentication) {
38         return ResponseEntity.ok(deviceService.getDevicesForLoggedInVendor(authentication));
39     }
40
41     @GetMapping("/my")
42     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
43     public ResponseEntity<List<DeviceResponseDTO>> getMyAssignedDevices(Authentication authentication) {
44         return ResponseEntity.ok(deviceService.getDevicesForLoggedInEmployee(authentication));
45     }
46
47     @GetMapping("/unassigned")
48     @PreAuthorize("hasRole('ADMIN')")
49     public ResponseEntity<List<DeviceResponseDTO>> getUnassignedDevices() {
50         return ResponseEntity.ok(deviceService.getUnassignedDevices());
51     }
52
53     @PostMapping("/{deviceId}/assign")
54     @PreAuthorize("hasRole('ADMIN')")
55     public ResponseEntity<String> assignDevice(@PathVariable Long deviceId,
56                                              @RequestBody AssignDeviceRequest request) {
57         DeviceAssignment assignment = deviceService.assignDevice(deviceId, request);
58         return ResponseEntity.status(HttpStatus.CREATED)
59             .body("Device " + assignment.getDevice().getId()
60                 + " assigned to employee " + assignment.getAssignedTo().getEmployeeId());
61     }
62
63     @GetMapping("/assignments")
64     @PreAuthorize("hasRole('TECH_VENDOR')")
65     public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfVendor(Authentication authentication){
66         return deviceService.getDeviceAssignmentOfVendor(authentication);
67     }
68 }
69
70 }
71

```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.EmployeeAttendanceResponseDTO;
4 import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Service.EmployeeService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12 @RestController
13 @RequestMapping("/employee")
14 public class EmployeeController {
15     private final EmployeeService employeeService;
16
17     public EmployeeController(EmployeeService employeeService) {
18         this.employeeService = employeeService;
19     }
20
21     @GetMapping
22     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN', 'EMPLOYEE', 'VENDOR')")
23     public ResponseEntity<?> getAllEmployees(){
24         return ResponseEntity.ok(employeeService.getAllEmployees());
25     }
26
27     @GetMapping("/me")
28     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
29     public ResponseEntity<Employee> myAccount(Authentication authentication){
30         return ResponseEntity.ok(employeeService.getAccount(authentication));
31     }
32
33     @GetMapping("/attendance")
34     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE', 'VENDOR')")
35     public ResponseEntity<EmployeeAttendanceResponseDTO> getAttendanceOverview() {
36         return ResponseEntity.ok(employeeService.getEmployeeAttendanceOverview());
37     }
38
39     @PutMapping("/{id}")
40     @PreAuthorize("hasRole('ADMIN')")
41     public ResponseEntity<Employee> updateEmployee(@PathVariable long id, @RequestBody EmployeeDTO
employeeDTO){
42         return ResponseEntity.ok(employeeService.updateEmployee(id, employeeDTO));
43     }
44
45     @DeleteMapping("/{id}")
46     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
47     public ResponseEntity<?> deleteEmployee(@PathVariable long id){
48         employeeService.deleteEmployee(id);
49         return ResponseEntity.ok("Employee with id: "+id+" is deleted");
50     }
51
52     //should be done by manager
53     @PutMapping("/inactive")
54     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
55     public ResponseEntity<String> inactivateEmployee(@RequestParam long employeeId) {
56         return employeeService.inactivateUser(employeeId);
57     }
58 }
59

```

16 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Exception.*;
4 import org.springframework.dao.DataIntegrityViolationException;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.ExceptionHandler;
8 import org.springframework.web.bind.annotation.RestControllerAdvice;
9 import org.springframework.web.context.request.WebRequest;
10 import org.springframework.security.access.AccessDeniedException;
11
12 import java.time.Instant;
13 import java.time.LocalDateTime;
14 import java.util.LinkedHashMap;
15 import java.util.Map;
16
17 @RestControllerAdvice
18 public class GlobalExceptionHandler {
19
20     private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
21         Map<String, Object> body = new LinkedHashMap<>();
22         body.put("timestamp", Instant.now());
23         body.put("status", status.value());
24         body.put("error", status.getReasonPhrase());
25         body.put("message", message);
26         body.put("path", request.getDescription(false).replace("uri=", ""));
27         return body;
28     }
29
30     @ExceptionHandler(EmployeeNotFound.class)
31     public ResponseEntity<Map<String, Object>> handleEmployeeNotFound(
32         EmployeeNotFound ex, WebRequest request) {
33         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
34             HttpStatus.NOT_FOUND);
35     }
36
37     @ExceptionHandler(InvalidStartDateException.class)
38     public ResponseEntity<Map<String, Object>> handleInvalidStartDate(
39         InvalidStartDateException ex, WebRequest request) {
40         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
41             HttpStatus.BAD_REQUEST);
42     }
43
44     @ExceptionHandler(InvalidEndDateException.class)
45     public ResponseEntity<Map<String, Object>> handleInvalidEndDate(
46         InvalidEndDateException ex, WebRequest request) {
47         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
48             HttpStatus.BAD_REQUEST);
49     }
50
51     @ExceptionHandler(OverlappingLeaveException.class)
52     public ResponseEntity<Map<String, Object>> handleOverlappingLeave(
53         OverlappingLeaveException ex, WebRequest request) {
54         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
55             HttpStatus.BAD_REQUEST);
56     }
57
58     @ExceptionHandler(DuplicateRequestException.class)
59     public ResponseEntity<Map<String, Object>> handleDuplicateRequest(
60         DuplicateRequestException ex, WebRequest request) {
61         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
62             HttpStatus.BAD_REQUEST);
63     }
64
65     @ExceptionHandler(LeaveRequestNotFoundException.class)
66     public ResponseEntity<Map<String, Object>> handleLeaveNotFound(
67         LeaveRequestNotFoundException ex, WebRequest request) {
68         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
69             HttpStatus.NOT_FOUND);
70     }
71 }
```

```

72     @ExceptionHandler(InvalidManagerException.class)
73     public ResponseEntity<Map<String, Object>> handleInvalidManager(
74         InvalidManagerException ex, WebRequest request) {
75         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
76             HttpStatus.FORBIDDEN);
77     }
78
79     @ExceptionHandler(SubscriptionAlreadyExists.class)
80     public ResponseEntity<Map<String, Object>> handleSubscriptionExists(
81         SubscriptionAlreadyExists ex, WebRequest request) {
82         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
83             HttpStatus.CONFLICT);
84     }
85
86
87     @ExceptionHandler(VendorNotFoundException.class)
88     public ResponseEntity<Map<String, Object>> handleVendorNotFound(
89         VendorNotFoundException ex, WebRequest request) {
90         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
91             HttpStatus.NOT_FOUND);
92     }
93
94     @ExceptionHandler(RestaurantNotFoundException.class)
95     public ResponseEntity<Map<String, Object>> handleRestaurantNotFound(
96         RestaurantNotFoundException ex, WebRequest request) {
97         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
98             HttpStatus.NOT_FOUND);
99     }
100
101     @ExceptionHandler(UnauthorizedAccessException.class)
102     public ResponseEntity<Map<String, Object>> handleUnauthorized(
103         UnauthorizedAccessException ex, WebRequest request) {
104         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
105             HttpStatus.FORBIDDEN);
106     }
107
108     @ExceptionHandler(IllegalArgumentException.class)
109     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
110         IllegalArgumentException ex, WebRequest request) {
111         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
112             HttpStatus.BAD_REQUEST);
113     }
114
115     @ExceptionHandler(DataIntegrityViolationException.class)
116     public ResponseEntity<Map<String, Object>> handleDataIntegrity(
117         DataIntegrityViolationException ex, WebRequest request) {
118         return new ResponseEntity<>(
119             buildError(HttpStatus.CONFLICT, "Cannot delete: related records exist", request),
120             HttpStatus.CONFLICT);
121     }
122
123     @ExceptionHandler(AccessDeniedException.class)
124     public ResponseEntity<Map<String, Object>> handleAccessDenied(AccessDeniedException ex) {
125         return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(
126             "error", "Access denied",
127             "message", ex.getMessage(),
128             "timestamp", LocalDateTime.now().toString()
129         ));
130     }
131
132 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.HolidayDTO;
4 import com.example.EmployeeManagementSystem.Service.HolidayService;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.security.access.prepost.PreAuthorize;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.LocalDate;
10
11 @RestController
12 @RequestMapping("/holidays")
13 public class HolidayController {
14     private final HolidayService holidayService;
15
16     public HolidayController(HolidayService holidayService) {
17         this.holidayService = holidayService;
18     }
19
20     @PostMapping
21     @PreAuthorize("hasRole('ADMIN')")
22     public ResponseEntity<?> addHoliday(@RequestBody HolidayDTO dto) {
23         return ResponseEntity.ok(holidayService.addHoliday(dto));
24     }
25
26     @GetMapping
27     public ResponseEntity<?> getHolidays(
28         @RequestParam(defaultValue = "0") int year
29     ) {
30         int targetYear = (year == 0) ? LocalDate.now().getYear() : year;
31         return ResponseEntity.ok(holidayService.getHolidaysByYear(targetYear));
32     }
33
34     @DeleteMapping("/{id}")
35     @PreAuthorize("hasRole('ADMIN')")
36     public ResponseEntity<?> deleteHoliday(@PathVariable Long id) {
37         holidayService.deleteHoliday(id);
38         return ResponseEntity.ok("Holiday deleted");
39     }
40 }
```

18 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Service.LeaveDashboardService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.core.Authentication;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.oauth2.core.user.OAuth2User;
10 import org.springframework.web.server.ResponseStatusException;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14
15 import static org.springframework.http.HttpStatus.UNAUTHORIZED;
16
17 @RestController
18 @RequestMapping("/api")
19 public class LeaveDashboardController {
20
21     private final LeaveDashboardService dashboardService;
22     private final EmployeeRepo employeeRepository;
23
24     public LeaveDashboardController(LeaveDashboardService dashboardService,
25                                     EmployeeRepo employeeRepository) {
26         this.dashboardService = dashboardService;
27         this.employeeRepository = employeeRepository;
28     }
29
30     // GET own dashboard (any authenticated employee)
31     // GET /api/employees/me/leave-dashboard
32     @GetMapping("/employees/me/leave-dashboard")
33     public ResponseEntity<LeaveDashboardResponse> getMyDashboard(
34         Authentication authentication) {
35
36         Long callerId = resolveEmployeeId(authentication);
37         return ResponseEntity.ok(
38             dashboardService.getDashboard(callerId, callerId, resolveRole(authentication)));
39     }
40
41     // GET any employee's dashboard (manager/admin only)
42     // GET /api/employees/5/leave-dashboard
43     @GetMapping("/employees/{id}/leave-dashboard")
44     public ResponseEntity<LeaveDashboardResponse> getEmployeeDashboard(
45         @PathVariable Long id,
46         Authentication authentication) {
47
48         Long callerId = resolveEmployeeId(authentication);
49         String callerRole = resolveRole(authentication);
50         return ResponseEntity.ok(
51             dashboardService.getDashboard(id, callerId, callerRole));
52     }
53
54     // GET team dashboard (manager/admin only)
55     // GET /api/employees/me/team-dashboard
56     @GetMapping("/employees/me/team-dashboard")
57     public ResponseEntity<List<LeaveDashboardResponse>> getTeamDashboard(
58         Authentication authentication) {
59
60         Long callerId = resolveEmployeeId(authentication);
61         String callerRole = resolveRole(authentication);
62         return ResponseEntity.ok(
63             dashboardService.getTeamDashboard(callerId, callerRole));
64     }
65
66     // PATCH mark warning read
67     // PATCH /api/warnings/42/read
68     @PatchMapping("/warnings/{warningId}/read")
69     public ResponseEntity<Void> markWarningRead(
70         @PathVariable Long warningId,
71         Authentication authentication) {
```

```

72
73     Long callerId = resolveEmployeeId(authentication);
74     String callerRole = resolveRole(authentication);
75     dashboardService.markWarningRead(warningId, callerId, callerRole);
76     return ResponseEntity.noContent().build();
77 }
78
79 // Supports both JWT/session UserDetails and Google OAuth2 principals.
80 private Long resolveEmployeeId(Authentication authentication) {
81     String email = resolveEmail(authentication);
82     return employeeRepository
83         .findByEmail(email)
84         .orElseThrow(() -> new IllegalStateException("Authenticated user not found"))
85         .getEmployeeId();
86 }
87
88 private String resolveRole(Authentication authentication) {
89     if (authentication == null || !authentication.isAuthenticated()) {
90         throw new ResponseStatusException(UNAUTHORIZED, "Authentication required");
91     }
92     return authentication.getAuthorities().stream()
93         .map(a -> a.getAuthority())
94         .filter(authority -> authority.startsWith("ROLE_"))
95         .findFirst()
96         .map(authority -> authority.replace("ROLE_", ""))
97         .orElse("EMPLOYEE");
98 }
99
100 private String resolveEmail(Authentication authentication) {
101     if (authentication == null || !authentication.isAuthenticated()) {
102         throw new ResponseStatusException(UNAUTHORIZED, "Authentication required");
103     }
104
105     Object principal = authentication.getPrincipal();
106     if (principal instanceof UserDetails userDetails) {
107         return userDetails.getUsername();
108     }
109     if (principal instanceof OAuth2User oAuth2User) {
110         String email = oAuth2User.getAttribute("email");
111         if (email != null && !email.isBlank()) {
112             return email;
113         }
114     }
115
116     throw new ResponseStatusException(UNAUTHORIZED, "Authenticated email not found");
117 }
118 }
119

```

19 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.ActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
5 import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
6 import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("leave_requests")
16 public class LeaveRequestController {
17     private final LeaveRequestService leaveRequestService;
18
19     public LeaveRequestController(LeaveRequestService leaveRequestService) {
20         this.leaveRequestService = leaveRequestService;
21     }
22
23     @GetMapping
24     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
25     public ResponseEntity<List<LeaveResponseDTO>> getAllLeaveRequest() {
26         return ResponseEntity.ok(leaveRequestService.getAllTheLeaveRequest());
27     }
28
29     @PostMapping
30     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
31     public ResponseEntity<?> addLeaveRequest(Authentication authentication, @RequestBody LeaveRequestDTO
32     dto){
33         return leaveRequestService.createRequest(authentication, dto);
34     }
35
36     @GetMapping("/pending")
37     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
38     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests() {
39         return ResponseEntity.ok(leaveRequestService.getAllThePendingLeaveRequests());
40     }
41
42     @PutMapping("/approval")
43     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
44     public ResponseEntity<?> updateLeaveRequestStatus(@RequestBody ActionDTO actionDTO, Authentication
45     authentication){
46         return leaveRequestService.updateLeaveRequestStatus(actionDTO, authentication);
47     }
48
49     @PutMapping("/cancel/{leaveId}")
50     @PreAuthorize("hasAuthority('LEAVE_CANCEL')")
51     public ResponseEntity<?> cancelLeaveRequest(Authentication authentication, @PathVariable long leaveId){
52         return leaveRequestService.cancelLeaveRequest(authentication, leaveId);
53     }
54
55     @GetMapping("/employee")
56     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
57     public ResponseEntity<?> getEmployeeLeaves(Authentication authentication) {
58         return leaveRequestService.getLeaveRequestsByEmployee(authentication);
59     }
60 }
```



```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 import java.util.Map;
8
9 @RestController
10 public class LoginController {
11
12     @GetMapping("/login")
13     public ResponseEntity<?> login() {
14         return ResponseEntity.ok().body(Map.of("message", "Please use POST to /api/auth/login with
credentials"));
15     }
16 }
17
```

```

1  package com.example.EmployeeManagementSystem.Controller;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Service.ManagerService;
6  import org.springframework.http.ResponseEntity;
7  import org.springframework.security.access.prepost.PreAuthorize;
8  import org.springframework.security.core.Authentication;
9  import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/manager")
15 public class ManagerController {
16     private final ManagerService manangerService;
17
18     public ManagerController(ManagerService manangerService) {
19         this.manangerService = manangerService;
20     }
21
22     @PostMapping("/createAdmin")
23     public Employee createAdmin(@RequestBody EmployeeDetails employeeDetails){
24         return manangerService.createAdmin(employeeDetails);
25     }
26
27     @PostMapping("/employees")
28     @PreAuthorize("hasAnyRole('MANAGER')")
29     public Employee createEmployee(@RequestBody EmployeeDetails employeeDetails, Authentication
authentication){
30         return manangerService.createEmp(employeeDetails,authentication);
31     }
32
33     @GetMapping("/employees")
34     @PreAuthorize("hasRole('MANAGER')")
35     public List<Employee> getAllEmployeeByManager(Authentication authentication){
36         return manangerService.getAllEmployeeByManager(authentication);
37     }
38
39     @GetMapping("/requests")
40     @PreAuthorize("hasRole('MANAGER')")
41     public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication){
42         return manangerService.getAllLeaveRequestByManager(authentication);
43     }
44
45     @GetMapping("/Users")
46     @PreAuthorize("hasAnyRole('MANAGER','ADMIN')")
47     public List<EmployeeDTO> getUsers(){
48         return manangerService.getUsers();
49     }
50
51     @PutMapping("/promote")
52     @PreAuthorize("hasRole('MANAGER')")
53     public ResponseEntity<?> promoteUser(Authentication authentication, @RequestBody PromoteRequest
promoteRequest){
54         return manangerService.promoteUser(authentication,promoteRequest);
55     }
56
57     @GetMapping("/pending")
58     @PreAuthorize("hasAnyRole('MANAGER')")
59     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests(Authentication authentication) {
60         return ResponseEntity.ok(manangerService.getAllThePendingLeaveRequests(authentication));
61     }
62
63     @GetMapping("/employees/search")
64     @PreAuthorize("hasAnyRole('MANAGER')")
65     public ResponseEntity<ManagerEmployeeDetailsDTO> getEmployeeDetails( @RequestParam String name){
66         return ResponseEntity.ok(manangerService.getEmployeeDetailsByName(name));
67     }
68
69

```

70

71 }

72

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Notification;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Service.NotificationService;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.core.Authentication;
9 import org.springframework.security.oauth2.core.user.OAuth2User;
10 import org.springframework.security.oauth2.client.authentication.OAuth2AuthenticationToken;
11 import org.springframework.web.bind.annotation.*;
12 import java.util.List;
13 import java.util.Map;
14
15 @RestController
16 @RequestMapping("/notifications")
17 public class NotificationController {
18
19     private final NotificationService notificationService;
20     private final EmployeeRepo employeeRepo;
21
22     public NotificationController(NotificationService notificationService,
23                                 EmployeeRepo employeeRepo) {
24         this.notificationService = notificationService;
25         this.employeeRepo = employeeRepo;
26     }
27
28     /** GET /notifications – all notifications for logged-in user */
29     @GetMapping
30     public ResponseEntity<List<Notification>> getAll(Authentication authentication) {
31         Employee employee = getEmployee(authentication);
32         return ResponseEntity.ok(notificationService.getForEmployee(employee));
33     }
34
35     /** GET /notifications/unread-count – for the badge number */
36     @GetMapping("/unread-count")
37     public ResponseEntity<?> unreadCount(Authentication authentication) {
38         Employee employee = getEmployee(authentication);
39         return ResponseEntity.ok(Map.of("count",
40                                         notificationService.getUnreadCount(employee)));
41     }
42
43     /** POST /notifications/mark-read – mark all as read when bell is opened */
44     @PostMapping("/mark-read")
45     public ResponseEntity<?> markRead(Authentication authentication) {
46         Employee employee = getEmployee(authentication);
47         notificationService.markAllRead(employee);
48         return ResponseEntity.ok(Map.of("message", "All notifications marked as read"));
49     }
50
51     private Employee getEmployee(Authentication authentication) {
52         final String email = (authentication instanceof OAuth2AuthenticationToken oauth)
53             ? oauth.getPrincipal().<String>getAttribute("email")
54             : authentication.getName(); // JWT path – getName() returns email
55
56         return employeeRepo.findByEmail(email)
57             .orElseThrow(() -> new RuntimeException("Employee not found: " + email));
58     }
59 }
60

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AuthResponse;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.Role;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
10 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
11 import com.example.EmployeeManagementSystem.Service.CombinedUserDetailsService;
12 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
13 import com.example.EmployeeManagementSystem.Util.JWTUtil;
14 import jakarta.servlet.http.HttpServletRequest;
15 import jakarta.servlet.http.HttpServletResponse;
16 import jakarta.servlet.http.HttpSession;
17 import org.springframework.beans.factory.annotation.Autowired;
18 import org.springframework.beans.factory.annotation.Value;
19 import org.springframework.http.*;
20 import org.springframework.security.core.userdetails.UserDetails;
21 import org.springframework.security.core.userdetails.UsernameNotFoundException;
22 import org.springframework.security.crypto.password.PasswordEncoder;
23 import org.springframework.util.LinkedMultiValueMap;
24 import org.springframework.util.MultiValueMap;
25 import org.springframework.web.bind.annotation.GetMapping;
26 import org.springframework.web.bind.annotation.RequestMapping;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29 import org.springframework.web.client.RestTemplate;
30
31 import java.io.IOException;
32 import java.util.Collections;
33 import java.util.Map;
34 import java.util.Optional;
35 import java.util.UUID;
36
37 @RestController
38 @RequestMapping("/auth/google")
39 public class OAuthController {
40
41     @GetMapping("/init")
42     public void initOAuth(@RequestParam(defaultValue = "EMPLOYEE") String role,
43                          @RequestParam(defaultValue = "Asia/Kolkata") String timezone,
44                          HttpServletRequest request,
45                          HttpServletResponse response) throws IOException {
46
47         // Now hand off to Spring's built-in OAuth2 flow
48         System.out.println("Now hand off to Spring's built-in OAuth2 flow");
49         response.sendRedirect("/oauth2/authorization/google");
50     }
51 }
52
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
4 import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
5 import com.example.EmployeeManagementSystem.Enum.MealSlot;
6 import com.example.EmployeeManagementSystem.Service.RestaurantService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14 import java.util.stream.Collectors;
15
16 /**
17  * Restaurant management endpoints.
18  *
19  * VENDOR-ONLY (write operations)
20  * POST /restaurants - Create a restaurant (vendor supplies vendorId in body)
21  * PUT /restaurants/{id} - Update restaurant details (vendor ownership verified)
22  * PUT /restaurants/{id}/deactivate - Soft-delete a restaurant (vendor ownership verified)
23  * GET /restaurants/vendor/{vendorId} - List MY restaurants (vendor views their own)
24  *
25  * EMPLOYEE / MANAGER (read-only browse)
26  * GET /restaurants - Browse all active restaurants
27  * GET /restaurants/{id} - Get a specific restaurant
28  * GET /restaurants/by-slot?slot=LUNCH - Find restaurants that serve a given meal slot
29  *
30  * Employees use the browse endpoints to pick a restaurant when creating a subscription.
31  * Employees and managers cannot create, update, or deactivate restaurants.
32  */
33 @RestController
34 @RequestMapping("/restaurants")
35 public class RestaurantController {
36
37     private final RestaurantService restaurantService;
38
39     public RestaurantController(RestaurantService restaurantService) {
40         this.restaurantService = restaurantService;
41     }
42
43     // VENDOR-ONLY write operations
44
45     /**
46      * Create a new restaurant.
47      * Request body must include vendorId – the service validates the vendor exists.
48      */
49     @PostMapping
50     @PreAuthorize("hasRole('FOOD_VENDOR')")
51     public ResponseEntity<RestaurantDTO> createRestaurant(@RequestBody RestaurantRequest
52 request, Authentication authentication) {
53         return ResponseEntity.status(HttpStatus.CREATED)
54             .body(restaurantService.createRestaurant(request, authentication));
55     }
56
57     /**
58      * Update a restaurant's name, address, description, or supported meal slots.
59      * vendorId in the request body is used to verify ownership before any change is made.
60      */
61     @PutMapping("/{id}")
62     @PreAuthorize("hasRole('FOOD_VENDOR')")
63     public ResponseEntity<RestaurantDTO> updateRestaurant(@PathVariable Long id,
64 @RequestBody RestaurantRequest request,
65 Authentication authentication) {
66         return ResponseEntity.ok(restaurantService.updateRestaurant(id, request, authentication));
67     }
68
69     /**
70      * Soft-deactivate a restaurant. Active subscriptions remain in the DB but
71      * new deliveries will stop since the restaurant is inactive.

```

```

71     * Pass vendorId as a query param so ownership can be verified.
72     */
73     @PutMapping("/{id}/deactivate")
74     @PreAuthorize("hasRole('FOOD_VENDOR')")
75     public ResponseEntity<String> deactivateRestaurant(@PathVariable Long id,
76                                                         Authentication authentication) {
77         restaurantService.deactivateRestaurant(id, authentication);
78         return ResponseEntity.ok("Restaurant " + id + " has been deactivated.");
79     }
80
81     /**
82     * List all active restaurants owned by a specific vendor.
83     * Only the vendor themselves should call this to see their portfolio.
84     */
85     @GetMapping("/vendor")
86     @PreAuthorize("hasRole('FOOD_VENDOR')")
87     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByVendor(Authentication authentication) {
88         return ResponseEntity.ok(restaurantService.getRestaurantsByVendor(authentication));
89     }
90
91     // EMPLOYEE / MANAGER read-only browse
92
93     /**
94     * Browse all active restaurants.
95     * Employees use this to discover available restaurants before subscribing.
96     */
97     @GetMapping
98     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
99     public ResponseEntity<List<RestaurantDTO>> getAllActiveRestaurants() {
100         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
101     }
102
103     /**
104     * Get a specific restaurant by id.
105     */
106     @GetMapping("/{id}")
107     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
108     public ResponseEntity<RestaurantDTO> getRestaurant(@PathVariable Long id) {
109         return ResponseEntity.ok(restaurantService.getRestaurant(id));
110     }
111
112     /**
113     * Find all active restaurants that serve a specific meal slot.
114     * Example: GET /restaurants/by-slot?slot=LUNCH
115     * Employees call this when building their subscription to see valid options for each slot.
116     */
117     @GetMapping("/by-slot")
118     @PreAuthorize("hasAnyRole('ADMIN', 'VENDOR', 'EMPLOYEE', 'MANAGER')")
119     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByMealSlot(@RequestParam MealSlot slot) {
120         return ResponseEntity.ok(restaurantService.getRestaurantsByMealSlot(slot));
121     }
122
123 }
124

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Scheduler.LeaveStatusScheduler;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.access.prepost.PreAuthorize;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.Map;
11
12 /**
13  * Controller for Scheduler Operations
14  */
15 @RestController
16 @RequestMapping("/scheduler")
17 public class SchedulerController {
18
19     private static final Logger log = LoggerFactory.getLogger(SchedulerController.class);
20
21     private final LeaveStatusScheduler leaveStatusScheduler;
22
23     public SchedulerController(LeaveStatusScheduler leaveStatusScheduler) {
24         this.leaveStatusScheduler = leaveStatusScheduler;
25     }
26
27     /**
28      * Manual trigger for leave scheduler
29      */
30     @PostMapping("/run")
31     @PreAuthorize("hasRole('ADMIN') or hasAuthority('SCHEDULER_TRIGGER')")
32     public ResponseEntity<?> runScheduler() {
33         try {
34             leaveStatusScheduler.runManually();
35
36             return ResponseEntity.ok().body(
37                 Map.of(
38                     "status", "success",
39                     "message", "Scheduler executed successfully"
40                 )
41             );
42
43         } catch (Exception e) {
44             return ResponseEntity.internalServerError().body(
45                 Map.of(
46                     "status", "error",
47                     "message", e.getMessage()
48                 )
49             );
50         }
51     }
52 }
```


26 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AdminActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.RepairDTO;
5 import com.example.EmployeeManagementSystem.DTO.ServiceRequestDTO;
6 import com.example.EmployeeManagementSystem.DTO.ServiceRequestResponseDTO;
7 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
8 import com.example.EmployeeManagementSystem.Service.ServiceRequestService;
9 import org.springframework.http.HttpStatus;
10 import org.springframework.http.ResponseEntity;
11 import org.springframework.security.access.prepost.PreAuthorize;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.web.bind.annotation.GetMapping;
14 import org.springframework.web.bind.annotation.PostMapping;
15 import org.springframework.web.bind.annotation.PutMapping;
16 import org.springframework.web.bind.annotation.RequestBody;
17 import org.springframework.web.bind.annotation.RequestMapping;
18 import org.springframework.web.bind.annotation.RestController;
19
20 import java.util.List;
21
22 @RestController
23 @RequestMapping("/service-requests")
24 public class ServiceRequestController {
25     private final ServiceRequestService serviceRequestService;
26
27     public ServiceRequestController(ServiceRequestService serviceRequestService) {
28         this.serviceRequestService = serviceRequestService;
29     }
30
31     @PostMapping
32     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN')")
33     public ResponseEntity<ServiceRequest> createServiceRequest(
34         @RequestBody ServiceRequestDTO serviceRequestDTO,
35         Authentication authentication) {
36         return ResponseEntity.status(HttpStatus.CREATED)
37             .body(serviceRequestService.createServiceRequest(serviceRequestDTO, authentication));
38     }
39
40     @GetMapping("/my")
41     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN')")
42     public ResponseEntity<List<ServiceRequestResponseDTO>> getMyServiceRequests(Authentication
43 authentication) {
44         return ResponseEntity.ok(serviceRequestService.getMyServiceRequests(authentication));
45     }
46
47     @GetMapping("/open")
48     @PreAuthorize("hasRole('ADMIN')")
49     public ResponseEntity<List<ServiceRequestResponseDTO>> getAllOpenServiceRequests() {
50         return ResponseEntity.ok(serviceRequestService.getAllOpenServiceRequests());
51     }
52
53     @PutMapping("/admin")
54     @PreAuthorize("hasRole('ADMIN')")
55     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestByAdmin(
56         Authentication authentication,
57         @RequestBody AdminActionDTO actionDTO) {
58         return ResponseEntity.ok(serviceRequestService.updateServiceRequestByAdmin(authentication,
59 actionDTO));
60     }
61
62     @GetMapping("/vendor/me")
63     @PreAuthorize("hasRole('TECH_VENDOR')")
64     public ResponseEntity<List<ServiceRequestResponseDTO>> getAllServiceRequestByVendor(Authentication
65 authentication) {
66         return ResponseEntity.ok(serviceRequestService.getAllServiceRequestByVendor(authentication));
67     }
68
69     @PutMapping("/vendor")
70     @PreAuthorize("hasRole('TECH_VENDOR')")
71     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestByVendor(
```

```
69         Authentication authentication,
70         @RequestBody RepairDTO repairDTO) {
71         return ResponseEntity.ok(serviceRequestService.updateServiceRequestByVendor(authentication,
72         repairDTO));
73     }
74 }
```

[illegible]

```
70         // Invalidate Spring Security context
71         SecurityContextHolder.clearContext();
72
73         // Invalidate HTTP session (kills OAuth2 session too)
74         session.invalidate();
75
76         // Clear the session cookie
77         Cookie cookie = new Cookie("JSESSIONID", null);
78         cookie.setMaxAge(0);
79         cookie.setPath("/");
80         response.addCookie(cookie);
81
82         return ResponseEntity.ok("Logged out");
83     }
84
85     @GetMapping("/me")
86     public ResponseEntity<?> me(Authentication authentication) {
87         if (authentication == null || !authentication.isAuthenticated()) {
88             return ResponseEntity.status(401).body(Map.of("error", "Not logged in"));
89         }
90         return ResponseEntity.ok(Map.of(
91             "username", authentication.getName(),
92             "roles", authentication.getAuthorities().toString(),
93             "authenticated", true
94         ));
95     }
96 }
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
4 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
5 import com.example.EmployeeManagementSystem.Entity.Subscription;
6 import com.example.EmployeeManagementSystem.Service.SubscriptionService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14
15 @RestController
16 @RequestMapping("/subscription")
17 public class SubscriptionController {
18     private final SubscriptionService subscriptionService;
19
20     public SubscriptionController(SubscriptionService subscriptionService) {
21         this.subscriptionService = subscriptionService;
22     }
23
24     //Authentication makes sure that whoever is logged can add subscription and cannot add other
25     //subscription
26     @PostMapping
27     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN','VENDOR')")
28     public ResponseEntity<?> addSubscription(@RequestBody SubscriptionRequest request, Authentication
29     authentication) {
30         subscriptionService.addSubscription(request, authentication);
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body("Added new subscription for user: " + authentication.getName());
33     }
34
35     @GetMapping
36     @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
37     public List<SubscriptionDTO> getAllSubscriptions() {
38         return subscriptionService.getAllSubscriptions();
39     }
40
41     @PutMapping("/{id}")
42     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
43     public ResponseEntity<Subscription> updateSubscription(
44         @PathVariable Long id,
45         @RequestBody SubscriptionRequest request) {
46         return ResponseEntity.ok(subscriptionService.updateSubscription(id, request));
47     }
48
49     @PutMapping("/{id}/pause")
50     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
51     public ResponseEntity<Subscription> pauseSubscription(@PathVariable Long id) {
52         return ResponseEntity.ok(subscriptionService.pauseSubscription(id));
53     }
54
55     @PutMapping("/{id}/resume")
56     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
57     public ResponseEntity<Subscription> resumeSubscription(@PathVariable Long id) {
58         return ResponseEntity.ok(subscriptionService.resumeSubscription(id));
59     }
60
61     @PutMapping("/{id}/expire")
62     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
63     public ResponseEntity<Subscription> expireSubscription(@PathVariable Long id) {
64         return ResponseEntity.ok(subscriptionService.expireSubscription(id));
65     }
66
67     @GetMapping("/user")
68     @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
69     public ResponseEntity<List<SubscriptionDTO>> getSubscriptionOfUser(Authentication authentication){
70         return ResponseEntity.ok(subscriptionService.getSubscriptionOfUser(authentication));
71     }

```

```
70     }
71
72 }
73
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Service.TotpService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.core.Authentication;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.Map;
11
12 @RestController
13 @RequestMapping("/totp")
14 public class TotpController {
15
16     private final TotpService totpService;
17     private final EmployeeRepo employeeRepo;
18
19     public TotpController(TotpService totpService, EmployeeRepo employeeRepo) {
20         this.totpService = totpService;
21         this.employeeRepo = employeeRepo;
22     }
23
24     /**
25      * GET /totp/status
26      * Returns whether TOTP is currently enabled for the logged-in employee.
27      * Used by the frontend to show the correct UI state on load.
28      */
29     @GetMapping("/status")
30     public ResponseEntity<?> status(Authentication authentication) {
31         Employee employee = getEmployee(authentication);
32         return ResponseEntity.ok(Map.of(
33             "totpEnabled", employee.isTotpEnabled()
34         ));
35     }
36
37     /**
38      * POST /totp/setup
39      * Step 1 – Generate a new TOTP secret and return the QR code URI.
40      * Employee scans the QR with Google Authenticator / Authy.
41      * TOTP is NOT enabled yet until /totp/verify-setup succeeds.
42      */
43     @PostMapping("/setup")
44     public ResponseEntity<?> setup(Authentication authentication) {
45         Employee employee = getEmployee(authentication);
46
47         String secret = totpService.generateSecret();
48         employee.setTotpSecret(secret);
49         employee.setTotpEnabled(false); // not active until confirmed
50         employeeRepo.save(employee);
51
52         String qrUri = totpService.getQrCodeUri(secret, employee.getName());
53
54         return ResponseEntity.ok(Map.of(
55             "secret", secret, // backup code for manual entry
56             "qrCodeUri", qrUri // embed in QR image on frontend
57         ));
58     }
59
60     /**
61      * POST /totp/verify-setup
62      * Step 2 – Employee scans QR and types their first code to confirm setup.
63      * Only after this succeeds is totpEnabled set to true.
64      * Body: { "code": "123456" }
65      */
66     @PostMapping("/verify-setup")
67     public ResponseEntity<?> verifySetup(@RequestBody Map<String, String> body,
68                                         Authentication authentication) {
69         Employee employee = getEmployee(authentication);
70
71         if (employee.getTotpSecret() == null) {

```

```

72         return ResponseEntity.badRequest().body(Map.of(
73             "error", "No TOTP secret found. Call /totp/setup first.");
74     }
75
76     String code = body.get("code");
77     if (code == null || code.isBlank()) {
78         return ResponseEntity.badRequest().body(Map.of("error", "code is required"));
79     }
80
81     if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
82         return ResponseEntity.badRequest().body(Map.of("error", "Invalid code. Try again.));
83     }
84
85     employee.setTotpEnabled(true);
86     employeeRepo.save(employee);
87
88     return ResponseEntity.ok(Map.of("message", "TOTP 2FA enabled successfully"));
89 }
90
91 /**
92  * POST /totp/validate
93  * Step 3 – On every login when totpEnabled=true, call this after password auth.
94  * In the new flow this is handled by /Authenticate/totp using a pre-auth token.
95  * This endpoint is kept for compatibility / manual use.
96  * Body: { "code": "123456" }
97  */
98 @PostMapping("/validate")
99 public ResponseEntity<?> validate(@RequestBody Map<String, String> body,
100     Authentication authentication) {
101     Employee employee = getEmployee(authentication);
102
103     if (!employee.isTotpEnabled()) {
104         return ResponseEntity.ok(Map.of("message", "TOTP not enabled for this user"));
105     }
106
107     String code = body.get("code");
108     if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
109         return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
110     }
111
112     return ResponseEntity.ok(Map.of("message", "TOTP verified", "authenticated", true));
113 }
114
115 /**
116  * DELETE /totp/disable
117  * Disable TOTP for the logged-in employee.
118  */
119 @DeleteMapping("/disable")
120 public ResponseEntity<?> disable(Authentication authentication) {
121     Employee employee = getEmployee(authentication);
122     employee.setTotpSecret(null);
123     employee.setTotpEnabled(false);
124     employeeRepo.save(employee);
125     return ResponseEntity.ok(Map.of("message", "TOTP disabled"));
126 }
127
128 // ---- helper ----
129
130 private Employee getEmployee(Authentication authentication) {
131     return employeeRepo.findByEmail(authentication.getName())
132         .orElseThrow(() -> new RuntimeException("Employee not found"));
133 }
134 }

```



```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorDTO;
4 import com.example.EmployeeManagementSystem.DTO.VendorRequest;
5 import com.example.EmployeeManagementSystem.Service.DeliveryService;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/vendors")
16 public class VendorController {
17
18     private final DeliveryService.VendorService vendorService;
19
20     public VendorController(DeliveryService.VendorService vendorService) {
21         this.vendorService = vendorService;
22     }
23
24     @PostMapping("/register")
25     @PreAuthorize("hasRole('ADMIN')")
26     public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
27         return ResponseEntity.status(HttpStatus.CREATED)
28             .body(vendorService.registerVendor(request));
29     }
30
31     @GetMapping
32     @PreAuthorize("hasAnyRole('FOOD_VENDOR', 'ADMIN', 'TECH_VENDOR')")
33     public ResponseEntity<List<VendorDTO>> getAllVendors() {
34         return ResponseEntity.ok(vendorService.getAllVendors());
35     }
36
37     @GetMapping("/me")
38     @PreAuthorize("hasAuthority('VENDOR_WRITE')")
39     public ResponseEntity<VendorDTO> getVendor(Authentication authentication) {
40         return ResponseEntity.ok(vendorService.getVendor(authentication));
41     }
42
43
44     @PutMapping("/update")
45     @PreAuthorize("hasAuthority('VENDOR_WRITE')")
46     public ResponseEntity<VendorDTO> updateVendor(
47         @RequestBody VendorRequest request,
48         Authentication authentication) {
49         return ResponseEntity.ok(vendorService.updateVendor(request, authentication));
50     }
51
52     @DeleteMapping
53     @PreAuthorize("hasRole('ADMIN')")
54     public ResponseEntity<String> deleteVendor(Authentication authentication) {
55         vendorService.deleteVendor(authentication);
56         return ResponseEntity.ok(
57             "Vendor account for " + authentication.getName() + " has been closed.");
58     }
59 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class AccessTokenRequest {
4     private String refreshToken;
5
6     public AccessTokenRequest(String refreshToken) {
7         this.refreshToken = refreshToken;
8     }
9
10    public void setRefreshToken(String refreshToken) {
11        this.refreshToken = refreshToken;
12    }
13
14    public String getRefreshToken() {
15        return refreshToken;
16    }
17 }
18
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 public class ActionDTO {
5     private long leaveRequestId;
6     private String action;
7     private String remarks;
8
9     public long getLeaveRequestId() {
10         return leaveRequestId;
11     }
12
13     public void setLeaveRequestId(long leaveRequestId) {
14         this.leaveRequestId = leaveRequestId;
15     }
16
17
18     public String getAction() {
19         return action;
20     }
21
22     public void setAction(String action) {
23         this.action = action;
24     }
25
26     public String getRemarks() {
27         return remarks;
28     }
29
30     public void setRemarks(String remarks) {
31         this.remarks = remarks;
32     }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5 public class AdminActionDTO {
6     private Long serviceRequestId;
7     private ServiceRequestStatus status;
8     private String adminRemarks;
9
10    public Long getServiceRequestId() {
11        return serviceRequestId;
12    }
13
14    public void setServiceRequestId(Long serviceRequestId) {
15        this.serviceRequestId = serviceRequestId;
16    }
17
18    public String getAdminRemarks() {
19        return adminRemarks;
20    }
21
22    public void setAdminRemarks(String adminRemarks) {
23        this.adminRemarks = adminRemarks;
24    }
25
26    public ServiceRequestStatus getStatus() {
27        return status;
28    }
29
30    public void setStatus(ServiceRequestStatus status) {
31        this.status = status;
32    }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4
5 public class AdminEmployeeDTO {
6     private String name;
7     private String email;
8     private String password;
9     private Gender gender;
10    private String timezone;
11    private long managerId;
12
13    public String getName() {
14        return name;
15    }
16
17    public String getPassword() {
18        return password;
19    }
20
21    public String getEmail() {
22        return email;
23    }
24
25    public String getTimezone() {
26        return timezone;
27    }
28
29    public Gender getGender() {
30        return gender;
31    }
32
33    public long getManagerId() {
34        return managerId;
35    }
36
37    public void setEmail(String email) {
38        this.email = email;
39    }
40
41    public void setPassword(String password) {
42        this.password = password;
43    }
44
45    public void setManagerId(long managerId) {
46        this.managerId = managerId;
47    }
48
49    public void setTimezone(String timezone) {
50        this.timezone = timezone;
51    }
52
53    public void setGender(Gender gender) {
54        this.gender = gender;
55    }
56
57    public void setName(String name) {
58        this.name = name;
59    }
60 }
61
```

```
1 package com.example.EmployeeManagementSystem.DTO;  
2  
3 public class AssignDeviceRequest {  
4     private Long employeeId;  
5  
6     public Long getEmployeeId() {  
7         return employeeId;  
8     }  
9  
10    public void setEmployeeId(Long employeeId) {  
11        this.employeeId = employeeId;  
12    }  
13 }  
14
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import com.fasterxml.jackson.annotation.JsonProperty;
5
6 public class AuthRequest {
7     @JsonProperty("username")
8     private String Username;
9     @JsonProperty("password")
10    private String Password;
11
12    public String getPassword() {
13        return Password;
14    }
15
16    public void setPassword(String password) {
17        Password = password;
18    }
19
20    public String getUsername() {
21        return Username;
22    }
23
24    public void setUsername(String username) {
25        Username = username;
26    }
27 }
28
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class AuthResponse {
4     private String accessToken;
5     private String refreshToken;
6
7     public AuthResponse(String accessToken, String token) {
8         this.accessToken=accessToken;
9         this.refreshToken=token;
10    }
11
12    public String getAccessToken() {
13        return accessToken;
14    }
15
16    public void setAccessToken(String accessToken) {
17        this.accessToken = accessToken;
18    }
19
20    public String getRefreshToken() {
21        return refreshToken;
22    }
23
24    public void setRefreshToken(String refreshToken) {
25        this.refreshToken = refreshToken;
26    }
27 }
28
```


38 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
4 import com.example.EmployeeManagementSystem.Enum.DeviceType;
5
6 import java.time.LocalDate;
7
8 public class DeviceAssignmentResponseDTO {
9     private Long id;
10    private Long deviceId;
11    private String deviceName;
12    private String deviceBrand;
13    private DeviceType deviceType;
14    private Long employeeId;
15    private String employeeName;
16    private AssignmentStatus status;
17    private LocalDate assignedDate;
18
19    public Long getId() {
20        return id;
21    }
22
23    public void setId(Long id) {
24        this.id = id;
25    }
26
27    public String getDeviceName() {
28        return deviceName;
29    }
30
31    public void setDeviceName(String deviceName) {
32        this.deviceName = deviceName;
33    }
34
35    public Long getDeviceId() {
36        return deviceId;
37    }
38
39    public void setDeviceId(Long deviceId) {
40        this.deviceId = deviceId;
41    }
42
43    public DeviceType getDeviceType() {
44        return deviceType;
45    }
46
47    public void setDeviceType(DeviceType deviceType) {
48        this.deviceType = deviceType;
49    }
50
51    public String getDeviceBrand() {
52        return deviceBrand;
53    }
54
55    public void setDeviceBrand(String deviceBrand) {
56        this.deviceBrand = deviceBrand;
57    }
58
59    public String getEmployeeName() {
60        return employeeName;
61    }
62
63    public void setEmployeeName(String employeeName) {
64        this.employeeName = employeeName;
65    }
66
67    public Long getEmployeeId() {
68        return employeeId;
69    }
70
71    public void setEmployeeId(Long employeeId) {
```

```
72         this.employeeId = employeeId;
73     }
74
75     public AssignmentStatus getStatus() {
76         return status;
77     }
78
79     public void setStatus(AssignmentStatus status) {
80         this.status = status;
81     }
82
83     public LocalDate getAssignedDate() {
84         return assignedDate;
85     }
86
87     public void setAssignedDate(LocalDate assignedDate) {
88         this.assignedDate = assignedDate;
89     }
90 }
91
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.DeviceType;
4
5 import java.time.LocalDate;
6
7 public class DeviceDTO {
8     private String deviceName;        // e.g. "Dell XPS 15"
9     private String brand;
10    private DeviceType deviceType;
11    private LocalDate warrantyExpiryDate;
12
13    public String getDeviceName() {
14        return deviceName;
15    }
16
17    public void setDeviceName(String deviceName) {
18        this.deviceName = deviceName;
19    }
20
21    public String getBrand() {
22        return brand;
23    }
24
25    public void setBrand(String brand) {
26        this.brand = brand;
27    }
28
29    public DeviceType getDeviceType() {
30        return deviceType;
31    }
32
33    public void setDeviceType(DeviceType deviceType) {
34        this.deviceType = deviceType;
35    }
36
37    public LocalDate getWarrantyExpiryDate() {
38        return warrantyExpiryDate;
39    }
40
41    public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
42        this.warrantyExpiryDate = warrantyExpiryDate;
43    }
44 }
45
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
6 import com.example.EmployeeManagementSystem.Enum.DeviceType;
7
8 import java.time.LocalDate;
9
10 public class DeviceResponseDTO {
11     private Long id;
12     private String deviceName;
13     private String brand;
14     private DeviceType deviceType;
15     private DeviceStatus deviceStatus;
16     private String vendorName;
17     private long assignedEmployeeId;
18     private String assignedEmployeeName;
19     private LocalDate assignedDate;
20
21     private LocalDate warrantyExpiryDate;
22
23     public Long getId() {
24         return id;
25     }
26
27     public void setId(Long id) {
28         this.id = id;
29     }
30
31     public String getDeviceName() {
32         return deviceName;
33     }
34
35     public void setDeviceName(String deviceName) {
36         this.deviceName = deviceName;
37     }
38
39     public String getBrand() {
40         return brand;
41     }
42
43     public void setBrand(String brand) {
44         this.brand = brand;
45     }
46
47     public DeviceType getDeviceType() {
48         return deviceType;
49     }
50
51     public void setDeviceType(DeviceType deviceType) {
52         this.deviceType = deviceType;
53     }
54
55     public DeviceStatus getDeviceStatus() {
56         return deviceStatus;
57     }
58
59     public void setDeviceStatus(DeviceStatus deviceStatus) {
60         this.deviceStatus = deviceStatus;
61     }
62
63     public String getVendorName() {
64         return vendorName;
65     }
66
67     public void setVendorName(String vendorName) {
68         this.vendorName = vendorName;
69     }
70
71     public long getAssignedEmployeeId() {
```

```
72         return assignedEmployeeId;
73     }
74
75     public void setAssignedEmployeeId(long assignedEmployeeId) {
76         this.assignedEmployeeId = assignedEmployeeId;
77     }
78
79     public String getAssignedEmployeeName() {
80         return assignedEmployeeName;
81     }
82
83     public void setAssignedEmployeeName(String assignedEmployeeName) {
84         this.assignedEmployeeName = assignedEmployeeName;
85     }
86
87     public LocalDate getAssignedDate() {
88         return assignedDate;
89     }
90
91     public void setAssignedDate(LocalDate assignedDate) {
92         this.assignedDate = assignedDate;
93     }
94
95     public LocalDate getWarrantyExpiryDate() {
96         return warrantyExpiryDate;
97     }
98
99     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
100         this.warrantyExpiryDate = warrantyExpiryDate;
101     }
102 }
103
```

41 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.util.List;
4
5 public class EmployeeAttendanceResponseDTO {
6     private long workingCount;
7     private long onLeaveCount;
8     private List<EmployeeStatusDTO> workingEmployees;
9     private List<EmployeeStatusDTO> onLeaveEmployees;
10
11     public long getWorkingCount() {
12         return workingCount;
13     }
14
15     public void setWorkingCount(long workingCount) {
16         this.workingCount = workingCount;
17     }
18
19     public long getOnLeaveCount() {
20         return onLeaveCount;
21     }
22
23     public void setOnLeaveCount(long onLeaveCount) {
24         this.onLeaveCount = onLeaveCount;
25     }
26
27     public List<EmployeeStatusDTO> getWorkingEmployees() {
28         return workingEmployees;
29     }
30
31     public void setWorkingEmployees(List<EmployeeStatusDTO> workingEmployees) {
32         this.workingEmployees = workingEmployees;
33     }
34
35     public List<EmployeeStatusDTO> getOnLeaveEmployees() {
36         return onLeaveEmployees;
37     }
38
39     public void setOnLeaveEmployees(List<EmployeeStatusDTO> onLeaveEmployees) {
40         this.onLeaveEmployees = onLeaveEmployees;
41     }
42 }
43
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4
5 public class EmployeeDetails {
6     private String name;
7     private String email;
8     private String password;
9     private String timezone;
10    private Gender gender;
11
12    public String getName() {
13        return name;
14    }
15
16    public void setName(String name) {
17        this.name = name;
18    }
19
20    public String getPassword() {
21        return password;
22    }
23
24    public void setPassword(String password) {
25        this.password = password;
26    }
27
28    public String getEmail() {
29        return email;
30    }
31
32    public void setEmail(String email) {
33        this.email = email;
34    }
35
36    public String getTimezone() {
37        return timezone;
38    }
39
40    public void setTimezone(String timezone) {
41        this.timezone = timezone;
42    }
43
44    public Gender getGender() {
45        return gender;
46    }
47
48    public void setGender(Gender gender) {
49        this.gender = gender;
50    }
51 }
52
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import com.example.EmployeeManagementSystem.Enum.Gender;
5
6 public class EmployeeDTO {
7     private String name;
8     private String email;
9     private String dept;
10    private String password;
11    private Gender gender;
12    private String timezone;
13
14    public String getTimezone() {
15        return timezone;
16    }
17
18    public void setTimezone(String timezone) {
19        this.timezone = timezone;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public void setName(String name) {
27        this.name = name;
28    }
29
30    public String getEmail() {
31        return email;
32    }
33
34    public void setEmail(String email) {
35        this.email = email;
36    }
37
38    public String getDept() {
39        return dept;
40    }
41
42    public void setDept(String dept) {
43        this.dept = dept;
44    }
45
46    public String getPassword() {
47        return password;
48    }
49
50    public void setPassword(String password) {
51        this.password = password;
52    }
53
54    public Gender getGender() {
55        return gender;
56    }
57
58    public void setGender(Gender gender) {
59        this.gender = gender;
60    }
61 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import com.example.EmployeeManagementSystem.Enum.Status;
5
6 public class EmployeeStatusDTO {
7     private long employeeId;
8     private String name;
9     private String email;
10    private String dept;
11    private Role role;
12    private Status status;
13    private String timezone;
14
15    public long getEmployeeId() {
16        return employeeId;
17    }
18
19    public void setEmployeeId(long employeeId) {
20        this.employeeId = employeeId;
21    }
22
23    public String getName() {
24        return name;
25    }
26
27    public void setName(String name) {
28        this.name = name;
29    }
30
31    public String getEmail() {
32        return email;
33    }
34
35    public void setEmail(String email) {
36        this.email = email;
37    }
38
39    public String getDept() {
40        return dept;
41    }
42
43    public void setDept(String dept) {
44        this.dept = dept;
45    }
46
47    public Role getRole() {
48        return role;
49    }
50
51    public void setRole(Role role) {
52        this.role = role;
53    }
54
55    public Status getStatus() {
56        return status;
57    }
58
59    public void setStatus(Status status) {
60        this.status = status;
61    }
62
63    public String getTimezone() {
64        return timezone;
65    }
66
67    public void setTimezone(String timezone) {
68        this.timezone = timezone;
69    }
70 }
71
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class HolidayDTO {
4     private Long id;
5     private String name;
6     private String date;
7     private String description;
8     private int year;
9
10    public Long getId() {
11        return id;
12    }
13
14    public void setId(Long id) {
15        this.id = id;
16    }
17
18    public String getName() {
19        return name;
20    }
21
22    public void setName(String name) {
23        this.name = name;
24    }
25
26    public String getDate() {
27        return date;
28    }
29
30    public void setDate(String date) {
31        this.date = date;
32    }
33
34    public String getDescription() {
35        return description;
36    }
37
38    public void setDescription(String description) {
39        this.description = description;
40    }
41
42    public int getYear() {
43        return year;
44    }
45
46    public void setYear(int year) {
47        this.year = year;
48    }
49 }
50
```

```

1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.math.BigDecimal;
4 import java.util.List;
5 import java.util.Map;
6
7 public class LeaveDashboardResponse {
8
9     private Long    employeeId;
10    private String  employeeName;
11    private int     year;
12
13    // Individual leave balances
14    private LeaveBalanceDto sick;
15    private LeaveBalanceDto casual;
16    private LeaveBalanceDto maternity; // null for non-female employees
17
18    // Carry-forward meter (casual + maternity combined)
19    private CarryForwardMeterDto carryForwardMeter;
20
21    // Generic balances keyed by leave type name, e.g. SICK, CASUAL, MATERNITY.
22    private Map<String, LeaveBalanceDto> leaveBalances;
23
24    // Active warnings (unread)
25    private List<WarningDto> warnings;
26
27    // nested DTOs
28
29    public static class LeaveBalanceDto {
30        private String    leaveType;
31        private BigDecimal opening;
32        private BigDecimal accrued;
33        private BigDecimal used;
34        private BigDecimal available;
35        private boolean    carriesForward;
36
37        public LeaveBalanceDto(String leaveType, BigDecimal opening,
38                                BigDecimal accrued, BigDecimal used,
39                                BigDecimal available, boolean carriesForward) {
40            this.leaveType    = leaveType;
41            this.opening      = opening;
42            this.accrued      = accrued;
43            this.used         = used;
44            this.available    = available;
45            this.carriesForward = carriesForward;
46        }
47
48        public String    getLeaveType()    { return leaveType; }
49        public BigDecimal getOpening()     { return opening; }
50        public BigDecimal getAccrued()     { return accrued; }
51        public BigDecimal getUsed()        { return used; }
52        public BigDecimal getAvailable()   { return available; }
53        public boolean    isCarriesForward() { return carriesForward; }
54    }
55
56    public static class CarryForwardMeterDto {
57        private BigDecimal current; // e.g. 28.00
58        private int        cap;    // always 30
59        private String     label;   // "28/30"
60        private String     status;  // SAFE | WARNING | CRITICAL | CAPPED
61
62        public CarryForwardMeterDto(BigDecimal current, int cap) {
63            this.current = current;
64            this.cap     = cap;
65            this.label   = current.toPlainString() + "/" + cap;
66            this.status  = resolveStatus(current, cap);
67        }
68
69        private String resolveStatus(BigDecimal current, int cap) {
70            int cmp = current.compareTo(BigDecimal.valueOf(cap));
71            if (cmp >= 0) return "CAPPED";

```

```

72         if (current.compareTo(BigDecimal.valueOf(cap - 1)) >= 0) return "CRITICAL";
73         if (current.compareTo(BigDecimal.valueOf(cap - 2)) >= 0) return "WARNING";
74         return "SAFE";
75     }
76
77     public BigDecimal getCurrent() { return current; }
78     public int         getCap()      { return cap; }
79     public String      getLabel()    { return label; }
80     public String      getStatus()   { return status; }
81 }
82
83 public static class WarningDto {
84     private Long id;
85     private String type;
86     private String message;
87     private String date;
88
89     public WarningDto(Long id, String type, String message, String date) {
90         this.id = id; this.type = type;
91         this.message = message; this.date = date;
92     }
93
94     public Long getId()      { return id; }
95     public String getType()  { return type; }
96     public String getMessage() { return message; }
97     public String getDate()  { return date; }
98 }
99
100 // root getters & setters
101
102 public Long         getEmployeeId()      { return employeeId; }
103 public String       getEmployeeName()    { return employeeName; }
104 public int          getYear()             { return year; }
105 public LeaveBalanceDto getSick()          { return sick; }
106 public LeaveBalanceDto getCasual()        { return casual; }
107 public LeaveBalanceDto getMaternity()     { return maternity; }
108 public CarryForwardMeterDto getCarryForwardMeter(){ return carryForwardMeter; }
109 public Map<String, LeaveBalanceDto> getLeaveBalances() { return leaveBalances; }
110 public List<WarningDto> getWarnings()     { return warnings; }
111
112 public void setEmployeeId(Long id)        { this.employeeId = id; }
113 public void setEmployeeName(String n)     { this.employeeName = n; }
114 public void setYear(int y)                { this.year = y; }
115 public void setSick(LeaveBalanceDto b)     { this.sick = b; }
116 public void setCasual(LeaveBalanceDto b)   { this.casual = b; }
117 public void setMaternity(LeaveBalanceDto b) { this.maternity = b; }
118 public void setCarryForwardMeter(CarryForwardMeterDto m) { this.carryForwardMeter = m; }
119 public void setLeaveBalances(Map<String, LeaveBalanceDto> b) { this.leaveBalances = b; }
120 public void setWarnings(List<WarningDto> w) { this.warnings = w; }
121 }
122

```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveType;
4
5 import java.time.LocalDate;
6
7 // DTO used when employee applies for leave.
8 // Contains only input fields required from employee.
9 public class LeaveRequestDTO {
10     private LeaveType leaveType;
11     private LocalDate startDate;
12     private LocalDate endDate;
13     private String reason;
14
15
16     public LeaveType getLeaveType() {
17         return leaveType;
18     }
19
20     public void setLeaveType(LeaveType leaveType) {
21         this.leaveType = leaveType;
22     }
23
24     public LocalDate getStartDate() {
25         return startDate;
26     }
27
28     public void setStartDate(LocalDate startDate) {
29         this.startDate = startDate;
30     }
31
32     public LocalDate getEndDate() {
33         return endDate;
34     }
35
36     public void setEndDate(LocalDate endDate) {
37         this.endDate = endDate;
38     }
39
40     public String getReason() {
41         return reason;
42     }
43
44     public void setReason(String reason) {
45         this.reason = reason;
46     }
47 }
48
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5
6 import java.time.LocalDate;
7
8 public class LeaveResponseDTO {
9     private long leaveRequestId;
10    private long employeeId;
11    private LeaveType leaveType;
12    private LocalDate startDate;
13    private LocalDate endDate;
14    private long numberOfDays;
15    private String reason;
16    private LeaveStatus status;
17
18    public long getLeaveRequestId() {
19        return leaveRequestId;
20    }
21
22    public void setLeaveRequestId(long leaveRequestId) {
23        this.leaveRequestId = leaveRequestId;
24    }
25
26    public long getEmployeeId() {
27        return employeeId;
28    }
29
30    public void setEmployeeId(long employeeId) {
31        this.employeeId = employeeId;
32    }
33
34    public LocalDate getStartDate() {
35        return startDate;
36    }
37
38    public void setStartDate(LocalDate startDate) {
39        this.startDate = startDate;
40    }
41
42    public LeaveType getLeaveType() {
43        return leaveType;
44    }
45
46    public void setLeaveType(LeaveType leaveType) {
47        this.leaveType = leaveType;
48    }
49
50    public LocalDate getEndDate() {
51        return endDate;
52    }
53
54    public void setEndDate(LocalDate endDate) {
55        this.endDate = endDate;
56    }
57
58    public long getNumberOfDays() {
59        return numberOfDays;
60    }
61
62    public void setNumberOfDays(long numberOfDays) {
63        this.numberOfDays = numberOfDays;
64    }
65
66    public String getReason() {
67        return reason;
68    }
69
70    public void setReason(String reason) {
71        this.reason = reason;
```

```
72     }
73
74     public LeaveStatus getStatus() {
75         return status;
76     }
77
78     public void setStatus(L leaveStatus status) {
79         this.status = status;
80     }
81 }
82
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import org.bouncycastle.crypto.signers.ISOTrailers;
4
5 public class ManagerDTO {
6     private long managerId;
7     private String name;
8     private String email;
9     private String dept;
10
11     public long getManagerId() {
12         return managerId;
13     }
14
15     public void setManagerId(long managerId) {
16         this.managerId = managerId;
17     }
18
19     public String getName() {
20         return name;
21     }
22
23     public void setName(String name) {
24         this.name = name;
25     }
26
27     public String getEmail() {
28         return email;
29     }
30
31     public void setEmail(String email) {
32         this.email = email;
33     }
34
35     public String getDept() {
36         return dept;
37     }
38
39     public void setDept(String dept) {
40         this.dept = dept;
41     }
42 }
43
```


50 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerEmployeeDetailsDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Status;
4
5 import java.util.List;
6
7 public class ManagerEmployeeDetailsDTO {
8     private Long employeeId;
9     private String name;
10    private String email;
11    private String dept;
12    private Status status;
13    private String attendance;
14    private List<DeviceResponseDTO> devices;
15
16    public Long getEmployeeId() {
17        return employeeId;
18    }
19
20    public void setEmployeeId(Long employeeId) {
21        this.employeeId = employeeId;
22    }
23
24    public String getName() {
25        return name;
26    }
27
28    public void setName(String name) {
29        this.name = name;
30    }
31
32    public String getEmail() {
33        return email;
34    }
35
36    public void setEmail(String email) {
37        this.email = email;
38    }
39
40    public String getDept() {
41        return dept;
42    }
43
44    public void setDept(String dept) {
45        this.dept = dept;
46    }
47
48    public Status getStatus() {
49        return status;
50    }
51
52    public void setStatus(Status status) {
53        this.status = status;
54    }
55
56    public String getAttendance() {
57        return attendance;
58    }
59
60    public void setAttendance(String attendance) {
61        this.attendance = attendance;
62    }
63
64    public List<DeviceResponseDTO> getDevices() {
65        return devices;
66    }
67
68    public void setDevices(List<DeviceResponseDTO> devices) {
69        this.devices = devices;
70    }
71 }
```


51 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5
6 import java.time.LocalDate;
7
8 public class ManagerLeaveResponseDTO {
9     private String employeeName;
10    private LeaveType leaveType;
11    private LocalDate startDate;
12    private LocalDate endDate;
13    private long numberOfDays;
14    private String reason;
15    private LeaveStatus status;
16
17    public String getEmployeeName() {
18        return employeeName;
19    }
20
21    public void setEmployeeName(String employeeName) {
22        this.employeeName = employeeName;
23    }
24
25    public LeaveType getLeaveType() {
26        return leaveType;
27    }
28
29    public void setLeaveType(LeaveType leaveType) {
30        this.leaveType = leaveType;
31    }
32
33    public LocalDate getStartDate() {
34        return startDate;
35    }
36
37    public void setStartDate(LocalDate startDate) {
38        this.startDate = startDate;
39    }
40
41    public LocalDate getEndDate() {
42        return endDate;
43    }
44
45    public void setEndDate(LocalDate endDate) {
46        this.endDate = endDate;
47    }
48
49    public long getNumberOfDays() {
50        return numberOfDays;
51    }
52
53    public void setNumberOfDays(long numberOfDays) {
54        this.numberOfDays = numberOfDays;
55    }
56
57    public String getReason() {
58        return reason;
59    }
60
61    public void setReason(String reason) {
62        this.reason = reason;
63    }
64
65    public LeaveStatus getStatus() {
66        return status;
67    }
68
69    public void setStatus(LeaveStatus status) {
70        this.status = status;
71    }
}
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class PromoteRequest {
4     private String email;
5     private String password;
6
7     public String getEmail() {
8         return email;
9     }
10
11     public void setEmail(String email) {
12         this.email = email;
13     }
14
15     public String getPassword() {
16         return password;
17     }
18
19     public void setPassword(String password) {
20         this.password = password;
21     }
22 }
23
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5 public class RepairDTO {
6     private Long requestId;
7     private String resolution;
8     private ServiceRequestStatus status;
9
10    public Long getRequestId() {
11        return requestId;
12    }
13
14    public void setRequestId(Long requestId) {
15        this.requestId = requestId;
16    }
17
18    public String getResolution() {
19        return resolution;
20    }
21
22    public void setResolution(String resolution) {
23        this.resolution = resolution;
24    }
25
26    public ServiceRequestStatus getStatus() {
27        return status;
28    }
29
30    public void setStatus(ServiceRequestStatus status) {
31        this.status = status;
32    }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4
5 import java.util.Set;
6
7 public class RestaurantDTO {
8     private Long id;
9     private String name;
10    private String address;
11    private String description;
12    private Set<MealSlot> supportedMealSlots;
13    private boolean active;
14    private Long vendorId;
15    private String vendorName;
16
17    public Long getId() { return id; }
18    public void setId(Long id) { this.id = id; }
19
20    public String getName() { return name; }
21    public void setName(String name) { this.name = name; }
22
23    public String getAddress() { return address; }
24    public void setAddress(String address) { this.address = address; }
25
26    public String getDescription() { return description; }
27    public void setDescription(String description) { this.description = description; }
28
29    public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
30    public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
31        this.supportedMealSlots = supportedMealSlots;
32    }
33
34    public boolean isActive() { return active; }
35    public void setActive(boolean active) { this.active = active; }
36
37    public Long getVendorId() { return vendorId; }
38    public void setVendorId(Long vendorId) { this.vendorId = vendorId; }
39
40    public String getVendorName() { return vendorName; }
41    public void setVendorName(String vendorName) { this.vendorName = vendorName; }
42 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7
8 import java.util.Set;
9
10 public class RestaurantRequest {
11
12     @NotBlank
13     private String name;
14
15     @NotBlank
16     private String address;
17
18     private String description;
19
20     /**
21      * Which meal slots this restaurant serves.
22      * Must be at least one of: BREAKFAST, LUNCH, DINNER.
23      */
24     @NotNull
25     @NotEmpty
26     private Set<MealSlot> supportedMealSlots;
27
28     /**
29      * The vendor ID of the owner making this request.
30      * Used to verify ownership on updates/deletes.
31      */
32
33     public String getName() { return name; }
34     public void setName(String name) { this.name = name; }
35
36     public String getAddress() { return address; }
37     public void setAddress(String address) { this.address = address; }
38
39     public String getDescription() { return description; }
40     public void setDescription(String description) { this.description = description; }
41
42     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
43     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
44         this.supportedMealSlots = supportedMealSlots;
45     }
46 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
4
5 public class ServiceRequestDTO {
6
7     private Long deviceId;
8     private ServiceRequestType requestType;
9     private String issueDescription;
10    private boolean urgent;
11
12    public Long getDeviceId() {
13        return deviceId;
14    }
15
16    public void setDeviceId(Long deviceId) {
17        this.deviceId = deviceId;
18    }
19
20    public ServiceRequestType getRequestType() {
21        return requestType;
22    }
23
24    public void setRequestType(ServiceRequestType requestType) {
25        this.requestType = requestType;
26    }
27
28    public String getIssueDescription() {
29        return issueDescription;
30    }
31
32    public void setIssueDescription(String issueDescription) {
33        this.issueDescription = issueDescription;
34    }
35
36    public boolean isUrgent() {
37        return urgent;
38    }
39
40    public void setUrgent(boolean urgent) {
41        this.urgent = urgent;
42    }
43 }
44
```

57 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDateTime;
4
5 public class ServiceRequestResponseDTO
6 {
7     private Long id;
8     private String requestType;
9     private String status;
10    private String issueDescription;
11    private String adminRemarks;
12    private String resolution;
13    private boolean urgent;
14    private LocalDateTime raisedAt;
15    private LocalDateTime resolvedAt;
16    private String deviceName;
17    private String raisedByName;
18    private String reviewedByName;
19
20    public String getDeviceName() {
21        return deviceName;
22    }
23
24    public void setDeviceName(String deviceName) {
25        this.deviceName = deviceName;
26    }
27
28    public String getReviewedByName() {
29        return reviewedByName;
30    }
31
32    public void setReviewedByName(String reviewedByName) {
33        this.reviewedByName = reviewedByName;
34    }
35
36    public LocalDateTime getResolvedAt() {
37        return resolvedAt;
38    }
39
40    public void setResolvedAt(LocalDateTime resolvedAt) {
41        this.resolvedAt = resolvedAt;
42    }
43
44    public LocalDateTime getRaisedAt() {
45        return raisedAt;
46    }
47
48    public void setRaisedAt(LocalDateTime raisedAt) {
49        this.raisedAt = raisedAt;
50    }
51
52    public boolean isUrgent() {
53        return urgent;
54    }
55
56    public void setUrgent(boolean urgent) {
57        this.urgent = urgent;
58    }
59
60    public String getResolution() {
61        return resolution;
62    }
63
64    public void setResolution(String resolution) {
65        this.resolution = resolution;
66    }
67
68    public Long getId() {
69        return id;
70    }
71}
```

```
72     public void setId(Long id) {
73         this.id = id;
74     }
75
76     public String getRequestType() {
77         return requestType;
78     }
79
80     public void setRequestType(String requestType) {
81         this.requestType = requestType;
82     }
83
84     public String getStatus() {
85         return status;
86     }
87
88     public void setStatus(String status) {
89         this.status = status;
90     }
91
92     public String getAdminRemarks() {
93         return adminRemarks;
94     }
95
96     public void setAdminRemarks(String adminRemarks) {
97         this.adminRemarks = adminRemarks;
98     }
99
100    public String getIssueDescription() {
101        return issueDescription;
102    }
103
104    public void setIssueDescription(String issueDescription) {
105        this.issueDescription = issueDescription;
106    }
107
108    public String getRaisedByName() {
109        return raisedByName;
110    }
111
112    public void setRaisedByName(String raisedByName) {
113        this.raisedByName = raisedByName;
114    }
115 }
116
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
6
7 import java.time.DayOfWeek;
8 import java.time.ZonedDateTime;
9
10 public class SubscriptionDTO {
11     private long subscriptionId;
12     private long userId;
13     private MealSlot mealSlot;
14     private ScheduleType scheduleType;
15     private DayOfWeek dayOfWeek;
16     private SubscriptionStatus status;
17     private ZonedDateTime nextDeliveryTime;
18
19     private Long restaurantId;
20     private String restaurantName;
21     private String restaurantAddress;
22
23     public void setRestaurantId(Long restaurantId) {
24         this.restaurantId = restaurantId;
25     }
26
27     public void setRestaurantName(String restaurantName) {
28         this.restaurantName = restaurantName;
29     }
30
31     public void setRestaurantAddress(String restaurantAddress) {
32         this.restaurantAddress = restaurantAddress;
33     }
34
35     public Long getRestaurantId() {
36         return restaurantId;
37     }
38
39     public String getRestaurantName() {
40         return restaurantName;
41     }
42
43     public String getRestaurantAddress() {
44         return restaurantAddress;
45     }
46
47     public long getSubscriptionId() {
48         return subscriptionId;
49     }
50
51     public void setSubscriptionId(long subscriptionId) {
52         this.subscriptionId = subscriptionId;
53     }
54
55     public long getUserId() {
56         return userId;
57     }
58
59     public void setUserId(long userId) {
60         this.userId = userId;
61     }
62
63     public MealSlot getMealSlot() {
64         return mealSlot;
65     }
66
67     public void setMealSlot(MealSlot mealSlot) {
68         this.mealSlot = mealSlot;
69     }
70
71     public ScheduleType getScheduleType() {
```

```
72         return scheduleType;
73     }
74
75     public void setScheduleType(ScheduleType scheduleType) {
76         this.scheduleType = scheduleType;
77     }
78
79     public DayOfWeek getDayOfWeek() {
80         return dayOfWeek;
81     }
82
83     public void setDayOfWeek(DayOfWeek dayOfWeek) {
84         this.dayOfWeek = dayOfWeek;
85     }
86
87     public SubscriptionStatus getStatus() {
88         return status;
89     }
90
91     public void setStatus(SubscriptionStatus status) {
92         this.status = status;
93     }
94
95     public ZonedDateTime getNextDeliveryTime() {
96         return nextDeliveryTime;
97     }
98
99     public void setNextDeliveryTime(ZonedDateTime nextDeliveryTime) {
100         this.nextDeliveryTime = nextDeliveryTime;
101     }
102 }
103
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7
8 import java.time.DayOfWeek;
9 import java.util.List;
10
11 public class SubscriptionRequest {
12     private ScheduleType scheduleType;
13     private DayOfWeek dayOfWeek;
14     @NotNull
15     @NotEmpty
16     private List<MealSlot> mealSlots;
17     @NotNull
18     private List<SlotOrder> slotOrders;
19     public static class SlotOrder {
20         @NotNull
21         private MealSlot mealSlot;
22
23         @NotNull
24         private Long restaurantId;
25
26         public MealSlot getMealSlot() {
27             return mealSlot;
28         }
29
30         public void setMealSlot(MealSlot mealSlot) {
31             this.mealSlot = mealSlot;
32         }
33
34         public Long getRestaurantId() {
35             return restaurantId;
36         }
37
38         public void setRestaurantId(Long restaurantId) {
39             this.restaurantId = restaurantId;
40         }
41     }
42
43     public ScheduleType getScheduleType() {
44         return scheduleType;
45     }
46
47     public void setScheduleType(ScheduleType scheduleType) {
48         this.scheduleType = scheduleType;
49     }
50
51     public List<MealSlot> getMealSlots() {
52         return mealSlots;
53     }
54
55     public void setMealSlots(List<MealSlot> mealSlots) {
56         this.mealSlots = mealSlots;
57     }
58
59     public DayOfWeek getDayOfWeek() {
60         return dayOfWeek;
61     }
62
63     public void setDayOfWeek(DayOfWeek dayOfWeek) {
64         this.dayOfWeek = dayOfWeek;
65     }
66
67
68
69     public List<SlotOrder> getSlotOrders() { return slotOrders; }
70     public void setSlotOrders(List<SlotOrder> slotOrders) { this.slotOrders = slotOrders; }
71 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDate;
4
5 public class VendorDTO {
6     private Long id;
7     private String name;
8     private String email;
9     private String phone;
10    private LocalDate registeredAt;
11
12    public Long getId() { return id; }
13    public void setId(Long id) { this.id = id; }
14
15    public String getName() { return name; }
16    public void setName(String name) { this.name = name; }
17
18    public String getEmail() { return email; }
19    public void setEmail(String email) { this.email = email; }
20
21    public String getPhone() { return phone; }
22    public void setPhone(String phone) { this.phone = phone; }
23
24    public LocalDate getRegisteredAt() { return registeredAt; }
25    public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
26 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6
7 public class VendorRequest {
8
9     @NotBlank
10     private String name;
11
12     @Email
13     @NotBlank
14     private String email;
15
16     @NotBlank
17     private String phone;
18
19     private String password;
20
21     private Role role;
22
23     public String getName() { return name; }
24     public void setName(String name) { this.name = name; }
25
26     public String getEmail() { return email; }
27     public void setEmail(String email) { this.email = email; }
28
29     public String getPhone() { return phone; }
30     public void setPhone(String phone) { this.phone = phone; }
31
32     public String getPassword() {
33         return password;
34     }
35
36     public void setPassword(String password) {
37         this.password = password;
38     }
39
40     public Role getRole() {
41         return role;
42     }
43
44     public void setRole(Role role) {
45         this.role = role;
46     }
47 }
```

62 Current:

src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
1 package com.example.EmployeeManagementSystem;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.scheduling.annotation.EnableScheduling;
6
7 @SpringBootApplication
8 @EnableScheduling
9 public class EmployeeManagementSystemApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(EmployeeManagementSystemApplication.class, args);
13     }
14
15 }
16
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/Entity/ApiKey.java
2 package com.example.EmployeeManagementSystem.Entity;
3
4 import jakarta.persistence.*;
5 import java.time.Instant;
6 import java.util.UUID;
7
8 @Entity
9 @Table(name = "api_keys")
10 public class ApiKey {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @Column(nullable = false, unique = true)
17     private String apiKey;
18
19     @Column(nullable = false)
20     private String keyName;
21
22     @ManyToOne
23     @JoinColumn(name = "employee_id")
24     private Employee employee;
25
26     @ManyToOne
27     @JoinColumn(name = "vendor_id")
28     private Vendor vendor;
29
30     @Column(nullable = false)
31     private String permissions; // Comma-separated permissions
32
33     @Column(nullable = false)
34     private Instant createdAt;
35
36     private Instant expiresAt;
37
38     private Instant lastUsedAt;
39
40     @Column(nullable = false)
41     private boolean active = true;
42
43     @PrePersist
44     public void prePersist() {
45         this.apiKey = "ems_" + UUID.randomUUID().toString().replace("-", "");
46         this.createdAt = Instant.now();
47     }
48
49     // Getters and setters
50     public Long getId() { return id; }
51     public void setId(Long id) { this.id = id; }
52
53     public String getApiKey() { return apiKey; }
54     public void setApiKey(String apiKey) { this.apiKey = apiKey; }
55
56     public String getKeyName() { return keyName; }
57     public void setKeyName(String keyName) { this.keyName = keyName; }
58
59     public Employee getEmployee() { return employee; }
60     public void setEmployee(Employee employee) { this.employee = employee; }
61
62     public Vendor getVendor() { return vendor; }
63     public void setVendor(Vendor vendor) { this.vendor = vendor; }
64
65     public String getPermissions() { return permissions; }
66     public void setPermissions(String permissions) { this.permissions = permissions; }
67
68     public Instant getCreatedAt() { return createdAt; }
69     public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
70
71     public Instant getExpiresAt() { return expiresAt; }
```

```
72     public void setExpiresAt(Instant expiresAt) { this.expiresAt = expiresAt; }
73
74     public Instant getLastUsedAt() { return lastUsedAt; }
75     public void setLastUsedAt(Instant lastUsedAt) { this.lastUsedAt = lastUsedAt; }
76
77     public boolean isActive() { return active; }
78     public void setActive(boolean active) { this.active = active; }
79 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
4 import com.example.EmployeeManagementSystem.Enum.MealSlot;
5 import jakarta.persistence.*;
6
7 import java.time.Instant;
8
9 @Entity
10 @Table(name = "delivery", uniqueConstraints = { @UniqueConstraint(name = "uq_subscription_scheduled_time",
11     columnNames = {"subscription_id", "scheduledDeliveryTime"})})
12 public class Delivery {
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @ManyToOne
18     @JoinColumn(name = "subscription_id")
19     private Subscription subscription;
20     private MealSlot mealSlot;
21     private Instant scheduledDeliveryTime;
22     private Instant actualDeliveryTime;
23
24     @Enumerated(EnumType.STRING)
25     private DeliveryStatus status;
26
27     private Instant createdAt;
28
29     @PrePersist
30     public void prePersist() {
31         createdAt = Instant.now();
32         if (status == null) {
33             status = DeliveryStatus.SCHEDULED;
34         }
35     }
36
37     public MealSlot getMealSlot() {
38         return mealSlot;
39     }
40
41     public void setMealSlot(MealSlot mealSlot) {
42         this.mealSlot = mealSlot;
43     }
44
45     public Subscription getSubscription() {
46         return subscription;
47     }
48
49     public void setSubscription(Subscription subscription) {
50         this.subscription = subscription;
51     }
52
53     public Long getId() {
54         return id;
55     }
56
57     public void setId(Long id) {
58         this.id = id;
59     }
60
61     public Instant getScheduledDeliveryTime() {
62         return scheduledDeliveryTime;
63     }
64
65     public void setScheduledDeliveryTime(Instant scheduledDeliveryTime) {
66         this.scheduledDeliveryTime = scheduledDeliveryTime;
67     }
68
69     public Instant getActualDeliveryTime() {
70         return actualDeliveryTime;
71     }
```

```
72
73     public void setActualDeliveryTime(Instant actualDeliveryTime) {
74         this.actualDeliveryTime = actualDeliveryTime;
75     }
76
77     public DeliveryStatus getStatus() {
78         return status;
79     }
80
81     public void setStatus(DeliveryStatus status) {
82         this.status = status;
83     }
84
85     public Instant getCreatedAt() {
86         return createdAt;
87     }
88
89     public void setCreatedAt(Instant createdAt) {
90         this.createdAt = createdAt;
91     }
92 }
93
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
5 import com.example.EmployeeManagementSystem.Enum.DeviceType;
6 import jakarta.persistence.*;
7
8 import java.time.LocalDate;
9 import java.time.LocalDateTime;
10
11 @Entity
12 public class Device {
13     @Id
14     @GeneratedValue(strategy = GenerationType.AUTO)
15     private Long id;
16
17     private String deviceName;        // e.g. "Dell XPS 15"
18     private String brand;
19
20     @ManyToOne
21     @JoinColumn(name = "vendor_id")
22     private Vendor techVendor;
23
24     @OneToOne(mappedBy = "device")
25     private DeviceAssignment currentAssignment;
26
27     private DeviceStatus deviceStatus;
28
29     private DeviceType deviceType;
30
31     private LocalDateTime createdAt;
32
33     private LocalDate warrantyExpiryDate;
34
35     public Long getId() {
36         return id;
37     }
38
39     public void setId(Long id) {
40         this.id = id;
41     }
42
43     public String getDeviceName() {
44         return deviceName;
45     }
46
47     public void setDeviceName(String deviceName) {
48         this.deviceName = deviceName;
49     }
50
51     public String getBrand() {
52         return brand;
53     }
54
55     public void setBrand(String brand) {
56         this.brand = brand;
57     }
58
59     public Vendor getTechVendor() {
60         return techVendor;
61     }
62
63     public void setTechVendor(Vendor techVendor) {
64         this.techVendor = techVendor;
65     }
66
67     public DeviceType getDeviceType() {
68         return deviceType;
69     }
70
71     public void setDeviceType(DeviceType deviceType) {
```

```
72         this.deviceType = deviceType;
73     }
74
75     public LocalDateTime getCreatedAt() {
76         return createdAt;
77     }
78
79     public void setCreatedAt(LocalDateTime createdAt) {
80         this.createdAt = createdAt;
81     }
82
83     public DeviceStatus getDeviceStatus() {
84         return deviceStatus;
85     }
86
87     public void setDeviceStatus(DeviceStatus deviceStatus) {
88         this.deviceStatus = deviceStatus;
89     }
90
91     public DeviceAssignment getCurrentAssignment() {
92         return currentAssignment;
93     }
94
95     public void setCurrentAssignment(DeviceAssignment currentAssignment) {
96         this.currentAssignment = currentAssignment;
97     }
98
99     public LocalDate getWarrantyExpiryDate() {
100         return warrantyExpiryDate;
101     }
102
103     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
104         this.warrantyExpiryDate = warrantyExpiryDate;
105     }
106 }
107
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
4 import jakarta.persistence.*;
5
6 import java.time.LocalDate;
7
8 @Entity
9 public class DeviceAssignment {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @OneToOne
16     @JoinColumn(name = "device_id")
17     private Device device;
18
19     @ManyToOne
20     @JoinColumn(name = "employee_id")
21     private Employee assignedTo;
22
23     @Enumerated(EnumType.STRING)
24     private AssignmentStatus status;
25
26     private LocalDate assignedDate;
27
28     public Long getId() {
29         return id;
30     }
31
32     public void setId(Long id) {
33         this.id = id;
34     }
35
36     public Device getDevice() {
37         return device;
38     }
39
40     public void setDevice(Device device) {
41         this.device = device;
42     }
43
44     public Employee getAssignedTo() {
45         return assignedTo;
46     }
47
48     public void setAssignedTo(Employee assignedTo) {
49         this.assignedTo = assignedTo;
50     }
51
52     public LocalDate getAssignedDate() {
53         return assignedDate;
54     }
55
56     public void setAssignedDate(LocalDate assignedDate) {
57         this.assignedDate = assignedDate;
58     }
59
60     public AssignmentStatus getStatus() {
61         return status;
62     }
63
64     public void setStatus(AssignmentStatus status) {
65         this.status = status;
66     }
67 }
68
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Enum.Status;
6 import com.fasterxml.jackson.annotation.JsonIgnore;
7 import jakarta.persistence.*;
8 import org.springframework.security.core.GrantedAuthority;
9 import org.springframework.security.core.authority.SimpleGrantedAuthority;
10 import org.springframework.security.core.userdetails.UserDetails;
11
12 import java.time.LocalDate;
13 import java.util.Collection;
14 import java.util.HashSet;
15 import java.util.List;
16 import java.util.Set;
17 import java.util.stream.Collectors;
18
19 @Entity
20 public class Employee implements UserDetails {
21     @Id
22     @GeneratedValue(strategy = GenerationType.IDENTITY)
23     private long employeeId;
24     @Column(nullable = true, unique = false)
25     private String name;
26     @Column(unique = true, nullable = false)
27     private String email;
28     @Column(nullable = true)
29     private String dept;
30     @Enumerated(EnumType.STRING)
31     private Status status;
32     private String password;
33
34     // Self-referential – replaces `String manager`
35     @ManyToOne(fetch = FetchType.LAZY)
36     @JoinColumn(name = "manager_id", nullable = true)
37     @JsonIgnore // prevent infinite recursion in serialization
38     private Employee manager;
39
40
41     private LocalDate joined_at;
42     @Enumerated(EnumType.STRING)
43     private Role role;
44     @Column(nullable = true)
45     private String totpSecret;
46     private boolean totpEnabled = false;
47     /** IANA timezone id, e.g. "Asia/Kolkata", "America/New_York". Defaults to UTC. */
48     @Column(nullable = false)
49     private String timezone = "UTC";
50     @Enumerated(EnumType.STRING)
51     @Column(nullable = true)
52     private Gender gender;
53
54     public String getTimezone() {
55         return timezone;
56     }
57
58     public void setTimezone(String timezone) {
59         this.timezone = timezone;
60     }
61
62     public Role getRole() {
63         return role;
64     }
65
66     public void setRole(Role role) {
67         this.role = role;
68     }
69
70     public long getEmployeeId() {
71         return employeeId;
```

```

72     }
73
74     public void setEmployeeId(long employeeId) {
75         this.employeeId = employeeId;
76     }
77
78     public String getName() {
79         return name;
80     }
81
82     public void setName(String name) {
83         this.name = name;
84     }
85
86     public String getEmail() {
87         return email;
88     }
89
90     public void setEmail(String email) {
91         this.email = email;
92     }
93
94     public String getDept() {
95         return dept;
96     }
97
98     public void setDept(String dept) {
99         this.dept = dept;
100    }
101
102    public Status getStatus() {
103        return status;
104    }
105
106    public void setStatus(Status status) {
107        this.status = status;
108    }
109
110    public LocalDate getJoined_at() {
111        return joined_at;
112    }
113
114    public void setJoined_at(LocalDate joined_at) {
115        this.joined_at = joined_at;
116    }
117
118    public Gender getGender() { return gender; }
119    public void setGender(Gender gender) { this.gender = gender; }
120
121    @PrePersist
122    public void initialSetup() {
123        this.status = Status.ACTIVE;
124        this.joined_at = LocalDate.now();
125        if (this.timezone == null || this.timezone.isBlank()) {
126            this.timezone = "UTC";
127        }
128    }
129
130    @Override
131    public Collection<? extends GrantedAuthority> getAuthorities() {
132        Set<SimpleGrantedAuthority> authorities=new HashSet<>();
133        authorities.add(new SimpleGrantedAuthority("ROLE_"+role.name()));
134        authorities.addAll(role.getPermissions().stream()
135            .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
136            .collect(Collectors.toSet()));
137        return authorities;
138    }
139
140    public String getPassword() {
141        return password;
142    }
143
144    @Override
145    public String getUsername() {
146        return email;
147    }

```

```
148
149     @Override
150     public boolean isAccountNonExpired() {
151         return UserDetails.super.isAccountNonExpired();
152     }
153
154     @Override
155     public boolean isAccountNonLocked() {
156         return UserDetails.super.isAccountNonLocked();
157     }
158
159     @Override
160     public boolean isCredentialsNonExpired() {
161         return UserDetails.super.isCredentialsNonExpired();
162     }
163
164     @Override
165     public boolean isEnabled() {
166         return UserDetails.super.isEnabled();
167     }
168
169     public void setPassword(String password) {
170         this.password = password;
171     }
172
173     public String getTotpSecret() {
174         return totpSecret;
175     }
176
177     public boolean isTotpEnabled() {
178         return totpEnabled;
179     }
180
181     public void setTotpSecret(String totpSecret) {
182         this.totpSecret = totpSecret;
183     }
184
185     public void setTotpEnabled(boolean totpEnabled) {
186         this.totpEnabled = totpEnabled;
187     }
188
189     public Employee getManager() {
190         return manager;
191     }
192
193     public void setManager(Employee manager) {
194         this.manager = manager;
195     }
196
197 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.LocalDate;
6
7 @Entity
8 public class FixedHoliday {
9
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private String name;           // "Diwali", "Christmas"
16     private LocalDate date;       // 2026-10-20
17     private String description;   // optional note
18     private int year;
19
20
21     public Long getId() {
22         return id;
23     }
24
25     public void setId(Long id) {
26         this.id = id;
27     }
28
29     public String getName() {
30         return name;
31     }
32
33     public void setName(String name) {
34         this.name = name;
35     }
36
37     public LocalDate getDate() {
38         return date;
39     }
40
41     public void setDate(LocalDate date) {
42         this.date = date;
43     }
44
45     public int getYear() {
46         return year;
47     }
48
49     public void setYear(int year) {
50         this.year = year;
51     }
52
53     public String getDescription() {
54         return description;
55     }
56
57     public void setDescription(String description) {
58         this.description = description;
59     }
60 }
61
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.math.BigDecimal;
6
7 @Entity
8 @Table(name = "leave_entitlement", uniqueConstraints = @UniqueConstraint(columnNames = {"employee_id",
9 "leave_type_id", "year"}))
10 public class LeaveEntitlement {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne(fetch = FetchType.LAZY, optional = false)
17     @JoinColumn(name = "employee_id", nullable = false)
18     private Employee employee;
19
20     @ManyToOne(fetch = FetchType.LAZY, optional = false)
21     @JoinColumn(name = "leave_type_id", nullable = false)
22     private LeaveType leaveType;
23
24     @Column(nullable = false)
25     private Integer year;
26
27     @Column(nullable = false, precision = 5, scale = 2)
28     private BigDecimal openingBalance = BigDecimal.ZERO;
29
30     @Column(nullable = false, precision = 5, scale = 2)
31     private BigDecimal accruedThisYear = BigDecimal.ZERO;
32
33     @Column(nullable = false, precision = 5, scale = 2)
34     private BigDecimal usedThisYear = BigDecimal.ZERO;
35
36     @Column(nullable = false, precision = 5, scale = 2)
37     private BigDecimal closingBalance = BigDecimal.ZERO;
38
39     // Convenience: current available = opening + accrued - used
40     @Transient
41     public BigDecimal getAvailableBalance() {
42         return openingBalance.add(accruedThisYear).subtract(usedThisYear);
43     }
44
45     // --- getters & setters ---
46     public Long getId() {
47         return id;
48     }
49
50     public Employee getEmployee() {
51         return employee;
52     }
53
54     public LeaveType getLeaveType() {
55         return leaveType;
56     }
57
58     public Integer getYear() {
59         return year;
60     }
61
62     public BigDecimal getOpeningBalance() {
63         return openingBalance;
64     }
65
66     public BigDecimal getAccruedThisYear() {
67         return accruedThisYear;
68     }
69
70     public BigDecimal getUsedThisYear() {
71         return usedThisYear;
```

```
71     }
72
73     public BigDecimal getClosingBalance() {
74         return closingBalance;
75     }
76
77     public void setEmployee(Employee e) {
78         this.employee = e;
79     }
80
81     public void setLeaveType(LeaveType lt) {
82         this.leaveType = lt;
83     }
84
85     public void setYear(Integer y) {
86         this.year = y;
87     }
88
89     public void setOpeningBalance(BigDecimal b) {
90         this.openingBalance = b;
91     }
92
93     public void setAccruedThisYear(BigDecimal a) {
94         this.accruedThisYear = a;
95     }
96
97     public void setUsedThisYear(BigDecimal u) {
98         this.usedThisYear = u;
99     }
100
101     public void setClosingBalance(BigDecimal c) {
102         this.closingBalance = c;
103     }
104 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5 import jakarta.persistence.*;
6
7 import java.time.LocalDate;
8
9 @Entity
10 public class LeaveRequest {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private long id;
14     @Enumerated(EnumType.STRING)
15     private LeaveType leaveType;
16     private LocalDate startDate;
17     private LocalDate endDate;
18     private String reason;
19     @Enumerated(EnumType.STRING)
20     private LeaveStatus status;
21     private String Manager;
22     private String remarks;
23     @ManyToOne
24     @JoinColumn(name = "employee_id")
25     private Employee employee;
26
27
28     public long getId() {
29         return id;
30     }
31
32     public void setId(long id) {
33         this.id = id;
34     }
35
36     public LeaveType getLeaveType() {
37         return leaveType;
38     }
39
40     public void setLeaveType(LeaveType leaveType) {
41         this.leaveType = leaveType;
42     }
43
44     public LocalDate getStartDate() {
45         return startDate;
46     }
47
48     public void setStartDate(LocalDate startDate) {
49         this.startDate = startDate;
50     }
51
52     public LocalDate getEndDate() {
53         return endDate;
54     }
55
56     public void setEndDate(LocalDate endDate) {
57         this.endDate = endDate;
58     }
59
60     public LeaveStatus getStatus() {
61         return status;
62     }
63
64     public void setStatus(LeaveStatus status) {
65         this.status = status;
66     }
67
68     public String getManager() {
69         return Manager;
70     }
71 }
```



```
72     public void setManager(String manager) {
73         Manager = manager;
74     }
75
76     public String getRemarks() {
77         return remarks;
78     }
79
80     public void setRemarks(String remarks) {
81         this.remarks = remarks;
82     }
83
84     public Employee getEmployee() {
85         return employee;
86     }
87
88     public void setEmployee(Employee employee) {
89         this.employee = employee;
90     }
91
92     public String getReason() {
93         return reason;
94     }
95
96     public void setReason(String reason) {
97         this.reason = reason;
98     }
99
100     @PrePersist
101     public void init(){
102         this.status=LeaveStatus.PENDING;
103     }
104 }
105
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4 import jakarta.persistence.*;
5
6 @Entity
7 @Table(name = "leave_type")
8 public class LeaveType {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Integer id;
13
14     @Column(nullable = false, unique = true, length = 50)
15     private String name;           // SICK, CASUAL, MATERNITY
16
17     @Column(nullable = false)
18     private boolean carriesForward;
19
20     @Column(nullable = false)
21     private boolean monthlyReset;   // true = sick leave wipes at month end
22
23     @Column(nullable = false)
24     private int maxCarryForward;    // 30 for carry-forwardable types
25
26     @Column(nullable = false)
27     private boolean genderRestricted;
28
29     @Enumerated(EnumType.STRING)
30     @Column(nullable = true)
31     private Gender genderRestriction; // null = open to all
32
33     // --- getters & setters ---
34     public Integer getId() { return id; }
35     public String getName() { return name; }
36     public boolean isCarriesForward() { return carriesForward; }
37     public boolean isMonthlyReset() { return monthlyReset; }
38     public int getMaxCarryForward() { return maxCarryForward; }
39     public boolean isGenderRestricted() { return genderRestricted; }
40     public Gender getGenderRestriction() { return genderRestriction; }
41
42     public void setId(Integer id) { this.id = id; }
43     public void setName(String name) { this.name = name; }
44     public void setCarriesForward(boolean cf) { this.carriesForward = cf; }
45     public void setMonthlyReset(boolean mr) { this.monthlyReset = mr; }
46     public void setMaxCarryForward(int max) { this.maxCarryForward = max; }
47     public void setGenderRestricted(boolean gr) { this.genderRestricted = gr; }
48     public void setGenderRestriction(Gender g) { this.genderRestriction = g; }
49 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDate;
7
8 @Entity
9 @Table(name = "leave_warning")
10 public class LeaveWarning {
11
12     public enum WarningType {
13         APPROACHING_28, // 2 months before cap
14         APPROACHING_29, // 1 month before cap
15         CAP_REACHED,    // exactly at 30
16         DAY_WASTED      // already at 30, day lost
17     }
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long id;
22
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "employee_id", nullable = false)
25     private Employee employee;
26
27     @Enumerated(EnumType.STRING)
28     @Column(nullable = false, length = 30)
29     private WarningType warningType;
30
31     @Column(nullable = false, columnDefinition = "TEXT")
32     private String message;
33
34     @Column(nullable = false, precision = 5, scale = 2)
35     private BigDecimal carryForwardBalance;
36
37     @Column(nullable = false)
38     private LocalDate warningDate;
39
40     @Column(nullable = false)
41     private boolean isRead = false;
42
43     // --- getters & setters ---
44     public Long getId() { return id; }
45     public Employee getEmployee() { return employee; }
46     public WarningType getWarningType() { return warningType; }
47     public String getMessage() { return message; }
48     public BigDecimal getCarryForwardBalance() { return carryForwardBalance; }
49     public LocalDate getWarningDate() { return warningDate; }
50     public boolean isRead() { return isRead; }
51
52     public void setEmployee(Employee e) { this.employee = e; }
53     public void setWarningType(WarningType wt) { this.warningType = wt; }
54     public void setMessage(String m) { this.message = m; }
55     public void setCarryForwardBalance(BigDecimal b) { this.carryForwardBalance = b; }
56     public void setWarningDate(LocalDate d) { this.warningDate = d; }
57     public void setRead(boolean r) { this.isRead = r; }
58 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "notification")
8 public class Notification {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     @ManyToOne(fetch = FetchType.LAZY)
15     @JoinColumn(name = "employee_id", nullable = false)
16     private Employee recipient;
17
18     @Column(nullable = false)
19     private String message;
20
21     @Column(nullable = false)
22     private String type; // LEAVE_APPROVED, LEAVE_REJECTED, LEAVE_REQUEST, HOLIDAY_ADDED
23
24     @Column(nullable = false)
25     private boolean isRead = false;
26
27     @Column(nullable = false)
28     private LocalDateTime createdAt = LocalDateTime.now();
29
30     // getters and setters
31     public Long getId() { return id; }
32     public Employee getRecipient() { return recipient; }
33     public void setRecipient(Employee recipient) { this.recipient = recipient; }
34     public String getMessage() { return message; }
35     public void setMessage(String message) { this.message = message; }
36     public String getType() { return type; }
37     public void setType(String type) { this.type = type; }
38     public boolean isRead() { return isRead; }
39     public void setRead(boolean read) { isRead = read; }
40     public LocalDateTime getCreatedAt() { return createdAt; }
41 }
42
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.Instant;
6 @Entity
7 public class RefreshToken {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    @Column(nullable = false, unique = true)
13    private String token;
14
15    @Column(nullable = false)
16    private Instant expiryTime;
17
18    @ManyToOne
19    @JoinColumn(name = "employee_id")
20    private Employee employee;
21
22    @ManyToOne
23    @JoinColumn(name = "vendor_id")
24    private Vendor vendor;
25
26    public Long getId() {
27        return id;
28    }
29
30    public String getToken() {
31        return token;
32    }
33
34    public void setToken(String token) {
35        this.token = token;
36    }
37
38    public Instant getExpiryTime() {
39        return expiryTime;
40    }
41
42    public void setExpiryTime(Instant expiryTime) {
43        this.expiryTime = expiryTime;
44    }
45
46    public Employee getEmployee() {
47        return employee;
48    }
49
50    public void setEmployee(Employee employee) {
51        this.employee = employee;
52    }
53
54    public Vendor getVendor() {
55        return vendor;
56    }
57
58    public void setVendor(Vendor vendor) {
59        this.vendor = vendor;
60    }
61 }
62
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import jakarta.persistence.*;
5 import java.util.Set;
6
7
8 @Entity
9 public class Restaurant {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false)
16     private String name;
17
18     @Column(nullable = false)
19     private String address;
20
21     private String description;
22
23     @ElementCollection(targetClass = MealSlot.class, fetch = FetchType.EAGER)
24     @CollectionTable(name = "restaurant_meal_slots",
25         joinColumns = @JoinColumn(name = "restaurant_id"))
26     @Enumerated(EnumType.STRING)
27     @Column(name = "meal_slot")
28     private Set<MealSlot> supportedMealSlots;
29
30     private boolean active = true;
31
32     @ManyToOne(fetch = FetchType.LAZY)
33     @JoinColumn(name = "vendor_id", nullable = false)
34     private Vendor vendor;
35
36     public Long getId() { return id; }
37     public void setId(Long id) { this.id = id; }
38
39     public String getName() { return name; }
40     public void setName(String name) { this.name = name; }
41
42     public String getAddress() { return address; }
43     public void setAddress(String address) { this.address = address; }
44
45     public String getDescription() { return description; }
46     public void setDescription(String description) { this.description = description; }
47
48     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
49     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
50         this.supportedMealSlots = supportedMealSlots;
51     }
52
53     public boolean isActive() { return active; }
54     public void setActive(boolean active) { this.active = active; }
55
56     public Vendor getVendor() { return vendor; }
57     public void setVendor(Vendor vendor) { this.vendor = vendor; }
58 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4 import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
5 import jakarta.persistence.*;
6
7 import java.time.LocalDateTime;
8
9 @Entity
10 public class ServiceRequest {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne
16     @JoinColumn(name = "device_id")
17     private Device device;
18
19     @ManyToOne
20     @JoinColumn(name = "raised_by")
21     private Employee raisedBy; // employee reporting issue
22
23     @ManyToOne
24     @JoinColumn(name = "reviewed_by")
25     private Employee reviewedBy; // admin who reviewed
26
27     @Enumerated(EnumType.STRING)
28     private ServiceRequestType requestType;
29
30     @Enumerated(EnumType.STRING)
31     private ServiceRequestStatus status;
32
33     private String issueDescription; // "screen cracked", "won't boot"
34     private String adminRemarks; // admin notes on decision
35     private String resolution; // what was done finally
36
37     private boolean urgent; // employee can flag as urgent
38
39     private LocalDateTime raisedAt;
40     private LocalDateTime resolvedAt;
41
42     @PrePersist
43     void onCreate() { this.raisedAt = LocalDateTime.now(); }
44
45     public Employee getRaisedBy() {
46         return raisedBy;
47     }
48
49     public void setRaisedBy(Employee raisedBy) {
50         this.raisedBy = raisedBy;
51     }
52
53     public Device getDevice() {
54         return device;
55     }
56
57     public void setDevice(Device device) {
58         this.device = device;
59     }
60
61     public Long getId() {
62         return id;
63     }
64
65     public void setId(Long id) {
66         this.id = id;
67     }
68
69     public Employee getReviewedBy() {
70         return reviewedBy;
71     }
```

```
72
73     public void setReviewedBy(Employee reviewedBy) {
74         this.reviewedBy = reviewedBy;
75     }
76
77     public ServiceRequestStatus getStatus() {
78         return status;
79     }
80
81     public void setStatus(ServiceRequestStatus status) {
82         this.status = status;
83     }
84
85     public ServiceRequestType getRequestType() {
86         return requestType;
87     }
88
89     public void setRequestType(ServiceRequestType requestType) {
90         this.requestType = requestType;
91     }
92
93     public String getIssueDescription() {
94         return issueDescription;
95     }
96
97     public void setIssueDescription(String issueDescription) {
98         this.issueDescription = issueDescription;
99     }
100
101     public String getAdminRemarks() {
102         return adminRemarks;
103     }
104
105     public void setAdminRemarks(String adminRemarks) {
106         this.adminRemarks = adminRemarks;
107     }
108
109     public String getResolution() {
110         return resolution;
111     }
112
113     public void setResolution(String resolution) {
114         this.resolution = resolution;
115     }
116
117     public boolean isUrgent() {
118         return urgent;
119     }
120
121     public void setUrgent(boolean urgent) {
122         this.urgent = urgent;
123     }
124
125     public LocalDateTime getRaisedAt() {
126         return raisedAt;
127     }
128
129     public void setRaisedAt(LocalDateTime raisedAt) {
130         this.raisedAt = raisedAt;
131     }
132
133     public LocalDateTime getResolvedAt() {
134         return resolvedAt;
135     }
136
137     public void setResolvedAt(LocalDateTime resolvedAt) {
138         this.resolvedAt = resolvedAt;
139     }
140 }
141
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
6 import jakarta.persistence.*;
7
8 import java.time.DayOfWeek;
9 import java.time.Instant;
10 import java.time.LocalDate;
11 import java.time.LocalTime;
12 @Entity
13 public class Subscription {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private long id;
17     @Enumerated(EnumType.STRING)
18     private MealSlot slot;
19     private ScheduleType scheduleType;
20     private DayOfWeek dayOfWeek;
21     @Enumerated(EnumType.STRING)
22     private SubscriptionStatus status;
23     private Instant nextDeliveryTime;
24     private LocalTime created_at;
25
26     @ManyToOne
27     @JoinColumn(name="employee_id")
28     private Employee employee;
29
30     @ManyToOne(fetch = FetchType.EAGER)
31     @JoinColumn(name = "restaurant_id", nullable = false)
32     private Restaurant restaurant;
33
34     @PrePersist
35     public void init(){
36         this.created_at=LocalTime.now();
37     }
38
39     public long getId() {
40         return id;
41     }
42
43     public void setId(long id) {
44         this.id = id;
45     }
46
47     public MealSlot getSlot() {
48         return slot;
49     }
50
51     public void setSlot(MealSlot slot) {
52         this.slot = slot;
53     }
54
55     public ScheduleType getScheduleType() {
56         return scheduleType;
57     }
58
59     public void setScheduleType(ScheduleType scheduleType) {
60         this.scheduleType = scheduleType;
61     }
62
63     public DayOfWeek getDayOfWeek() {
64         return dayOfWeek;
65     }
66
67     public void setDayOfWeek(DayOfWeek dayOfWeek) {
68         this.dayOfWeek = dayOfWeek;
69     }
70
71     public SubscriptionStatus getStatus() {
```

```
72         return status;
73     }
74
75     public void setStatus(SubscriptionStatus status) {
76         this.status = status;
77     }
78
79     public LocalTime getCreated_at() {
80         return created_at;
81     }
82
83     public void setCreated_at(LocalTime created_at) {
84         this.created_at = created_at;
85     }
86
87     public Instant getNextDeliveryTime() {
88         return nextDeliveryTime;
89     }
90
91     public void setNextDeliveryTime(Instant nextDeliveryTime) {
92         this.nextDeliveryTime = nextDeliveryTime;
93     }
94
95     public Employee getEmployee() {
96         return employee;
97     }
98
99     public void setEmployee(Employee employee) {
100         this.employee = employee;
101     }
102     public Restaurant getRestaurant() { return restaurant; }
103     public void setRestaurant(Restaurant restaurant) { this.restaurant = restaurant; }
104 }
105
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import jakarta.persistence.*;
5 import jakarta.xml.bind.annotation.XmlAttribute;
6 import org.jspecify.annotations.Nullable;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.authority.SimpleGrantedAuthority;
9 import org.springframework.security.core.userdetails.UserDetails;
10
11 import java.time.LocalDate;
12 import java.util.Collection;
13 import java.util.HashSet;
14 import java.util.List;
15 import java.util.Set;
16 import java.util.stream.Collectors;
17
18 /**
19  * Represents a restaurant vendor/owner.
20  * Vendors can register and manage their restaurants.
21  * They cannot access employee, leave, or subscription APIs.
22  */
23 @Entity
24 @Table(name = "vendor")
25 public class Vendor implements UserDetails {
26
27     @Id
28     @GeneratedValue(strategy = GenerationType.IDENTITY)
29     private Long id;
30
31     @Column(nullable = false)
32     private String name;
33
34     @Column(unique = true, nullable = false)
35     private String email;
36
37     @Column(nullable = false)
38     private String phone;
39
40     @Column(nullable = false)
41     private String password;
42
43     @Enumerated(EnumType.STRING)
44     private Role role;
45
46     private LocalDate registeredAt;
47
48     @PrePersist
49     public void init() {
50         if (this.registeredAt == null) {
51             this.registeredAt = LocalDate.now();
52         }
53     }
54
55     public Long getId() { return id; }
56     public void setId(Long id) { this.id = id; }
57
58     public String getName() { return name; }
59     public void setName(String name) { this.name = name; }
60
61     public String getEmail() { return email; }
62     public void setEmail(String email) { this.email = email; }
63
64     public String getPhone() { return phone; }
65     public void setPhone(String phone) { this.phone = phone; }
66
67     public LocalDate getRegisteredAt() { return registeredAt; }
68     public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
69
70
71     @Override
```

```
72     public Collection<? extends GrantedAuthority> getAuthorities() {
73         Role effectiveRole = role;
74         Set<SimpleGrantedAuthority> authorities=new HashSet<>();
75         authorities.add(new SimpleGrantedAuthority("ROLE_"+effectiveRole.name()));
76         authorities.addAll(effectiveRole.getPermissions().stream()
77             .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
78             .collect(Collectors.toSet()));
79         return authorities;
80     }
81
82     @Override
83     public @Nullable String getPassword() {
84         return password;
85     }
86
87     @Override
88     public String getUsername() {
89         return email;
90     }
91
92     @Override
93     public boolean isAccountNonExpired() {
94         return UserDetails.super.isAccountNonExpired();
95     }
96
97     @Override
98     public boolean isAccountNonLocked() {
99         return UserDetails.super.isAccountNonLocked();
100    }
101
102    @Override
103    public boolean isCredentialsNonExpired() {
104        return UserDetails.super.isCredentialsNonExpired();
105    }
106
107    @Override
108    public boolean isEnabled() {
109        return UserDetails.super.isEnabled();
110    }
111
112    public void setPassword(String password) {
113        this.password = password;
114    }
115
116    public Role getRole() {
117        return role;
118    }
119
120    public void setRole(Role role) {
121        this.role = role;
122    }
123 }
124
```

```

1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7
8 @Entity
9 @Table(name = "year_end_carry_forward_log")
10 public class YearEndCarryForwardLog {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne(fetch = FetchType.LAZY, optional = false)
17     @JoinColumn(name = "employee_id", nullable = false)
18     private Employee employee;
19
20     @Column(nullable = false)
21     private Integer processedYear;
22
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "leave_type_id", nullable = false)
25     private LeaveType leaveType;
26
27     @Column(nullable = false, precision = 5, scale = 2)
28     private BigDecimal closingBalance; // raw closing before cap
29
30     @Column(nullable = false, precision = 5, scale = 2)
31     private BigDecimal cappedBalance; // after global 30-day cap applied
32
33     @Column(nullable = false, precision = 5, scale = 2)
34     private BigDecimal daysWasted = BigDecimal.ZERO;
35
36     @Column(nullable = false, precision = 5, scale = 2)
37     private BigDecimal openingNextYear; // written into next year's entitlement
38
39     @Column(nullable = false)
40     private LocalDateTime processedAt = LocalDateTime.now();
41
42     // --- getters & setters ---
43     public Long getId() { return id; }
44     public Employee getEmployee() { return employee; }
45     public Integer getProcessedYear() { return processedYear; }
46     public LeaveType getLeaveType() { return leaveType; }
47     public BigDecimal getClosingBalance() { return closingBalance; }
48     public BigDecimal getCappedBalance() { return cappedBalance; }
49     public BigDecimal getDaysWasted() { return daysWasted; }
50     public BigDecimal getOpeningNextYear() { return openingNextYear; }
51     public LocalDateTime getProcessedAt() { return processedAt; }
52
53     public void setEmployee(Employee e) { this.employee = e; }
54     public void setProcessedYear(Integer y) { this.processedYear = y; }
55     public void setLeaveType(LeaveType lt) { this.leaveType = lt; }
56     public void setClosingBalance(BigDecimal b) { this.closingBalance = b; }
57     public void setCappedBalance(BigDecimal b) { this.cappedBalance = b; }
58     public void setDaysWasted(BigDecimal d) { this.daysWasted = d; }
59     public void setOpeningNextYear(BigDecimal b) { this.openingNextYear = b; }
60     public void setProcessedAt(LocalDateTime dt) { this.processedAt = dt; }
61 }
62

```

80 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java

8
lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum AssignmentStatus {
```

```
4     ACTIVE,
```

```
5     RETURNED,
```

```
6     REVOKED
```

```
7 }
```

```
8
```

81

Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java

9

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeliveryStatus {  
4     SCHEDULED,  
5     IN_PROGRESS,  
6     DELIVERED,  
7     MISSED  
8 }  
9
```

82

Current:

10

src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeviceStatus {  
4     AVAILABLE,  
5     ASSIGNED,  
6     UNDER_REPAIR,  
7     CONDEMNED,  
8     RETURNED_TO_VENDOR  
9 }  
10
```



```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeviceType {  
4     LAPTOP,  
5     DESKTOP,  
6     PHONE,  
7     TABLET,  
8     MONITOR,  
9     KEYBOARD,  
10    MOUSE,  
11    HEADSET,  
12    OTHER  
13  
14 }  
15
```

84

Current:

5

src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum Gender {  
4     M, F, O  
5 }
```

85

Current:

9

src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum LeaveStatus {  
4     PENDING,  
5     APPROVED,  
6     REJECTED,  
7     CANCELLED  
8 }  
9
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum LeaveType {  
4     SICK,  
5     CASUAL,  
6     MATERNITY,  
7     UNPAID  
8 }  
9
```

87

Current:

8

src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum MealSlot {
```

```
4     BREAKFAST,
```

```
5     LUNCH,
```

```
6     DINNER
```

```
7 }
```

```
8
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 import java.util.Set;  
4  
5 public enum Permission {  
6     LEAVE_READ,  
7     LEAVE_WRITE,  
8     LEAVE_UPDATE,  
9     LEAVE_CANCEL,  
10    SUBSCRIPTION_WRITE,  
11    SUBSCRIPTION_READ,  
12    SUBSCRIPTION_UPDATE,  
13    VENDOR_UPDATE,  
14    VENDOR_DELETE,  
15    RESTAURANTS_ADD,  
16    RESTAURANTS_READ,  
17    RESTAURANTS_DELETE,  
18    PROFILE_READ,  
19    VENDOR_WRITE;  
20 }  
21
```

```
1 package com.example.EmployeeManagementSystem.Enum;
2
3 import java.util.Collection;
4 import java.util.Set;
5
6 import static com.example.EmployeeManagementSystem.Enum.Permission.*;
7
8
9 public enum Role {
10     EMPLOYEE(Set.of(LEAVE_WRITE,
11         LEAVE_CANCEL,
12         SUBSCRIPTION_WRITE,
13         SUBSCRIPTION_UPDATE,
14         RESTAURANTS_READ,
15         SUBSCRIPTION_READ,
16         PROFILE_READ)),
17     MANAGER(Set.of(LEAVE_UPDATE,
18         LEAVE_WRITE,
19         SUBSCRIPTION_WRITE,
20         RESTAURANTS_READ,
21         SUBSCRIPTION_UPDATE,
22         LEAVE_READ,
23         SUBSCRIPTION_READ,
24         PROFILE_READ)),
25     FOOD_VENDOR(Set.of(
26         VENDOR_WRITE,
27         VENDOR_DELETE,
28         RESTAURANTS_ADD,
29         RESTAURANTS_DELETE,
30         RESTAURANTS_READ,
31         PROFILE_READ
32     )),
33     ADMIN(Set.of(
34         LEAVE_READ,
35         LEAVE_WRITE,
36         LEAVE_UPDATE,
37         LEAVE_CANCEL,
38         RESTAURANTS_READ,
39         SUBSCRIPTION_READ,
40         SUBSCRIPTION_WRITE,
41         SUBSCRIPTION_UPDATE,
42         VENDOR_WRITE,
43         VENDOR_UPDATE,
44         VENDOR_DELETE,
45         PROFILE_READ
46     )),
47     TECH_VENDOR(Set.of(VENDOR_WRITE)),
48     USER(Set.of(PROFILE_READ));
49     private final Set<Permission> permissions;
50
51     Role(Set<Permission> permissions) {
52         this.permissions = permissions;
53     }
54
55     public Set<Permission> getPermissions() {
56         return permissions;
57     }
58 }
59
60
```

90

Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java

7

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ScheduleType {  
4     DAILY,  
5     WEEKLY  
6 }  
7
```



```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ServiceRequestStatus {  
4     OPEN,  
5     SENT_FOR_REPAIR,  
6     REPAIR_DONE,  
7     REJECTED  
8 }  
9
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ServiceRequestType {  
4     REPAIR,  
5     REPLACEMENT,  
6     UPGRADE,  
7     MAINTENANCE,  
8     RETURN  
9 }  
10
```

93

Current:

8

src\main\java\com\example\EmployeeManagementSystem\Enum\Status.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum Status {
```

```
4     ACTIVE,
```

```
5     INACTIVE,
```

```
6     ON_LEAVE
```

```
7 }
```

```
8
```

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum SubscriptionStatus {
```

```
4     ACTIVE,
```

```
5     PAUSED,
```

```
6     EXPIRED
```

```
7 }
```

```
8
```

95 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class DuplicateRequestException extends RuntimeException {
4     public DuplicateRequestException(String message) {
5         super(message);
6     }
7 }
8
```

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 import org.springframework.security.core.userdetails.UsernameNotFoundException;
4
5 public class EmployeeNotFound extends UsernameNotFoundException {
6     public EmployeeNotFound(String message) {
7         super(message);
8     }
9 }
10
```

97 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidEndDateException extends RuntimeException{
4     public InvalidEndDateException(String message) {
5         super(message);
6     }
7 }
8
```

98 Current:**src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java**

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidManagerException extends RuntimeException{
4     public InvalidManagerException(String message) {
5         super(message);
6     }
7 }
8
```


99 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidStartDateException extends RuntimeException{
4     public InvalidStartDateException(String message) {
5         super(message);
6     }
7 }
8
```

100 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundExcep

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class LeaveRequestNotFoundException extends RuntimeException{
4     public LeaveRequestNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

101 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.j

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class OverlappingLeaveException extends RuntimeException{
4     public OverlappingLeaveException(String message) {
5         super(message);
6     }
7 }
8
```

102 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class RestaurantNotFoundException extends RuntimeException {
4     public RestaurantNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

103 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class SubscriptionAlreadyExists extends RuntimeException{
4     public SubscriptionAlreadyExists(String message) {
5         super(message);
6     }
7 }
8
```

104 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class UnauthorizedAccessException extends RuntimeException {
4     public UnauthorizedAccessException(String message) {
5         super(message);
6     }
7 }
```

105 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class VendorNotFoundException extends RuntimeException {
4     public VendorNotFoundException(String message) {
5         super(message);
6     }
7 }
```

```

1 // src/main/java/com/example/EmployeeManagementSystem/Filter/ApiKeyFilter.java
2 package com.example.EmployeeManagementSystem.Filter;
3
4 import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationToken;
5 import jakarta.servlet.FilterChain;
6 import jakarta.servlet.ServletException;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.beans.factory.annotation.Qualifier;
11 import org.springframework.security.authentication.AuthenticationManager;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.security.core.context.SecurityContextHolder;
14 import org.springframework.stereotype.Component;
15 import org.springframework.web.filter.OncePerRequestFilter;
16
17 import java.io.IOException;
18 @Component
19 public class ApiKeyFilter extends OncePerRequestFilter {
20
21     private static final String API_KEY_HEADER = "X-API-Key";
22
23     @Autowired
24     @Qualifier("apiKeyAuthenticationManager") // or just use ProviderManager directly
25     private AuthenticationManager authenticationManager;
26
27     @Override
28     protected void doFilterInternal(HttpServletRequest request,
29                                     HttpServletResponse response,
30                                     FilterChain filterChain) throws ServletException, IOException {
31
32         String apiKey = request.getHeader(API_KEY_HEADER);
33
34         if (apiKey != null && !apiKey.isEmpty()) {
35             try {
36                 // Create the unauthenticated token
37                 ApiKeyAuthenticationToken authRequest = new ApiKeyAuthenticationToken(apiKey);
38                 // Run it through the provider – this hits the DB, checks expiry, loads roles
39                 Authentication authenticated = authenticationManager.authenticate(authRequest);
40                 // Only set it if validation passed
41                 SecurityContextHolder.getContext().setAuthentication(authenticated);
42             } catch (RuntimeException e) {
43                 // Invalid/expired key – clear context and return 401
44                 SecurityContextHolder.clearContext();
45                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
46                 response.setContentType("application/json");
47                 response.getWriter().write("{\"error\": \"" + e.getMessage() + "\"}");
48                 return;
49             }
50         }
51
52         filterChain.doFilter(request, response);
53     }
54
55     @Override
56     protected boolean shouldNotFilter(HttpServletRequest request) {
57         String path = request.getRequestURI();
58         return path.startsWith("/Authenticate") ||
59             path.startsWith("/session/login") ||
60             path.startsWith("/session/logout") ||
61             path.startsWith("/oauth2") ||
62             path.startsWith("/login/oauth2") ||
63             path.startsWith("/swagger-ui") ||
64             path.startsWith("/v3/api-docs") ||
65             path.startsWith("/api-keys/generate"); // Allow API key generation without API key
66     }
67 }

```



```
1 package com.example.EmployeeManagementSystem.Filter;
2
3 import com.example.EmployeeManagementSystem.Util.JWTUtil;
4 import io.jsonwebtoken.ExpiredJwtException;
5 import io.jsonwebtoken.MalformedJwtException;
6 import io.jsonwebtoken.UnsupportedJwtException;
7 import jakarta.servlet.FilterChain;
8 import jakarta.servlet.ServletException;
9 import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.beans.factory.annotation.Qualifier;
13 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
14 import org.springframework.security.core.context.SecurityContextHolder;
15 import org.springframework.security.core.userdetails.UserDetails;
16 import org.springframework.security.core.userdetails.UserDetailsService;
17 import org.springframework.stereotype.Component;
18 import org.springframework.web.filter.OncePerRequestFilter;
19
20 import java.io.IOException;
21
22 @Component
23 public class JwtAuthFilter extends OncePerRequestFilter {
24
25     @Autowired
26     @Qualifier("combinedUserDetailsService")
27     private UserDetailsService customUserDetailsService;
28
29     @Autowired
30     private JWTUtil jwtUtil;
31
32     @Override
33     protected void doFilterInternal(HttpServletRequest request,
34                                     HttpServletResponse response,
35                                     FilterChain filterChain) throws ServletException, IOException {
36         String token = null;
37         String username = null;
38
39         String authHeader = request.getHeader("Authorization");
40
41         if (authHeader != null && authHeader.startsWith("Bearer ")) {
42             token = authHeader.substring(7).trim();
43
44             // Reject obviously invalid values before hitting JWT
45             if (token.isEmpty() || token.equals("null") || token.equals("undefined")) {
46                 filterChain.doFilter(request, response);
47                 return;
48             }
49
50             try {
51                 username = jwtUtil.extractUsernameFromToken(token);
52             } catch (ExpiredJwtException e) {
53                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
54                 response.setContentType("application/json");
55                 response.getWriter().write("{\""
56                     {
57                         "error": "TOKEN_EXPIRED",
58                         "message": "JWT expired. Please login again to get a new token."
59                     }
60                     "\"");
61                 return;
62             } catch (MalformedJwtException | UnsupportedJwtException | IllegalArgumentException e) {
63                 // Token is garbage (e.g. "null" string, garbled value) –
64                 // don't crash. OAuth2 session auth will still work fine.
65                 logger.debug("JwtAuthFilter: skipping malformed token – " + e.getMessage());
66                 filterChain.doFilter(request, response);
67                 return;
68             }
69         }
70
71         if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
```

```
72         UserDetails userDetails = customUserDetailService.loadUserByUsername(username);
73         if (userDetails != null && jwtUtil.validateToken(username, userDetails, token)) {
74             UsernamePasswordAuthenticationToken authenticationToken =
75                 new UsernamePasswordAuthenticationToken(
76                     userDetails, null, userDetails.getAuthorities());
77             SecurityContextHolder.getContext().setAuthentication(authenticationToken);
78         }
79     }
80
81     filterChain.doFilter(request, response);
82 }
83 }
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/Repository/ApiKeyRepository.java
2 package com.example.EmployeeManagementSystem.Repository;
3
4 import com.example.EmployeeManagementSystem.Entity.ApiKey;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Modifying;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9 import org.springframework.transaction.annotation.Transactional;
10
11 import java.time.Instant;
12 import java.util.List;
13 import java.util.Optional;
14
15 public interface ApiKeyRepository extends JpaRepository<ApiKey, Long> {
16
17     Optional<ApiKey> findByApiKeyAndActiveTrue(String apiKey);
18
19
20     List<ApiKey> findByEmployee_EmployeeIdAndActiveTrue(Long employeeId);
21
22
23     List<ApiKey> findByVendor_IdAndActiveTrue(Long vendorId);
24
25     @Modifying
26     @Transactional
27     @Query("UPDATE ApiKey a SET a.lastUsedAt = :now WHERE a.apiKey = :apiKey")
28     void updateLastUsed(@Param("apiKey") String apiKey, @Param("now") Instant now);
29
30     @Modifying
31     @Transactional
32     @Query("UPDATE ApiKey a SET a.active = false WHERE a.expiresAt < :now AND a.active = true")
33     int expireOldKeys(@Param("now") Instant now);
34 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Delivery;
4 import com.example.EmployeeManagementSystem.Entity.Subscription;
5 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9
10 import java.time.Instant;
11 import java.util.List;
12
13 public interface DeliveryRepository extends JpaRepository<Delivery, Long> {
14     List<Delivery> findByStatusAndScheduledDeliveryTimeLessThanEqual(
15         DeliveryStatus status, Instant time);
16
17     @Query("SELECT d FROM Delivery d WHERE d.subscription = :subscription " +
18         "AND d.status != :status ORDER BY d.scheduledDeliveryTime DESC")
19     List<Delivery> findRecentDeliveriesBySubscription(
20         @Param("subscription") Subscription subscription,
21         @Param("status") DeliveryStatus status);
22 }
23
```

110 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
4 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8
9 import java.util.List;
10 import java.util.Optional;
11
12 public interface DeviceAssignmentRepo extends JpaRepository<DeviceAssignment, Long> {
13     Optional<DeviceAssignment> findByDeviceIdAndStatus(Long deviceId, AssignmentStatus status);
14     List<DeviceAssignment> findByAssignedToEmployeeIdAndStatus(Long employeeId, AssignmentStatus status);
15
16     @Query(value = "SELECT da.* FROM device_assignment da JOIN device d ON da.device_id = d.id WHERE d.vendor_id = :vendorId", nativeQuery = true)
17     List<DeviceAssignment> getAllAssignmentsByVendorId(@Param("vendorId") long vendorId);
18
19
20 }
21
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Device;
4 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8
9 public interface DeviceRepository extends JpaRepository<Device, Long> {
10     List<Device> findByTechVendorId(Long vendorId);
11     List<Device> findByDeviceStatus(DeviceStatus deviceStatus);
12 }
13
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Enum.Status;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9 import org.springframework.stereotype.Repository;
10
11 import java.util.List;
12 import java.util.Optional;
13
14 @Repository
15 public interface EmployeeRepo extends JpaRepository<Employee, Long> {
16     Optional<Employee> findByEmail(String email);
17     Optional<Employee> findByName(String name);
18     long countByRole(Role role);
19
20     List<Employee> findByManager(Employee manager);
21     List<Employee> findByStatus(Status status);
22     List<Employee> findByRole(Role role);
23     @Query("SELECT e FROM Employee e WHERE e.manager.employeeId = :managerId")
24     List<Employee> findByManagerEmployeeId(@Param("managerId") Long managerId);
25 }
26
```

113 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\FixedHolidayRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.FixedHoliday;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface FixedHolidayRepository extends JpaRepository<FixedHoliday, Long> {
10     List<FixedHoliday> findByYear(int year);
11     List<FixedHoliday> findByDateBetween(LocalDate start, LocalDate end);
12 }
13
```


114 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveEntitlementRepository.

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7
8 import java.math.BigDecimal;
9 import java.util.List;
10 import java.util.Optional;
11
12 public interface LeaveEntitlementRepository extends JpaRepository<LeaveEntitlement, Long> {
13
14     Optional<LeaveEntitlement> findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
15         Long employeeId, Integer leaveTypeId, Integer year);
16
17     List<LeaveEntitlement> findByEmployeeEmployeeIdAndYear(Long employeeId, Integer year);
18
19     @Query("""
20         SELECT COALESCE(SUM(e.closingBalance), 0)
21         FROM LeaveEntitlement e
22         WHERE e.employee.employeeId = :empId
23             AND e.leaveType.carriesForward = true
24             AND e.year IN :years
25         """)
26     BigDecimal sumCarryForwardBalances(
27         @Param("empId") Long employeeId,
28         @Param("years") List<Integer> years);
29
30     List<LeaveEntitlement> findByLeaveTypeIdAndYear(Integer leaveTypeId, Integer year);
31 }
```

```

1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
5 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
6 import com.example.EmployeeManagementSystem.Enum.LeaveType;
7 import org.springframework.data.jpa.repository.JpaRepository;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10
11 import java.time.LocalDate;
12 import java.util.List;
13
14 public interface LeaveRequestRepo extends JpaRepository<LeaveRequest, Long> {
15
16     List<LeaveRequest> findByStatus(LeaveStatus status);
17
18     /**
19      * Find leave requests by start date and status
20      * Used by scheduler to find leaves starting today
21      */
22     List<LeaveRequest> findByStartDateAndStatus(LocalDate startDate, LeaveStatus status);
23
24     // Fix the checkDuplicate query (fix parameter names)
25     @Query(value = "SELECT COUNT(*) " +
26         "FROM leave_request l " +
27         "WHERE l.employee_id = :employeeId " +
28         "AND l.start_date = :startDate " +
29         "AND l.end_date = :endDate " +
30         "AND l.status=:status",nativeQuery = true)
31     long checkDuplicate(@Param("employeeId") long employeeId,
32         @Param("startDate") LocalDate startDate,
33         @Param("endDate") LocalDate endDate,
34         @Param("status") String status);
35
36     // Fix overlapping leave query (was commented in your code)
37     @Query(value = "select COUNT(*) " +
38         "from leave_request l " +
39         "where l.employee_id=:employeeId " +
40         "AND l.status IN ('APPROVED','PENDING') " +
41         "AND l.start_date <= :endDate " +
42         "AND l.end_date >= :startDate",nativeQuery = true)
43     long countOverlappingLeave(
44         @Param("employeeId") long employeeId,
45         @Param("startDate") LocalDate startDate,
46         @Param("endDate") LocalDate endDate
47     );
48
49     // Add missing methods
50     List<LeaveRequest> findByEmployeeOrderByStartDateDesc(Employee employee);
51
52
53     List<LeaveRequest> findByEndDateAndStatus(LocalDate endDate, LeaveStatus status);
54
55     // Add for leave duration validation
56     @Query("SELECT COALESCE(SUM(DATEDIFF(l.endDate, l.startDate) + 1), 0) " +
57         "FROM LeaveRequest l WHERE l.employee = :employee " +
58         "AND l.leaveType = :leaveType " +
59         "AND YEAR(l.startDate) = :year " +
60         "AND l.status IN ('APPROVED', 'PENDING')")
61     long countDaysByEmployeeAndLeaveTypeAndYear(@Param("employee") Employee employee,
62         @Param("leaveType") LeaveType leaveType,
63         @Param("year") int year);
64
65     @Query("SELECT lr FROM LeaveRequest lr JOIN FETCH lr.employee WHERE lr.status = :status")
66     List<LeaveRequest> findApprovedLeavesWithEmployee(LeaveStatus status);
67
68
69     List<LeaveRequest> findByEmployee_Manager(Employee manager);
70
71     List<LeaveRequest> findByStatusAndEmployee_Manager(LeaveStatus status, Employee manager);

```

```
72
73     @Query("""
74     SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
75     FROM LeaveRequest lr
76     WHERE lr.employee = :employee
77           AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.SICK
78           AND lr.status = com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED
79           AND YEAR(lr.startDate) = :year
80           AND MONTH(lr.startDate) = :month
81     """)
82     long countApprovedSickDaysInMonth(
83         @Param("employee") Employee employee,
84         @Param("year") int year,
85         @Param("month") int month);
86
87 }
88
```

116 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import com.example.EmployeeManagementSystem.Entity.LeaveType;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public interface LeaveTypeRepository extends JpaRepository<LeaveType, Integer> {
11     Optional<LeaveType> findByName(String name);
12     List<LeaveType> findByMonthlyResetTrue();
13 }
```

117 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface LeaveWarningRepository extends JpaRepository<LeaveWarning, Long> {
10
11     // All unread warnings for an employee – shown on dashboard
12     List<LeaveWarning> findByEmployeeEmployeeIdAndIsReadFalseOrderByWarningDateDesc(Long employeeId);
13
14     // All warnings for an employee (read + unread) – for history view
15     List<LeaveWarning> findByEmployeeEmployeeIdOrderByWarningDateDesc(Long employeeId);
16
17     // Prevent duplicate warnings for same type on same date
18     boolean existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(
19         Long employeeId, LeaveWarning.WarningType warningType, LocalDate date
20     );
21 }
```

118 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\NotificationRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Notification;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import java.util.List;
7
8 public interface NotificationRepository extends JpaRepository<Notification, Long> {
9     List<Notification> findByRecipientOrderByCreatedAtDesc(Employee recipient);
10    long countByRecipientAndIsReadFalse(Employee recipient);
11 }
12
```

119 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import org.springframework.data.jpa.repository.JpaRepository;
7
8 import java.util.Optional;
9
10 public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {
11
12     Optional<RefreshToken> findByToken(String token);
13
14     // Needed so EmployeeService can clean up tokens before deleting an employee
15     void deleteByEmployee(Employee employee);
16
17     // Needed so VendorService can clean up tokens before deleting a vendor
18     void deleteByVendor(Vendor vendor);
19 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Restaurant;
4 import com.example.EmployeeManagementSystem.Enum.MealSlot;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8 import org.springframework.stereotype.Repository;
9
10 import java.util.List;
11
12 @Repository
13 public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {
14
15     /** All active restaurants owned by a vendor. */
16     List<Restaurant> findByVendor_EmailAndActiveTrue(String email);
17
18     /** All active restaurants (for employees browsing available options). */
19     List<Restaurant> findByActiveTrue();
20
21     /**
22      * Active restaurants that support a given meal slot.
23      * Employees use this to discover which restaurants they can subscribe to for a given slot.
24      */
25     @Query("SELECT r FROM Restaurant r JOIN r.supportedMealSlots s " +
26           "WHERE r.active = true AND s = :mealSlot")
27     List<Restaurant> findActiveByMealSlot(@Param("mealSlot") MealSlot mealSlot);
28
29     /** Check if a restaurant belongs to a vendor (ownership guard). */
30     boolean existsByIdAndVendor_Email(Long restaurantId, String email);
31 }
```


121 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3
4 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
8 import jdk.dynalink.linker.LinkerServices;
9 import org.springframework.data.jpa.repository.JpaRepository;
10
11 import java.util.List;
12
13 public interface ServiceRequestRepository extends JpaRepository<ServiceRequest, Long> {
14
15     List<ServiceRequest> findByStatus(ServiceRequestStatus status);
16     List<ServiceRequest> findByRaisedByOrderByRaisedAtDesc(Employee employee);
17     List<ServiceRequest> findByDeviceTechVendorAndStatus(
18         Vendor vendor,
19         ServiceRequestStatus status
20     );
21
22 }
23
```

122 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Subscription;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7 import org.springframework.stereotype.Repository;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @Repository
13 public interface SubscriptionRepository extends JpaRepository<Subscription, Long> {
14     @Query(value = "select count(*) from subscription s where s.employee_id=:employeeId and s.slot=:slot and s.status=:status", nativeQuery = true)
15     int checkSubscriptionExists(
16         @Param("employeeId") long employeeId,
17         @Param("slot") String slot,
18         @Param("status") String status
19     );
20
21     List<Subscription> findByEmployee_employeeId(long id);
22
23     @Query(value = "SELECT * FROM subscription s WHERE s.status = :status AND s.next_delivery_time IS NOT NULL AND s.next_delivery_time <= :currentTime", nativeQuery = true)
24     List<Subscription> findDueSubscriptions(
25         @Param("status") String status,
26         @Param("currentTime") Instant currentTime // was LocalDateTime
27     );
28
29     List<Subscription> findByEmployee_name(String name);
30     List<Subscription> findByEmployee_Email(String email);
31 }
32
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Vendor;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface VendorRepo extends JpaRepository<Vendor, Long> {
11     Optional<Vendor> findByEmail(String email);
12     boolean existsByEmail(String email);
13 }
```

124 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRe

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3
4 import com.example.EmployeeManagementSystem.Entity.YearEndCarryForwardLog;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8
9 public interface YearEndCarryForwardLogRepository
10     extends JpaRepository<YearEndCarryForwardLog, Long> {
11
12     List<YearEndCarryForwardLog> findByEmployeeEmployeeIdAndProcessedYear(
13         Long employeeId, Integer year
14     );
15
16     // Check if year-end has already been run for this year (idempotency guard)
17     boolean existsByProcessedYear(Integer year);
18 }
```

```
1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Service.DeliveryService;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.scheduling.annotation.Scheduled;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class DeliveryScheduler {
11     private static final Logger logger = LoggerFactory.getLogger(DeliveryScheduler.class);
12     private final DeliveryService deliveryService;
13
14     public DeliveryScheduler(DeliveryService deliveryService) {
15         this.deliveryService = deliveryService;
16     }
17
18     // Runs every hour
19     @Scheduled(cron = "0 0 * * * *")
20     public void processDeliveries() {
21         logger.info("Starting delivery processing scheduler...");
22         try {
23             deliveryService.processDueDeliveries();
24             logger.info("Delivery processing completed successfully");
25         } catch (Exception e) {
26             logger.error("Error processing deliveries: ", e);
27         }
28     }
29 }
30
```

```

1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Service.CarryForwardWarningService;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
6 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
7 import org.slf4j.Logger;
8 import org.slf4j.LoggerFactory;
9 import org.springframework.scheduling.annotation.Scheduled;
10 import org.springframework.stereotype.Component;
11
12 import java.time.LocalDate;
13
14 @Component
15 public class LeaveAccrualScheduler {
16
17     private static final Logger log = LoggerFactory.getLogger(LeaveAccrualScheduler.class);
18
19     private final LeaveAccrualService accrualService;
20     private final SickLeaveResetService resetService;
21     private final CarryForwardWarningService warningService;
22     private final YearEndCarryForwardService yearEndService;
23
24     public LeaveAccrualScheduler(LeaveAccrualService accrualService,
25                                 SickLeaveResetService resetService,
26                                 CarryForwardWarningService warningService,
27                                 YearEndCarryForwardService yearEndService) {
28         this.accrualService = accrualService;
29         this.resetService = resetService;
30         this.warningService = warningService;
31         this.yearEndService = yearEndService;
32     }
33
34     // 1st of month – accrue then warn
35     @Scheduled(cron = "0 5 0 1 * ?")
36     public void runMonthlyAccrual() {
37         LocalDate today = LocalDate.now();
38         accrualService.runMonthlyAccrual(today);
39         warningService.runWarningCheck(today);
40     }
41
42     // Last day of month 23:55 – sick leave reset
43     @Scheduled(cron = "0 55 23 L * ?")
44     public void runSickLeaveReset() {
45         resetService.runMonthlyReset(LocalDate.now());
46     }
47
48     // Dec 31 at 22:00 – year-end carry-forward cap
49     @Scheduled(cron = "0 0 22 31 12 ?")
50     public void runYearEnd() {
51         int closingYear = LocalDate.now().getYear();
52         yearEndService.runYearEnd(closingYear);
53     }
54 }

```

```

1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
5 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.scheduling.annotation.Scheduled;
12 import org.springframework.stereotype.Component;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.time.LocalDate;
16 import java.time.ZoneId;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.stream.Collectors;
20
21 @Component
22 public class LeaveStatusScheduler {
23
24     private static final Logger log = LoggerFactory.getLogger(LeaveStatusScheduler.class);
25
26     private final LeaveRequestRepo leaveRequestRepo;
27     private final EmployeeRepo employeeRepo;
28
29     public LeaveStatusScheduler(LeaveRequestRepo leaveRequestRepo,
30                               EmployeeRepo employeeRepo) {
31         this.leaveRequestRepo = leaveRequestRepo;
32         this.employeeRepo = employeeRepo;
33     }
34
35     /**
36      * Runs every day at midnight UTC
37      */
38     @Scheduled(cron = "0 0 0 * * *", zone = "UTC")
39     @Transactional
40     public void updateEmployeeLeaveStatus() {
41
42         log.info("=== Starting Timezone-Aware Leave Scheduler ===");
43
44         List<LeaveRequest> leaves =
45             leaveRequestRepo.findApprovedLeavesWithEmployee(LeaveStatus.APPROVED);
46
47         Map<Long, List<LeaveRequest>> employeeLeavesMap = leaves.stream()
48             .collect(Collectors.groupingBy(lr -> lr.getEmployee().getEmployeeId()));
49
50         int onLeaveCount = 0;
51         int activeCount = 0;
52
53         for (Map.Entry<Long, List<LeaveRequest>> entry : employeeLeavesMap.entrySet()) {
54
55             Employee employee = entry.getValue().get(0).getEmployee();
56
57             ZoneId zone = ZoneId.of(
58                 employee.getTimezone() != null ? employee.getTimezone() : "UTC"
59             );
60
61             LocalDate today = LocalDate.now(zone);
62
63             boolean isOnLeaveToday = entry.getValue().stream().anyMatch(leave ->
64                 !today.isBefore(leave.getStartDate()) &&
65                 !today.isAfter(leave.getEndDate())
66             );
67
68             // CASE 1: Should be ON_LEAVE
69             if (isOnLeaveToday && employee.getStatus() != Status.ON_LEAVE) {
70
71                 employee.setStatus(Status.ON_LEAVE);

```

```

72         employeeRepo.save(employee);
73         onLeaveCount++;
74
75         log.info("Employee {} ({} marked ON_LEAVE | TZ: {}",
76                 employee.getEmployeeId(),
77                 employee.getName(),
78                 zone);
79     }
80
81     // CASE 2: Should be ACTIVE
82     if (!isOnLeaveToday && employee.getStatus() == Status.ON_LEAVE) {
83
84         employee.setStatus(Status.ACTIVE);
85         employeeRepo.save(employee);
86         activeCount++;
87
88         log.info("Employee {} ({} marked ACTIVE | TZ: {}",
89                 employee.getEmployeeId(),
90                 employee.getName(),
91                 zone);
92     }
93 }
94
95     log.info("=== Scheduler Completed ===");
96     log.info("ON_LEAVE updated: {}", onLeaveCount);
97     log.info("ACTIVE updated: {}", activeCount);
98 }
99
100 /**
101  * Manual trigger (for testing)
102  */
103 public void runManually() {
104     log.info("Manual scheduler trigger started...");
105     updateEmployeeLeaveStatus();
106 }
107 }

```



```

1 // src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationProvider.java
2 package com.example.EmployeeManagementSystem.security;
3
4 import com.example.EmployeeManagementSystem.Entity.ApiKey;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
8 import org.springframework.security.authentication.AuthenticationProvider;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.security.core.AuthenticationException;
11 import org.springframework.security.core.GrantedAuthority;
12 import org.springframework.security.core.authority.SimpleGrantedAuthority;
13 import org.springframework.stereotype.Component;
14 import org.springframework.transaction.annotation.Transactional;
15
16 import java.time.Instant;
17 import java.util.ArrayList;
18 import java.util.Arrays;
19 import java.util.Collection;
20 import java.util.List;
21 import java.util.stream.Collectors;
22
23 @Component
24 public class ApiKeyAuthenticationProvider implements AuthenticationProvider {
25
26     private final ApiKeyRepository apiKeyRepository;
27
28     public ApiKeyAuthenticationProvider(ApiKeyRepository apiKeyRepository) {
29         this.apiKeyRepository = apiKeyRepository;
30     }
31
32     @Override
33     @Transactional
34     public Authentication authenticate(Authentication authentication) throws AuthenticationException {
35         String apiKey = null;
36
37         // Extract API key from the authentication object
38         if (authentication instanceof ApiKeyAuthenticationToken) {
39             apiKey = ((ApiKeyAuthenticationToken) authentication).getApiKey();
40         }
41
42         if (apiKey == null || apiKey.isEmpty()) {
43             throw new RuntimeException("Invalid API key");
44         }
45
46         ApiKey key = apiKeyRepository.findByApiKeyAndActiveTrue(apiKey)
47             .orElseThrow(() -> new RuntimeException("Invalid or inactive API key"));
48
49         // Check expiration
50         if (key.getExpiresAt() != null && key.getExpiresAt().isBefore(Instant.now())) {
51             throw new RuntimeException("API key has expired");
52         }
53
54         // Update last used timestamp
55         apiKeyRepository.updateLastUsed(apiKey, Instant.now());
56
57         // Build authorities from permissions
58         List<GrantedAuthority> authorities = new ArrayList<>();
59
60         if (key.getPermissions() != null && !key.getPermissions().isEmpty()) {
61             authorities.addAll(Arrays.stream(key.getPermissions().split(","))
62                 .map(String::trim)
63                 .map(SimpleGrantedAuthority::new)
64                 .collect(Collectors.toList()));
65         }
66
67         Object principal;
68         if (key.getEmployee() != null) {
69             principal = key.getEmployee();
70             authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getEmployee().getRole().name()));
71         } else if (key.getVendor() != null) {

```

```
72         principal = key.getVendor();
73         authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
74     } else {
75         throw new RuntimeException("No user associated with API key");
76     }
77
78     return new ApiKeyAuthenticationToken(principal, apiKey, authorities);
79 }
80
81 @Override
82 public boolean supports(Class<?> authentication) {
83     return ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
84 }
85 }
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationToken.java
2 package com.example.EmployeeManagementSystem.security;
3
4 import org.springframework.security.core.Authentication;
5 import org.springframework.security.core.GrantedAuthority;
6 import java.util.Collection;
7
8 public class ApiKeyAuthenticationToken implements Authentication {
9
10     private static final long serialVersionUID = 1L;
11
12     private final Object principal;
13     private final Object credentials;
14     private final String apiKey;
15     private final Collection<? extends GrantedAuthority> authorities;
16     private boolean authenticated;
17
18     // Constructor for unauthenticated API key (before validation)
19     public ApiKeyAuthenticationToken(String apiKey) {
20         this.apiKey = apiKey;
21         this.principal = null;
22         this.credentials = null;
23         this.authorities = null;
24         this.authenticated = false;
25     }
26
27     // Constructor for authenticated API key (after validation)
28     public ApiKeyAuthenticationToken(Object principal, Object credentials,
29                                     Collection<? extends GrantedAuthority> authorities) {
30         this.apiKey = null;
31         this.principal = principal;
32         this.credentials = credentials;
33         this.authorities = authorities;
34         this.authenticated = true;
35     }
36
37     @Override
38     public Collection<? extends GrantedAuthority> getAuthorities() {
39         return authorities;
40     }
41
42     @Override
43     public Object getCredentials() {
44         return credentials;
45     }
46
47     @Override
48     public Object getDetails() {
49         return null;
50     }
51
52     @Override
53     public Object getPrincipal() {
54         return principal;
55     }
56
57     @Override
58     public boolean isAuthenticated() {
59         return authenticated;
60     }
61
62     @Override
63     public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
64         this.authenticated = isAuthenticated;
65     }
66
67     @Override
68     public String getName() {
69         if (principal instanceof org.springframework.security.core.userdetails.UserDetails) {
70             return ((org.springframework.security.core.userdetails.UserDetails) principal).getUsername();
71         }
```

```
72         if (principal instanceof com.example.EmployeeManagementSystem.Entity.Employee) {
73             return ((com.example.EmployeeManagementSystem.Entity.Employee) principal).getUsername();
74         }
75         if (principal instanceof com.example.EmployeeManagementSystem.Entity.Vendor) {
76             return ((com.example.EmployeeManagementSystem.Entity.Vendor) principal).getUsername();
77         }
78         return principal != null ? principal.toString() : apiKey;
79     }
80
81     public String getApiKey() {
82         return apiKey;
83     }
84 }
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.ApiKey;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.time.Instant;
13 import java.time.temporal.ChronoUnit;
14 import java.util.List;
15
16 @Service
17 public class ApiKeyService {
18
19     @Autowired
20     private ApiKeyRepository apiKeyRepository;
21     @Autowired private EmployeeRepo employeeRepo;
22     @Autowired private VendorRepo vendorRepo;
23
24     @Transactional
25     public ApiKey generateForVendor(Long vendorId, String name,
26                                     String permissions, int validDays) {
27         Vendor vendor = vendorRepo.findById(vendorId)
28             .orElseThrow(() -> new RuntimeException("Vendor not found"));
29
30         ApiKey key = new ApiKey(); // constructor auto-generates "ems_<uuid>"
31         key.setVendor(vendor);
32         key.setKeyName(name);
33         key.setPermissions(permissions); // e.g. "VENDOR_WRITE" or "TECH_VENDOR"
34         key.setExpiresAt(Instant.now().plus(validDays, ChronoUnit.DAYS));
35         key.setActive(true);
36         return apiKeyRepository.save(key);
37     }
38
39     // Same for employee-issued keys if needed
40     public List<ApiKey> getActiveKeysForVendor(Long vendorId) {
41         return apiKeyRepository.findByVendor_IdAndActiveTrue(vendorId);
42     }
43
44     @Transactional
45     public void revokeKey(Long keyId) {
46         ApiKey key = apiKeyRepository.findById(keyId)
47             .orElseThrow(() -> new RuntimeException("Key not found"));
48         key.setActive(false);
49         apiKeyRepository.save(key);
50     }
51 }
52
```

131 Current:

src\main\java\com\example\EmployeeManagementSystem\Service\CarryForwardWarningService.java

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
9 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
10 import org.slf4j.Logger;
11 import org.slf4j.LoggerFactory;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.math.BigDecimal;
16 import java.time.LocalDate;
17 import java.util.List;
18
19 @Service
20 public class CarryForwardWarningService {
21
22     private static final Logger log = LoggerFactory.getLogger(CarryForwardWarningService.class);
23
24     private static final BigDecimal CAP = BigDecimal.valueOf(30);
25     private static final BigDecimal WARN_AT_29 = BigDecimal.valueOf(29);
26     private static final BigDecimal WARN_AT_28 = BigDecimal.valueOf(28);
27
28     private final EmployeeRepo employeeRepository;
29     private final LeaveEntitlementRepository entitlementRepository;
30     private final LeaveWarningRepository warningRepository;
31
32     public CarryForwardWarningService(EmployeeRepo employeeRepository,
33                                     LeaveEntitlementRepository entitlementRepository,
34                                     LeaveWarningRepository warningRepository) {
35         this.employeeRepository = employeeRepository;
36         this.entitlementRepository = entitlementRepository;
37         this.warningRepository = warningRepository;
38     }
39
40     @Transactional
41     public void runWarningCheck(LocalDate checkDate) {
42         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
43
44         for (Employee employee : activeEmployees) {
45             BigDecimal carryForwardTotal = calculateCarryForwardBalance(employee, checkDate);
46             evaluateAndWarn(employee, carryForwardTotal, checkDate);
47         }
48
49         log.info("Warning check complete for {} – {} employees evaluated",
50               checkDate, activeEmployees.size());
51     }
52
53     // -----
54     // Sum available balance across all carry-forwardable types
55     // for the current year (this is what will carry forward if unused)
56     // -----
57     private BigDecimal calculateCarryForwardBalance(Employee employee, LocalDate date) {
58         int year = date.getYear();
59
60         List<LeaveEntitlement> entitlements =
61             entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(), year);
62
63         return entitlements.stream()
64             .filter(e -> e.getLeaveType().isCarriesForward())
65             .map(LeaveEntitlement::getAvailableBalance)
66             .reduce(BigDecimal.ZERO, BigDecimal::add);
67     }
68
69     // -----
70     // Decide which warning to issue based on carry-forward total
71     // -----

```

```

72     private void evaluateAndWarn(Employee employee, BigDecimal balance, LocalDate date) {
73         int cmp30 = balance.compareTo(CAP);
74         int cmp29 = balance.compareTo(WARN_AT_29);
75         int cmp28 = balance.compareTo(WARN_AT_28);
76
77         if (cmp30 > 0) {
78             // Balance EXCEEDS cap – a day was wasted (cap enforcement happens in Phase 5,
79             // but we flag it here so the warning appears immediately after accrual)
80             issueWarning(employee, LeaveWarning.WarningType.DAY_WASTED, balance, date,
81                 "A carry-forward day was wasted this month because your balance exceeded the 30-day
82 cap.");
83         } else if (cmp30 == 0) {
84             issueWarning(employee, LeaveWarning.WarningType.CAP_REACHED, balance, date,
85                 "Carry-forward cap reached (30 days). Next month's unused leave will be wasted.");
86         } else if (cmp29 >= 0) {
87             issueWarning(employee, LeaveWarning.WarningType.APPROACHING_29, balance, date,
88                 "You have " + balance.toPlainString() + " carry-forward days. "
89                 + "1 month until cap. Use leave before it stops accumulating.");
90         } else if (cmp28 >= 0) {
91             issueWarning(employee, LeaveWarning.WarningType.APPROACHING_28, balance, date,
92                 "You have " + balance.toPlainString() + " carry-forward days. "
93                 + "2 months until cap (30). Plan to use leave soon.");
94         }
95         // Below 28 – no warning needed
96     }
97 }
98
99 // -----
100 // Persist warning – skip if same type already issued today
101 // -----
102 private void issueWarning(Employee employee, LeaveWarning.WarningType type,
103     BigDecimal balance, LocalDate date, String message) {
104     boolean alreadyIssued = warningRepository
105         .existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(employee.getEmployeeId(), type,
106 date);
107     if (alreadyIssued) {
108         log.debug("Warning {} already issued for employee {} on {}", type, employee.getEmployeeId(),
109 date);
110         return;
111     }
112
113     LeaveWarning warning = new LeaveWarning();
114     warning.setEmployee(employee);
115     warning.setWarningType(type);
116     warning.setMessage(message);
117     warning.setCarryForwardBalance(balance);
118     warning.setWarningDate(date);
119
120     warningRepository.save(warning);
121
122     log.info("Warning issued – employee={} type={} balance={} date={}",
123         employee.getEmployeeId(), type, balance, date);
124 }
125 }
126 }

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import org.springframework.security.core.userdetails.UserDetails;
8 import org.springframework.security.core.userdetails.UserDetailsService;
9 import org.springframework.security.core.userdetails.UsernameNotFoundException;
10 import org.springframework.stereotype.Service;
11
12 import java.util.Optional;
13
14 @Service("combinedUserDetailsService")
15 public class CombinedUserDetailsService implements UserDetailsService {
16
17     private final VendorRepo vendorRepository;
18     private final EmployeeRepo employeeRepository;
19
20     public CombinedUserDetailsService(VendorRepo vendorRepository, EmployeeRepo employeeRepository) {
21         this.vendorRepository = vendorRepository;
22         this.employeeRepository = employeeRepository;
23     }
24
25     @Override
26     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
27         Optional<Employee> employee = employeeRepository.findByEmail(username);
28         if (employee.isPresent()) {
29             return employee.get(); // must implement UserDetails
30         }
31
32         // Then try Vendor
33         Optional<Vendor> vendor = vendorRepository.findByEmail(username);
34         if (vendor.isPresent()) {
35             return vendor.get();
36         }
37
38         throw new UsernameNotFoundException("User not found: " + username);
39     }
40 }
41
```



```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository VendorRepo;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.authority.SimpleGrantedAuthority;
9 import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
11 import org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
12 import org.springframework.security.oauth2.core.user.DefaultOAuth2User;
13 import org.springframework.security.oauth2.core.user.OAuth2User;
14 import org.springframework.stereotype.Service;
15
16 import java.util.ArrayList;
17 import java.util.List;
18 import java.util.UUID;
19
20 @Service
21 public class CustomOAuth2UserService extends DefaultOAuth2UserService {
22
23     private final EmployeeRepo employeeRepo;
24     private final VendorRepo vendorRepo;
25     private final PasswordEncoder passwordEncoder;
26
27     public CustomOAuth2UserService(EmployeeRepo employeeRepo,
28                                   VendorRepo vendorRepo,
29                                   PasswordEncoder passwordEncoder) {
30         this.employeeRepo = employeeRepo;
31         this.vendorRepo = vendorRepo;
32         this.passwordEncoder = passwordEncoder;
33     }
34
35     @Override
36     public OAuth2User loadUser(OAuth2UserRequest userRequest) {
37         System.out.println("=== CustomOAuth2UserService.loadUser() called ===");
38         OAuth2User oAuth2User = super.loadUser(userRequest);
39
40         String email = oAuth2User.getAttribute("email");
41         String name = oAuth2User.getAttribute("name");
42         System.out.println("OAuth email: " + email);
43
44         boolean employeeExists = employeeRepo.findByEmail(email).isPresent();
45         boolean vendorExists = vendorRepo.findByEmail(email).isPresent();
46         System.out.println("employeeExists=" + employeeExists + ", vendorExists=" + vendorExists);
47
48         List<GrantedAuthority> authorities = new ArrayList<>(oAuth2User.getAuthorities());
49
50         if (employeeExists) {
51             Employee employee = employeeRepo.findByEmail(email).get();
52             authorities.add(new SimpleGrantedAuthority("ROLE_" + employee.getRole().name()));
53
54             // ADD THIS – loads LEAVE_CANCEL, SUBSCRIPTION_READ, etc. into the session
55             employee.getRole().getPermissions().forEach(permission ->
56                 authorities.add(new SimpleGrantedAuthority(permission.name()))
57             );
58
59             System.out.println("Returning existing employee with role: " + employee.getRole());
60             return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
61         }
62
63         if (vendorExists) {
64             // add vendor role if needed
65             authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
66             System.out.println("Returning existing vendor");
67             return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
68         }
69
70         // New user
71         registerAsUser(email, name);

```

```
72     authorities.add(new SimpleGrantedAuthority("ROLE_USER"));
73     return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
74 }
75
76 private void registerAsUser(String email, String name) {
77     Employee e = new Employee();
78     e.setEmail(email);
79     e.setName(name);
80     e.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
81     e.setRole(Role.USER);
82     e.setTimezone("Asia/Kolkata");
83     e.setDept("UNASSIGNED");
84     employeeRepo.saveAndFlush(e); // flush immediately so OAuth2SuccessHandler sees the record
85     System.out.println("New USER registered and flushed for: " + email);
86 }
87 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorDTO;
4 import com.example.EmployeeManagementSystem.DTO.VendorRequest;
5 import com.example.EmployeeManagementSystem.Entity.Delivery;
6 import com.example.EmployeeManagementSystem.Entity.Subscription;
7 import com.example.EmployeeManagementSystem.Entity.Vendor;
8 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
9 import com.example.EmployeeManagementSystem.Enum.Role;
10 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
11 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
12 import com.example.EmployeeManagementSystem.Repository.DeliveryRepository;
13 import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
14 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
15 import com.example.EmployeeManagementSystem.Util.AuthUtil;
16 import org.slf4j.Logger;
17 import org.slf4j.LoggerFactory;
18 import org.springframework.dao.DataIntegrityViolationException;
19 import org.springframework.security.core.Authentication;
20 import org.springframework.security.crypto.password.PasswordEncoder;
21 import org.springframework.stereotype.Service;
22 import org.springframework.transaction.annotation.Transactional;
23
24 import java.time.Instant;
25 import java.util.List;
26 import java.util.stream.Collectors;
27
28 @Service
29 public class DeliveryService {
30     private final SubscriptionRepository subscriptionRepository;
31     private final DeliveryRepository deliveryRepository;
32     private final DeliveryTimeService deliveryTimeService;
33     private static final Logger logger = LoggerFactory.getLogger(DeliveryService.class);
34
35     public DeliveryService(SubscriptionRepository subscriptionRepository,
36                           DeliveryRepository deliveryRepository,
37                           DeliveryTimeService deliveryTimeService) {
38         this.subscriptionRepository = subscriptionRepository;
39         this.deliveryRepository = deliveryRepository;
40         this.deliveryTimeService = deliveryTimeService;
41     }
42
43     // AFTER
44     @Transactional
45     public void processDueDeliveries() {
46         Instant now = Instant.now();
47         List<Subscription> dueSubscriptions = subscriptionRepository
48             .findDueSubscriptions(SubscriptionStatus.ACTIVE.name(), now);
49
50         for (Subscription subscription : dueSubscriptions) {
51             try {
52                 Delivery delivery = new Delivery();
53                 delivery.setSubscription(subscription);
54                 delivery.setMealSlot(subscription.getSlot());
55                 delivery.setScheduledDeliveryTime(subscription.getNextDeliveryTime());
56                 delivery.setStatus(DeliveryStatus.IN_PROGRESS);
57                 deliveryRepository.save(delivery);
58
59                 Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
60                 subscription.setNextDeliveryTime(nextDelivery);
61                 subscriptionRepository.save(subscription);
62
63             } catch (DataIntegrityViolationException e) {
64                 // Duplicate delivery already exists – just advance the next delivery time
65                 logger.warn("Duplicate delivery skipped for subscription ID: {}", subscription.getId());
66                 Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
67                 subscription.setNextDeliveryTime(nextDelivery);
68                 subscriptionRepository.save(subscription);
69             }
70         }
71     }

```

```

72
73     @Transactional
74     public Delivery updateDeliveryStatus(Long deliveryId, DeliveryStatus status) {
75         Delivery delivery = deliveryRepository.findById(deliveryId)
76             .orElseThrow(() -> new RuntimeException("Delivery not found"));
77
78         delivery.setStatus(status);
79         if (status == DeliveryStatus.DELIVERED) {
80             delivery.setActualDeliveryTime(Instant.now());
81         }
82
83         return deliveryRepository.save(delivery);
84     }
85
86     public List<Delivery> getDeliveriesBySubscription(Long subscriptionId) {
87         Subscription subscription = subscriptionRepository.findById(subscriptionId)
88             .orElseThrow(() -> new RuntimeException("Subscription not found"));
89         return deliveryRepository.findRecentDeliveriesBySubscription(subscription,
DeliveryStatus.SCHEDULED);
90     }
91
92     @Service
93     public static class VendorService {
94
95         private static final Logger log = LoggerFactory.getLogger(VendorService.class);
96         private final VendorRepo vendorRepo;
97         private final PasswordEncoder passwordEncoder;
98
99         public VendorService(VendorRepo vendorRepo, PasswordEncoder passwordEncoder) {
100             this.vendorRepo = vendorRepo;
101             this.passwordEncoder = passwordEncoder;
102         }
103
104         public VendorDTO registerVendor(VendorRequest request) {
105             if (vendorRepo.existsByEmail(request.getEmail())) {
106                 throw new IllegalArgumentException(
107                     "A vendor with email " + request.getEmail() + " already exists");
108             }
109             Vendor vendor = new Vendor();
110             vendor.setName(request.getName());
111             vendor.setEmail(request.getEmail());
112             vendor.setPhone(request.getPhone());
113             vendor.setRole(request.getRole());
114             vendor.setPassword(passwordEncoder.encode(request.getPassword()));
115             Vendor saved = vendorRepo.save(vendor);
116             log.info("Vendor registered: {} (id={})", saved.getName(), saved.getId());
117             return toDTO(saved);
118         }
119
120         public VendorDTO getVendor(Authentication authentication) {
121             String email= AuthUtil.extractEmail(authentication);
122             return toDTO(vendorRepo.findById(email).orElseThrow(
123                 ()->new VendorNotFoundException("Cannot get others account")
124             ));
125         }
126
127         public List<VendorDTO> getAllVendors() {
128             return vendorRepo.findAll().stream()
129                 .map(this::toDTO)
130                 .collect(Collectors.toList());
131         }
132
133         public VendorDTO updateVendor(VendorRequest request, Authentication authentication) {
134             String email= AuthUtil.extractEmail(authentication);
135             Vendor vendor = vendorRepo.findById(email).orElseThrow(
136                 ()->new VendorNotFoundException("Cannot update others account")
137             );
138             if (request.getName() != null) vendor.setName(request.getName());
139             if (request.getPhone() != null) vendor.setPhone(request.getPhone());
140             if (request.getPassword() != null)
vendor.setPassword(passwordEncoder.encode(request.getPassword()));
141             // Email update not allowed to avoid breaking restaurant ownership references
142             return toDTO(vendorRepo.save(vendor));
143         }
144
145         public void deleteVendor(Authentication authentication) {

```

```
146         String email= AuthUtil.extractEmail(authentication);
147         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
148             ()->new VendorNotFoundException("Cannot delete others account")
149         );
150         vendorRepo.delete(vendor);
151         log.info("Vendor deleted: email={}", email);
152     }
153
154     public void deleteVendorById(Long vendorId) {
155         Vendor vendor = findOrThrow(vendorId);
156         vendorRepo.delete(vendor);
157         log.info("Vendor deleted by admin: id={}, email={}", vendor.getId(), vendor.getEmail());
158     }
159
160     // helpers
161
162     private Vendor findOrThrow(Long id) {
163         return vendorRepo.findById(id)
164             .orElseThrow(() -> new VendorNotFoundException("Vendor with id " + id + " not found"));
165     }
166
167     private VendorDTO toDTO(Vendor vendor) {
168         VendorDTO dto = new VendorDTO();
169         dto.setId(vendor.getId());
170         dto.setName(vendor.getName());
171         dto.setEmail(vendor.getEmail());
172         dto.setPhone(vendor.getPhone());
173         dto.setRegisteredAt(vendor.getRegisteredAt());
174         return dto;
175     }
176 }
177 }
178
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.Subscription;
6 import com.example.EmployeeManagementSystem.Enum.MealSlot;
7 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
8 import org.springframework.stereotype.Service;
9
10 import java.time.*;
11
12 @Service
13 public class DeliveryTimeService {
14
15     private final MealSlotService mealSlotService;
16
17     public DeliveryTimeService(MealSlotService mealSlotService) {
18         this.mealSlotService = mealSlotService;
19     }
20     public Instant calculateNextDelivery(SubscriptionsRequest request, MealSlot slot, Employee employee) {
21         ZoneId userZone=ZoneId.of(employee.getTimezone());
22         if (request.getScheduleType() == ScheduleType.DAILY) {
23             return calculateDailyNextDelivery(slot,userZone);
24         } else {
25             return calculateWeeklyNextDelivery(slot, request.getDayOfWeek(),userZone);
26         }
27     }
28
29     private Instant calculateDailyNextDelivery(MealSlot slot, ZoneId userZone) {
30         LocalDateTime slotTime = mealSlotService.getTime(slot);
31         ZonedDateTime now = ZonedDateTime.now(userZone);
32         ZonedDateTime deliveryTime;
33
34         if (now.toLocalTime().isBefore(slotTime)) {
35             deliveryTime=ZonedDateTime.of(now.toLocalDate(),slotTime,userZone);
36         } else {
37             deliveryTime=ZonedDateTime.of(now.toLocalDate().plusDays(1), slotTime,userZone);
38         }
39         return deliveryTime.toInstant();
40     }
41
42     private Instant calculateWeeklyNextDelivery(MealSlot slot, DayOfWeek dayOfWeek,ZoneId userZone) {
43         LocalDateTime slotTime = mealSlotService.getTime(slot);
44         ZonedDateTime now=ZonedDateTime.now(userZone);
45         DayOfWeek today = now.getDayOfWeek();
46
47         int daysToAdd = dayOfWeek.getValue() - today.getValue();
48
49         if (daysToAdd == 0) {
50             if (now.toLocalTime().isBefore(slotTime)) {
51                 return ZonedDateTime.of(now.toLocalDate(), slotTime,userZone).toInstant();
52             } else {
53                 return ZonedDateTime.of(now.toLocalDate().plusDays(7), slotTime,userZone).toInstant();
54             }
55         }
56         if (daysToAdd < 0) {
57             daysToAdd += 7;
58         }
59
60         ZonedDateTime deliveryTime = ZonedDateTime.of(
61             now.toLocalDate().plusDays(daysToAdd),slotTime,userZone);
62         return deliveryTime.toInstant();
63     }
64
65     public Instant getNextDeliveryTime(Subscriptions subscription) {
66         ZoneId userZone=ZoneId.of(subscription.getEmployee().getTimezone());
67         if (subscription.getScheduleType() == ScheduleType.DAILY) {
68             return calculateDailyNextDelivery(subscription.getSlot(),userZone);
69         } else {
70             return calculateWeeklyNextDelivery(subscription.getSlot(),
71 subscription.getDayOfWeek(),userZone);
72         }
73     }
74 }
```

```
71     }
```

```
72 }
```

```
73 }
```

```
74
```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
4  import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
5  import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
6  import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
7  import com.example.EmployeeManagementSystem.Entity.Device;
8  import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
9  import com.example.EmployeeManagementSystem.Entity.Employee;
10 import com.example.EmployeeManagementSystem.Entity.Vendor;
11 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
12 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
13 import com.example.EmployeeManagementSystem.Enum.Role;
14 import com.example.EmployeeManagementSystem.Enum.Status;
15 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
16 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
17 import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
18 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
19 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
20 import com.example.EmployeeManagementSystem.Util.AuthUtil;
21 import org.springframework.security.core.Authentication;
22 import org.springframework.stereotype.Service;
23 import org.springframework.transaction.annotation.Transactional;
24
25 import java.time.LocalDate;
26 import java.time.LocalDateTime;
27 import java.util.List;
28 import java.util.Set;
29
30 @Service
31 public class DeviceService {
32     private final DeviceRepository deviceRepository;
33     private final VendorRepo vendorRepo;
34     private final DeviceAssignmentRepo deviceAssignmentRepo;
35     private final EmployeeRepo employeeRepo;
36
37     public DeviceService(DeviceRepository deviceRepository,
38                         VendorRepo vendorRepo,
39                         DeviceAssignmentRepo deviceAssignmentRepo,
40                         EmployeeRepo employeeRepo) {
41         this.deviceRepository = deviceRepository;
42         this.vendorRepo = vendorRepo;
43         this.deviceAssignmentRepo = deviceAssignmentRepo;
44         this.employeeRepo = employeeRepo;
45     }
46
47     public Device addDevice(DeviceDTO deviceDTO, Authentication authentication) {
48         String email = AuthUtil.extractEmail(authentication);
49         Vendor vendor = vendorRepo.findByEmail(email)
50             .orElseThrow(() -> new RuntimeException("Vendor not found"));
51
52         Device device = new Device();
53         device.setDeviceName(deviceDTO.getDeviceName());
54         device.setWarrantyExpiryDate(deviceDTO.getWarrantyExpiryDate());
55         device.setBrand(deviceDTO.getBrand());
56         device.setDeviceStatus(DeviceStatus.AVAILABLE);
57         device.setDeviceType(deviceDTO.getDeviceType());
58         device.setTechVendor(vendor);
59         device.setCreatedAt(LocalDateTime.now());
60         return deviceRepository.save(device);
61     }
62
63     public List<DeviceResponseDTO> getDevicesForLoggedInVendor(Authentication authentication) {
64         String email = AuthUtil.extractEmail(authentication);
65         Vendor vendor = vendorRepo.findByEmail(email)
66             .orElseThrow(() -> new RuntimeException("Vendor not found"));
67
68         return deviceRepository.findByTechVendorId(vendor.getId()).stream()
69             .map(this::toDeviceResponse)
70             .toList();
71     }

```



```

72
73     public List<DeviceResponseDTO> getDevicesForLoggedInEmployee(Authentication authentication) {
74         String email = AuthUtil.extractEmail(authentication);
75         Employee employee = employeeRepo.findByEmail(email)
76             .orElseThrow(() -> new RuntimeException("Employee not found"));
77
78         return deviceAssignmentRepo.findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(),
AssignmentStatus.ACTIVE)
79             .stream()
80             .map(DeviceAssignment::getDevice)
81             .map(this::toDeviceResponse)
82             .toList();
83     }
84
85     public List<DeviceResponseDTO> getUnassignedDevices() {
86         return deviceRepository.findByDeviceStatus(DeviceStatus.AVAILABLE).stream()
87             .map(this::toDeviceResponse)
88             .toList();
89     }
90
91     @Transactional
92     public DeviceAssignment assignDevice(Long deviceId, AssignDeviceRequest request) {
93         if (request.getEmployeeId() == null) {
94             throw new IllegalArgumentException("employeeId is required");
95         }
96
97         Device device = deviceRepository.findById(deviceId)
98             .orElseThrow(() -> new RuntimeException("Device not found"));
99
100        Employee employee = employeeRepo.findById(request.getEmployeeId())
101            .orElseThrow(() -> new RuntimeException("Employee not found"));
102
103        if (employee.getStatus() != Status.ACTIVE) {
104            throw new IllegalArgumentException("Device can only be assigned to an active user");
105        }
106
107        if (!Set.of(Role.EMPLOYEE, Role.MANAGER, Role.ADMIN).contains(employee.getRole())) {
108            throw new IllegalArgumentException("Devices can only be assigned to employees, managers, or
admins");
109        }
110
111        if (device.getDeviceStatus() != DeviceStatus.AVAILABLE) {
112            throw new IllegalArgumentException("Only available devices can be assigned");
113        }
114
115        deviceAssignmentRepo.findByDeviceIdAndStatus(deviceId, AssignmentStatus.ACTIVE)
116            .ifPresent(existingAssignment -> {
117                throw new IllegalArgumentException("Device is already assigned");
118            });
119
120        DeviceAssignment assignment = new DeviceAssignment();
121        assignment.setDevice(device);
122        assignment.setAssignedTo(employee);
123        assignment.setStatus(AssignmentStatus.ACTIVE);
124        assignment.setAssignedDate(LocalDate.now());
125
126        DeviceAssignment savedAssignment = deviceAssignmentRepo.save(assignment);
127        device.setCurrentAssignment(savedAssignment);
128        device.setDeviceStatus(DeviceStatus.ASSIGNED);
129        deviceRepository.save(device);
130
131        return savedAssignment;
132    }
133
134
135
136    private DeviceResponseDTO toDeviceResponse(Device device) {
137        DeviceResponseDTO response = new DeviceResponseDTO();
138        response.setId(device.getId());
139        response.setDeviceName(device.getDeviceName());
140        response.setBrand(device.getBrand());
141        response.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
142        response.setDeviceType(device.getDeviceType());
143        response.setDeviceStatus(device.getDeviceStatus());
144
145        if (device.getTechVendor() != null) {

```

```

146         response.setVendorName(device.getTechVendor().getName());
147     }
148
149     DeviceAssignment currentAssignment = device.getCurrentAssignment();
150     if (currentAssignment != null) {
151         response.setAssignedDate(currentAssignment.getAssignedDate());
152
153         Employee assignedEmployee = currentAssignment.getAssignedTo();
154         if (assignedEmployee != null) {
155             response.setAssignedEmployeeId(assignedEmployee.getEmployeeId());
156             response.setAssignedEmployeeName(assignedEmployee.getName());
157         }
158     }
159
160     return response;
161 }
162
163 public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfVendor(Authentication authentication) {
164     String email=AuthUtil.extractEmail(authentication);
165     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
166         ()->new VendorNotFoundException("Vendor:"+email+" not found")
167     );
168     List<DeviceAssignment>
deviceAssignments=deviceAssignmentRepo.getAllAssignmentsByVendorId(vendor.getId());
169     return deviceAssignments.stream().map(deviceAssignment -> toAssignmentDTO(deviceAssignment)).toList();
170 }
171
172 private DeviceAssignmentResponseDTO toAssignmentDTO(DeviceAssignment a) {
173     DeviceAssignmentResponseDTO dto = new DeviceAssignmentResponseDTO();
174     dto.setId(a.getId());
175     dto.setStatus(a.getStatus());
176     dto.setAssignedDate(a.getAssignedDate());
177
178     if (a.getDevice() != null) {
179         dto.setDeviceId(a.getDevice().getId());
180         dto.setDeviceName(a.getDevice().getDeviceName());
181         dto.setDeviceBrand(a.getDevice().getBrand());
182         dto.setDeviceType(a.getDevice().getDeviceType());
183     }
184
185     if (a.getAssignedTo() != null) {
186         dto.setEmployeeId(a.getAssignedTo().getEmployeeId());
187         dto.setEmployeeName(a.getAssignedTo().getName()); // adjust to your Employee field
188     }
189
190     return dto;
191 }
192 }
193

```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
6  import com.example.EmployeeManagementSystem.Enum.Role;
7  import com.example.EmployeeManagementSystem.Enum.Status;
8  import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
9  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
11 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
12 import com.example.EmployeeManagementSystem.Util.AuthUtil;
13 import org.slf4j.Logger;
14 import org.slf4j.LoggerFactory;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.security.crypto.password.PasswordEncoder;
18 import org.springframework.stereotype.Service;
19 import org.springframework.transaction.annotation.Transactional;
20
21 import java.time.ZoneId;
22 import java.util.List;
23 import java.util.stream.Collectors;
24
25 @Service
26 public class EmployeeService {
27
28     private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);
29
30     private final EmployeeRepo employeeRepo;
31     private final LeaveRequestRepo leaveRequestRepo;
32     private final PasswordEncoder passwordEncoder;
33     private final RefreshTokenRepository refreshTokenRepository; // FIX: added
34
35     public EmployeeService(EmployeeRepo employeeRepo,
36                             LeaveRequestRepo leaveRequestRepo,
37                             PasswordEncoder passwordEncoder,
38                             RefreshTokenRepository refreshTokenRepository) {
39         this.employeeRepo = employeeRepo;
40         this.leaveRequestRepo = leaveRequestRepo;
41         this.passwordEncoder = passwordEncoder;
42         this.refreshTokenRepository = refreshTokenRepository; // FIX: injected
43     }
44
45     public List<Employee> getAllEmployees() {
46         return employeeRepo.findAll();
47     }
48
49     public Employee createEmployee(AdminEmployeeDTO employeeDTO) {
50         log.info("Creating new employee with email: {}", employeeDTO.getEmail());
51         var employee = new Employee();
52         if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
53         if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
54         if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
55             employee.setTimezone(employeeDTO.getTimezone());
56         } else {
57             employee.setTimezone("UTC");
58         }
59         employee.setRole(Role.EMPLOYEE);
60         if (employeeDTO.getPassword() == null) {
61             throw new RuntimeException("Password is required");
62         }
63         employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
64         employee.setGender(employeeDTO.getGender());
65         Employee manager=employeeRepo.findById(employeeDTO.getManagerId()).orElseThrow(
66             ()->new EmployeeNotFound("Employee with not found"+ employeeDTO.getManagerId())
67         );
68         employee.setDept(manager.getDept());
69         employee.setManager(manager);
70         return employeeRepo.save(employee);
71     }

```

```

72
73     public Employee createManager(EmployeeDTO employeeDTO) {
74         log.info("Creating new manager with email: {}", employeeDTO.getEmail());
75         var employee = new Employee();
76         if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
77         if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
78         if (employeeDTO.getDept() != null) employee.setDept(employeeDTO.getDept());
79         if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
80             employee.setTimezone(employeeDTO.getTimezone());
81         } else {
82             employee.setTimezone("UTC");
83         }
84         employee.setRole(Role.MANAGER);
85         if (employeeDTO.getPassword() == null) {
86             throw new RuntimeException("Password is required");
87         }
88         employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
89         employee.setGender(employeeDTO.getGender());
90         return employeeRepo.save(employee);
91     }
92
93     public Employee updateEmployee(long id, EmployeeDTO employee) {
94         var updateEmployee = employeeRepo.findById(id).orElseThrow(
95             () -> new EmployeeNotFound("Employee with id:" + id + " not found")
96         );
97         if (employee.getName() != null)
98             updateEmployee.setName(employee.getName());
99         if (employee.getEmail() != null)
100             updateEmployee.setEmail(employee.getEmail());
101         if (employee.getDept() != null)
102             updateEmployee.setDept(employee.getDept());
103         if (employee.getTimezone() != null && isValidTimezone(employee.getTimezone())) {
104             updateEmployee.setTimezone(employee.getTimezone());
105         }
106         return employeeRepo.save(updateEmployee);
107     }
108
109     /**
110      * FIX: Delete order must be:
111      * 1. refresh_token (FK employee)
112      * 2. leave_request (FK employee)
113      * 3. employee
114      *
115      * Previously only leave_request was deleted first, causing a
116      * FK constraint failure on refresh_token when deleting the employee.
117      */
118     @Transactional
119     public void deleteEmployee(long id) {
120         var employee = employeeRepo.findById(id).orElseThrow(
121             () -> new EmployeeNotFound("Employee with id:" + id + " not found")
122         );
123
124         // Step 1: delete refresh tokens for this employee
125         refreshTokenRepository.deleteByEmployee(employee);
126
127         // Step 2: delete leave requests for this employee
128         List<LeaveRequest> leaveRequests =
129             leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
130         leaveRequestRepo.deleteAll(leaveRequests);
131
132         // Step 3: delete the employee
133         employeeRepo.delete(employee);
134     }
135
136     public ResponseEntity<String> inactivateUser(long employeeId) {
137         var employee = employeeRepo.findById(employeeId).orElseThrow(
138             () -> new EmployeeNotFound("Employee with id:" + employeeId + " not found")
139         );
140         if (employee.getStatus() == Status.INACTIVE) {
141             return ResponseEntity.badRequest()
142                 .body("Status of the employee with id: " + employeeId + " is already set to inactive");
143         }
144         employee.setStatus(Status.INACTIVE);
145         employeeRepo.save(employee);
146         return ResponseEntity.ok("Status of the employee with id: " + employeeId + " is set to inactive");
147     }

```

```

148
149  /** Validate that the given string is a recognized IANA timezone id. */
150  private boolean isValidTimezone(String tz) {
151      try {
152          ZoneId.of(tz);
153          return true;
154      } catch (Exception e) {
155          return false;
156      }
157  }
158
159  public Employee getAccount(Authentication authentication) {
160      String email = AuthUtil.extractEmail(authentication);
161      Employee employee = employeeRepo.findByEmail(email).orElseThrow(
162          () -> new EmployeeNotFound("Employee :" + email + " NOT FOUND")
163      );
164      return employee;
165  }
166
167  public EmployeeAttendanceResponseDTO getEmployeeAttendanceOverview() {
168      List<Employee> employees = employeeRepo.findAll();
169
170      List<EmployeeStatusDTO> workingEmployees = employees.stream()
171          .filter(e -> e.getStatus() == Status.ACTIVE)
172          .map(this::toStatusDTO)
173          .collect(Collectors.toList());
174
175      List<EmployeeStatusDTO> onLeaveEmployees = employees.stream()
176          .filter(e -> e.getStatus() == Status.ON_LEAVE)
177          .map(this::toStatusDTO)
178          .collect(Collectors.toList());
179
180      EmployeeAttendanceResponseDTO response = new EmployeeAttendanceResponseDTO();
181      response.setWorkingCount(workingEmployees.size());
182      response.setOnLeaveCount(onLeaveEmployees.size());
183      response.setWorkingEmployees(workingEmployees);
184      response.setOnLeaveEmployees(onLeaveEmployees);
185      return response;
186  }
187
188  private EmployeeStatusDTO toStatusDTO(Employee employee) {
189      EmployeeStatusDTO dto = new EmployeeStatusDTO();
190      dto.setEmployeeId(employee.getEmployeeId());
191      dto.setName(employee.getName());
192      dto.setEmail(employee.getEmail());
193      dto.setDept(employee.getDept());
194      dto.setRole(employee.getRole());
195      return dto;
196  }
197
198  public ResponseEntity<List<ManagerDTO>> getAllManagers() {
199      List<ManagerDTO> managers = employeeRepo.findByRole(Role.MANAGER).stream()
200          .map(this::toManagerDTO)
201          .collect(Collectors.toList());
202      return ResponseEntity.ok(managers);
203  }
204
205  private ManagerDTO toManagerDTO(Employee employee) {
206      ManagerDTO dto = new ManagerDTO();
207      dto.setManagerId(employee.getEmployeeId());
208      dto.setName(employee.getName());
209      dto.setEmail(employee.getEmail());
210      dto.setDept(employee.getDept());
211      return dto;
212  }
213 }
214

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.HolidayDTO;
4 import com.example.EmployeeManagementSystem.Entity.FixedHoliday;
5 import com.example.EmployeeManagementSystem.Repository.FixedHolidayRepository;
6 import org.springframework.stereotype.Service;
7
8 import java.time.LocalDate;
9 import java.util.List;
10 @Service
11 public class HolidayService {
12     private final FixedHolidayRepository holidayRepository;
13
14     public HolidayService(FixedHolidayRepository holidayRepository) {
15         this.holidayRepository = holidayRepository;
16     }
17
18     // Admin adds a holiday
19     public FixedHoliday addHoliday(HolidayDTO dto) {
20         FixedHoliday h = new FixedHoliday();
21         h.setName(dto.getName());
22         h.setDate(LocalDate.parse(dto.getDate()));
23         h.setDescription(dto.getDescription());
24         h.setYear(LocalDate.parse(dto.getDate()).getYear());
25         return holidayRepository.save(h);
26     }
27
28     // Everyone can view holidays
29     public List<FixedHoliday> getHolidaysByYear(int year) {
30         return holidayRepository.findByYear(year);
31     }
32
33     // Admin deletes a holiday
34     public void deleteHoliday(Long id) {
35         holidayRepository.deleteById(id);
36     }
37 }
38
39
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveType;
6 import com.example.EmployeeManagementSystem.Enum.Gender;
7 import com.example.EmployeeManagementSystem.Enum.Status;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
10 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
11 import org.slf4j.Logger;
12 import org.slf4j.LoggerFactory;
13 import org.springframework.stereotype.Service;
14 import org.springframework.transaction.annotation.Transactional;
15
16 import java.math.BigDecimal;
17 import java.time.LocalDate;
18 import java.util.List;
19 import java.util.Set;
20
21 @Service
22 public class LeaveAccrualService {
23
24     private static final Logger log = LoggerFactory.getLogger(LeaveAccrualService.class);
25     private static final BigDecimal ONE_DAY = BigDecimal.ONE;
26     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
27
28     private final EmployeeRepo employeeRepository;
29     private final LeaveTypeRepository leaveTypeRepository;
30     private final LeaveEntitlementRepository entitlementRepository;
31
32     public LeaveAccrualService(EmployeeRepo employeeRepository,
33                               LeaveTypeRepository leaveTypeRepository,
34                               LeaveEntitlementRepository entitlementRepository) {
35         this.employeeRepository = employeeRepository;
36         this.leaveTypeRepository = leaveTypeRepository;
37         this.entitlementRepository = entitlementRepository;
38     }
39
40     @Transactional
41     public void runMonthlyAccrual(LocalDate accrualDate) {
42         int year = accrualDate.getYear();
43
44         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
45         List<LeaveType> allLeaveTypes = leaveTypeRepository.findAll();
46
47         log.info("Starting monthly accrual for {} - {} active employees",
48                 accrualDate, activeEmployees.size());
49
50         for (Employee employee : activeEmployees) {
51             for (LeaveType leaveType : allLeaveTypes) {
52
53                 if (!SUPPORTED_LEAVE_TYPES.contains(leaveType.getName())) {
54                     continue;
55                 }
56                 if (!isEligible(employee, leaveType, accrualDate)) {
57                     continue;
58                 }
59
60                 // Find or create entitlement row for this employee/type/year
61                 LeaveEntitlement entitlement = entitlementRepository
62                     .findByEmployeeIdAndLeaveTypeIdAndYear(
63                         employee.getEmployeeId(), leaveType.getId(), year)
64                     .orElseGet(() -> createNewEntitlement(employee, leaveType, year));
65
66                 // Add 1 day accrual
67                 entitlement.setAccruedThisYear(
68                     entitlement.getAccruedThisYear().add(ONE_DAY)
69                 );
70
71                 entitlementRepository.save(entitlement);

```

```

72         log.debug("Accrued 1 {} day for employee {} (year {})",
73             leaveType.getName(), employee.getEmployeeId(), year);
74     }
75 }
76
77     log.info("Monthly accrual complete for {}", accrualDate);
78 }
79
80 // -----
81 // Eligibility check – single place for all rules
82 // -----
83 private boolean isEligible(Employee employee, LeaveType leaveType, LocalDate accrualDate) {
84
85     // 1. Gender restriction (Maternity = female only)
86     if (leaveType.isGenderRestricted()) {
87         Gender restriction = leaveType.getGenderRestriction();
88         if (restriction != null && restriction != employee.getGender()) {
89             return false;
90         }
91     }
92
93     // 2. Employee must have joined ON OR BEFORE the accrual date
94     // (no accrual for employees joining after the 1st of this month)
95     if (employee.getJoined_at() != null && employee.getJoined_at().isAfter(accrualDate)) {
96         return false;
97     }
98
99     return true;
100 }
101
102 // -----
103 // Creates a fresh entitlement row, carrying forward previous year's
104 // closing balance if this is the first row of the year
105 // -----
106 private LeaveEntitlement createNewEntitlement(Employee employee,
107     LeaveType leaveType,
108     int year) {
109     LeaveEntitlement entitlement = new LeaveEntitlement();
110     entitlement.setEmployee(employee);
111     entitlement.setLeaveType(leaveType);
112     entitlement.setYear(year);
113
114     // Carry forward last year's closing balance (if the type supports it)
115     if (leaveType.isCarriesForward()) {
116         BigDecimal lastYearClosing = entitlementRepository
117             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
118                 employee.getEmployeeId(), leaveType.getId(), year - 1)
119             .map(LeaveEntitlement::getClosingBalance)
120             .orElse(BigDecimal.ZERO);
121
122         entitlement.setOpeningBalance(lastYearClosing);
123     }
124
125     return entitlementRepository.save(entitlement);
126 }
127 }
128

```



```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6 import com.example.EmployeeManagementSystem.Entity.LeaveType;
7 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
8 import com.example.EmployeeManagementSystem.Enum.Gender;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
13 import org.springframework.security.access.AccessDeniedException;
14 import org.springframework.stereotype.Service;
15 import org.springframework.transaction.annotation.Transactional;
16
17 import java.math.BigDecimal;
18 import java.time.LocalDate;
19 import java.util.Set;
20 import java.util.LinkedHashMap;
21 import java.util.List;
22 import java.util.Map;
23
24 @Service
25 public class LeaveDashboardService {
26
27     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
28
29     private final EmployeeRepo employeeRepository;
30     private final LeaveEntitlementRepository leaveEntitlementRepository;
31     private final LeaveTypeRepository leaveTypeRepository;
32     private final LeaveWarningRepository warningRepository;
33
34     public LeaveDashboardService(EmployeeRepo employeeRepository,
35                                 LeaveEntitlementRepository leaveEntitlementRepository,
36                                 LeaveTypeRepository leaveTypeRepository,
37                                 LeaveWarningRepository warningRepository) {
38         this.employeeRepository = employeeRepository;
39         this.leaveEntitlementRepository = leaveEntitlementRepository;
40         this.leaveTypeRepository = leaveTypeRepository;
41         this.warningRepository = warningRepository;
42     }
43
44     @Transactional
45     public LeaveDashboardResponse getDashboard(Long requestedEmployeeId,
46                                              Long callerEmployeeId,
47                                              String callerRole) {
48         assertAccess(requestedEmployeeId, callerEmployeeId, callerRole);
49
50         Employee employee = employeeRepository.findById(requestedEmployeeId)
51             .orElseThrow(() -> new IllegalArgumentException(
52                 "Employee not found: " + requestedEmployeeId));
53
54         int year = LocalDate.now().getYear();
55
56         ensureCurrentYearEntitlements(employee, year);
57         List<LeaveEntitlement> entitlements =
58             leaveEntitlementRepository.findByEmployeeIdAndYear(requestedEmployeeId, year);
59
60         List<LeaveWarning> warnings =
61             warningRepository.findByEmployeeIdAndIsReadFalseOrderByWarningDateDesc(
62                 requestedEmployeeId);
63
64         LeaveDashboardResponse.LeaveBalanceDto sick = buildBalanceDto(entitlements, "SICK");
65         LeaveDashboardResponse.LeaveBalanceDto casual = buildBalanceDto(entitlements, "CASUAL");
66         LeaveDashboardResponse.LeaveBalanceDto maternity = buildBalanceDto(entitlements, "MATERNITY");
67
68         Map<String, LeaveDashboardResponse.LeaveBalanceDto> leaveBalances = buildBalanceMap(entitlements);
69         BigDecimal carryTotal = entitlements.stream()
70             .filter(e -> isSupportedLeaveType(e.getLeaveType()))
71             .filter(e -> e.getLeaveType().isCarriesForward())

```

```

72         .map(LeaveEntitlement::getAvailableBalance)
73         .reduce(BigDecimal.ZERO, BigDecimal::add);
74
75     LeaveDashboardResponse response = new LeaveDashboardResponse();
76     response.setEmployeeId(employee.getEmployeeId());
77     response.setEmployeeName(employee.getName());
78     response.setYear(year);
79     response.setSick(sick);
80     response.setCasual(casual);
81     response.setMaternity(maternity);
82     response.setLeaveBalances(leaveBalances);
83     response.setCarryForwardMeter(new LeaveDashboardResponse.CarryForwardMeterDto(carryTotal, 30));
84     response.setWarnings(warnings.stream()
85         .map(w -> new LeaveDashboardResponse.WarningDto(
86             w.getId(),
87             w.getWarningType().name(),
88             w.getMessage(),
89             w.getWarningDate().toString()))
90         .toList());
91
92     return response;
93 }
94
95 // Mark warning read
96
97 @Transactional
98 public void markWarningRead(Long warningId, Long callerEmployeeId, String callerRole) {
99     LeaveWarning warning = warningRepository.findById(warningId)
100         .orElseThrow(() -> new IllegalArgumentException("Warning not found: " + warningId));
101
102     assertAccess(warning.getEmployee().getEmployeeId(), callerEmployeeId, callerRole);
103
104     warning.setRead(true);
105     warningRepository.save(warning);
106 }
107
108 // Team dashboard (manager/admin)
109
110 @Transactional
111 public List<LeaveDashboardResponse> getTeamDashboard(Long managerId, String callerRole) {
112     if (!"MANAGER".equals(callerRole) && !"ADMIN".equals(callerRole)) {
113         throw new AccessDeniedException("Only managers and admins can view team dashboards");
114     }
115
116     List<Employee> team = employeeRepository.findByManagerEmployeeId(managerId);
117     return team.stream()
118         .map(emp -> getDashboard(emp.getEmployeeId(), managerId, callerRole))
119         .toList();
120 }
121
122 // Helpers
123
124 private void ensureCurrentYearEntitlements(Employee employee, int year) {
125     List<LeaveType> leaveTypes = leaveTypeRepository.findAll();
126
127     for (LeaveType leaveType : leaveTypes) {
128         if (!isSupportedLeaveType(leaveType)) {
129             continue;
130         }
131         if (!isEligible(employee, leaveType)) {
132             continue;
133         }
134
135         leaveEntitlementRepository
136             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
137                 employee.getEmployeeId(), leaveType.getId(), year)
138             .orElseGet(() -> createEntitlement(employee, leaveType, year));
139     }
140 }
141
142 private boolean isEligible(Employee employee, LeaveType leaveType) {
143     if (!leaveType.isGenderRestricted()) {
144         return true;
145     }
146
147     Gender restriction = leaveType.getGenderRestriction();

```

```

148         return restriction == null || restriction == employee.getGender();
149     }
150
151     private LeaveEntitlement createEntitlement(Employee employee, LeaveType leaveType, int year) {
152         LeaveEntitlement entitlement = new LeaveEntitlement();
153         entitlement.setEmployee(employee);
154         entitlement.setLeaveType(leaveType);
155         entitlement.setYear(year);
156
157         if (leaveType.isCarriesForward()) {
158             BigDecimal lastYearClosing = leaveEntitlementRepository
159                 .findByEmployeeIdAndLeaveTypeIdAndYear(
160                     employee.getId(), leaveType.getId(), year - 1)
161                 .map(LeaveEntitlement::getClosingBalance)
162                 .orElse(BigDecimal.ZERO);
163             entitlement.setOpeningBalance(lastYearClosing);
164         }
165
166         return leaveEntitlementRepository.save(entitlement);
167     }
168
169     private Map<String, LeaveDashboardResponse.LeaveBalanceDto> buildBalanceMap(
170         List<LeaveEntitlement> entitlements) {
171         Map<String, LeaveDashboardResponse.LeaveBalanceDto> balances = new LinkedHashMap<>();
172         for (LeaveEntitlement entitlement : entitlements) {
173             String typeName = entitlement.getLeaveType().getName();
174             if (!SUPPORTED_LEAVE_TYPES.contains(typeName)) {
175                 continue;
176             }
177             balances.put(typeName, toBalanceDto(entitlement, typeName));
178         }
179         return balances;
180     }
181
182     private LeaveDashboardResponse.LeaveBalanceDto buildBalanceDto(List<LeaveEntitlement> entitlements,
183                                                                     String typeName) {
184         return entitlements.stream()
185             .filter(e -> typeName.equalsIgnoreCase(e.getLeaveType().getName()))
186             .findFirst()
187             .map(e -> toBalanceDto(e, typeName))
188             .orElse(null); // null = leave type not applicable for this employee
189     }
190
191     private LeaveDashboardResponse.LeaveBalanceDto toBalanceDto(LeaveEntitlement entitlement,
192                                                                 String typeName) {
193         return new LeaveDashboardResponse.LeaveBalanceDto(
194             typeName,
195             entitlement.getOpeningBalance(),
196             entitlement.getAccruedThisYear(),
197             entitlement.getUsedThisYear(),
198             entitlement.getAvailableBalance(),
199             entitlement.getLeaveType().isCarriesForward());
200     }
201
202     private boolean isSupportedLeaveType(LeaveType leaveType) {
203         return leaveType != null && SUPPORTED_LEAVE_TYPES.contains(leaveType.getName());
204     }
205
206     private void assertAccess(Long targetId, Long callerId, String role) {
207         boolean isSelf = targetId.equals(callerId);
208         boolean isManager = "MANAGER".equals(role) || "ADMIN".equals(role);
209         if (!isSelf && !isManager) {
210             throw new AccessDeniedException(
211                 "Employee " + callerId + " cannot view dashboard for " + targetId);
212         }
213     }
214 }
215

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.ActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.AuthRequest;
5 import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
6 import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
7 import com.example.EmployeeManagementSystem.Entity.Employee;
8 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
9 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
10 import com.example.EmployeeManagementSystem.Enum.LeaveType;
11 import com.example.EmployeeManagementSystem.Enum.Role;
12 import com.example.EmployeeManagementSystem.Enum.Status;
13 import com.example.EmployeeManagementSystem.Exception.*;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
16 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
17 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
18 import com.example.EmployeeManagementSystem.Util.AuthUtil;
19 import org.slf4j.Logger;
20 import org.slf4j.LoggerFactory;
21 import org.springframework.http.HttpStatus;
22 import org.springframework.http.ResponseEntity;
23 import org.springframework.security.core.Authentication;
24 import org.springframework.stereotype.Service;
25 import org.springframework.transaction.annotation.Transactional;
26
27 import java.math.BigDecimal;
28 import java.time.LocalDate;
29 import java.time.ZoneId;
30 import java.time.temporal.ChronoUnit;
31 import java.util.ArrayList;
32 import java.util.List;
33 import java.util.Map;
34 import java.util.stream.Collectors;
35
36 @Service
37 public class LeaveRequestService {
38     private final LeaveRequestRepo leaveRequestRepo;
39     private final EmployeeRepo employeeRepo;
40     private final LeaveTypeRepository leaveTypeRepository;
41     private final LeaveEntitlementRepository leaveEntitlementRepository;
42     private final NotificationService notificationService;
43     private static final Logger log = LoggerFactory.getLogger(LeaveRequestService.class);
44
45     public LeaveRequestService(LeaveRequestRepo leaveRequestRepo,
46                               EmployeeRepo employeeRepo,
47                               LeaveTypeRepository leaveTypeRepository,
48                               LeaveEntitlementRepository leaveEntitlementRepository,
49                               NotificationService notificationService) {
50         this.leaveRequestRepo = leaveRequestRepo;
51         this.employeeRepo = employeeRepo;
52         this.leaveTypeRepository = leaveTypeRepository;
53         this.leaveEntitlementRepository = leaveEntitlementRepository;
54         this.notificationService=notificationService;
55     }
56
57     public List<LeaveResponseDTO> getAllTheLeaveRequest(){
58         List<LeaveRequest> requestList=leaveRequestRepo.findAll();
59         List<LeaveResponseDTO> dtos=new ArrayList<>();
60         for(LeaveRequest request:requestList){
61             dtos.add(convertToDTO(request));
62         }
63         return dtos;
64     }
65
66     public ResponseEntity<?> createRequest(Authentication authentication, LeaveRequestDTO requestDTO){
67         var leaveRequest=new LeaveRequest();
68         String email= AuthUtil.extractEmail(authentication);
69         //Validate employee exists and is active
70         var employee=employeeRepo.findByEmail(email).orElseThrow(()->

```

```

72         new EmployeeNotFound("Employee with name: "+email+" is not found")
73     );
74     if (employee.getStatus() != Status.ACTIVE) {
75         return ResponseEntity.badRequest()
76             .body(Map.of("error", "Only active employees can apply for leave"));
77     }
78
79     leaveRequest.setEmployee(employee);
80     //Checking if the start date is before today
81     String timezone= employee.getTimezone();
82     ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
83         ? timezone
84         : "UTC");
85     LocalDate today = LocalDate.now(zoneId);
86     if (requestDTO.getStartDate().isBefore(today)) {
87         throw new InvalidStartDateException("Start date cannot be before current date");
88     }
89     //Checking if the end date is before start date
90     if (requestDTO.getEndDate().isBefore(requestDTO.getStartDate())) {
91         throw new InvalidEndDateException("End date must be equal to or greater than start date");
92     }
93     //optional
94     long daysRequested = ChronoUnit.DAYS.between(requestDTO.getStartDate(), requestDTO.getEndDate()) +
95     1;
96     if (daysRequested > 30) {
97         return ResponseEntity.badRequest()
98             .body(Map.of("error", "Leave cannot exceed 30 consecutive days"));
99     }
100    //optional
101    if (requestDTO.getLeaveType() == LeaveType.SICK) {
102        // Only 1 sick day allowed per calendar month
103        long sickDaysUsedThisMonth = leaveRequestRepo.countApprovedSickDaysInMonth(
104            employee, today.getYear(), today.getMonthValue());
105        if (sickDaysUsedThisMonth >= 1) {
106            return ResponseEntity.badRequest()
107                .body(Map.of(
108                    "error", "You have already used your sick leave for this month. Please
109                    apply for Unpaid leave instead."
110                ));
111        }
112        if (daysRequested > 1) {
113            return ResponseEntity.badRequest()
114                .body(Map.of("error", "Sick leave is limited to 1 day per month."));
115        }
116    }
117    long duplicateCount= leaveRequestRepo.checkDuplicate(
118        employee.getEmployeeId(),
119        requestDTO.getStartDate(),
120        requestDTO.getEndDate(),
121        LeaveStatus.PENDING.name()
122    );
123    if(duplicateCount>0){
124        throw new DuplicateRequestException("Duplicate leave request");
125    }
126
127    //checking if there is any overlapping leave request of that employee
128    long count= leaveRequestRepo.countOverlappingLeave(
129        employee.getEmployeeId(),
130        requestDTO.getStartDate(),
131        requestDTO.getEndDate()
132    );
133    if(count>0){
134        throw new OverlappingLeaveException("Overlapping leave exists");
135    }
136    leaveRequest.setStartDate(requestDTO.getStartDate());
137    leaveRequest.setEndDate(requestDTO.getEndDate());
138
139    if(requestDTO.getLeaveType()==null){
140        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
141            .body("Leave type is required..");
142    }
143    leaveRequest.setLeaveType(requestDTO.getLeaveType());
144    if(requestDTO.getReason()!=null){
145        leaveRequest.setReason(requestDTO.getReason());

```

```

146     }
147     LeaveRequest savedRequest=leaveRequestRepo.save(leaveRequest);
148
149     Employee manager = employee.getManager(); // assuming getManager() exists
150     if (manager != null) {
151         notificationService.notify(
152             manager,
153             employee.getName() + " has submitted a leave request.",
154             "LEAVE_REQUEST"
155         );
156     }
157     // After the existing manager notification block
158     employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
159         notificationService.notify(
160             admin,
161             employee.getName() + " submitted a " + requestDTO.getLeaveType() + " leave request
162 (" +
163             requestDTO.getStartDate() + " " + requestDTO.getEndDate() + ")",
164             "LEAVE_REQUEST"
165         )
166     );
167     return ResponseEntity.status(HttpStatus.CREATED).body(convertToDTO(savedRequest));
168 }
169 //Only for manager
170 public List<LeaveResponseDTO> getAllThePendingLeaveRequests(){
171     List<LeaveRequest> requestList=leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
172     List<LeaveResponseDTO> responseDTOS=new ArrayList<>();
173     for(LeaveRequest request:requestList){
174         responseDTOS.add(convertToDTO(request));
175     }
176     return responseDTOS;
177 }
178
179 @Transactional
180 public ResponseEntity<?> updateLeaveRequestStatus(ActionDTO actionDTO,Authentication authentication){
181     String email= AuthUtil.extractEmail(authentication);
182     Employee manager=employeeRepo.findByEmail(email).orElseThrow(
183         ()->new EmployeeNotFound("Manager with name: "+email+" Not Found")
184     );
185     if (actionDTO.getAction() == null) {
186         return ResponseEntity.badRequest().body("Action is required");
187     }
188     log.info("Updating leave request status for id: {}", actionDTO.getLeaveRequestId());
189
190     var leaveRequest=leaveRequestRepo.findById(actionDTO.getLeaveRequestId()).orElseThrow(
191         ()-> new LeaveRequestNotFoundException("LeaveRequest with
192 id:"+actionDTO.getLeaveRequestId()+" not found")
193     );
194     if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
195         return ResponseEntity.badRequest()
196             .body("Only PENDING requests can be approved or rejected. Current status: "
197                 + leaveRequest.getStatus());
198     }
199
200     if(actionDTO.getAction().equalsIgnoreCase("approved")){
201         leaveRequest.setStatus(LeaveStatus.APPROVED);
202         applyApprovedLeaveToEntitlement(leaveRequest);
203         notificationService.notify(
204             leaveRequest.getEmployee(),
205             "Your leave request from " + leaveRequest.getStartDate() + " to " +
206 leaveRequest.getEndDate() + " has been approved.",
207             "LEAVE_APPROVED"
208         );
209         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
210             notificationService.notify(
211                 admin,
212                 leaveRequest.getEmployee().getName() + "'s leave request has been approved by
213 " + manager.getName(),
214                 "LEAVE_APPROVED"
215             )
216         );
217     }
218     else if(actionDTO.getAction().equalsIgnoreCase("rejected")){
219         leaveRequest.setStatus(LeaveStatus.REJECTED);

```

```

218         notificationService.notify(
219             leaveRequest.getEmployee(),
220             "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " was rejected." +
221             (actionDTO.getRemarks() != null ? " Reason: " + actionDTO.getRemarks() : ""),
222             "LEAVE_REJECTED"
223         );
224         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
225             notificationService.notify(
226                 admin,
227                 leaveRequest.getEmployee().getName() + "'s leave request has been rejected by
" + manager.getName(),
228                 "LEAVE_REJECTED"
229             )
230         );
231     }
232     else{
233         return ResponseEntity.badRequest().body("Invalid action,Use APPROVED or REJECTED");
234     }
235     leaveRequest.setManager(manager.getEmail());
236     if(actionDTO.getRemarks()!=null){
237         leaveRequest.setRemarks(actionDTO.getRemarks());
238     }
239     return ResponseEntity.ok(leaveRequestRepo.save(leaveRequest));
240 }
241
242 @Transactional
243 public ResponseEntity<?> cancelLeaveRequest(Authentication authentication, long leaveId){
244     String email= AuthUtil.extractEmail(authentication);
245     var employee=employeeRepo.findByEmail(email).orElseThrow(
246         ()->new EmployeeNotFound("Employee with email:"+email+" not found")
247     );
248     var leaveRequest=leaveRequestRepo.findById(leaveId).orElseThrow(
249         ()->new LeaveRequestNotFoundException("Leave request with id:"+leaveId+" is not found")
250     );
251     if(employee.getEmployeeId()!=leaveRequest.getEmployee().getEmployeeId()){
252         return ResponseEntity.badRequest().body("You cannot cancel others Leave request..");
253     }
254     String timezone= employee.getTimezone();
255     ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
256         ? timezone
257         : "UTC");
258     LocalDate today = LocalDate.now(zoneId);
259     if(leaveRequest.getStartDate().isEqual(today)||leaveRequest.getStartDate().isBefore(today)){
260         return ResponseEntity.badRequest().body("You cannot cancel leave request after the leave has
started");
261     }
262     LeaveStatus previousStatus = leaveRequest.getStatus();
263     leaveRequest.setStatus(LeaveStatus.CANCELLED);
264     if (previousStatus == LeaveStatus.APPROVED) {
265         reverseApprovedLeaveFromEntitlement(leaveRequest);
266     }
267     leaveRequestRepo.save(leaveRequest);
268     return ResponseEntity.ok("Leave Request with id:"+ leaveId+" Successfully cancelled");
269 }
270
271 private LeaveResponseDTO convertToDTO(LeaveRequest request){
272     var responseDTO=new LeaveResponseDTO();
273     responseDTO.setLeaveRequestId(request.getId());
274     responseDTO.setEmployeeId(request.getEmployee().getEmployeeId());
275     responseDTO.setLeaveType(request.getLeaveType());
276     responseDTO.setStartDate(request.getStartDate());
277     responseDTO.setEndDate(request.getEndDate());
278     responseDTO.setNumberOfDays(requestedDays(request).longValue());
279     responseDTO.setReason(request.getReason());
280     responseDTO.setStatus(request.getStatus());
281     return responseDTO;
282 }
283
284 public ResponseEntity<?> getLeaveRequestsByEmployee(Authentication authentication) {
285     String email= AuthUtil.extractEmail(authentication);
286     log.info("Fetching leave requests for employee: {}", email);
287
288     Employee employee = employeeRepo.findByEmail(email)
289         .orElseThrow(() -> new EmployeeNotFound("Employee not found: " +email ));
290

```

```

291     List<LeaveRequest> requests = leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
292     List<LeaveResponseDTO> response = requests.stream()
293         .map(this::convertToDTO)
294         .collect(Collectors.toList());
295
296     return ResponseEntity.ok(Map.of(
297         "employeeName", email,
298         "totalRequests", response.size(),
299         "requests", response
300     ));
301 }
302 public List<LeaveResponseDTO> getPendingLeaves() {
303     List<LeaveRequest> pendingRequests = leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
304     return pendingRequests.stream()
305         .map(this::convertToDTO)
306         .collect(Collectors.toList());
307 }
308
309 /**
310  * Approve a leave request by ID (simplified version for AdminController).
311  * Returns LeaveResponseDTO directly.
312  */
313 @Transactional
314 public LeaveResponseDTO approveLeave(Long id) {
315     LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
316         .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not
found"));
317
318     if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
319         throw new IllegalStateException("Only PENDING requests can be approved. Current status: " +
leaveRequest.getStatus());
320     }
321
322     leaveRequest.setStatus(LeaveStatus.APPROVED);
323     leaveRequest.setManager("ADMIN"); // Or get from security context if needed
324     applyApprovedLeaveToEntitlement(leaveRequest);
325     notificationService.notify(
326         leaveRequest.getEmployee(),
327         "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " has been approved by admin.",
328         "LEAVE_APPROVED"
329     );
330
331     LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
332     return convertToDTO(saved);
333 }
334
335 /**
336  * Reject a leave request by ID (simplified version for AdminController).
337  * Returns LeaveResponseDTO directly.
338  */
339 @Transactional
340 public LeaveResponseDTO rejectLeave(Long id) {
341     LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
342         .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not
found"));
343
344     if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
345         throw new IllegalStateException("Only PENDING requests can be rejected. Current status: " +
leaveRequest.getStatus());
346     }
347
348     leaveRequest.setStatus(LeaveStatus.REJECTED);
349     leaveRequest.setManager("ADMIN");
350     notificationService.notify(
351         leaveRequest.getEmployee(),
352         "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " was rejected by admin.",
353         "LEAVE_REJECTED"
354     );
355
356     LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
357     return convertToDTO(saved);
358 }
359
360 private void applyApprovedLeaveToEntitlement(LeaveRequest leaveRequest) {

```



```

361         updateUsedLeave(leaveRequest, requestedDays(leaveRequest));
362     }
363
364     private void reverseApprovedLeaveFromEntitlement(LeaveRequest leaveRequest) {
365         updateUsedLeave(leaveRequest, requestedDays(leaveRequest).negate());
366     }
367
368     private void updateUsedLeave(LeaveRequest leaveRequest, BigDecimal delta) {
369         int year = leaveRequest.getStartDate().getYear();
370         String leaveTypeName = leaveRequest.getLeaveType().name();
371         com.example.EmployeeManagementSystem.Entity.LeaveType leaveType = leaveTypeRepository
372             .findByName(leaveTypeName)
373             .orElseThrow(() -> new IllegalStateException(
374                 "Leave type not configured: " + leaveTypeName));
375
376         var entitlement = leaveEntitlementRepository
377             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
378                 leaveRequest.getEmployee().getEmployeeId(), leaveType.getId(), year)
379             .orElseGet(() -> createEntitlement(leaveRequest.getEmployee(), leaveType, year));
380
381         BigDecimal used = entitlement.getUsedThisYear().add(delta);
382         entitlement.setUsedThisYear(used.max(BigDecimal.ZERO));
383         leaveEntitlementRepository.save(entitlement);
384     }
385
386     private com.example.EmployeeManagementSystem.Entity.LeaveEntitlement createEntitlement(
387         Employee employee,
388         com.example.EmployeeManagementSystem.Entity.LeaveType leaveType,
389         int year) {
390         var entitlement = new com.example.EmployeeManagementSystem.Entity.LeaveEntitlement();
391         entitlement.setEmployee(employee);
392         entitlement.setLeaveType(leaveType);
393         entitlement.setYear(year);
394
395         if (leaveType.isCarriesForward()) {
396             BigDecimal lastYearClosing = leaveEntitlementRepository
397                 .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
398                     employee.getEmployeeId(), leaveType.getId(), year - 1)
399                 .map(com.example.EmployeeManagementSystem.Entity.LeaveEntitlement::getClosingBalance)
400                 .orElse(BigDecimal.ZERO);
401             entitlement.setOpeningBalance(lastYearClosing);
402         }
403
404         return leaveEntitlementRepository.save(entitlement);
405     }
406
407     private BigDecimal requestedDays(LeaveRequest leaveRequest) {
408         long days = ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1;
409         return BigDecimal.valueOf(days);
410     }
411 }
412

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.Device;
5 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
6 import com.example.EmployeeManagementSystem.Entity.Employee;
7 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
8 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
9 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
10 import com.example.EmployeeManagementSystem.Enum.Role;
11 import com.example.EmployeeManagementSystem.Enum.Status;
12 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
13 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
16 import com.example.EmployeeManagementSystem.Util.AuthUtil;
17 import org.springframework.http.ResponseEntity;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.security.crypto.password.PasswordEncoder;
20 import org.springframework.stereotype.Service;
21
22 import java.time.temporal.ChronoUnit;
23 import java.util.List;
24 import java.util.stream.Collectors;
25
26 @Service
27 public class ManagerService {
28     private final EmployeeRepo employeeRepo;
29     private final PasswordEncoder passwordEncoder;
30     private final LeaveRequestRepo leaveRequestRepo;
31     private final DeviceAssignmentRepo deviceAssignmentRepo;
32
33     public ManagerService(EmployeeRepo employeeRepo,
34                           PasswordEncoder passwordEncoder,
35                           LeaveRequestRepo leaveRequestRepo,
36                           DeviceAssignmentRepo deviceAssignmentRepo) {
37         this.employeeRepo = employeeRepo;
38         this.passwordEncoder = passwordEncoder;
39         this.leaveRequestRepo = leaveRequestRepo;
40         this.deviceAssignmentRepo = deviceAssignmentRepo;
41     }
42
43     public Employee createEmp(EmployeeDetails employeeDetails, Authentication authentication) {
44         String email= AuthUtil.extractEmail(authentication);
45         Employee manager= employeeRepo.findByEmail(email).orElseThrow(
46             ()->new EmployeeNotFound("Manager :"+email+"Not found")
47         );
48         Employee employee=new Employee();
49         employee.setName(employeeDetails.getName());
50         employee.setEmail(employeeDetails.getEmail());
51         employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
52         employee.setGender(employeeDetails.getGender());
53         employee.setDept(manager.getDept());
54         employee.setRole(Role.EMPLOYEE);
55         employee.setManager(manager);
56         employee.setTimezone(employeeDetails.getTimezone());
57         return employeeRepo.save(employee);
58     }
59
60
61     public List<Employee> getAllEmployeeByManager(Authentication authentication) {
62         String email=AuthUtil.extractEmail(authentication);
63         Employee manager=employeeRepo.findByEmail(email).orElseThrow(
64             ()->new EmployeeNotFound("Manger:"+email+" not found")
65         );
66         return employeeRepo.findByManager(manager);
67     }
68
69     public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication) {
70         String email=AuthUtil.extractEmail(authentication);
71         Employee manager=employeeRepo.findByEmail(email).orElseThrow(

```

```

72         ()->new EmployeeNotFound("Manager with "+email+" not found")
73     };
74     List<LeaveRequest> requests=leaveRequestRepo.findByEmployee_Manager(manager);
75     return requests.stream().map(request->toManagerLeaveResponseDTO(request)).toList();
76 }
77
78 public List<EmployeeDTO> getUsers(){
79     List<Employee> employeeList=employeeRepo.findByRole(Role.USER);
80     return employeeList.stream().map(this::toDTO).toList();
81 }
82
83 public ResponseEntity<?> promoteUser(Authentication authentication, PromoteRequest promoteRequest){
84     Employee user=employeeRepo.findByEmail(promoteRequest.getEmail()).orElseThrow(
85         ()->new EmployeeNotFound("Employee:"+promoteRequest.getEmail()+" not found")
86     );
87     if(user.getRole()!=Role.USER){
88         return ResponseEntity.ok("Employee already promoted");
89     }
90     String managerEmail=AuthUtil.extractEmail(authentication);
91     Employee manager=employeeRepo.findByEmail(managerEmail).orElseThrow(
92         ()->new EmployeeNotFound("Manager:"+managerEmail+" not found")
93     );
94
95     user.setRole(Role.EMPLOYEE);
96     user.setManager(manager);
97     user.setDept(manager.getDept());
98     user.setPassword(passwordEncoder.encode(promoteRequest.getPassword()));
99     Employee promotedEmployee=employeeRepo.save(user);
100    return ResponseEntity.ok(toDTO(promotedEmployee));
101 }
102
103 public EmployeeDTO toDTO(Employee employee){
104     EmployeeDTO employeeDTO=new EmployeeDTO();
105     employeeDTO.setName(employee.getName());
106     employeeDTO.setEmail(employee.getEmail());
107     employeeDTO.setDept(employee.getDept());
108     employeeDTO.setPassword(employee.getPassword());
109     employeeDTO.setTimezone(employee.getTimezone());
110     return employeeDTO;
111 }
112
113 public Employee createAdmin(EmployeeDetails employeeDetails) {
114     Employee employee=new Employee();
115     employee.setName(employeeDetails.getName());
116     employee.setEmail(employeeDetails.getEmail());
117     employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
118     employee.setDept("Main");
119     employee.setRole(Role.ADMIN);
120     employee.setTimezone(employeeDetails.getTimezone());
121     return employeeRepo.save(employee);
122 }
123
124 public List<LeaveResponseDTO> getAllThePendingLeaveRequests(Authentication authentication) {
125     String email=AuthUtil.extractEmail(authentication);
126     Employee manager=employeeRepo.findByEmail(email).orElseThrow(
127         ()->new EmployeeNotFound("Manager with "+email+" not found")
128     );
129     List<LeaveRequest>
leaveRequests=leaveRequestRepo.findByStatusAndEmployee_Manager(LeaveStatus.PENDING,manager);
130     return leaveRequests.stream()
131         .map(this::convertToLeaveResponseDTO)
132         .collect(Collectors.toList());
133 }
134
135 public ManagerEmployeeDetailsDTO getEmployeeDetailsByName( String name) {
136
137     Employee employee = employeeRepo.findByName(name).orElseThrow(
138         () -> new EmployeeNotFound("Employee:" + name + " not found")
139     );
140
141
142
143     ManagerEmployeeDetailsDTO dto = new ManagerEmployeeDetailsDTO();
144     dto.setEmployeeId(employee.getEmployeeId());
145     dto.setName(employee.getName());
146     dto.setEmail(employee.getEmail());

```

```

147         dto.setDept(employee.getDept());
148         dto.setStatus(employee.getStatus());
149         dto.setAttendance(toAttendance(employee.getStatus()));
150         dto.setDevices(deviceAssignmentRepo
151             .findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(), AssignmentStatus.ACTIVE)
152             .stream()
153             .map(DeviceAssignment::getDevice)
154             .map(this::toDeviceResponseDTO)
155             .toList());
156         return dto;
157     }
158
159     private LeaveResponseDTO convertToLeaveResponseDTO(LeaveRequest leaveRequest) {
160         LeaveResponseDTO dto = new LeaveResponseDTO();
161         dto.setLeaveRequestId(leaveRequest.getId());
162         dto.setEmployeeId(leaveRequest.getEmployee().getEmployeeId());
163         dto.setLeaveType(leaveRequest.getLeaveType());
164         dto.setStartDate(leaveRequest.getStartDate());
165         dto.setEndDate(leaveRequest.getEndDate());
166         dto.setReason(leaveRequest.getReason());
167         dto.setStatus(leaveRequest.getStatus());
168         return dto;
169     }
170
171     private ManagerLeaveResponseDTO toManagerLeaveResponseDTO(LeaveRequest leaveRequest){
172         ManagerLeaveResponseDTO managerLeaveResponseDTO=new ManagerLeaveResponseDTO();
173         managerLeaveResponseDTO.setEmployeeName(leaveRequest.getEmployee().getName());
174         managerLeaveResponseDTO.setLeaveType(leaveRequest.getLeaveType());
175         managerLeaveResponseDTO.setStartDate(leaveRequest.getStartDate());
176         managerLeaveResponseDTO.setEndDate(leaveRequest.getEndDate());
177         managerLeaveResponseDTO.setNumberOfDays(
178             ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1);
179         managerLeaveResponseDTO.setStatus(leaveRequest.getStatus());
180         managerLeaveResponseDTO.setReason(leaveRequest.getReason());
181         return managerLeaveResponseDTO;
182     }
183
184     private String toAttendance(Status status) {
185         if (status == Status.ON_LEAVE) {
186             return "ON_LEAVE";
187         }
188         if (status == Status.ACTIVE) {
189             return "PRESENT";
190         }
191         return status.name();
192     }
193
194     private DeviceResponseDTO toDeviceResponseDTO(Device device) {
195         DeviceResponseDTO dto = new DeviceResponseDTO();
196         dto.setId(device.getId());
197         dto.setDeviceName(device.getDeviceName());
198         dto.setBrand(device.getBrand());
199         dto.setDeviceType(device.getDeviceType());
200         dto.setDeviceStatus(device.getDeviceStatus());
201         dto.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
202
203         if (device.getTechVendor() != null) {
204             dto.setVendorName(device.getTechVendor().getName());
205         }
206
207         DeviceAssignment assignment = device.getCurrentAssignment();
208         if (assignment != null && assignment.getStatus() == AssignmentStatus.ACTIVE) {
209             dto.setAssignedDate(assignment.getAssignedDate());
210             if (assignment.getAssignedTo() != null) {
211                 dto.setAssignedEmployeeId(assignment.getAssignedTo().getEmployeeId());
212                 dto.setAssignedEmployeeName(assignment.getAssignedTo().getName());
213             }
214         }
215
216         return dto;
217     }
218 }
219

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import org.springframework.stereotype.Service;
5
6 import java.time.LocalDateTime;
7 import java.util.Map;
8
9 @Service
10 public class MealSlotService {
11
12     private final Map<MealSlot, LocalDateTime> slotLocalTimeMap = Map.of(
13         MealSlot.BREAKFAST, LocalDateTime.of(8, 0),
14         MealSlot.LUNCH, LocalDateTime.of(13, 0),
15         MealSlot.DINNER, LocalDateTime.of(21, 0)
16     );
17
18     public LocalDateTime getTime(MealSlot slot) {
19         return slotLocalTimeMap.get(slot);
20     }
21 }
22
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Notification;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.NotificationRepository;
7 import org.springframework.stereotype.Service;
8 import java.util.List;
9
10 @Service
11 public class NotificationService {
12
13     private final NotificationRepository notificationRepo;
14     private final EmployeeRepo employeeRepo;
15
16     public NotificationService(NotificationRepository notificationRepo,
17                               EmployeeRepo employeeRepo) {
18         this.notificationRepo = notificationRepo;
19         this.employeeRepo = employeeRepo;
20     }
21
22     /** Create a notification for a specific employee */
23     public void notify(Employee recipient, String message, String type) {
24         Notification n = new Notification();
25         n.setRecipient(recipient);
26         n.setMessage(message);
27         n.setType(type);
28         notificationRepo.save(n);
29     }
30
31     /** Notify every employee in the system (e.g. new public holiday) */
32     public void notifyAll(String message, String type) {
33         List<Employee> all = employeeRepo.findAll();
34         for (Employee e : all) {
35             notify(e, message, type);
36         }
37     }
38
39     public List<Notification> getForEmployee(Employee employee) {
40         return notificationRepo.findByRecipientOrderByCreatedAtDesc(employee);
41     }
42
43     public long getUnreadCount(Employee employee) {
44         return notificationRepo.countByRecipientAndIsReadFalse(employee);
45     }
46
47     public void markAllRead(Employee employee) {
48         List<Notification> unread = notificationRepo
49             .findByRecipientOrderByCreatedAtDesc(employee)
50             .stream()
51             .filter(n -> !n.isRead())
52             .toList();
53         unread.forEach(n -> n.setRead(true));
54         notificationRepo.saveAll(unread);
55     }
56 }
57
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
8 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
9 import org.springframework.stereotype.Service;
10
11 import java.time.Instant;
12 import java.time.temporal.ChronoUnit;
13 import java.util.UUID;
14
15 @Service
16 public class RefreshTokenService {
17     private final EmployeeRepo employeeRepo;
18     private final VendorRepo vendorRepo;
19
20     public RefreshTokenService(RefreshTokenRepository refreshTokenRepository, EmployeeRepo employeeRepo,
21     VendorRepo vendorRepo) {
22         this.employeeRepo = employeeRepo;
23         this.vendorRepo = vendorRepo;
24     }
25
26     public RefreshToken createForEmployee(Employee employee) {
27         RefreshToken refreshToken = new RefreshToken();
28         refreshToken.setToken(UUID.randomUUID().toString());
29         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
30         refreshToken.setEmployee(employee);
31         return refreshToken;
32     }
33
34     public RefreshToken createForVendor(Vendor vendor) {
35         RefreshToken refreshToken = new RefreshToken();
36         refreshToken.setToken(UUID.randomUUID().toString());
37         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
38         refreshToken.setVendor(vendor);
39         return refreshToken;
40     }
41 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
4 import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
5 import com.example.EmployeeManagementSystem.Entity.Restaurant;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.MealsSlot;
8 import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
9 import com.example.EmployeeManagementSystem.Exception.UnauthorizedAccessException;
10 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
11 import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
12 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.slf4j.Logger;
15 import org.slf4j.LoggerFactory;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.stereotype.Service;
18
19 import java.util.List;
20 import java.util.stream.Collectors;
21
22 @Service
23 public class RestaurantService {
24
25     private static final Logger log = LoggerFactory.getLogger(RestaurantService.class);
26
27     private final RestaurantRepository restaurantRepository;
28     private final VendorRepo vendorRepo;
29
30     public RestaurantService(RestaurantRepository restaurantRepository, VendorRepo vendorRepo) {
31         this.restaurantRepository = restaurantRepository;
32         this.vendorRepo = vendorRepo;
33     }
34
35     /**
36      * Create a new restaurant.
37      * Only the owning vendor (matched by vendorId in the request) may do this.
38      */
39     public RestaurantDTO createRestaurant(RestaurantRequest request, Authentication authentication) {
40         String email = AuthUtil.extractEmail(authentication);
41         Vendor vendor = vendorRepo.findByEmail(email)
42             .orElseThrow(() -> new VendorNotFoundException(
43                 "Vendor with email " + email + " not found"));
44
45         Restaurant restaurant = new Restaurant();
46         restaurant.setName(request.getName());
47         restaurant.setAddress(request.getAddress());
48         restaurant.setDescription(request.getDescription());
49         restaurant.setSupportedMealsSlots(request.getSupportedMealsSlots());
50         restaurant.setVendor(vendor);
51         restaurant.setActive(true);
52
53         Restaurant saved = restaurantRepository.save(restaurant);
54         log.info("Restaurant created: {} by vendor {}", saved.getName(), vendor.getId());
55         return toDTO(saved);
56     }
57
58     /**
59      * Update restaurant details.
60      * Vendor must own this restaurant – otherwise 403.
61      */
62     public RestaurantDTO updateRestaurant(Long restaurantId, RestaurantRequest request, Authentication authentication) {
63         Restaurant restaurant = findOrThrow(restaurantId);
64         String email = AuthUtil.extractEmail(authentication);
65         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
66             () -> new VendorNotFoundException("Vendor Not found")
67         );
68         assertOwnership(restaurantId, vendor.getEmail());
69
70         if (request.getName() != null) restaurant.setName(request.getName());

```



```

71     if (request.getAddress() != null) restaurant.setAddress(request.getAddress());
72     if (request.getDescription() != null) restaurant.setDescription(request.getDescription());
73     if (request.getSupportedMealSlots() != null && !request.getSupportedMealSlots().isEmpty()) {
74         restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
75     }
76     return toDTO(restaurantRepository.save(restaurant));
77 }
78
79 /**
80  * Deactivate a restaurant (soft delete).
81  * Only the owning vendor may do this.
82  * Active subscriptions to this restaurant will no longer receive new deliveries
83  * – the Subscription status should be handled separately.
84  */
85 public void deactivateRestaurant(Long restaurantId, Authentication authentication) {
86     Restaurant restaurant = findOrThrow(restaurantId);
87     String email= AuthUtil.extractEmail(authentication);
88     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
89         ()->new VendorNotFoundException("Vendor Not found")
90     );
91     assertOwnership(restaurantId, vendor.getEmail());
92     restaurant.setActive(false);
93     restaurantRepository.save(restaurant);
94     log.info("Restaurant deactivated: id={}", restaurantId);
95 }
96
97 /**
98  * Get all restaurants owned by a vendor.
99  * VENDOR-only endpoint.
100  */
101 public List<RestaurantDTO> getRestaurantsByVendor(Authentication authentication) {
102     String email= AuthUtil.extractEmail(authentication);
103     vendorRepo.findByEmail(email)
104         .orElseThrow(() -> new VendorNotFoundException("Vendor " + email + " not found"));
105     return restaurantRepository.findByVendor_EmailAndActiveTrue(email)
106         .stream().map(this::toDTO).collect(Collectors.toList());
107 }
108
109 /**
110  * Get all active restaurants.
111  * Available to EMPLOYEE so they can browse and choose.
112  */
113 public List<RestaurantDTO> getAllActiveRestaurants() {
114     return restaurantRepository.findByActiveTrue()
115         .stream().map(this::toDTO).collect(Collectors.toList());
116 }
117
118 /**
119  * Get active restaurants that serve a specific meal slot.
120  * Employees call this when subscribing, to see valid restaurant options for a slot.
121  */
122 public List<RestaurantDTO> getRestaurantsByMealSlot(MealSlot mealSlot) {
123     return restaurantRepository.findActiveByMealSlot(mealSlot)
124         .stream().map(this::toDTO).collect(Collectors.toList());
125 }
126
127 public RestaurantDTO getRestaurant(Long id) {
128     return toDTO(findOrThrow(id));
129 }
130
131 // helpers
132
133 private Restaurant findOrThrow(Long id) {
134     return restaurantRepository.findById(id)
135         .orElseThrow(() -> new RestaurantNotFoundException(
136             "Restaurant with id " + id + " not found"));
137 }
138
139 /**
140  * Throws UnauthorizedAccessException if the restaurant does not belong to the vendor.
141  * This is the core access-control guard for all mutating vendor operations.
142  */
143 private void assertOwnership(Long restaurantId, String email) {
144     if (!restaurantRepository.existsByIdAndVendor_Email(restaurantId, email)) {
145         throw new UnauthorizedAccessException(
146             "Vendor " + email + " does not own restaurant " + restaurantId);

```

```
147     }
148 }
149
150 public RestaurantDTO toDTO(Restaurant r) {
151     RestaurantDTO dto = new RestaurantDTO();
152     dto.setId(r.getId());
153     dto.setName(r.getName());
154     dto.setAddress(r.getAddress());
155     dto.setDescription(r.getDescription());
156     dto.setSupportedMealSlots(r.getSupportedMealSlots());
157     dto.setActive(r.isActive());
158     dto.setVendorId(r.getVendor().getId());
159     dto.setVendorName(r.getVendor().getName());
160     return dto;
161 }
162 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.AdminActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.RepairDTO;
5 import com.example.EmployeeManagementSystem.DTO.ServiceRequestDTO;
6 import com.example.EmployeeManagementSystem.DTO.ServiceRequestResponseDTO;
7 import com.example.EmployeeManagementSystem.Entity.Device;
8 import com.example.EmployeeManagementSystem.Entity.Employee;
9 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
10 import com.example.EmployeeManagementSystem.Entity.Vendor;
11 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
12 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
13 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
14 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
15 import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
16 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
17 import com.example.EmployeeManagementSystem.Repository.ServiceRequestRepository;
18 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
19 import com.example.EmployeeManagementSystem.Util.AuthUtil;
20 import org.springframework.security.core.Authentication;
21 import org.springframework.stereotype.Service;
22 import org.springframework.transaction.annotation.Transactional;
23 import org.w3c.dom.stylesheets.LinkStyle;
24
25 import java.time.LocalDateTime;
26 import java.util.List;
27 import java.util.Vector;
28
29 @Service
30 public class ServiceRequestService {
31     private final ServiceRequestRepository serviceRequestRepository;
32     private final EmployeeRepo employeeRepo;
33     private final DeviceRepository deviceRepository;
34     private final VendorRepo vendorRepo;
35
36     public ServiceRequestService(ServiceRequestRepository serviceRequestRepository, EmployeeRepo employeeRepo, DeviceRepository deviceRepository, VendorRepo vendorRepo) {
37         this.serviceRequestRepository = serviceRequestRepository;
38         this.employeeRepo = employeeRepo;
39         this.deviceRepository = deviceRepository;
40         this.vendorRepo = vendorRepo;
41     }
42
43     public ServiceRequest createServiceRequest(ServiceRequestDTO dto, Authentication authentication) {
44         String email= AuthUtil.extractEmail(authentication);
45         Employee employee = employeeRepo.findByEmail(email)
46             .orElseThrow(() -> new RuntimeException("Employee not found"));
47
48         // 2. Fetch device
49         Device device = deviceRepository.findById(dto.getDeviceId())
50             .orElseThrow(() -> new RuntimeException("Device not found"));
51
52         // 3. Validate device is assigned to employee
53         if (device.getCurrentAssignment() == null ||
54             !device.getCurrentAssignment().getAssignedTo().equals(employee)) {
55             throw new RuntimeException("Device not assigned to this employee");
56         }
57
58         // 4. Validate device status
59         if (device.getDeviceStatus() != DeviceStatus.ASSIGNED) {
60             throw new RuntimeException("Device is not in usable state");
61         }
62
63         // 5. Create Service Request
64         ServiceRequest request = new ServiceRequest();
65         request.setDevice(device);
66         request.setRaisedBy(employee);
67         request.setRequestType(dto.getRequestType());
68         request.setIssueDescription(dto.getIssueDescription());
69         request.setUrgent(dto.isUrgent());
70         request.setStatus(ServiceRequestStatus.OPEN);

```

```

71         return serviceRequestRepository.save(request);
72     }
73
74
75     public List<ServiceRequestResponseDTO> getAllOpenServiceRequests(){
76         List<ServiceRequest>
serviceRequestList=serviceRequestRepository.findByStatus(ServiceRequestStatus.OPEN);
77         return serviceRequestList.stream().map(serviceRequest ->
toServiceRequestResponseDTO(serviceRequest)).toList();
78
79     }
80
81     public List<ServiceRequestResponseDTO> getMyServiceRequests(Authentication authentication) {
82         String email = AuthUtil.extractEmail(authentication);
83         Employee employee = employeeRepo.findByEmail(email)
84             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " + email));
85
86         List<ServiceRequest> serviceRequestList =
serviceRequestRepository.findByRaisedByOrderByRaisedAtDesc(employee);
87         return serviceRequestList.stream()
88             .map(this::toServiceRequestResponseDTO)
89             .toList();
90     }
91
92     private ServiceRequestDTO toServiceRequestDTO(ServiceRequest serviceRequest){
93         ServiceRequestDTO requestDTO=new ServiceRequestDTO();
94         requestDTO.setDeviceId(serviceRequest.getDevice().getId());
95         requestDTO.setRequestType(serviceRequest.getRequestType());
96         requestDTO.setIssueDescription(serviceRequest.getIssueDescription());
97         requestDTO.setUrgent(serviceRequest.isUrgent());
98         return requestDTO;
99     }
100
101     private ServiceRequestResponseDTO toServiceRequestResponseDTO(ServiceRequest serviceRequest) {
102         ServiceRequestResponseDTO responseDTO = new ServiceRequestResponseDTO();
103         responseDTO.setId(serviceRequest.getId());
104         responseDTO.setRequestType(serviceRequest.getRequestType() != null
105             ? serviceRequest.getRequestType().name()
106             : null);
107         responseDTO.setStatus(serviceRequest.getStatus() != null
108             ? serviceRequest.getStatus().name()
109             : null);
110         responseDTO.setIssueDescription(serviceRequest.getIssueDescription());
111         responseDTO.setAdminRemarks(serviceRequest.getAdminRemarks());
112         responseDTO.setResolution(serviceRequest.getResolution());
113         responseDTO.setUrgent(serviceRequest.isUrgent());
114         responseDTO.setRaisedAt(serviceRequest.getRaisedAt());
115         responseDTO.setResolvedAt(serviceRequest.getResolvedAt());
116         responseDTO.setDeviceName(serviceRequest.getDevice() != null
117             ? serviceRequest.getDevice().getDeviceName()
118             : null);
119         responseDTO.setReviewedByName(serviceRequest.getReviewedBy() != null
120             ? serviceRequest.getReviewedBy().getName()
121             : null);
122         responseDTO.setRaisedByName(serviceRequest.getRaisedBy().getName());
123         return responseDTO;
124     }
125
126
127     @Transactional
128     public ServiceRequestResponseDTO updateServiceRequestByAdmin(Authentication authentication,
AdminActionDTO actionDTO){
129         ServiceRequest
serviceRequest=serviceRequestRepository.findById(actionDTO.getServiceRequestId()).orElseThrow(
130             ()->new RuntimeException("Service Request Not Found")
131         );
132         String email=AuthUtil.extractEmail(authentication);
133         Employee admin=employeeRepo.findByEmail(email).orElseThrow(
134             ()->new EmployeeNotFound("Admin not found:"+email)
135         );
136         Device device=serviceRequest.getDevice();
137         if (actionDTO.getStatus() == ServiceRequestStatus.SENT_FOR_REPAIR) {
138             device.setDeviceStatus(DeviceStatus.UNDER_REPAIR);
139             deviceRepository.save(device);
140         } else if (actionDTO.getStatus() == ServiceRequestStatus.REJECTED) {
141             device.setDeviceStatus(DeviceStatus.ASSIGNED);

```

```

142     }
143     serviceRequest.setReviewedBy(admin);
144     serviceRequest.setAdminRemarks(actionDTO.getAdminRemarks());
145     serviceRequest.setStatus(actionDTO.getStatus());
146     ServiceRequest saved=serviceRequestRepository.save(serviceRequest);
147     return toServiceRequestResponseDTO(saved);
148 }
149
150 public List<ServiceRequestResponseDTO> getAllServiceRequestByVendor(Authentication authentication){
151     String email=AuthUtil.extractEmail(authentication);
152     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
153         ()->new VendorNotFoundException("Vendor not found: "+email)
154     );
155
156     List<ServiceRequest>
serviceRequestList=serviceRequestRepository.findByDeviceTechVendorAndStatus(vendor, ServiceRequestStatus.SENT_FOR
_REPAIR);
157     return serviceRequestList.stream()
158         .map(serviceRequest -> toServiceRequestResponseDTO(serviceRequest))
159         .toList();
160 }
161
162 public ServiceRequestResponseDTO updateServiceRequestByVendor(
163     Authentication authentication, RepairDTO repairDTO
164 ) {
165
166     // 1. Get vendor
167     String email = AuthUtil.extractEmail(authentication);
168     Vendor vendor = vendorRepo.findByEmail(email)
169         .orElseThrow(() -> new RuntimeException("Vendor not found"));
170
171     // 2. Get request
172     ServiceRequest request = serviceRequestRepository.findById(repairDTO.getRequestId())
173         .orElseThrow(() -> new RuntimeException("Request not found"));
174
175     // 3. Validate request belongs to vendor
176     if (!request.getDevice().getTechVendor().getId().equals(vendor.getId())) {
177         throw new RuntimeException("Unauthorized access");
178     }
179
180     // 4. Validate status
181     if (request.getStatus() != ServiceRequestStatus.SENT_FOR_REPAIR) {
182         throw new RuntimeException("Request is not in repair stage");
183     }
184
185     Device device = request.getDevice();
186
187     // 5. Update based on repair result
188     if (repairDTO.getStatus()==ServiceRequestStatus.REPAIR_DONE) {
189         request.setResolution(repairDTO.getResolution());
190         request.setStatus(repairDTO.getStatus());
191
192         device.setDeviceStatus(DeviceStatus.ASSIGNED); // back to employee
193     } else {
194         request.setResolution(repairDTO.getResolution());
195         request.setStatus(ServiceRequestStatus.REPAIR_DONE);
196
197         device.setDeviceStatus(DeviceStatus.CONDEMNED); // not usable
198     }
199
200     request.setResolvedAt(LocalDateTime.now());
201
202     deviceRepository.save(device);
203     ServiceRequest serviceRequest=serviceRequestRepository.save(request);
204     return toServiceRequestResponseDTO(serviceRequest);
205 }
206 }
207
208
209

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import com.example.EmployeeManagementSystem.Entity.LeaveType;
5 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
6 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
7 import org.slf4j.Logger;
8 import org.slf4j.LoggerFactory;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.math.BigDecimal;
13 import java.time.LocalDate;
14 import java.util.List;
15
16 @Service
17 public class SickLeaveResetService {
18
19     private static final Logger log = LoggerFactory.getLogger(SickLeaveResetService.class);
20
21     private final LeaveTypeRepository leaveTypeRepository;
22     private final LeaveEntitlementRepository entitlementRepository;
23
24     public SickLeaveResetService(LeaveTypeRepository leaveTypeRepository,
25                               LeaveEntitlementRepository entitlementRepository) {
26         this.leaveTypeRepository = leaveTypeRepository;
27         this.entitlementRepository = entitlementRepository;
28     }
29
30     @Transactional
31     public void runMonthlyReset(LocalDate resetDate) {
32         // resetDate = last day of the month being reset (e.g. 2025-06-30)
33         int year = resetDate.getYear();
34
35         // Find all leave types that reset monthly (only SICK in your design)
36         List<LeaveType> resetTypes = leaveTypeRepository.findByMonthlyResetTrue();
37
38         for (LeaveType leaveType : resetTypes) {
39             List<LeaveEntitlement> entitlements =
40                 entitlementRepository.findByLeaveTypeIdAndYear(leaveType.getId(), year);
41
42             for (LeaveEntitlement entitlement : entitlements) {
43
44                 BigDecimal available = entitlement.getAvailableBalance();
45
46                 if (available.compareTo(BigDecimal.ZERO) > 0) {
47                     // Wipe unused days – move accrued back to zero effect
48                     // We do this by setting accrued = used (so available = 0)
49                     // This preserves the audit trail of what was used
50                     entitlement.setAccruedThisYear(entitlement.getUsedThisYear());
51
52                     log.info("Sick leave reset for employee {} – {} unused day(s) expired on {}",
53                           entitlement.getEmployee().getEmployeeId(), available, resetDate);
54                 } else {
55                     log.debug("Sick leave reset for employee {} – nothing to expire",
56                             entitlement.getEmployee().getEmployeeId());
57                 }
58
59                 entitlementRepository.save(entitlement);
60             }
61         }
62
63         log.info("Sick leave monthly reset complete for {}", resetDate);
64     }
65 }

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
4 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Restaurant;
7 import com.example.EmployeeManagementSystem.Entity.Subscription;
8 import com.example.EmployeeManagementSystem.Enum.MealSlot;
9 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
10 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
11 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
12 import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
13 import com.example.EmployeeManagementSystem.Exception.SubscriptionAlreadyExists;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
16 import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
17 import com.example.EmployeeManagementSystem.Util.AuthUtil;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.stereotype.Service;
20
21 import java.time.ZoneId;
22 import java.time.ZonedDateTime;
23 import java.util.ArrayList;
24 import java.util.List;
25
26 @Service
27 public class SubscriptionService {
28
29     private final SubscriptionRepository subscriptionRepository;
30     private final EmployeeRepo employeeRepo;
31     private final DeliveryTimeService deliveryTimeService;
32     private final RestaurantRepository restaurantRepository;
33
34     public SubscriptionService(SubscriptionRepository subscriptionRepository,
35                               EmployeeRepo employeeRepo,
36                               DeliveryTimeService deliveryTimeService,
37                               RestaurantRepository restaurantRepository) {
38         this.subscriptionRepository = subscriptionRepository;
39         this.employeeRepo = employeeRepo;
40         this.deliveryTimeService = deliveryTimeService;
41         this.restaurantRepository = restaurantRepository;
42     }
43
44     /**
45      * Add subscriptions for an employee.
46      *
47      * Each SlotOrder specifies a meal slot AND the restaurant for that slot,
48      * so an employee can have breakfast from Restaurant A and lunch from Restaurant B.
49      *
50      * Validations:
51      * - Employee must exist and be ACTIVE
52      * - Restaurant must exist and be active
53      * - Restaurant must support the requested meal slot
54      * - No duplicate active subscription for the same employee + slot combination
55      */
56     public void addSubscription(SubscriptionsRequest request, Authentication authentication) {
57         String email = AuthUtil.extractEmail(authentication);
58         Employee employee = employeeRepo.findByEmail(email)
59             .orElseThrow(() -> new EmployeeNotFound(
60                 "Employee with id " + email + " not found"));
61
62         if (request.getSlotOrders() == null || request.getSlotOrders().isEmpty()) {
63             throw new IllegalArgumentException("slotOrders cannot be null or empty");
64         }
65
66         if (request.getScheduleType() == ScheduleType.DAILY) {
67             request.setDayOfWeek(null);
68         } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
69             if (request.getDayOfWeek() == null) {
70                 throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
71             }
72         }
73     }
74 }
```

```

72     }
73
74     for (SubscriptionRequest.SlotOrder slotOrder : request.getSlotOrders()) {
75         MealSlot slot = slotOrder.getMealSlot();
76         Long restaurantId = slotOrder.getRestaurantId();
77
78         // 1. Validate restaurant exists and is active
79         Restaurant restaurant = restaurantRepository.findById(restaurantId)
80             .orElseThrow(() -> new RestaurantNotFoundException(
81                 "Restaurant with id " + restaurantId + " not found"));
82         if (!restaurant.isActive()) {
83             throw new IllegalArgumentException(
84                 "Restaurant '" + restaurant.getName() + "' is not currently active");
85         }
86
87         // 2. Validate the restaurant actually serves this meal slot
88         if (!restaurant.getSupportedMealSlots().contains(slot)) {
89             throw new IllegalArgumentException(
90                 "Restaurant '" + restaurant.getName() + "' does not serve " + slot +
91                 ". Supported slots: " + restaurant.getSupportedMealSlots());
92         }
93
94         // 3. Check for duplicate active subscription (same employee + slot)
95         if (subscriptionRepository.checkSubscriptionExists(
96             employee.getEmployeeId(), slot.name(), SubscriptionStatus.ACTIVE.name()) > 0) {
97             throw new SubscriptionAlreadyExists(
98                 "Employee " + employee.getEmployeeId() +
99                 " already has an active " + slot + " subscription. " +
100                 "Cancel the existing one before subscribing to a new restaurant.");
101         }
102
103         // 4. Create subscription
104         Subscription subscription = new Subscription();
105         subscription.setEmployee(employee);
106         subscription.setSlot(slot);
107         subscription.setRestaurant(restaurant);
108         subscription.setScheduleType(request.getScheduleType());
109         subscription.setDayOfWeek(request.getDayOfWeek());
110         subscription.setStatus(SubscriptionStatus.ACTIVE);
111         subscription.setNextDeliveryTime(
112             deliveryTimeService.calculateNextDelivery(request, slot, employee));
113
114         subscriptionRepository.save(subscription);
115     }
116 }
117
118 public List<SubscriptionDTO> getAllSubscriptions() {
119     List<Subscription> subscriptions = subscriptionRepository.findAll();
120     List<SubscriptionDTO> dtoList = new ArrayList<>();
121     for (Subscription s : subscriptions) {
122         dtoList.add(convertToDTO(s));
123     }
124     return dtoList;
125 }
126
127 public Subscription updateSubscription(Long id, SubscriptionRequest request) {
128     Subscription subscription = subscriptionRepository.findById(id)
129         .orElseThrow(() -> new RuntimeException("Subscription not found"));
130     Employee employee = subscription.getEmployee();
131
132     if (request.getScheduleType() == ScheduleType.DAILY) {
133         subscription.setDayOfWeek(null);
134     } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
135         if (request.getDayOfWeek() == null) {
136             throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
137         }
138         subscription.setDayOfWeek(request.getDayOfWeek());
139     }
140     subscription.setScheduleType(request.getScheduleType());
141
142     // Update slot & restaurant if a single slotOrder is provided
143     if (request.getSlotOrders() != null && !request.getSlotOrders().isEmpty()) {
144         if (request.getSlotOrders().size() > 1) {
145             throw new IllegalArgumentException(
146                 "Update supports one slot at a time. Use separate subscriptions for multiple
slots.");

```



```

147     }
148     SubscriptionRequest.SlotOrder slotOrder = request.getSlotOrders().get(0);
149     MealSlot newSlot = slotOrder.getMealSlot();
150     Restaurant restaurant = restaurantRepository.findById(slotOrder.getRestaurantId())
151         .orElseThrow(() -> new RestaurantNotFoundException(
152             "Restaurant " + slotOrder.getRestaurantId() + " not found"));
153     if (!restaurant.isActive()) {
154         throw new IllegalArgumentException("Restaurant '" + restaurant.getName() + "' is not
active");
155     }
156     if (!restaurant.getSupportedMealSlots().contains(newSlot)) {
157         throw new IllegalArgumentException(
158             "Restaurant '" + restaurant.getName() + "' does not serve " + newSlot);
159     }
160     subscription.setSlot(newSlot);
161     subscription.setRestaurant(restaurant);
162 }
163
164     subscription.setNextDeliveryTime(
165         deliveryTimeService.calculateNextDelivery(request, subscription.getSlot(), employee));
166
167     return subscriptionRepository.save(subscription);
168 }
169
170 public Subscription pauseSubscription(Long id) {
171     Subscription subscription = subscriptionRepository.findById(id)
172         .orElseThrow(() -> new RuntimeException("Subscription not found"));
173     subscription.setStatus(SubscriptionStatus.PAUSED);
174     subscription.setNextDeliveryTime(null);
175     return subscriptionRepository.save(subscription);
176 }
177
178 public Subscription resumeSubscription(Long id) {
179     Subscription subscription = subscriptionRepository.findById(id)
180         .orElseThrow(() -> new RuntimeException("Subscription not found"));
181     subscription.setStatus(SubscriptionStatus.ACTIVE);
182     subscription.setNextDeliveryTime(deliveryTimeService.getNextDeliveryTime(subscription));
183     return subscriptionRepository.save(subscription);
184 }
185
186 public Subscription expireSubscription(Long id) {
187     Subscription subscription = subscriptionRepository.findById(id)
188         .orElseThrow(() -> new RuntimeException("Subscription not found"));
189     subscription.setStatus(SubscriptionStatus.EXPIRED);
190     subscription.setNextDeliveryTime(null);
191     return subscriptionRepository.save(subscription);
192 }
193
194 public List<SubscriptionDTO> getSubscriptionOfUser(Authentication authentication) {
195     String email= AuthUtil.extractEmail(authentication);
196     List<Subscription> subscriptions = subscriptionRepository.findByEmployee_Email(email);
197     List<SubscriptionDTO> dtoList = new ArrayList<>();
198     for (Subscription s : subscriptions) {
199         dtoList.add(convertToDTO(s));
200     }
201     return dtoList;
202 }
203
204 // helpers
205
206 private SubscriptionDTO convertToDTO(Subscription s) {
207     SubscriptionDTO dto = new SubscriptionDTO();
208     dto.setSubscriptionId(s.getId());
209     dto.setUserId(s.getEmployee().getEmployeeId());
210     dto.setMealSlot(s.getSlot());
211     dto.setScheduleType(s.getScheduleType());
212     if (s.getScheduleType() == ScheduleType.WEEKLY) {
213         dto.setDayOfWeek(s.getDayOfWeek());
214     }
215     dto.setStatus(s.getStatus());
216     if (s.getNextDeliveryTime() != null) {
217         ZoneId userZone = ZoneId.of(s.getEmployee().getTimezone());
218         dto.setNextDeliveryTime(s.getNextDeliveryTime().atZone(userZone));
219     }
220     // Restaurant info
221     if (s.getRestaurant() != null) {

```

```
222         dto.setRestaurantId(s.getRestaurant().getId());
223         dto.setRestaurantName(s.getRestaurant().getName());
224         dto.setRestaurantAddress(s.getRestaurant().getAddress());
225     }
226     return dto;
227 }
228 }
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import dev.samstevens.totp.code.*;
4 import dev.samstevens.totp.secret.DefaultSecretGenerator;
5 import dev.samstevens.totp.secret.SecretGenerator;
6 import dev.samstevens.totp.time.SystemTimeProvider;
7 import dev.samstevens.totp.time.TimeProvider;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 public class TotpService {
12
13     private final SecretGenerator secretGenerator = new DefaultSecretGenerator();
14     private final TimeProvider timeProvider = new SystemTimeProvider();
15     private final CodeGenerator codeGenerator = new DefaultCodeGenerator();
16     private final CodeVerifier codeVerifier = new DefaultCodeVerifier(codeGenerator, timeProvider);
17
18     /** Generate a new Base32 secret for a user */
19     public String generateSecret() {
20         return secretGenerator.generate();
21     }
22
23     /**
24      * Generate a QR code URI – user scans this with Google Authenticator / Authy.
25      * Format: otpauth://totp/{issuer}:{username}?secret={secret}&issuer={issuer}
26      */
27     public String getQrCodeUri(String secret, String username) {
28         return String.format(
29             "otpauth://totp/EmployeeMS:%s?secret=%s&issuer=EmployeeManagementSystem",
30             username, secret
31         );
32     }
33
34     /**
35      * Verify the 6-digit code the user typed.
36      * DefaultCodeVerifier already allows ±1 time step (±30 seconds) for clock skew.
37      */
38     public boolean verifyCode(String secret, String code) {
39         try {
40             return codeVerifier.isValidCode(secret, code);
41         } catch (Exception e) {
42             return false;
43         }
44     }
45 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.*;
4 import com.example.EmployeeManagementSystem.Enum.Status;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
7 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
8 import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.math.BigDecimal;
15 import java.math.RoundingMode;
16 import java.time.LocalDate;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.Set;
20 import java.util.stream.Collectors;
21
22 @Service
23 public class YearEndCarryForwardService {
24
25     private static final Logger log = LoggerFactory.getLogger(YearEndCarryForwardService.class);
26     private static final BigDecimal CAP = BigDecimal.valueOf(30);
27     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
28
29     private final EmployeeRepo employeeRepository;
30     private final LeaveEntitlementRepository entitlementRepository;
31     private final YearEndCarryForwardLogRepository logRepository;
32     private final LeaveWarningRepository warningRepository;
33
34     public YearEndCarryForwardService(
35         EmployeeRepo employeeRepository,
36         LeaveEntitlementRepository entitlementRepository,
37         YearEndCarryForwardLogRepository logRepository,
38         LeaveWarningRepository warningRepository) {
39         this.employeeRepository = employeeRepository;
40         this.entitlementRepository = entitlementRepository;
41         this.logRepository = logRepository;
42         this.warningRepository = warningRepository;
43     }
44
45     // -----
46     // Main entry point – called by scheduler on Dec 31 at 22:00
47     // -----
48     @Transactional
49     public void runYearEnd(int closingYear) {
50
51         // Idempotency guard – never run twice for same year
52         if (logRepository.existsByProcessedYear(closingYear)) {
53             log.warn("Year-end carry-forward already processed for {}. Skipping.", closingYear);
54             return;
55         }
56
57         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
58         log.info("Starting year-end carry-forward for {} – {} employees", closingYear,
59             activeEmployees.size());
60         for (Employee employee : activeEmployees) {
61             processEmployeeYearEnd(employee, closingYear);
62         }
63
64         log.info("Year-end carry-forward complete for {}", closingYear);
65     }
66
67     // -----
68     // Per-employee processing
69     // -----
70     private void processEmployeeYearEnd(Employee employee, int closingYear) {

```

```

71     int nextYear = closingYear + 1;
72
73     // Fetch all entitlements for the closing year
74     List<LeaveEntitlement> entitlements =
75         entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(),
closingYear);
76
77     // Separate carry-forwardable from non-carry-forwardable
78     List<LeaveEntitlement> carryForwardable = entitlements.stream()
79         .filter(e -> SUPPORTED_LEAVE_TYPES.contains(e.getLeaveType().getName()))
80         .filter(e -> e.getLeaveType().isCarriesForward())
81         .collect(Collectors.toList());
82
83     // Step 1 – compute raw closing balance for each type
84     //     closing = opening + accrued - used
85     Map<LeaveEntitlement, BigDecimal> rawClosing = carryForwardable.stream()
86         .collect(Collectors.toMap(
87             e -> e,
88             e -> e.getOpeningBalance()
89                 .add(e.getAccruedThisYear())
90                 .subtract(e.getUsedThisYear())
91                 .max(BigDecimal.ZERO) // never negative
92         ));
93
94     // Step 2 – sum all carry-forwardable closing balances
95     BigDecimal totalRaw = rawClosing.values().stream()
96         .reduce(BigDecimal.ZERO, BigDecimal::add);
97
98     // Step 3 – apply cap proportionally if total exceeds 30
99     Map<LeaveEntitlement, BigDecimal> cappedClosing = applyCap(rawClosing, totalRaw);
100
101     // Step 4 – persist: update closing balance on current year row,
102     //     create/update opening balance on next year row, write log
103     for (LeaveEntitlement entitlement : carryForwardable) {
104         BigDecimal raw = rawClosing.get(entitlement);
105         BigDecimal capped = cappedClosing.get(entitlement);
106         BigDecimal wasted = raw.subtract(capped);
107
108         // Update closing balance on the current year's row
109         entitlement.setClosingBalance(capped);
110         entitlementRepository.save(entitlement);
111
112         // Write or update next year's opening balance
113         LeaveEntitlement nextYearRow = entitlementRepository
114             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
115                 employee.getEmployeeId(), entitlement.getLeaveType().getId(), nextYear)
116             .orElseGet(() -> buildNextYearRow(employee, entitlement.getLeaveType(), nextYear));
117
118         nextYearRow.setOpeningBalance(capped);
119         entitlementRepository.save(nextYearRow);
120
121         // Write audit log
122         writeLog(employee, closingYear, entitlement.getLeaveType(), raw, capped, wasted);
123
124         // Issue DAY_WASTED warning if any days were lost
125         if (wasted.compareTo(BigDecimal.ZERO) > 0) {
126             issueWastedWarning(employee, wasted, capped);
127             log.info("Employee {} lost {} day(s) of {} to cap at year end",
128                 employee.getEmployeeId(), wasted, entitlement.getLeaveType().getName());
129         }
130     }
131
132     // Step 5 – zero out sick leave closing balance (it never carries)
133     entitlements.stream()
134         .filter(e -> SUPPORTED_LEAVE_TYPES.contains(e.getLeaveType().getName()))
135         .filter(e -> !e.getLeaveType().isCarriesForward())
136         .forEach(e -> {
137             e.setClosingBalance(BigDecimal.ZERO);
138             entitlementRepository.save(e);
139         });
140
141     log.info("Year-end processed for employee {} – total raw={{} capped={}",
142         employee.getEmployeeId(), totalRaw, CAP.min(totalRaw));
143 }
144
145 // -----

```

```

146 // Cap logic – distribute cap proportionally across leave types
147 //
148 // Example: casual=20, maternity=16 total=36, over cap by 6
149 // casual gets: 20/36 * 30 = 16.67
150 // maternity gets: 16/36 * 30 = 13.33
151 // total = 30 exactly
152 // -----
153 private Map<LeaveEntitlement, BigDecimal> applyCap(
154     Map<LeaveEntitlement, BigDecimal> rawClosing,
155     BigDecimal totalRaw) {
156
157     if (totalRaw.compareTo(CAP) <= 0) {
158         // Under cap – no adjustment needed
159         return rawClosing;
160     }
161
162     return rawClosing.entrySet().stream()
163         .collect(Collectors.toMap(
164             Map.Entry::getKey,
165             entry -> {
166                 BigDecimal raw = entry.getValue();
167                 // proportion = raw / totalRaw * CAP
168                 return raw
169                     .divide(totalRaw, 10, RoundingMode.HALF_UP)
170                     .multiply(CAP)
171                     .setScale(2, RoundingMode.HALF_UP);
172             }
173         ));
174 }
175
176 // -----
177 // Build a blank next-year entitlement row
178 // -----
179 private LeaveEntitlement buildNextYearRow(Employee employee, LeaveType leaveType, int year) {
180     LeaveEntitlement row = new LeaveEntitlement();
181     row.setEmployee(employee);
182     row.setLeaveType(leaveType);
183     row.setYear(year);
184     row.setOpeningBalance(BigDecimal.ZERO);
185     row.setAccruedThisYear(BigDecimal.ZERO);
186     row.setUsedThisYear(BigDecimal.ZERO);
187     row.setClosingBalance(BigDecimal.ZERO);
188     return entitlementRepository.save(row);
189 }
190
191 // -----
192 // Write audit log entry
193 // -----
194 private void writeLog(Employee employee, int year, LeaveType leaveType,
195     BigDecimal raw, BigDecimal capped, BigDecimal wasted) {
196     YearEndCarryForwardLog entry = new YearEndCarryForwardLog();
197     entry.setEmployee(employee);
198     entry.setProcessedYear(year);
199     entry.setLeaveType(leaveType);
200     entry.setClosingBalance(raw);
201     entry.setCappedBalance(capped);
202     entry.setDaysWasted(wasted);
203     entry.setOpeningNextYear(capped);
204     logRepository.save(entry);
205 }
206
207 // -----
208 // Issue a DAY_WASTED warning on the dashboard
209 // -----
210 private void issueWastedWarning(Employee employee, BigDecimal wasted, BigDecimal newBalance) {
211     LeaveWarning warning = new LeaveWarning();
212     warning.setEmployee(employee);
213     warning.setWarningType(LeaveWarning.WarningType.DAY_WASTED);
214     warning.setCarryForwardBalance(newBalance);
215     warning.setWarningDate(LocalDate.now());
216     warning.setMessage(
217         wasted.toPlainString() + " carry-forward day(s) were lost at year-end " +
218         "because your balance exceeded the 30-day cap. " +
219         "Your opening balance for next year is " + newBalance.toPlainString() + " days."
220     );
221     warningRepository.save(warning);

```

```
222     }
223 }
224
```

```
1 package com.example.EmployeeManagementSystem.Util;
2
3 import org.springframework.security.core.Authentication;
4 import org.springframework.security.oauth2.core.user.OAuth2User;
5 import org.springframework.stereotype.Component;
6
7 @Component
8 public class AuthUtil {
9     public static String extractEmail(Authentication authentication) {
10         if (authentication == null) return null;
11
12         // OAuth2 login (session-based) – principal is OAuth2User
13         if (authentication.getPrincipal() instanceof OAuth2User oAuth2User) {
14             return oAuth2User.getAttribute("email"); // always the actual email
15         }
16
17         // JWT or session username/password login – getName() is already the email
18         return authentication.getName();
19     }
20 }
21
```



```
1 package com.example.EmployeeManagementSystem.Util;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.Role;
6 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
7 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
10 import io.jsonwebtoken.Claims;
11 import io.jsonwebtoken.Jwts;
12 import io.jsonwebtoken.security.Keys;
13 import jakarta.annotation.PostConstruct;
14 import org.springframework.beans.factory.annotation.Value;
15 import org.springframework.security.core.userdetails.UserDetails;
16 import org.springframework.stereotype.Component;
17
18 import javax.crypto.SecretKey;
19 import java.util.Date;
20 import java.util.Optional;
21
22 @Component
23 public class JWTUtil {
24
25     private final EmployeeRepo employeeRepo;
26     private final VendorRepo vendorRepo;
27
28     // FIX 1: Was 1000*60*60 (1 hour) – caused token expiry mid-session.
29     // Extended to 24 hours to match the session timeout in application.yaml.
30     private final long Expiration_time = 1000L * 60 * 60 * 24;
31
32     @Value("${jwt.secret}")
33     private String secret;
34     private SecretKey key;
35
36     public JWTUtil(EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
37         this.employeeRepo = employeeRepo;
38         this.vendorRepo = vendorRepo;
39     }
40
41     @PostConstruct
42     public void init() {
43         key = Keys.hmacShaKeyFor(secret.getBytes());
44     }
45
46     public String generateToken(String username, String role) {
47         return Jwts.builder()
48             .subject(username)
49             .claim("role", role)
50             .issuedAt(new Date())
51             .expiration(new Date(System.currentTimeMillis() + Expiration_time))
52             .signWith(key)
53             .compact();
54     }
55
56     public String extractUsernameFromToken(String token) {
57         Claims body = extractClaims(token);
58         return body.getSubject();
59     }
60
61     private Claims extractClaims(String token) {
62         return Jwts.parser()
63             .verifyWith(key)
64             .build()
65             .parseSignedClaims(token)
66             .getPayload();
67     }
68
69     public boolean validateToken(String username, UserDetails userDetails, String token) {
70         return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
71     }
```

```
72
73     private boolean isTokenExpired(String token) {
74         return extractClaims(token).getExpiration().before(new Date());
75     }
76 }
```

```

1 package com.example.EmployeeManagementSystem.Util;
2
3 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
8 import jakarta.servlet.http.HttpServletRequest;
9 import jakarta.servlet.http.HttpServletResponse;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.oauth2.core.user.OAuth2User;
12 import org.springframework.security.web.authentication.SimpleUrlAuthenticationSuccessHandler;
13 import org.springframework.stereotype.Service;
14
15 import java.io.IOException;
16 import java.net.URLEncoder;
17 import java.nio.charset.StandardCharsets;
18
19 @Service
20 public class OAuth2SuccessHandler extends SimpleUrlAuthenticationSuccessHandler{
21     private final JWTUtil jwtUtil;
22     private final EmployeeRepo employeeRepo;
23     private final VendorRepo vendorRepo;
24     private final RefreshTokenService refreshTokenService;
25     private final RefreshTokenRepository refreshTokenRepository;
26
27     public OAuth2SuccessHandler(JWTUtil jwtUtil,
28                               EmployeeRepo employeeRepo,
29                               VendorRepo vendorRepo,
30                               RefreshTokenService refreshTokenService,
31                               RefreshTokenRepository refreshTokenRepository) {
32         this.jwtUtil = jwtUtil;
33         this.employeeRepo = employeeRepo;
34         this.vendorRepo = vendorRepo;
35         this.refreshTokenService = refreshTokenService;
36         this.refreshTokenRepository = refreshTokenRepository;
37     }
38
39     @Override
40     public void onAuthenticationSuccess(HttpServletRequest request,
41                                       HttpServletResponse response,
42                                       Authentication authentication) throws IOException {
43
44         try{
45             OAuth2User oAuth2User = (OAuth2User) authentication.getPrincipal();
46             String email = oAuth2User.getAttribute("email");
47
48             // Generate refresh token – check which entity type owns this email
49             RefreshToken refreshToken;
50             String role;
51             var employeeOpt = employeeRepo.findByEmail(email);
52             var vendorOpt = vendorRepo.findByEmail(email);
53
54             if (employeeOpt.isPresent()) {
55                 role = employeeOpt.get().getRole().name();
56                 refreshToken = refreshTokenService.createForEmployee(employeeOpt.get());
57             } else if (vendorOpt.isPresent()) {
58                 role = vendorOpt.get().getRole().name();
59                 refreshToken = refreshTokenService.createForVendor(vendorOpt.get());
60             } else {
61                 response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "User not found post-
62 OAuth");
63                 return;
64             }
65             String accessToken = jwtUtil.generateToken(email, role);
66             refreshTokenRepository.save(refreshToken);
67
68             // Redirect frontend to a callback page with tokens as query params
69             // (or use a fragment # so tokens don't hit server logs)
70             String redirectUrl = "http://localhost:8080/google.html" // your frontend page

```

```
71         + "?accessToken=" + URLEncoder.encode(accessToken, StandardCharsets.UTF_8)
72         + "&refreshToken=" + URLEncoder.encode(refreshToken.getToken(),
StandardCharsets.UTF_8);
73
74         getRedirectStrategy().sendRedirect(request, response, redirectUrl);
75     }
76     catch (Exception e) {
77         // This will now print the REAL error in your console
78         System.err.println("=== OAuth2SuccessHandler FAILED ===");
79         e.printStackTrace();
80         response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
81             "OAuth2 login failed: " + e.getMessage());
82     }
83 }
84 }
85
```

```
1  spring:
2    application:
3      name: EmployeeManagementSystem
4
5    datasource :
6      url : jdbc:mysql://localhost:3306/employee_management_db
7      username : ${USER_NAME}
8      password : ${PASSWORD}
9
10   jpa:
11     hibernate :
12       ddl-auto: update
13     show-sql: true
14
15   springdoc:
16     api-docs:
17       path: /v3/api-docs
18       enabled: true
19     swagger-ui:
20       path: /swagger-ui.html
21       enabled: true
22       operations-sorter: method
23       tags-sorter: alpha
24       try-it-out-enabled: true
25       filter: true
26     show-actuator: true
27   mvc:
28     static-path-pattern: /**
29   session:
30     tracking-modes: COOKIE
31   web:
32     resources:
33       static-locations: classpath:/static/
34       add-mappings: true
35   session:
36     timeout: 86400s # 24 hours in seconds (was 1440m which is incorrect format)
37     store-type: none # Use default cookie-based session
38
39   servlet:
40     session:
41       cookie:
42         http-only: true
43         same-site: strict
44         secure: false # set true in production with HTTPS
45
46   security:
47     debug: true
48     oauth2:
49       client:
50         registration:
51           google:
52             client-id: ${client_id}
53             client-secret: ${client_secret}
54             scope:
55               - email
56               - profile
57
58   logging:
59     level:
60       org.springframework.security: DEBUG
61       com.example.EmployeeManagementSystem: DEBUG
62   jwt:
63     secret: ${JWT_SECRET}
64   # Add this to your application.yaml
65   api:
66     key:
67       enabled: true
68       header-name: X-API-Key
69
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Admin Console</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <style>
9          * { margin: 0; padding: 0; box-sizing: border-box; }
10         :root {
11             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
12             --panel: #1e2436; --panel2: #252c40;
13             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
14             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
15             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
16             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
17             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
18             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
19             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
20             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
21         }
22         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
23         .app { display: flex; height: 100vh; }
24         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
25         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
26         .logo h2 { font-size: 20px; color: var(--text); }
27         .logo span { color: var(--accent); }
28         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
29         .nav-item { padding: 12px 20px; margin: 4px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; }
30         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
31         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
32         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
33         .page-title { font-size: 20px; font-weight: 600; }
34         .user-info { display: flex; align-items: center; gap: 16px; }
35         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; }
36         .content { flex: 1; overflow-y: auto; padding: 24px; }
37         .view { display: none; }
38         .view.active { display: block; }
39         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
40         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); }
41         .stat-label { font-size: 11px; color: var(--text3); text-transform: uppercase; margin-bottom: 8px; }
42         .stat-value { font-size: 28px; font-weight: 600; }
43         .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 600px; }
44         .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
45         .form-group { display: flex; flex-direction: column; gap: 6px; }
46         .form-group.full { grid-column: 1 / -1; }
47         label { font-size: 11px; font-weight: 600; color: var(--text3); text-transform: uppercase; }
48         input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; }
49         input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
50         .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow-x: auto; }
51         table { width: 100%; border-collapse: collapse; }
52         th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text3); font-size:
11px; font-weight: 600; text-transform: uppercase; }
53         td { padding: 12px 16px; border-top: 1px solid var(--border); }
54         .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-b
lock; }
55         .chip.active, .chip.approved, .chip.delivered, .chip.repair_done { background: var(--green-dim);
color: var(--green); }
56         .chip.pending, .chip.scheduled, .chip.sent_for_repair { background: var(--amber-dim); color: var(--
amber); }

```

```

57     .chip.open { background: var(--accent-dim); color: var(--accent); }
58     .chip.inactive, .chip.rejected, .chip.cancelled, .chip.expired { background: var(--red-dim); color:
var(--red); }
59     .chip.on_leave { background: var(--violet-dim); color: var(--violet); }
60     .chip.employee { background: var(--accent-dim); color: var(--accent); }
61     .chip.manager { background: var(--violet-dim); color: var(--violet); }
62     .chip.vendor { background: var(--red-dim); color: var(--red); }
63     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
64     .chip.unpaid, .chip.UNPAID { background: var(--violet-dim); color: var(--violet); }
65     .chip.assigned { background: var(--green-dim); color: var(--green); }
66     .chip.condemned { background: var(--red-dim); color: var(--red); }
67     .chip.admin { background: var(--amber-dim); color: var(--amber); }
68     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
69     .btn-primary { background: var(--accent); color: white; }
70     .btn-primary:hover { background: #6ba0ff; }
71     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
72     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
73     .btn-sm { padding: 4px 10px; font-size: 11px; margin: 2px; }
74     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; }
75     .section-title { font-size: 16px; font-weight: 600; }
76     .empty-state { text-align: center; padding: 40px; color: var(--text3); }
77     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
78     .modal-overlay.open { display: flex; }
79     .modal { background: var(--panel); border-radius: 12px; padding: 24px; width: 450px; max-width: 90%;
}
80     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
81     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
82     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; }
83     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid var(--green);
}
84     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid var(--red); }
85     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
86     .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
87     .notif-wrapper { position: relative; cursor: pointer; }
88
89     .notif-badge {
90         position: absolute; top: -5px; right: -6px;
91         background: #e53e3e; color: white;
92         font-size: 10px; font-weight: 600;
93         min-width: 16px; height: 16px;
94         border-radius: 50%; display: flex;
95         align-items: center; justify-content: center;
96         padding: 0 3px;
97     }
98     .notif-panel {
99         position: absolute; top: 48px; right: 0;
100        width: 320px; max-height: 400px; overflow-y: auto;
101        background: var(--panel);
102        border: 1px solid var(--border2);
103        border-radius: 10px;
104        box-shadow: 0 4px 24px rgba(0,0,0,0.4);
105        z-index: 999;
106    }
107    .notif-header {
108        padding: 12px 16px; font-weight: 600;
109        color: var(--text);
110        border-bottom: 1px solid var(--border);
111    }
112    .notif-item {
113        padding: 12px 16px; font-size: 13px;
114        color: var(--text2);
115        border-bottom: 1px solid var(--border);
116    }
117    .notif-item.unread { background: var(--accent-dim); color: var(--text); }
118    .notif-time { font-size: 11px; color: var(--text3); margin-top: 4px; }
119 </style>
120 </head>
121 <body>
122 <div class="app">
123     <!-- Sidebar -->

```

```

124     <div class="sidebar">
125         <div class="logo">
126             <h2>E<span>M</span>S</h2>
127             <p>Admin Control Center</p>
128         </div>
129         <div style="flex: 1;">
130             <div class="nav-item active" onclick="showView('dashboard', event)">#9632; Dashboard</div>
131             <div class="nav-item" onclick="showView('employees', event)">#9632; Employees</div>
132             <div class="nav-item" onclick="showView('add-employee', event)">#9632; Add Employee</div>
133             <div class="nav-item" onclick="showView('add-manager', event)">#9632; Add Manager</div>
134             <div class="nav-item" onclick="showView('add-vendor', event)">#9632; Add Vendor</div>
135             <div class="nav-item" onclick="window.location.href='/leave_management.html'">#9632; Leave
Management</div>
136             <div class="nav-item" onclick="showView('holidays', event)">#9632; Holidays</div>
137             <div class="nav-item" onclick="showView('restaurants', event)">#9632; Restaurants</div>
138             <div class="nav-item" onclick="showView('subscriptions', event)">#9632; Subscriptions</div>
139             <div class="nav-item" onclick="showView('add-subscription', event)">#9632; Add
Subscription</div>
140             <div class="nav-item" onclick="showView('vendors', event)">#9632; Vendors</div>
141             <div class="nav-item" onclick="showView('devices', event)">#9632; Devices</div>
142             <div class="nav-item" onclick="showView('my-devices', event)">#9632; My Devices</div>
143             <div class="nav-item" onclick="showView('service-requests', event)">#9632; Service
Requests</div>
144             <div class="nav-item" onclick="showView('admin-service-requests', event)">#9632; All Service
Requests</div>
145         </div>
146     </div>
147 </div>
148
149 <!-- Main Content -->
150 <div class="main-content">
151     <div class="topbar">
152         <div class="page-title" id="pageTitle">Dashboard</div>
153         <!-- Bell icon with badge -->
154         <div class="notif-wrapper" onclick="toggleNotifPanel()">
155             <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round">
156                 <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
157                 <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
158             </svg>
159             <span id="notif-badge" class="notif-badge" style="display:none"></span>
160         </div>
161
162         <!-- Dropdown panel -->
163         <div id="notif-panel" class="notif-panel" style="display:none">
164             <div class="notif-header">Notifications</div>
165             <div id="notif-list"></div>
166         </div>
167         <div class="user-info">
168             <span id="username-display"></span>
169             <button class="logout-btn" onclick="logout()">Logout</button>
170         </div>
171     </div>
172 </div>
173 <div class="content" id="content">
174
175     <!-- DASHBOARD VIEW -->
176     <div id="dashboard-view" class="view active">
177         <div class="stats-grid">
178             <div class="stat-card"><div class="stat-label">Total Employees</div><div class="stat-
value" id="total-employees">--</div></div>
179             <div class="stat-card"><div class="stat-label">Active</div><div class="stat-value"
id="active-employees">--</div></div>
180             <div class="stat-card"><div class="stat-label">On Leave</div><div class="stat-value"
id="onleave-employees">--</div></div>
181             <div class="stat-card"><div class="stat-label">Pending Leaves</div><div class="stat-
value" id="pending-leaves">--</div></div>
182         </div>
183         <div class="section-head"><div class="section-title">Recent Leave Requests</div><button
class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button></div>
184         <div class="table-
wrap"><table><thead><tr><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Status</th></tr></thead><tbo
dy id="recent-leaves"></tbody></table></div>
185     </div>
186
187     <!-- EMPLOYEES VIEW -->
188     <div id="employees-view" class="view">

```



```

189         <div class="section-head"><div class="section-title">All Employees</div><button class="btn
btn-primary btn-sm" onclick="loadEmployees()">Refresh</button></div>
190         <div class="table-
wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>
>Timezone</th><th>Actions</th></tr></thead><tbody i...
191         </div>
192
193         <div id="add-employee-view" class="view">
194             <div class="section-title" style="margin-bottom: 20px;">Add Employee</div>
195             <div class="form-card">
196                 <div class="form-grid">
197                     <div class="form-group"><label>Name</label><input type="text" id="employee-name"
placeholder="Employee name"></div>
198                     <div class="form-group"><label>Email</label><input type="email" id="employee-email"
placeholder="employee@company.com"></div>
199                     <div class="form-group"><label>Password</label><input type="password" id="employee-
password" placeholder="Temporary password"></div>
200                     <div class="form-group">
201                         <label>Gender</label>
202                         <select id="employee-gender">
203                             <option value="M">Male</option>
204                             <option value="F">Female</option>
205                             <option value="O">Other</option>
206                         </select>
207                     </div>
208                     <div class="form-group full">
209                         <label>Manager</label>
210                         <select id="employee-manager-select">
211                             <option value="">-- Select Manager --</option>
212                         </select>
213                     </div>
214                     <div class="form-group full"><label>Timezone</label>
215                         <select id="employee-timezone">
216                             <option value="Asia/Kolkata">India – IST UTC+5:30</option>
217                             <option value="UTC">UTC</option>
218                             <option value="America/New_York">New York – EST UTC-5</option>
219                             <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
220                             <option value="Europe/London">London – GMT UTC+0</option>
221                             <option value="Europe/Paris">Paris – CET UTC+1</option>
222                             <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
223                             <option value="Asia/Dubai">Dubai – GST UTC+4</option>
224                             <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
225                             <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
226                         </select>
227                     </div>
228                 </div>
229                 <div class="modal-actions" style="margin-top: 20px;">
230                     <button class="btn btn-primary" onclick="createEmployee()">Create Employee</button>
231                     <button class="btn btn-secondary"
onclick="clearEmployeeForm('employee')">Clear</button>
232                 </div>
233             </div>
234         </div>
235
236         <div id="add-manager-view" class="view">
237             <div class="section-title" style="margin-bottom: 20px;">Add Manager</div>
238             <div class="form-card">
239                 <div class="form-grid">
240                     <div class="form-group"><label>Name</label><input type="text" id="manager-name-
create" placeholder="Manager name"></div>
241                     <div class="form-group"><label>Email</label><input type="email" id="manager-email-
create" placeholder="manager@company.com"></div>
242                     <div class="form-group"><label>Password</label><input type="password" id="manager-
password-create" placeholder="Temporary password"></div>
243                     <div class="form-group"><label>Department</label><input type="text" id="manager-
dept-create" placeholder="Department"></div>
244                     <div class="form-group full"><label>Timezone</label>
245                         <select id="manager-timezone-create">
246                             <option value="Asia/Kolkata">India – IST UTC+5:30</option>
247                             <option value="UTC">UTC</option>
248                             <option value="America/New_York">New York – EST UTC-5</option>
249                             <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
250                             <option value="Europe/London">London – GMT UTC+0</option>
251                             <option value="Europe/Paris">Paris – CET UTC+1</option>
252                             <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
253                             <option value="Asia/Dubai">Dubai – GST UTC+4</option>

```

```

254             <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
255             <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
256         </select>
257     </div>
258     <div class="form-group"><label>Gender</label>
259         <select id="manager-gender">
260             <option value="M">Male</option>
261             <option value="F">Female</option>
262             <option value="O">Other</option>
263         </select>
264     </div>
265 </div>
266     <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="createManager()">Create Manager</button><button class="btn btn-secondary" onclick="...
267     </div>
268 </div>
269
270     <div id="add-vendor-view" class="view">
271         <div class="section-title" style="margin-bottom: 20px;">Add Vendor</div>
272         <div class="form-card">
273             <div class="form-grid">
274                 <div class="form-group"><label>Name</label><input type="text" id="vendor-name-
create" placeholder="Vendor name"></div>
275                 <div class="form-group"><label>Email</label><input type="email" id="vendor-email-
create" placeholder="vendor@company.com"></div>
276                 <div class="form-group"><label>Password</label><input type="password" id="vendor-
password-create" placeholder="Temporary password"></div>
277                 <div class="form-group"><label>Phone</label><input type="text" id="vendor-phone-
create" placeholder="+91 90000 00000"></div>
278                 <div class="form-group">
279                     <label>Role</label>
280                     <select id="vendor-role">
281                         <option value="TECH_VENDOR">tech vendor</option>
282                         <option value="FOOD_VENDOR">Food Vendor</option>
283                     </select>
284                 </div>
285             </div>
286             <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="createVendor()">Create Vendor</button><button class="btn btn-secondary" onclick="cl...
287         </div>
288     </div>
289
290     <!-- APPLY LEAVE VIEW -->
291     <div id="apply-leave-view" class="view">
292         <div class="section-title" style="margin-bottom: 20px;">Apply for Leave</div>
293         <div class="form-card">
294             <div class="form-grid">
295                 <div class="form-group full"><label>Logged In User</label><input type="email"
id="leave-email" placeholder="your.email@company.com" readonly></div>
296                 <div class="form-group"><label>Leave Type</label><select id="leave-type"><option
value="SICK">Sick Leave</option><option value="CASUAL">Casual Leave</option><option value="M...
297                     Sick leave is limited to <strong>1 day per month</strong>. If already used
this month, please select Unpaid Leave.
298                 </div></div>
299                 <div class="form-group"><label>Start Date</label><input type="date" id="leave-
start"></div>
300                 <div class="form-group"><label>End Date</label><input type="date" id="leave-
end"></div>
301                 <div class="form-group full"><label>Reason</label><textarea id="leave-reason"
rows="3" placeholder="Please provide reason for leave..."></textarea></div>
302             </div>
303             <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="applyLeave()">Submit Leave Request</button><button class="btn btn-secondary" oncl...
304         </div>
305     </div>
306
307     <!-- HOLIDAYS VIEW -->
308     <div id="holidays-view" class="view">
309         <div class="section-head">
310             <div class="section-title">Fixed Holidays</div>
311             <button class="btn btn-primary btn-sm" onclick="loadHolidays()">Refresh</button>
312         </div>
313
314     <!-- Add Holiday Form -->
315     <div class="form-card" style="margin-bottom: 24px;">
316         <div class="form-grid">

```

```
317         <div class="form-group">
318             <label>Holiday Name</label>
319             <input type="text" id="h-name" placeholder="e.g. Diwali, Christmas">
320         </div>
321         <div class="form-group">
322             <label>Date</label>
323             <input type="date" id="h-date">
324         </div>
325         <div class="form-group full">
326             <label>Description (optional)</label>
327             <input type="text" id="h-desc" placeholder="Brief description">
328         </div>
329     </div>
330     <div class="modal-actions" style="margin-top: 20px;">
331         <button class="btn btn-primary" onclick="addHoliday()">Add Holiday</button>
332         <button class="btn btn-secondary" onclick="clearHolidayForm()">Clear</button>
333     </div>
334 </div>
335
336 <!-- Holidays Table -->
337 <div class="table-wrap">
338     <table>
339         <thead>
340             <tr>
341                 <th>ID</th>
342                 <th>Name</th>
343                 <th>Date</th>
344                 <th>Description</th>
345                 <th>Action</th>
346             </tr>
347         </thead>
348         <tbody id="holidays-list">
349             <tr><td colspan="5" class="empty-state">Loading...</td></tr>
350         </tbody>
351     </table>
352 </div>
353 </div>
354
355 <!-- MY LEAVES VIEW -->
356 <div id="my-leaves-view" class="view">
357     <div class="section-head"><div class="section-title">My Leave Requests</div><button
358 class="btn btn-primary btn-sm" onclick="loadMyLeaves()">Refresh</button></div>
359     <div class="table-wrap"><table><thead><tr><th>ID</th><th>Type</th><th>Start
360 Date</th><th>End Date</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead><tbody id="my-leaves-...
361 </div>
362 <!-- PENDING LEAVES VIEW -->
363 <div id="pending-leaves-view" class="view">
364     <div class="section-head"><div class="section-title">Pending Leave Approvals</div><button
365 class="btn btn-primary btn-sm" onclick="loadPendingLeaves()">Refresh</button></div>
366     <div class="table-wrap"><table><thead><tr><th>ID</th><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Reason</th><th>Ac
367 tions</th></tr></thead><tbody id="pending-leaves-li...
368 </div>
369 <!-- RESTAURANTS VIEW -->
370 <div id="restaurants-view" class="view">
371     <div class="section-head"><div class="section-title">Restaurants</div><div><select
372 id="slot-filter" onchange="loadRestaurants()"><option value="">All Slots</option><option value="BR...
373 <div class="table-wrap"><table><thead><tr><th>Name</th><th>Address</th><th>Meal
374 Slots</th><th>Status</th><th>Vendor</th></tr></thead><tbody id="restaurants-list"></tbody></table></div>
375 </div>
376 <!-- MY SUBSCRIPTIONS VIEW -->
377 <div id="subscriptions-view" class="view">
378     <div class="section-head"><div class="section-title">My Meal Subscriptions</div><button
379 class="btn btn-primary btn-sm" onclick="loadSubscriptions()">Refresh</button></div>
380     <div class="table-wrap"><table><thead><tr><th>ID</th><th>Restaurant</th><th>Meal
381 Slot</th><th>Schedule</th><th>Status</th><th>Next Delivery</th><th>Actions</th></tr></thead><tbody i...
382 </div>
383 <!-- ADD SUBSCRIPTION VIEW -->
384 <div id="add-subscription-view" class="view">
385     <div class="section-title" style="margin-bottom: 20px;">Create Meal Subscription</div>
386     <div class="form-card">
387         <div class="form-grid">
```

```
384         <div class="form-group full">
385             <label>Employee Email</label>
386             <input type="email" id="sub-email" placeholder="employee@company.com">
387         </div>
388         <div class="form-group full">
389             <label>Restaurant</label>
390             <select id="sub-restaurant-select">
391                 <option value="">-- Select Restaurant --</option>
392             </select>
393         </div>
394         <div class="form-group full">
395             <label>Meal Slots</label>
396             <div id="meal-slot-checkboxes" style="display:flex; gap:16px; flex-wrap:wrap;
margin-top:6px;">
397                 <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
398                     <input type="checkbox" value="BREAKFAST"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Breakfast
399                 </label>
400                 <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
401                     <input type="checkbox" value="LUNCH"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Lunch
402                 </label>
403                 <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
404                     <input type="checkbox" value="DINNER"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Dinner
405                 </label>
406             </div>
407         </div>
408         <div class="form-group">
409             <label>Schedule Type</label>
410             <select id="sub-schedule">
411                 <option value="DAILY">Daily</option>
412                 <option value="WEEKLY">Weekly</option>
413             </select>
414         </div>
415         <div class="form-group" id="day-of-week-group" style="display:none;">
416             <label>Day of Week</label>
417             <select id="sub-day">
418                 <option value="MONDAY">Monday</option>
419                 <option value="TUESDAY">Tuesday</option>
420                 <option value="WEDNESDAY">Wednesday</option>
421                 <option value="THURSDAY">Thursday</option>
422                 <option value="FRIDAY">Friday</option>
423                 <option value="SATURDAY">Saturday</option>
424                 <option value="SUNDAY">Sunday</option>
425             </select>
426         </div>
427     </div>
428     <div class="modal-actions" style="margin-top: 20px;">
429         <button class="btn btn-primary" onclick="addSubscription()">Create
Subscription</button>
430         <button class="btn btn-secondary" onclick="clearSubscriptionForm()">Clear</button>
431     </div>
432 </div>
433 </div>
434
435 <!-- DELIVERIES VIEW -->
436 <div id="deliveries-view" class="view">
437     <div class="section-head"><div class="section-title">My Deliveries</div><button class="btn
btn-primary btn-sm" onclick="loadDeliveries()">Refresh</button></div>
438     <div class="table-wrap"><table><thead><tr><th>ID</th><th>Subscription ID</th><th>Meal
Slot</th><th>Scheduled Time</th><th>Status</th></tr></thead><tbody id="deliveries-list"></tbody>...
439     </div>
440
441 <!-- VENDORS VIEW -->
442 <div id="vendors-view" class="view">
443     <div class="section-head"><div class="section-title">Vendors</div><button class="btn btn-
primary btn-sm" onclick="loadVendors()">Refresh</button></div>
444     <div class="table-
wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Registered</th><th>Actions</th><
/tr></thead><tbody id="vendors-list"></tbody></table>...
445     </div>
446
```

```

447         <!-- DEVICES VIEW -->
448         <div id="devices-view" class="view">
449             <div class="section-head">
450                 <div class="section-title">Unassigned Devices</div>
451                 <button class="btn btn-primary btn-sm"
onclick="loadUnassignedDevices()">Refresh</button>
452             </div>
453             <div class="table-wrap">
454                 <table>
455                     <thead>
456                         <tr>
457                             <th>ID</th>
458                             <th>Device Name</th>
459                             <th>Brand</th>
460                             <th>Type</th>
461                             <th>Warranty</th>
462                             <th>Vendor</th>
463                             <th>Actions</th>
464                         </tr>
465                     </thead>
466                     <tbody id="devices-list"></tbody>
467                 </table>
468             </div>
469         </div>
470
471         <!-- ASSIGN DEVICE MODAL -->
472         <div id="assign-modal" class="modal-overlay">
473             <div class="modal">
474                 <div class="modal-title">Assign Device to Employee</div>
475                 <input type="hidden" id="assign-device-id">
476                 <div class="form-group" style="margin-bottom:12px">
477                     <label>Device</label>
478                     <input type="text" id="assign-device-name" readonly style="color:var(--text2)">
479                 </div>
480                 <div class="form-group" style="margin-bottom:12px">
481                     <label>Employee</label>
482                     <select id="assign-employee-select">
483                         <option value="">-- Select Employee --</option>
484                     </select>
485                 </div>
486                 <div class="modal-actions">
487                     <button class="btn btn-primary" onclick="confirmAssign()">Assign</button>
488                     <button class="btn btn-secondary" onclick="closeAssignModal()">Cancel</button>
489                 </div>
490             </div>
491         </div>
492
493         <!-- MY DEVICES VIEW -->
494         <div id="my-devices-view" class="view">
495             <div class="section-head">
496                 <div class="section-title">My Assigned Devices</div>
497                 <button class="btn btn-primary btn-sm"
onclick="loadMyAssignedDevices()">Refresh</button>
498             </div>
499             <div class="table-wrap">
500                 <table>
501                     <thead>
502                         <tr>
503                             <th>ID</th>
504                             <th>Device Name</th>
505                             <th>Brand</th>
506                             <th>Type</th>
507                             <th>Warranty</th>
508                             <th>Assigned On</th>
509                             <th>Status</th>
510                         </tr>
511                     </thead>
512                     <tbody id="my-devices-list"></tbody>
513                 </table>
514             </div>
515         </div>
516
517         <!-- MY SERVICE REQUESTS VIEW -->
518         <div id="service-requests-view" class="view">
519             <div class="section-head">
520                 <div class="section-title">My Service Requests</div>

```

```

521         <button class="btn btn-primary btn-sm" onclick="loadServiceRequests()">Refresh</button>
522     </div>
523     <div class="table-wrap" style="margin-bottom:24px">
524         <table>
525             <thead><tr>
526                 <th>ID</th><th>Device</th><th>Type</th>
527                 <th>Description</th><th>Urgent</th><th>Status</th><th>Raised At</th>
528             </tr></thead>
529             <tbody id="service-requests-list"></tbody>
530         </table>
531     </div>
532
533     <div class="section-title" style="margin-bottom:16px">Raise a New Request</div>
534     <div class="form-card">
535         <div class="form-grid">
536             <div class="form-group full">
537                 <label>Device</label>
538                 <select id="sr-device-select">
539                     <option value="">-- Select your device --</option>
540                 </select>
541             </div>
542             <div class="form-group">
543                 <label>Request Type</label>
544                 <select id="sr-type">
545                     <option value="REPAIR">Repair</option>
546                     <option value="REPLACEMENT">Replacement</option>
547                     <option value="UPGRADE">Upgrade</option>
548                     <option value="MAINTENANCE">Maintenance</option>
549                     <option value="RETURN">Return</option>
550                 </select>
551             </div>
552             <div class="form-group" style="display:flex;align-items:center;gap:10px;padding-
553 top:20px">
554                 <input type="checkbox" id="sr-urgent" style="width:16px;height:16px;padding:0">
555                 <label style="text-transform:none;font-size:13px;font-weight:400;color:var(--
556 text);cursor:pointer" for="sr-urgent">Mark as Urgent</label>
557             </div>
558             <div class="form-group full">
559                 <label>Issue Description</label>
560                 <textarea id="sr-description" rows="3" placeholder="Describe the issue in
561 detail..."></textarea>
562             </div>
563         </div>
564     </div>
565     <div class="modal-actions" style="margin-top:20px">
566         <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
567 Request</button>
568         <button class="btn btn-secondary"
569 onclick="clearServiceRequestForm()">Clear</button>
570     </div>
571 </div>
572
573 <!-- ===== ADMIN SERVICE REQUESTS VIEW ===== -->
574 <div id="admin-service-requests-view" class="view">
575     <div class="section-head">
576         <div class="section-title">All Open Service Requests</div>
577         <button class="btn btn-primary btn-sm"
578 onclick="loadAdminServiceRequests()">Refresh</button>
579     </div>
580     <div class="table-wrap">
581         <table>
582             <thead>
583                 <tr>
584                     <th>ID</th>
585                     <th>Employee</th>
586                     <th>Device</th>
587                     <th>Type</th>
588                     <th>Description</th>
589                     <th>Urgent</th>
590                     <th>Status</th>
591                     <th>Raised At</th>
592                     <th>Actions</th>
593                 </tr>
594             </thead>
595             <tbody id="admin-service-requests-list"></tbody>
596         </table>

```

```

591         </div>
592     </div>
593
594 </div>
595 </div>
596 </div>
597
598 <!-- Approve/Reject Modal -->
599 <div id="action-modal" class="modal-overlay">
600     <div class="modal">
601         <div class="modal-title" id="modal-title">Approve Leave Request</div>
602         <input type="hidden" id="action-leave-id">
603         <input type="hidden" id="action-type">
604         <div class="form-group"><label>Reviewed By</label><input type="email" id="manager-email"
placeholder="admin@company.com" readonly></div>
605         <div class="form-group"><label>Remarks</label><textarea id="action-remarks" rows="3"
placeholder="Add remarks..."></textarea></div>
606         <div class="modal-actions"><button class="btn btn-primary" id="action-btn"
onclick="submitApproval()">Approve</button><button class="btn btn-secondary"
onclick="closeModal()">Cancel</button>...
607     </div>
608 </div>
609
610 <!-- Cancel Leave Modal -->
611 <div id="cancel-modal" class="modal-overlay">
612     <div class="modal">
613         <div class="modal-title">Cancel Leave Request</div>
614         <input type="hidden" id="cancel-leave-id">
615         <div class="form-group"><label>Cancelled By</label><input type="email" id="cancel-email"
placeholder="admin@company.com" readonly></div>
616         <div class="modal-actions"><button class="btn btn-danger" onclick="confirmCancelLeave()">Cancel
Leave</button><button class="btn btn-secondary" onclick="closeCancelModal()">Close</button></div>
617     </div>
618 </div>
619
620 <!-- Admin Service Request Action Modal -->
621 <div id="admin-sr-modal" class="modal-overlay">
622     <div class="modal">
623         <div class="modal-title">Update Service Request <span id="admin-sr-modal-id" style="color:var(--
accent)"></span></div>
624         <input type="hidden" id="admin-sr-id">
625
626         <div class="form-group" style="margin-bottom:14px">
627             <label>Device</label>
628             <input type="text" id="admin-sr-device-display" readonly style="color:var(--text2)">
629         </div>
630         <div class="form-group" style="margin-bottom:14px">
631             <label>Employee</label>
632             <input type="text" id="admin-sr-employee-display" readonly style="color:var(--text2)">
633         </div>
634         <div class="form-group" style="margin-bottom:14px">
635             <label>Update Status</label>
636             <select id="admin-sr-status">
637                 <option value="SENT_FOR_REPAIR">Sent for Repair – device will be marked Under
Repair</option>
638                 <option value="REPAIR_DONE">Repair Done</option>
639                 <option value="REJECTED">Rejected</option>
640             </select>
641         </div>
642         <div class="form-group" style="margin-bottom:4px">
643             <label>Admin Remarks</label>
644             <textarea id="admin-sr-remarks" rows="3" placeholder="Add remarks or instructions for the
vendor..."></textarea>
645         </div>
646         <div id="admin-sr-repair-hint" style="font-size:11px;color:var(--amber);margin-
top:6px;display:none;">
647             Selecting "Sent for Repair" will automatically mark the device as <strong>Under
Repair</strong>.
648         </div>
649
650         <div class="modal-actions">
651             <button class="btn btn-primary" onclick="submitAdminSRUpdate()">Update Request</button>
652             <button class="btn btn-secondary" onclick="closeAdminSRModal()">Cancel</button>
653         </div>
654     </div>
655 </div>

```

```

656
657 <script>
658     const API = 'http://localhost:8080';
659     let currentUser = null;
660     const ROLE_DASHBOARDS = {
661         EMPLOYEE: '/employee-dashboard.html',
662         MANAGER: '/manager-dashboard.html',
663         VENDOR: '/vendor-dashboard.html',
664         ADMIN: '/dashboard.html'
665     };
666
667     (async function initAuth() {
668         const token = localStorage.getItem('jwt_token');
669         const authType = localStorage.getItem('auth_type');
670         if (!token && authType !== 'session') {
671             window.location.href = '/login.html';
672             return;
673         }
674
675         try {
676             const response = await fetch(`${API}/employee/me`, {
677                 headers: getAuthHeader(),
678                 credentials: 'include'
679             });
680
681             if (response.status === 401) { window.location.href = '/login.html'; return; }
682             if (!response.ok) { window.location.href = '/login.html'; return; }
683
684             const employee = await response.json();
685             const role = employee.role || 'EMPLOYEE';
686             if (role !== 'ADMIN') {
687                 window.location.href = ROLE_DASHBOARDS[role] || '/employee-dashboard.html';
688                 return;
689             }
690
691             currentUser = employee;
692             localStorage.setItem('last_role', role);
693             document.getElementById('username-display').textContent = employee.email || employee.name ||
694 'admin';
695             document.getElementById('leave-email').value = employee.email || '';
696             document.getElementById('manager-email').value = employee.email || '';
697             document.getElementById('cancel-email').value = employee.email || '';
698             loadDashboard();
699         } catch (e) {
700             window.location.href = '/login.html';
701         }
702     })();
703
704     function getAuthHeader() {
705         const token = localStorage.getItem('jwt_token');
706         return token ? { 'Authorization': `Bearer ${token}` } : {};
707     }
708
709     async function apiFetch(path, opts = {}) {
710         try {
711             const response = await fetch(`${API}${path}`, {
712                 ...opts,
713                 headers: {
714                     'Content-Type': 'application/json',
715                     ...getAuthHeader(),
716                     ...opts.headers
717                 },
718                 credentials: 'include'
719             });
720
721             if (response.status === 401) {
722                 localStorage.removeItem('jwt_token');
723                 localStorage.removeItem('auth_type');
724                 showAlert('Session expired. Please login again.', 'error');
725                 setTimeout(() => window.location.href = '/login.html', 1500);
726                 throw new Error('Unauthorized');
727             }
728
729             if (!response.ok) {
730                 const error = await response.text();
731                 throw new Error(error || 'Request failed');
732             }
733         }

```



```

731     }
732
733     const text = await response.text();
734     if (!text) return null;
735     try { return JSON.parse(text); } catch { return text; }
736   } catch (e) {
737     if (!e.message.includes('Unauthorized')) showAlert(e.message, 'error');
738     throw e;
739   }
740 }
741
742 function showAlert(message, type) {
743   const alert = document.createElement('div');
744   alert.className = `alert alert-${type}`;
745   alert.textContent = message;
746   document.body.appendChild(alert);
747   setTimeout(() => alert.remove(), 3000);
748 }
749
750 function showView(viewName, event) {
751   document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
752   document.getElementById(`${viewName}-view`).classList.add('active');
753   document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
754   if (event && event.target) event.target.classList.add('active');
755   document.getElementById('pageTitle').textContent = viewName.split('-').map(w =>
756     w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
757   const loaders = {
758     'dashboard': loadDashboard,
759     'employees': loadEmployees,
760     'add-employee': loadManagersDropdown,
761     'holidays': loadHolidays,
762     'add-manager': () => {},
763     'add-vendor': () => {},
764     'my-leaves': loadMyLeaves,
765     'pending-leaves': loadPendingLeaves,
766     'restaurants': loadRestaurants,
767     'subscriptions': loadSubscriptions,
768     'add-subscription': loadRestaurantDropdown,
769     'deliveries': loadDeliveries,
770     'vendors': loadVendors,
771     'devices': loadUnassignedDevices,
772     'my-devices': loadMyAssignedDevices,
773     'service-requests': loadServiceRequests,
774     'admin-service-requests': loadAdminServiceRequests
775   };
776   if (loaders[viewName]) loaders[viewName]();
777 }
778
779 // ===== DASHBOARD =====
780 async function loadDashboard() {
781   try {
782     const [employees, leaves] = await Promise.all([
783       apiFetch('/employee'),
784       apiFetch('/leave_requests')
785     ]);
786     document.getElementById('total-employees').textContent = employees.length;
787     document.getElementById('active-employees').textContent = employees.filter(e => e.status ===
788 'ACTIVE').length;
789     document.getElementById('onleave-employees').textContent = employees.filter(e => e.status ===
790 'ON_LEAVE').length;
791     document.getElementById('pending-leaves').textContent = leaves.filter(l => l.status ===
792 'PENDING').length;
793     const recent = leaves.slice(-5).reverse();
794     const tbody = document.getElementById('recent-leaves');
795     if (!recent.length) {
796       tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
797     } else {
798       tbody.innerHTML = recent.map(l => `<tr>
799         <td>${l.employeeId || 'N/A'}</td>
800         <td><span class="chip">${l.leaveType || 'N/A'}</span></td>
801         <td>${l.startDate || 'N/A'}</td>
802         <td>${l.endDate || 'N/A'}</td>
803         <td><span class="chip ${l.status?.toLowerCase()}>${l.status || 'N/A'}</span></td>
804       </tr>`).join('');
805     }
806   } catch (e) { console.error(e); }

```

```

804     }
805
806     // ===== EMPLOYEES =====
807     async function loadEmployees() {
808         try {
809             const employees = await apiFetch('/employee');
810             const tbody = document.getElementById('employees-list');
811             if (!employees.length) {
812                 tbody.innerHTML = '<tr><td colspan="8" class="empty-state">No employees found</td></tr>';
813             } else {
814                 tbody.innerHTML = employees.map(e => `<tr>
815                     <td>${e.employeeId}</td>
816                     <td>${e.name || 'N/A'}</td>
817                     <td>${e.email || 'N/A'}</td>
818                     <td>${e.dept || 'N/A'}</td>
819                     <td><span class="chip ${e.role?.toLowerCase()}>${e.role || 'N/A'}</span></td>
820                     <td><span class="chip ${e.status?.toLowerCase()}>${e.status || 'N/A'}</span></td>
821                     <td>${e.timezone || 'N/A'}</td>
822                     <td><button class="btn btn-danger btn-sm"
onclick="deleteEmployee(${e.employeeId})">Delete</button></td>
823                 </tr>`).join('');
824             }
825         } catch (e) { console.error(e); }
826     }
827
828     async function loadManagersDropdown() {
829         try {
830             const managers = await apiFetch('/admin/managers');
831             const select = document.getElementById('employee-manager-select');
832             select.innerHTML = '<option value="">-- Select Manager --</option>';
833             (managers || []).forEach(m => {
834                 const opt = document.createElement('option');
835                 opt.value = m.managerId; // matches ManagerDTO field
836                 opt.textContent = `${m.name} - ${m.dept || 'N/A'} (${m.email})`;
837                 select.appendChild(opt);
838             });
839         } catch (e) {
840             showAlert('Failed to load managers', 'error');
841         }
842     }
843
844
845
846     async function createEmployee() {
847         const managerId = document.getElementById('employee-manager-select').value;
848         const body = {
849             name: document.getElementById('employee-name').value.trim(),
850             email: document.getElementById('employee-email').value.trim(),
851             password: document.getElementById('employee-password').value,
852             gender: document.getElementById('employee-gender').value,
853             timezone: document.getElementById('employee-timezone').value || 'UTC',
854             managerId: parseInt(managerId)
855         };
856         if (!body.name || !body.email || !body.password) {
857             showAlert('Please fill all employee fields', 'error');
858             return;
859         }
860         if (!managerId) {
861             showAlert('Please select a manager', 'error');
862             return;
863         }
864         try {
865             await apiFetch('/admin/employees', { method: 'POST', body: JSON.stringify(body) });
866             showAlert('Employee created successfully!', 'success');
867             clearEmployeeForm('employee');
868             loadEmployees();
869         } catch (e) {
870             showAlert('Failed to create employee: ' + e.message, 'error');
871         }
872     }
873
874     async function createManager() {
875         const body = {
876             name: document.getElementById('manager-name-create').value.trim(),
877             email: document.getElementById('manager-email-create').value.trim(),
878             password: document.getElementById('manager-password-create').value,

```

```

879         dept: document.getElementById('manager-dept-create').value.trim(),
880         gender: document.getElementById('manager-gender').value,
881         timezone: document.getElementById('manager-timezone-create').value || 'UTC'
882     };
883     if (!body.name || !body.email || !body.password || !body.dept || !body.gender) {
884         showAlert('Please fill all manager fields', 'error');
885         return;
886     }
887     try {
888         await apiFetch('/admin/employees/manager', { method: 'POST', body: JSON.stringify(body) });
889         showAlert('Manager created successfully!', 'success');
890         clearEmployeeForm('manager');
891         loadEmployees();
892     } catch (e) { showAlert('Failed to create manager: ' + e.message, 'error'); }
893 }
894
895 function clearEmployeeForm(type) {
896     const fields = type === 'manager'
897         ? ['manager-name-create', 'manager-email-create', 'manager-password-create', 'manager-dept-create',
898           'manager-timezone-create']
899         : ['employee-name', 'employee-email', 'employee-password', 'employee-timezone'];
900     fields.forEach(id => document.getElementById(id).value = '');
901     if (type === 'employee') {
902         document.getElementById('employee-manager-select').value = '';
903     }
904 }
905
906 async function deleteEmployee(employeeId) {
907     if (!confirm(`Delete employee #${employeeId}?`)) return;
908     try {
909         await apiFetch(`/admin/employee/${employeeId}`, { method: 'DELETE' });
910         showAlert('Employee deleted successfully!', 'success');
911         loadEmployees();
912         loadDashboard();
913     } catch (e) { showAlert('Failed to delete employee: ' + e.message, 'error'); }
914 }
915
916 // ===== LEAVE REQUESTS =====
917 async function applyLeave() {
918     const leaveType = document.getElementById('leave-type').value;
919     const startDate = document.getElementById('leave-start').value;
920     const endDate = document.getElementById('leave-end').value;
921     const reason = document.getElementById('leave-reason').value.trim();
922     if (!startDate || !endDate) { showAlert('Please fill all required fields', 'error'); return; }
923     try {
924         await apiFetch('/leave_requests', {
925             method: 'POST',
926             body: JSON.stringify({ leaveType, startDate, endDate, reason })
927         });
928         showAlert('Leave request submitted successfully!', 'success');
929         clearLeaveForm();
930         loadMyLeaves();
931     } catch (e) { showAlert('Failed to submit: ' + e.message, 'error'); }
932 }
933
934 function toggleSickHint() {
935     const val = document.getElementById('leave-type').value;
936     document.getElementById('sick-leave-hint').style.display = val === 'SICK' ? 'block' : 'none';
937 }
938
939 async function loadMyLeaves() {
940     try {
941         const data = await apiFetch('/leave_requests/employee');
942         const leaves = data.requests || [];
943         const tbody = document.getElementById('my-leaves-list');
944         if (!leaves.length) {
945             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
946         } else {
947             tbody.innerHTML = leaves.map(l => `<tr>
948                 <td>${l.leaveRequestId}</td>
949                 <td><span class="chip">${l.leaveType}</span></td>
950                 <td>${l.startDate}</td>
951                 <td>${l.endDate}</td>
952                 <td><span class="chip ${l.status?.toLowerCase()}>${l.status}</span></td>
953                 <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-'}</td>

```

```

952         <td>${l.status === 'PENDING' ? `<button class="btn btn-danger btn-sm"
onclick="openCancelModal(${l.leaveRequestId})">Cancel</button>` : '-'}</td>
953     </tr>`).join('');
954     }
955     } catch (e) { console.error(e); }
956 }
957
958 async function loadPendingLeaves() {
959     try {
960         const leaves = await apiFetch('/leave_requests/pending');
961         const tbody = document.getElementById('pending-leaves-list');
962         if (!leaves.length) {
963             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No pending leave
requests</td></tr>';
964         } else {
965             tbody.innerHTML = leaves.map(l => `<tr>
966                 <td>${l.leaveRequestId}</td>
967                 <td>${l.employeeId}</td>
968                 <td><span class="chip">${l.leaveType}</span></td>
969                 <td>${l.startDate}</td>
970                 <td>${l.endDate}</td>
971                 <td style="max-width:150px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-
'}</td>
972                 <td class="action-row">
973                     <button class="btn btn-primary btn-sm"
onclick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
974                     <button class="btn btn-danger btn-sm" onclick="openActionModal(${l.leaveRequestId},
'REJECTED')">Reject</button>
975                 </td>
976             </tr>`).join('');
977         }
978     } catch (e) { console.error(e); }
979 }
980
981 function openActionModal(leaveId, action) {
982     document.getElementById('action-leave-id').value = leaveId;
983     document.getElementById('action-type').value = action;
984     document.getElementById('modal-title').textContent = `${action} Leave Request #${leaveId}`;
985     document.getElementById('action-btn').textContent = action;
986     document.getElementById('action-btn').className = action === 'APPROVED' ? 'btn btn-primary' : 'btn
btn-danger';
987     document.getElementById('action-modal').classList.add('open');
988 }
989
990 async function submitApproval() {
991     const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
992     const action = document.getElementById('action-type').value;
993     const remarks = document.getElementById('action-remarks').value;
994     try {
995         await apiFetch('/leave_requests/approval', {
996             method: 'PUT',
997             body: JSON.stringify({ leaveRequestId, action, remarks })
998         });
999         showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
1000         closeModal();
1001         loadPendingLeaves();
1002     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1003 }
1004
1005 function openCancelModal(leaveId) {
1006     document.getElementById('cancel-leave-id').value = leaveId;
1007     document.getElementById('cancel-modal').classList.add('open');
1008 }
1009
1010 async function confirmCancelLeave() {
1011     const leaveId = document.getElementById('cancel-leave-id').value;
1012     try {
1013         await apiFetch(`/leave_requests/cancel/${leaveId}`, { method: 'PUT' });
1014         showAlert('Leave cancelled successfully!', 'success');
1015         closeCancelModal();
1016         loadMyLeaves();
1017     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1018 }
1019
1020 function clearLeaveForm() {
1021     document.getElementById('leave-start').value = '';

```

```

1022     document.getElementById('leave-end').value = '';
1023     document.getElementById('leave-reason').value = '';
1024 }
1025
1026 // ===== RESTAURANTS =====
1027 async function loadRestaurants() {
1028     try {
1029         const slot = document.getElementById('slot-filter').value;
1030         const path = slot ? `/restaurants/by-slot?slot=${slot}` : '/restaurants';
1031         const restaurants = await apiFetch(path);
1032         const tbody = document.getElementById('restaurants-list');
1033         if (!restaurants.length) {
1034             tbody.innerHTML = `<tr><td colspan="5" class="empty-state">No restaurants
found</td></tr>`;
1035         } else {
1036             tbody.innerHTML = restaurants.map(r => `<tr>
1037                 <td><strong>${r.name}</strong></td>
1038                 <td>${r.address || 'N/A'}</td>
1039                 <td>${r.supportedMealSlots ? r.supportedMealSlots.map(s => `<span
class="chip">${s}</span>`).join(' ') : 'N/A'}</td>
1040                 <td><span class="chip ${r.active ? 'active' : 'inactive'}">${r.active ? 'ACTIVE' :
'INACTIVE'}</span></td>
1041                 <td>${r.vendorName || r.vendorId || 'N/A'}</td>
1042             </tr>`).join('');
1043         }
1044     } catch (e) { console.error(e); }
1045 }
1046
1047 // ===== SUBSCRIPTIONS =====
1048 async function loadRestaurantDropdown() {
1049     try {
1050         const restaurants = await apiFetch('/restaurants');
1051         const arr = Array.isArray(restaurants) ? restaurants : [];
1052         const select = document.getElementById('sub-restaurant-select');
1053         select.innerHTML = `<option value="">-- Select Restaurant --</option>`;
1054         arr.filter(r => r.active).forEach(r => {
1055             const opt = document.createElement('option');
1056             opt.value = r.id || r.restaurantId;
1057             opt.textContent = r.name;
1058             opt.dataset.slots = JSON.stringify(r.supportedMealSlots || []);
1059             select.appendChild(opt);
1060         });
1061     } catch (e) { console.error('Failed to load restaurants:', e); }
1062 }
1063
1064 document.getElementById('sub-restaurant-select')?.addEventListener('change', function () {
1065     const selected = this.options[this.selectedIndex];
1066     const supported = selected?.dataset?.slots ? JSON.parse(selected.dataset.slots) : [];
1067     const boxes = document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]');
1068     boxes.forEach(cb => {
1069         const wrapper = cb.parentElement;
1070         if (!supported.length || supported.includes(cb.value)) {
1071             wrapper.style.display = 'flex'; wrapper.style.opacity = '1'; cb.disabled = false;
1072         } else {
1073             cb.checked = false; cb.disabled = true; wrapper.style.opacity = '0.3';
1074         }
1075     });
1076 });
1077
1078 async function loadSubscriptions() {
1079     try {
1080         const subscriptions = await apiFetch('/subscription/user');
1081         const arr = Array.isArray(subscriptions) ? subscriptions : [];
1082         const tbody = document.getElementById('subscriptions-list');
1083         tbody.innerHTML = arr.length
1084             ? arr.map(s => `<tr>
1085                 <td>${s.subscriptionId}</td>
1086                 <td>${s.restaurantName || 'N/A'}</td>
1087                 <td><span class="chip">${s.mealSlot}</span></td>
1088                 <td>${s.scheduleType}</td>
1089                 <td><span class="chip ${s.status?.toLowerCase()}>${s.status}</span></td>
1090                 <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleDateString() : 'Not
scheduled'}</td>
1091                 <td class="action-row">
1092                     ${s.status === 'ACTIVE' ? `<button class="btn btn-secondary btn-sm"
onclick="pauseSubscription(${s.subscriptionId})">Pause</button>` : ''}

```

```

1093         ${s.status === 'PAUSED' ? `<button class="btn btn-primary btn-sm"
onclick="resumeSubscription(${s.subscriptionId})">Resume</button>` : ''}
1094         <button class="btn btn-danger btn-sm"
onclick="expireSubscription(${s.subscriptionId})">Expire</button>
1095     </td>
1096 </tr>`).join('')
1097     : `<tr><td colspan="7" class="empty-state">No subscriptions found</td></tr>`;
1098 } catch (e) { console.error(e); }
1099 }
1100
1101 async function addSubscription() {
1102     const email = document.getElementById('sub-email').value.trim();
1103     const restaurantId = parseInt(document.getElementById('sub-restaurant-select').value);
1104     const scheduleType = document.getElementById('sub-schedule').value;
1105     const dayOfWeek = document.getElementById('sub-day').value;
1106     const selectedSlots = [...document.querySelectorAll('#meal-slot-checkboxes
input[type=checkbox]:checked')].map(cb => cb.value);
1107     if (!email) { showAlert('Please enter employee email', 'error'); return; }
1108     if (!restaurantId) { showAlert('Please select a restaurant', 'error'); return; }
1109     if (!selectedSlots.length) { showAlert('Please select at least one meal slot', 'error'); return; }
1110     const body = {
1111         scheduleType,
1112         mealsSlots: selectedSlots,
1113         slotOrders: selectedSlots.map(slot => ({ mealSlot: slot, restaurantId })))
1114     };
1115     if (scheduleType === 'WEEKLY') body.dayOfWeek = dayOfWeek;
1116     try {
1117         await apiFetch('/subscription', { method: 'POST', body: JSON.stringify(body) });
1118         showAlert('Subscription created successfully!', 'success');
1119         clearSubscriptionForm();
1120         loadSubscriptions();
1121     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1122 }
1123
1124 async function pauseSubscription(id) {
1125     try { await apiFetch(`/subscription/${id}/pause`, { method: 'PUT' }); showAlert('Subscription
paused', 'success'); loadSubscriptions(); }
1126     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1127 }
1128
1129 async function resumeSubscription(id) {
1130     try { await apiFetch(`/subscription/${id}/resume`, { method: 'PUT' }); showAlert('Subscription
resumed', 'success'); loadSubscriptions(); }
1131     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1132 }
1133
1134 async function expireSubscription(id) {
1135     try { await apiFetch(`/subscription/${id}/expire`, { method: 'PUT' }); showAlert('Subscription
expired', 'success'); loadSubscriptions(); }
1136     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1137 }
1138
1139 function clearSubscriptionForm() {
1140     document.getElementById('sub-email').value = '';
1141     document.getElementById('sub-restaurant-select').value = '';
1142     document.getElementById('sub-schedule').value = 'DAILY';
1143     document.getElementById('day-of-week-group').style.display = 'none';
1144     document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]').forEach(cb => {
1145         cb.checked = false; cb.disabled = false;
1146         cb.parentElement.style.opacity = '1'; cb.parentElement.style.display = 'flex';
1147     });
1148 }
1149
1150 document.getElementById('sub-schedule')?.addEventListener('change', function () {
1151     document.getElementById('day-of-week-group').style.display = this.value === 'WEEKLY' ? 'block' :
'none';
1152 });
1153
1154 // ===== DELIVERIES =====
1155 async function loadDeliveries() {
1156     try {
1157         const subscriptions = await apiFetch('/subscription/user');
1158         let allDeliveries = [];
1159         for (const sub of subscriptions) {
1160             try {
1161                 const deliveries = await apiFetch(`/delivery/subscription/${sub.subscriptionId}`);

```

```

1162         allDeliveries.push(...deliveries);
1163     } catch (e) {}
1164 }
1165 const tbody = document.getElementById('deliveries-list');
1166 if (!allDeliveries.length) {
1167     tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No deliveries found</td></tr>';
1168 } else {
1169     tbody.innerHTML = allDeliveries.map(d => `<tr>
1170         <td>${d.id}</td>
1171         <td>${d.subscription?.id || 'N/A'}</td>
1172         <td><span class="chip">${d.mealSlot || 'N/A'}</span></td>
1173         <td>${d.scheduledDeliveryTime ? new Date(d.scheduledDeliveryTime).toLocaleString() :
1174 'N/A'}</td>
1175         <td><span class="chip ${d.status?.toLowerCase()}>${d.status ||
1176 'SCHEDULED'}</span></td>
1177     </tr>`).join('');
1178 } catch (e) { console.error(e); }
1179 }
1180 // ===== VENDORS =====
1181 async function loadVendors() {
1182     try {
1183         const vendors = await apiFetch('/vendors');
1184         const tbody = document.getElementById('vendors-list');
1185         if (!vendors.length) {
1186             tbody.innerHTML = '<tr><td colspan="6" class="empty-state">No vendors found</td></tr>';
1187         } else {
1188             tbody.innerHTML = vendors.map(v => `<tr>
1189                 <td>${v.id}</td>
1190                 <td><strong>${v.name}</strong></td>
1191                 <td>${v.email}</td>
1192                 <td>${v.phone || 'N/A'}</td>
1193                 <td>${v.registeredAt || 'N/A'}</td>
1194                 <td><button class="btn btn-danger btn-sm"
onclick="deleteVendor(${v.id})">Delete</button></td>
1195             </tr>`).join('');
1196         } catch (e) { console.error(e); }
1197     }
1198 }
1199
1200 async function createVendor() {
1201     const body = {
1202         name: document.getElementById('vendor-name-create').value.trim(),
1203         email: document.getElementById('vendor-email-create').value.trim(),
1204         password: document.getElementById('vendor-password-create').value,
1205         phone: document.getElementById('vendor-phone-create').value.trim(),
1206         role: document.getElementById('vendor-role').value
1207     };
1208     if (!body.name || !body.email || !body.password || !body.phone) {
1209         showAlert('Please fill all vendor fields', 'error'); return;
1210     }
1211     try {
1212         await apiFetch('/vendors/register', { method: 'POST', body: JSON.stringify(body) });
1213         showAlert('Vendor created successfully!', 'success');
1214         clearVendorForm();
1215         loadVendors();
1216     } catch (e) { showAlert('Failed to create vendor: ' + e.message, 'error'); }
1217 }
1218
1219 function clearVendorForm() {
1220     ['vendor-name-create', 'vendor-email-create', 'vendor-password-create', 'vendor-phone-create']
1221     .forEach(id => document.getElementById(id).value = '');
1222 }
1223
1224 async function deleteVendor(vendorId) {
1225     if (!confirm(`Delete vendor #${vendorId}?`)) return;
1226     try {
1227         await apiFetch(`/vendors/${vendorId}`, { method: 'DELETE' });
1228         showAlert('Vendor deleted successfully!', 'success');
1229         loadVendors();
1230     } catch (e) { showAlert('Failed to delete vendor: ' + e.message, 'error'); }
1231 }
1232
1233 // ===== MODAL HELPERS =====
1234 function closeModal() {

```

```

1235     document.getElementById('action-modal').classList.remove('open');
1236     document.getElementById('action-remarks').value = '';
1237     document.getElementById('manager-email').value = currentUser?.email || '';
1238 }
1239
1240 function closeCancelModal() {
1241     document.getElementById('cancel-modal').classList.remove('open');
1242     document.getElementById('cancel-email').value = currentUser?.email || '';
1243 }
1244
1245 // ===== DEVICES =====
1246 async function loadUnassignedDevices() {
1247     try {
1248         const devices = await apiFetch('/devices/unassigned');
1249         const tbody = document.getElementById('devices-list');
1250         if (!devices || !devices.length) {
1251             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No unassigned
devices</td></tr>';
1252             return;
1253         }
1254         const today = new Date();
1255         tbody.innerHTML = devices.map(d => {
1256             const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1257             const warrantyDisplay = warrantyExpiry
1258                 ? (warrantyExpiry > today
1259                     ? `<span style="color:var(--green);font-size:11px">
${d.warrantyExpiryDate}</span>`
1260                     : `<span style="color:var(--red);font-size:11px"> ${d.warrantyExpiryDate}</span>`)
1261                 : `<span style="color:var(--text3);font-size:11px">--</span>`;
1262             return `<tr>
<td>#${d.id}</td>
<td><strong>${d.deviceName || '-'}</strong></td>
<td>${d.brand || '-'}</td>
<td><span class="chip">${d.deviceType || '-'}</span></td>
<td>${warrantyDisplay}</td>
<td>${d.vendorName || '-'}</td>
<td><button class="btn btn-primary btn-sm" onclick="openAssignModal(${d.id},
'${d.deviceName}')">Assign</button></td>
</tr>`;
1270         }).join('');
1271     } catch (e) { console.error(e); }
1272 }
1273
1274
1275 async function openAssignModal(deviceId, deviceName) {
1276     document.getElementById('assign-device-id').value = deviceId;
1277     document.getElementById('assign-device-name').value = deviceName;
1278     try {
1279         const employees = await apiFetch('/employee');
1280         const select = document.getElementById('assign-employee-select');
1281         select.innerHTML = '<option value="">-- Select Employee --</option>';
1282         employees.filter(e => e.status === 'ACTIVE').forEach(e => {
1283             const opt = document.createElement('option');
1284             opt.value = e.employeeId;
1285             opt.textContent = `${e.name} (${e.email}) - ${e.dept || 'N/A'} `;
1286             select.appendChild(opt);
1287         });
1288     } catch (e) { showAlert('Failed to load employees', 'error'); return; }
1289     document.getElementById('assign-modal').classList.add('open');
1290 }
1291
1292 async function confirmAssign() {
1293     const deviceId = document.getElementById('assign-device-id').value;
1294     const employeeId = document.getElementById('assign-employee-select').value;
1295     if (!employeeId) { showAlert('Please select an employee', 'error'); return; }
1296     try {
1297         const message = await apiFetch(`/devices/${deviceId}/assign`, {
1298             method: 'POST',
1299             body: JSON.stringify({ employeeId: parseInt(employeeId) })
1300         });
1301         showAlert(message || 'Device assigned successfully!', 'success');
1302         closeAssignModal();
1303         loadUnassignedDevices();
1304     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1305 }
1306
1307 function closeAssignModal() {

```



```

1308     document.getElementById('assign-modal').classList.remove('open');
1309     document.getElementById('assign-employee-select').value = '';
1310     document.getElementById('assign-device-id').value = '';
1311     document.getElementById('assign-device-name').value = '';
1312 }
1313
1314 // ===== MY DEVICES =====
1315 async function loadMyAssignedDevices() {
1316     try {
1317         const devices = await apiFetch('/devices/my');
1318         const tbody = document.getElementById('my-devices-list');
1319         if (!devices || !devices.length) {
1320             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
1321             return;
1322         }
1323         const today = new Date();
1324         tbody.innerHTML = devices.map(d => {
1325             const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1326             const warrantyDisplay = warrantyExpiry
1327                 ? (warrantyExpiry > today
1328                     ? `<span style="color:var(--green);font-size:11px">
1329                         ${d.warrantyExpiryDate}</span>`
1330                     : `<span style="color:var(--red);font-size:11px"> Expired
1331                         ${d.warrantyExpiryDate}</span>`)
1332                 : `<span style="color:var(--text3);font-size:11px">--</span>`;
1333             return `<tr>
1334                 <td>#${d.id}</td>
1335                 <td><strong>${d.deviceName} || '-'</strong></td>
1336                 <td>${d.brand} || '-'</td>
1337                 <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
1338                 <td>${warrantyDisplay}</td>
1339                 <td style="font-size:12px;color:var(--text2)">${d.assignedDate} || '-'</td>
1340                 <td><span class="chip" ${((d.deviceStatus || '').toLowerCase())}>${d.deviceStatus} || '-'</span></td>
1341             </tr>`;
1342         }).join('');
1343     } catch (e) { console.error(e); }
1344 }
1345
1346 // ===== MY SERVICE REQUESTS =====
1347 async function loadServiceRequests() {
1348     await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
1349 }
1350
1351 async function loadServiceRequestsList() {
1352     try {
1353         const requests = await apiFetch('/service-requests/my');
1354         const tbody = document.getElementById('service-requests-list');
1355         if (!requests || !requests.length) {
1356             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No service requests
yet</td></tr>';
1357             return;
1358         }
1359         tbody.innerHTML = requests.map(r => `<tr>
1360             <td>#${r.id}</td>
1361             <td>${r.deviceName} || '-'</td>
1362             <td><span class="chip">${r.requestType}</span></td>
1363             <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap">${r.issueDescription} || '-'</td>
1364             <td>${r.urgent ? `<span class="chip pending">YES</span>` : `<span style="color:var(--
text2)">No</span>`}</td>
1365             <td><span class="chip" ${r.status?.toLowerCase()}>${r.status}</span></td>
1366             <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
Date(r.raisedAt).toLocaleDateString() : '-'}</td>
1367         </tr>`).join('');
1368     } catch (e) { console.error(e); }
1369 }
1370
1371 async function loadMyDevicesForSR() {
1372     try {
1373         const devices = await apiFetch('/devices/my');
1374         const select = document.getElementById('sr-device-select');
1375         select.innerHTML = '<option value="">-- Select your device --</option>';
1376         (devices || []).forEach(d => {
1377             const opt = document.createElement('option');
1378             opt.value = d.id;

```

```

1377         opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
1378         select.appendChild(opt);
1379     });
1380     } catch (e) { console.error('Failed to load devices for SR', e); }
1381 }
1382
1383 async function submitServiceRequest() {
1384     const deviceId = parseInt(document.getElementById('sr-device-select').value);
1385     const requestType = document.getElementById('sr-type').value;
1386     const issueDescription = document.getElementById('sr-description').value.trim();
1387     const urgent = document.getElementById('sr-urgent').checked;
1388     if (!deviceId) { showAlert('Please select a device', 'error'); return; }
1389     if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
1390     try {
1391         await apiFetch('/service-requests', {
1392             method: 'POST',
1393             body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
1394         });
1395         showAlert('Service request submitted!', 'success');
1396         clearServiceRequestForm();
1397         loadServiceRequestsList();
1398     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1399 }
1400
1401 function clearServiceRequestForm() {
1402     document.getElementById('sr-device-select').value = '';
1403     document.getElementById('sr-type').value = 'REPAIR';
1404     document.getElementById('sr-description').value = '';
1405     document.getElementById('sr-urgent').checked = false;
1406 }
1407
1408 // ===== ADMIN SERVICE REQUESTS =====
1409 async function loadAdminServiceRequests() {
1410     try {
1411         const requests = await apiFetch('/service-requests/open');
1412         const tbody = document.getElementById('admin-service-requests-list');
1413         if (!requests || !requests.length) {
1414             tbody.innerHTML = '<tr><td colspan="9" class="empty-state">No open service
requests</td></tr>';
1415             return;
1416         }
1417         tbody.innerHTML = requests.map(r => `<tr>
1418             <td>#${r.id}</td>
1419             <td style="font-size:12px;color:var(--text2)">${r.raisedByName || r.employeeId ||
'-'}</td>
1420             <td><strong>${r.deviceName || '-'}</strong></td>
1421             <td><span class="chip">${r.requestType}</span></td>
1422             <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;font-
size:12px;color:var(--text2)" title="${r.issueDescription || ''}">${r.issueDescription || '...
1423             <td>${r.urgent ? '<span class="chip pending">YES</span>' : '<span style="color:var(--
text2);font-size:12px">No</span>'}</td>
1424             <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
1425             <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
Date(r.raisedAt).toLocaleDateString() : '-'}</td>
1426             <td>
1427                 <button class="btn btn-primary btn-sm" onclick="openAdminSRModal(${r.id},
'${(r.deviceName || '').replace(/'/g, '\\')}', '${(r.raisedByName || r.employeeId || '').replace(/'/g,...
1428                 Update
1429             </button>
1430             </td>
1431         </tr>`).join('');
1432     } catch (e) { console.error(e); }
1433 }
1434
1435 function openAdminSRModal(requestId, deviceName, employeeName) {
1436     document.getElementById('admin-sr-id').value = requestId;
1437     document.getElementById('admin-sr-modal-id').textContent = `#${requestId}`;
1438     document.getElementById('admin-sr-device-display').value = deviceName;
1439     document.getElementById('admin-sr-employee-display').value = employeeName;
1440     document.getElementById('admin-sr-status').value = 'SENT_FOR_REPAIR';
1441     document.getElementById('admin-sr-remarks').value = '';
1442     // Show hint for default selection
1443     document.getElementById('admin-sr-repair-hint').style.display = 'block';
1444     document.getElementById('admin-sr-modal').classList.add('open');
1445 }
1446

```

```

1447 // Show/hide the repair hint based on status selection
1448 document.getElementById('admin-sr-status')?.addEventListener('change', function () {
1449     document.getElementById('admin-sr-repair-hint').style.display =
1450         this.value === 'SENT_FOR_REPAIR' ? 'block' : 'none';
1451 });
1452
1453 async function submitAdminSRUpdate() {
1454     const serviceRequestId = parseInt(document.getElementById('admin-sr-id').value);
1455     const status = document.getElementById('admin-sr-status').value;
1456     const adminRemarks = document.getElementById('admin-sr-remarks').value.trim();
1457
1458     if (!adminRemarks) {
1459         showAlert('Please add remarks before updating', 'error');
1460         return;
1461     }
1462
1463     try {
1464         await apiFetch('/service-requests/admin', {
1465             method: 'PUT',
1466             body: JSON.stringify({ serviceRequestId, status, adminRemarks })
1467         });
1468         const statusLabel = status.replace(/_/g, ' ').toLowerCase();
1469         showAlert('Service request marked as "${statusLabel}" successfully!', 'success');
1470         closeAdminSRModal();
1471         loadAdminServiceRequests();
1472     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1473 }
1474
1475 function closeAdminSRModal() {
1476     document.getElementById('admin-sr-modal').classList.remove('open');
1477     document.getElementById('admin-sr-id').value = '';
1478     document.getElementById('admin-sr-remarks').value = '';
1479     document.getElementById('admin-sr-repair-hint').style.display = 'none';
1480 }
1481
1482 // ===== LOGOUT =====
1483 function logout() {
1484     localStorage.removeItem('jwt_token');
1485     localStorage.removeItem('refresh_token');
1486     localStorage.removeItem('auth_type');
1487     localStorage.removeItem('last_role');
1488     fetch(`${API}/session/logout`, { method: 'POST', credentials: 'include' })
1489         .finally(() => {
1490             document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1491             window.location.href = '/google.html';
1492         });
1493 }
1494
1495 async function loadNotifCount() {
1496     try {
1497         const data = await apiFetch('/notifications/unread-count');
1498         const badge = document.getElementById('notif-badge');
1499
1500         // Handles: plain number, { count: N }, or { unreadCount: N }
1501         const count = typeof data === 'number'
1502             ? data
1503             : (data?.count ?? data?.unreadCount ?? 0);
1504
1505         if (count > 0) {
1506             badge.textContent = count;
1507             badge.style.display = 'flex';
1508         } else {
1509             badge.style.display = 'none';
1510         }
1511     } catch (e) { console.warn('Notif count error', e); }
1512 }
1513
1514 async function toggleNotifPanel() {
1515     const panel = document.getElementById('notif-panel');
1516     if (panel.style.display === 'none') {
1517         panel.style.display = 'block';
1518         await loadNotifications();
1519         try { await apiFetch('/notifications/mark-read', { method: 'POST' }); } catch (e) {}
1520         document.getElementById('notif-badge').style.display = 'none';
1521     } else {
1522         panel.style.display = 'none';
1523     }
1524 }

```

```

1523     }
1524
1525     async function loadNotifications() {
1526         try {
1527             const data = await apiFetch('/notifications');
1528             const list = document.getElementById('notif-list');
1529             list.innerHTML = (data && data.length)
1530                 ? data.map(n => `
1531                     <div class="notif-item ${n.read ? '' : 'unread'}">
1532                         <div>${n.message}</div>
1533                         <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
1534                     </div>`).join('')
1535                     : '<div class="notif-item">No notifications yet.</div>';
1536         } catch(e) { console.warn('Notif load error', e); }
1537     }
1538
1539     loadNotifCount();
1540     setInterval(loadNotifCount, 30000);
1541
1542     document.addEventListener('click', e => {
1543         const panel = document.getElementById('notif-panel');
1544         const wrapper = document.querySelector('.notif-wrapper');
1545         if (panel && wrapper && !panel.contains(e.target) && !wrapper.contains(e.target)) {
1546             panel.style.display = 'none';
1547         }
1548     });
1549
1550     // Set today's date as default for leave form
1551     document.getElementById('leave-start').valueAsDate = new Date();
1552     document.getElementById('leave-end').valueAsDate = new Date();
1553
1554     // ===== HOLIDAYS =====
1555     async function loadHolidays() {
1556         try {
1557             const year = new Date().getFullYear();
1558             const holidays = await apiFetch(`/holidays?year=${year}`);
1559             const tbody = document.getElementById('holidays-list');
1560             if (!holidays || !holidays.length) {
1561                 tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No holidays added yet</td></tr>';
1562                 return;
1563             }
1564             tbody.innerHTML = holidays.map(h => `<tr>
1565                 <td>#${h.id}</td>
1566                 <td><strong>${h.name}</strong></td>
1567                 <td>${h.date}</td>
1568                 <td>${h.description || '-'}</td>
1569                 <td>
1570                     <button class="btn btn-danger btn-sm" onclick="deleteHoliday(${h.id})">Delete</button>
1571                 </td>
1572             </tr>`).join('');
1573         } catch(e) { console.error(e); }
1574     }
1575
1576     async function addHoliday() {
1577         const name = document.getElementById('h-name').value.trim();
1578         const date = document.getElementById('h-date').value;
1579         const description = document.getElementById('h-desc').value.trim();
1580         if (!name || !date) { showAlert('Please enter name and date', 'error'); return; }
1581         try {
1582             await apiFetch('/holidays', {
1583                 method: 'POST',
1584                 body: JSON.stringify({ name, date, description })
1585             });
1586             showAlert('Holiday added!', 'success');
1587             clearHolidayForm();
1588             loadHolidays();
1589         } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1590     }
1591
1592     async function deleteHoliday(id) {
1593         if (!confirm('Delete this holiday?')) return;
1594         try {
1595             await apiFetch(`/holidays/${id}`, { method: 'DELETE' });
1596             showAlert('Holiday deleted', 'success');
1597             loadHolidays();
1598         } catch(e) { showAlert('Failed: ' + e.message, 'error'); }

```

```
1599 }
1600
1601 function clearHolidayForm() {
1602     document.getElementById('h-name').value = '';
1603     document.getElementById('h-date').value = '';
1604     document.getElementById('h-desc').value = '';
1605 }
1606 </script>
1607 </body>
1608 </html>
1609
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Employee Portal</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&
display=swap" rel="stylesheet">
8      <!-- FullCalendar CDN -->
9      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
10     <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
11     <style>
12         *{margin:0;padding:0;box-sizing:border-box}
13         :root{
14             --bg:#080c10;--bg2:#0d1117;--bg3:#111820;
15             --panel:#141c26;--panel2:#1a2332;
16             --border:rgba(99,179,237,0.08);--border2:rgba(99,179,237,0.15);
17             --text:#d4e6f1;--text2:#7a9bb5;--text3:#4a6d87;
18             --accent:#38bdf8;--accent-dim:rgba(56,189,248,0.1);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
20             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
21             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
22             --font:'Sora', sans-serif;--mono:'JetBrains Mono', monospace;
23             --sidebar:240px;
24         }
25         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
26         .app{display:flex;height:100vh}
27
28         /* SIDEBAR */
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;position:relative;flex-shrink:0}
30         .sidebar::before{content:'';position:absolute;top:0;left:0;right:0;height:1px;background:linear-
gradient(90deg,transparent,var(--accent),transparent)}
31         .logo{padding:24px 20px;border-bottom:1px solid var(--border)}
32         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
33         .logo-icon{width:32px;height:32px;background:linear-gradient(135deg,var(--accent),#0ea5e9);border-
radius:8px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size...
34         .logo h2{font-size:16px;font-weight:600;color:var(--text);letter-spacing:0.5px}
35         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono);margin-top:2px}
36         .role-badge{display:inline-flex;align-items:center;gap:5px;padding:4px 10px;background:var(--accent-dim);border:1px solid rgba(56,189,248,0.25);border-radius:20px;font-size:10px;font-weight...
37         .role-badge::before{content:'';width:6px;height:6px;background:var(--accent);border-
radius:50%;animation:pulse 2s infinite}
38         @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
39         .nav{flex:1;padding:12px 0;overflow-y:auto}
40         .nav-section{padding:4px 20px 8px;font-size:10px;font-weight:600;color:var(--text3);letter-
spacing:1.5px;text-transform:uppercase;margin-top:8px}
41         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
42         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
43         .nav-item.active{background:var(--accent-dim);color:var(--accent);border:1px solid
rgba(56,189,248,0.2)}
44         .nav-item .icon{width:20px;text-align:center;font-size:14px;flex-shrink:0}
45         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
46         .user-card{display:flex;align-items:center;gap:10px}
47         .avatar{width:32px;height:32px;background:linear-gradient(135deg,#0ea5e9,var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:13px;font-weight:6...
48         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
49         .user-info-side .email{font-size:10px;color:var(--text3);font-family:var(--mono);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:140px}
50
51         /* MAIN */
52         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
53         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
54         .page-title{font-size:16px;font-weight:600;color:var(--text)}
55         .topbar-right{display:flex;align-items:center;gap:12px}
56         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
57         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}

```

```

58     .notif-wrapper { position: relative; cursor: pointer; }
59
60     .notif-badge {
61         position: absolute; top: -5px; right: -6px;
62         background: #e53e3e; color: white;
63         font-size: 10px; font-weight: 600;
64         min-width: 16px; height: 16px;
65         border-radius: 50%; display: flex;
66         align-items: center; justify-content: center;
67         padding: 0 3px;
68     }
69
70     .notif-panel {
71         position: absolute; top: 48px; right: 16px;
72         width: 320px; max-height: 400px; overflow-y: auto;
73         background: white; border: 1px solid #e2e8f0;
74         border-radius: 10px; box-shadow: 0 4px 16px rgba(0,0,0,0.1);
75         z-index: 999;
76     }
77
78     .notif-header {
79         padding: 12px 16px; font-weight: 600;
80         border-bottom: 1px solid #e2e8f0;
81     }
82
83     .notif-item {
84         padding: 12px 16px; font-size: 14px;
85         border-bottom: 1px solid #f1f5f9;
86     }
87
88     .notif-item.unread { background: #eff6ff; }
89     .notif-time { font-size: 11px; color: #94a3b8; margin-top: 4px; }
90
91     /* CONTENT */
92     .content{flex:1;overflow-y:auto;padding:24px}
93     .view{display:none}
94     .view.active{display:block;animation:fadeIn 0.3s ease}
95     @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
96
97     /* CARDS */
98     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
99     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:12px;padding:20px;position:relative;overflow:hidden;transition:border-color 0.2s}
100     .stat-card:hover{border-color:var(--border2)}
101     .stat-card::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
102     .stat-card.blue::after{background:var(--accent)}
103     .stat-card.green::after{background:var(--green)}
104     .stat-card.amber::after{background:var(--amber)}
105     .stat-card.red::after{background:var(--red)}
106     .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;margin-
bottom:10px;font-weight:500}
107     .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
108     .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
109
110     /* SECTION */
111     .section{background:var(--panel);border:1px solid var(--border);border-radius:12px;margin-
bottom:20px;overflow:hidden}
112     .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
113     .section-title{font-size:14px;font-weight:600;color:var(--text)}
114     .section-sub{font-size:12px;color:var(--text3)}
115
116     /* TABLE */
117     .tablewrap{overflow-x:auto}
118     table{width:100%;border-collapse:collapse}
119     th{padding:10px 16px;text-align:left;font-size:11px;color:var(--text3);font-weight:600;letter-
spacing:0.8px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--bord...
120     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
121     tr:last-child td{border-bottom:none}
122     tr:hover td{background:rgba(56,189,248,0.03);color:var(--text)}
123
124     /* CHIPS */
125     .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600;letter-spacing:0.5px}
126     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}

```

```

127     .chip.open{background:var(--accent-dim);color:var(--accent)}
128     .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
129     .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
130     .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
131     .chip.sick{background:var(--red-dim);color:var(--red)}
132     .chip.casual{background:var(--amber-dim);color:var(--amber)}
133     .chip.maternity{background:var(--accent-dim);color:var(--accent)}
134     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
135     .chip.unpaid { background: rgba(139,92,246,0.15); color: #a78bfa; border: 1px solid
rgba(139,92,246,0.2); }
136     .chip.assigned{background:var(--green-dim);color:var(--green)}
137     .chip.condemned{background:var(--red-dim);color:var(--red)}
138
139     /* FORM */
140     .form-card{padding:24px}
141     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
142     .form-group{display:flex;flex-direction:column;gap:6px}
143     .form-group.full{grid-column:1/-1}
144     label{font-size:11px;font-weight:600;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
145     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
146     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent);background:var(--
bg2)}
147     input::placeholder,textarea::placeholder{color:var(--text3)}
148     select option{background:var(--bg2)}
149
150     /* BUTTONS */
151     .btn{padding:9px 18px;border-radius:8px;border:none;font-family:var(--font);font-size:13px;font-
weight:600;cursor:pointer;transition:all 0.2s}
152     .btn-primary{background:var(--accent);color:#fff}
153     .btn-primary:hover{background:#0ea5e9;transform:translateY(-1px);box-shadow:0 4px 12px
rgba(56,189,248,0.3)}
154     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
155     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
156     .btn-sm{padding:6px 12px;font-size:12px}
157     .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
158     .btn-danger:hover{background:var(--red);color:#fff}
159     .form-actions{display:flex;gap:10px;margin-top:20px}
160
161     /* ALERT */
162     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
163     .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
164     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
165     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
166
167     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
168     .profile-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px}
169     .profile-field{display:flex;flex-direction:column;gap:4px}
170     .profile-field .field-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-weight:600}
171     .profile-field .field-value{font-size:14px;color:var(--text);padding:10px 14px;background:var(--
bg3);border-radius:8px;border:1px solid var(--border);font-family:var(--mono)}
172
173     .subscription-card{display:grid;grid-template-columns:repeat(auto-
fill,minmax(200px,1fr));gap:12px;padding:20px}
174     .sub-item{background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px;transition:border-color 0.2s}
175     .sub-item:hover{border-color:var(--border2)}
176     .sub-item .meal-name{font-weight:600;color:var(--text);margin-bottom:6px}
177     .sub-item .meal-meta{font-size:11px;color:var(--text3)}
178
179     .chip.paused{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
180     .chip.expired{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
181
182     /* ===== */
183     /* CALENDAR VIEW STYLES */
184     /* ===== */
185
186     /* Leave balance summary cards above the calendar */
187     .leave-balance-grid {
188         display: grid;
189         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));

```



```

190         gap: 14px;
191         padding: 20px;
192         border-bottom: 1px solid var(--border);
193     }
194     .leave-bal-card {
195         background: var(--bg3);
196         border: 1px solid var(--border);
197         border-radius: 10px;
198         padding: 16px 18px;
199         display: flex;
200         flex-direction: column;
201         gap: 6px;
202         position: relative;
203         overflow: hidden;
204         transition: border-color 0.2s;
205     }
206     .leave-bal-card:hover { border-color: var(--border2); }
207     .leave-bal-card::before {
208         content: '';
209         position: absolute;
210         left: 0; top: 0; bottom: 0;
211         width: 3px;
212         border-radius: 3px 0 0 3px;
213     }
214     .leave-bal-card.sick::before { background: var(--red); }
215     .leave-bal-card.casual::before { background: var(--amber); }
216     .leave-bal-card.mat::before { background: var(--accent); }
217     .leave-bal-card.total::before { background: var(--green); }
218     .lbc-icon { font-size: 18px; }
219     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }
220     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-
height: 1; }
221     .lbc-sub { font-size: 11px; color: var(--text3); }
222
223     /* Legend row */
224     .cal-legend {
225         display: flex;
226         align-items: center;
227         gap: 20px;
228         padding: 12px 20px;
229         border-bottom: 1px solid var(--border);
230         flex-wrap: wrap;
231     }
232     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase;
letter-spacing: 0.8px; margin-right: 4px; }
233     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2);
}
234     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
235
236     /* FullCalendar dark-theme overrides */
237     #employee-calendar {
238         padding: 16px 20px 20px;
239     }
240     /* Toolbar */
241     #employee-calendar .fc .fc-toolbar-title {
242         font-family: var(--font);
243         font-size: 15px;
244         font-weight: 600;
245         color: var(--text);
246     }
247     #employee-calendar .fc .fc-button {
248         background: var(--panel2);
249         border: 1px solid var(--border2);
250         color: var(--text2);
251         font-family: var(--font);
252         font-size: 12px;
253         font-weight: 600;
254         border-radius: 6px;
255         padding: 5px 12px;
256         transition: all 0.2s;
257         box-shadow: none;
258         text-transform: capitalize;
259     }
260     #employee-calendar .fc .fc-button:hover,
261     #employee-calendar .fc .fc-button-active,

```

```

262     #employee-calendar .fc .fc-button:not(:disabled):active {
263         background: var(--accent-dim);
264         border-color: var(--accent);
265         color: var(--accent);
266         box-shadow: none;
267     }
268     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
269
270     /* Grid */
271     #employee-calendar .fc-theme-standard td,
272     #employee-calendar .fc-theme-standard th,
273     #employee-calendar .fc-theme-standard .fc-scrollgrid {
274         border-color: var(--border);
275     }
276     #employee-calendar .fc .fc-col-header-cell-cushion {
277         font-family: var(--font);
278         font-size: 11px;
279         font-weight: 600;
280         color: var(--text3);
281         text-transform: uppercase;
282         letter-spacing: 0.8px;
283         padding: 8px 4px;
284     }
285     #employee-calendar .fc .fc-daygrid-day-number {
286         font-family: var(--mono);
287         font-size: 12px;
288         color: var(--text3);
289         padding: 6px 8px;
290     }
291     #employee-calendar .fc .fc-daygrid-day.fc-day-today {
292         background: var(--accent-dim) !important;
293     }
294     #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number {
295         color: var(--accent);
296         font-weight: 700;
297     }
298     #employee-calendar .fc .fc-daygrid-day:hover {
299         background: rgba(56,189,248,0.03);
300     }
301     #employee-calendar .fc-daygrid-day-frame {
302         background: var(--bg3);
303     }
304     #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame {
305         background: var(--bg2);
306     }
307
308     /* Events */
309     #employee-calendar .fc-event {
310         border: none !important;
311         border-radius: 4px;
312         font-family: var(--font);
313         font-size: 11px;
314         font-weight: 600;
315         padding: 2px 6px;
316         cursor: pointer;
317     }
318     #employee-calendar .fc-event-title { font-weight: 600; }
319
320     /* Event tooltip */
321     .cal-tooltip {
322         position: fixed;
323         background: var(--panel2);
324         border: 1px solid var(--border2);
325         border-radius: 8px;
326         padding: 10px 14px;
327         font-size: 12px;
328         color: var(--text);
329         z-index: 9999;
330         pointer-events: none;
331         max-width: 200px;
332         box-shadow: 0 8px 24px rgba(0,0,0,0.4);
333     }
334     .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
335     .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
336
337     /* balance loading state */

```

```
338         .lbc-count.loading { color: var(--text3); font-size: 16px; }
339     </style>
340 </head>
341 <body>
342 <div class="app">
343     <!-- SIDEBAR -->
344     <div class="sidebar">
345         <div class="logo">
346             <div class="logo-mark">
347                 <div class="logo-icon">E</div>
348                 <h2>EMS Portal</h2>
349             </div>
350             <p>Employee Management</p>
351             <div class="role-badge">Employee</div>
352         </div>
353         <nav class="nav">
354             <div class="nav-section">Overview</div>
355             <div class="nav-item active" onclick="showView('dashboard',this)"><span class="icon"></span>
Dashboard</div>
356             <div class="nav-section">Leave</div>
357             <div class="nav-item" onclick="showView('apply-leave',this)"><span class="icon"></span> Apply
Leave</div>
358             <div class="nav-item" onclick="showView('my-leaves',this)"><span class="icon"></span> My Leave
History</div>
359             <div class="nav-item" onclick="showView('my-calendar',this)"><span class="icon"></span> My
Calendar</div>
360             <div class="nav-section">Meals</div>
361             <div class="nav-item" onclick="showView('restaurants',this)"><span class="icon"></span>
Restaurants</div>
362             <div class="nav-item" onclick="showView('subscriptions',this)"><span class="icon"></span> My
Subscriptions</div>
363             <div class="nav-item" onclick="showView('add-subscription',this)"><span class="icon"></span>
Add Subscription</div>
364             <div class="nav-section">Account</div>
365             <div class="nav-item" onclick="showView('profile',this)"><span class="icon"></span> My
Profile</div>
366             <div class="nav-item" onclick="showView('my-devices',this)"><span class="icon"></span> My
Devices</div>
367             <div class="nav-item" onclick="showView('service-requests',this)"><span class="icon"></span>
Service Requests</div>
368         </nav>
369         <div class="sidebar-footer">
370             <div class="user-card">
371                 <div class="avatar" id="avatar-initials">?</div>
372                 <div class="user-info-side">
373                     <div class="name" id="sidebar-name">Loading...</div>
374                     <div class="email" id="sidebar-email"></div>
375                 </div>
376             </div>
377         </div>
378     </div>
379
380     <!-- MAIN -->
381     <div class="main">
382         <div class="topbar">
383             <div class="page-title" id="pageTitle">Dashboard</div>
384             <!-- Bell icon with badge -->
385             <div class="notif-wrapper" onclick="toggleNotifPanel()">
386                 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
387                     stroke="currentColor" stroke-width="2" stroke-linecap="round">
388                     <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
389                     <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
390                 </svg>
391                 <span id="notif-badge" class="notif-badge" style="display:none"></span>
392             </div>
393
394             <!-- Dropdown panel -->
395             <div id="notif-panel" class="notif-panel" style="display:none">
396                 <div class="notif-header">Notifications</div>
397                 <div id="notif-list"></div>
398             </div>
399             <div class="topbar-right">
400                 <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
401                 <button class="logout-btn" onclick="logout()">Sign Out</button>
402             </div>
```

```
403     </div>
404     <div class="content">
405
406         <!-- DASHBOARD -->
407         <div id="dashboard-view" class="view active">
408             <div class="stats-grid">
409                 <div class="stat-card blue">
410                     <div class="stat-label">Leave Balance</div>
411                     <div class="stat-value" id="stat-balance">--</div>
412                     <div class="stat-sub">days remaining</div>
413                 </div>
414                 <div class="stat-card amber">
415                     <div class="stat-label">Pending Leaves</div>
416                     <div class="stat-value" id="stat-pending">--</div>
417                     <div class="stat-sub">awaiting approval</div>
418                 </div>
419                 <div class="stat-card green">
420                     <div class="stat-label">Approved Leaves</div>
421                     <div class="stat-value" id="stat-approved">--</div>
422                     <div class="stat-sub">this year</div>
423                 </div>
424                 <div class="stat-card red">
425                     <div class="stat-label">Rejected Leaves</div>
426                     <div class="stat-value" id="stat-rejected">--</div>
427                     <div class="stat-sub">this year</div>
428                 </div>
429             </div>
430             <div class="section">
431                 <div class="section-head">
432                     <div>
433                         <div class="section-title">Recent Leave Requests</div>
434                         <div class="section-sub">Your last 5 requests</div>
435                     </div>
436                     <button class="btn btn-secondary btn-sm" onclick="loadDashboard()">
Refresh</button>
437                 </div>
438                 <div class="tablewrap">
439                     <table>
440
441                         <thead><tr><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
442                         <tbody id="recent-leaves"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
443                     </table>
444                 </div>
445                 <div class="section">
446                     <div class="section-head">
447                         <div class="section-title">Active Subscriptions</div>
448                     </div>
449                     <div id="dashboard-subs" class="subscription-card"></div>
450                 </div>
451             </div>
452
453             <!-- APPLY LEAVE -->
454             <div id="apply-leave-view" class="view">
455                 <div class="section">
456                     <div class="section-head"><div class="section-title">Apply for Leave</div></div>
457                     <div class="form-card">
458                         <div class="form-grid">
459                             <div class="form-group">
460                                 <label>Leave Type</label>
461                                 <select id="leave-type">
462                                     <option value="SICK">Sick Leave</option>
463                                     <option value="CASUAL">Casual Leave</option>
464                                     <option value="MATERNITY">Maternity Leave</option>
465                                     <option value="UNPAID">Unpaid Leave</option>
466                                 </select>
467                                 <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(251,191,36,0.2);bord...
468                                     Sick leave is limited to <strong>1 day per month</strong>. If already
used this month, please select Unpaid Leave.
469                                 </div>
470                             </div>
471                             <div class="form-group">
472                                 <label>Start Date</label>
473                                 <input type="date" id="leave-start">
```

```

474                 </div>
475                 <div class="form-group">
476                     <label>End Date</label>
477                     <input type="date" id="leave-end">
478                 </div>
479                 <div class="form-group full">
480                     <label>Reason</label>
481                     <textarea id="leave-reason" rows="3" placeholder="Please describe the
reason for your leave..."></textarea>
482                 </div>
483             </div>
484             <div class="form-actions">
485                 <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
486                 <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
487             </div>
488         </div>
489     </div>
490 </div>
491
492     <!-- MY LEAVES -->
493     <div id="my-leaves-view" class="view">
494         <div class="section">
495             <div class="section-head">
496                 <div><div class="section-title">My Leave History</div><div class="section-sub">All
your leave requests</div></div>
497                 <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
498             </div>
499             <div class="tablewrap">
500                 <table>
501 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Status</th><th>Action</th></tr><
/thead>
502                 <tbody id="my-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
503             </table>
504         </div>
505     </div>
506 </div>
507
508     <!-- ===== -->
509     <!-- MY CALENDAR VIEW (NEW) -->
510     <!-- ===== -->
511     <div id="my-calendar-view" class="view">
512         <div class="section">
513             <div class="section-head">
514                 <div>
515                     <div class="section-title">My Leave Calendar</div>
516                     <div class="section-sub">All approved leaves and public holidays</div>
517                 </div>
518                 <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
519             </div>
520
521         <!-- Leave Balance Summary Cards -->
522         <div class="leave-balance-grid">
523             <!-- Sick Leave Card -->
524             <div class="leave-bal-card sick">
525                 <span class="lbc-icon"></span>
526                 <span class="lbc-label">Sick Leave</span>
527                 <span class="lbc-count" id="cal-sick-remaining">--</span>
528                 <span class="lbc-sub" id="cal-sick-sub">loading...</span>
529             </div>
530             <!-- Casual Leave Card -->
531             <div class="leave-bal-card casual">
532                 <span class="lbc-icon"></span>
533                 <span class="lbc-label">Casual Leave</span>
534                 <span class="lbc-count" id="cal-casual-remaining">--</span>
535                 <span class="lbc-sub" id="cal-casual-sub">loading...</span>
536             </div>
537             <!-- Maternity Leave Card -->
538             <div class="leave-bal-card mat">
539                 <span class="lbc-icon"></span>
540                 <span class="lbc-label">Maternity Leave</span>
541                 <span class="lbc-count" id="cal-mat-remaining">--</span>
542                 <span class="lbc-sub" id="cal-mat-sub">loading...</span>
543             </div>

```

```

544         <!-- Total Used Card -->
545         <div class="leave-bal-card total">
546             <span class="lbc-icon"></span>
547             <span class="lbc-label">Total Used</span>
548             <span class="lbc-count" id="cal-total-used">--</span>
549             <span class="lbc-sub" id="cal-total-sub">loading...</span>
550         </div>
551     </div>
552
553     <!-- Legend -->
554     <div class="cal-legend">
555         <span class="legend-label">Legend:</span>
556         <div class="legend-item">
557             <div class="legend-dot" style="background:#34d399"></div>
558             Approved Leave
559         </div>
560         <div class="legend-item">
561             <div class="legend-dot" style="background:#fbbf24"></div>
562             Pending Leave
563         </div>
564         <div class="legend-item">
565             <div class="legend-dot" style="background:#f87171"></div>
566             Rejected Leave
567         </div>
568         <div class="legend-item">
569             <div class="legend-dot" style="background:#38bdf8"></div>
570             Public Holiday
571         </div>
572     </div>
573
574     <!-- FullCalendar Mount Point -->
575     <div id="employee-calendar"></div>
576 </div>
577 </div>
578 <!-- END CALENDAR VIEW -->
579
580 <!-- RESTAURANTS -->
581 <div id="restaurants-view" class="view">
582     <div class="section">
583         <div class="section-head">
584             <div class="section-title">Available Restaurants</div>
585             <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
Refresh</button>
586         </div>
587         <div class="tablewrap">
588             <table>
589                 <thead><tr><th>Name</th><th>Meal
Slot</th><th>Status</th><th>Action</th></tr></thead>
590                 <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
591             </table>
592         </div>
593     </div>
594 </div>
595
596 <!-- SUBSCRIPTIONS -->
597 <div id="subscriptions-view" class="view">
598     <div class="section">
599         <div class="section-head">
600             <div class="section-title">My Meal Subscriptions</div>
601             <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
Refresh</button>
602         </div>
603         <div class="tablewrap">
604             <table>
605                 <thead>
606                     <tr>
607                         <th>Restaurant</th><th>Meal Slot</th><th>Schedule</th>
608                         <th>Next Delivery</th><th>Status</th><th>Action</th>
609                     </tr>
610                 </thead>
611                 <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-
state">Loading...</td></tr></tbody>
612             </table>
613         </div>
614     </div>

```

```
615         </div>
616
617         <!-- ADD SUBSCRIPTION -->
618         <div id="add-subscription-view" class="view">
619             <div class="section">
620                 <div class="section-head"><div class="section-title">Subscribe to Meal Plan</div></div>
621                 <div class="form-card">
622                     <div class="form-grid">
623                         <div class="form-group">
624                             <label>Meal Slot</label>
625                             <select id="sub-meal-slot">
626                                 <option value="BREAKFAST">Breakfast</option>
627                                 <option value="LUNCH">Lunch</option>
628                                 <option value="DINNER">Dinner</option>
629                             </select>
630                         </div>
631                         <div class="form-group">
632                             <label>Restaurant</label>
633                             <select id="sub-restaurant-id">
634                                 <option value="">Select a restaurant</option>
635                             </select>
636                         </div>
637                         <div class="form-group">
638                             <label>Schedule Type</label>
639                             <select id="sub-schedule-type">
640                                 <option value="DAILY">Daily</option>
641                                 <option value="WEEKLY">Weekly</option>
642                             </select>
643                         </div>
644                         <div class="form-group" id="day-of-week-group" style="display:none">
645                             <label>Day of Week</label>
646                             <select id="sub-day-of-week">
647                                 <option value="MONDAY">Monday</option>
648                                 <option value="TUESDAY">Tuesday</option>
649                                 <option value="WEDNESDAY">Wednesday</option>
650                                 <option value="THURSDAY">Thursday</option>
651                                 <option value="FRIDAY">Friday</option>
652                                 <option value="SATURDAY">Saturday</option>
653                                 <option value="SUNDAY">Sunday</option>
654                             </select>
655                         </div>
656                     </div>
657                     <div class="form-actions">
658                         <button class="btn btn-primary" onclick="addSubscription()">Subscribe</button>
659                     </div>
660                 </div>
661             </div>
662         </div>
663
664         <!-- MY DEVICES -->
665         <div id="my-devices-view" class="view">
666             <div class="section">
667                 <div class="section-head">
668                     <div>
669                         <div class="section-title">My Assigned Devices</div>
670                         <div class="section-sub">Devices currently assigned to you</div>
671                     </div>
672                     <button class="btn btn-secondary btn-sm" onclick="loadMyAssignedDevices()">
Refresh</button>
673                 </div>
674                 <div class="tablewrap">
675                     <table>
676                         <thead>
677                             <tr>
678                                 <th>ID</th><th>Device Name</th><th>Brand</th>
679                                 <th>Type</th><th>Warranty</th><th>Assigned On</th><th>Status</th>
680                             </tr>
681                         </thead>
682                         <tbody id="my-devices-list"></tbody>
683                     </table>
684                 </div>
685             </div>
686         </div>
687
688         <!-- SERVICE REQUESTS -->
689         <div id="service-requests-view" class="view">
```

```

690         <div class="section" style="margin-bottom:20px">
691             <div class="section-head">
692                 <div>
693                     <div class="section-title">My Service Requests</div>
694                     <div class="section-sub">All requests you have raised</div>
695                 </div>
696                 <button class="btn btn-secondary btn-sm" onclick="loadServiceRequestsList()">
Refresh</button>
697             </div>
698             <div class="tablewrap">
699                 <table>
700                     <thead>
701                         <tr>
702                             <th>ID</th><th>Device</th><th>Type</th>
703                             <th>Description</th><th>Urgent</th><th>Status</th><th>Raised At</th>
704                         </tr>
705                     </thead>
706                     <tbody id="service-requests-list">
707                         <tr><td colspan="7" class="empty-state">Loading...</td></tr>
708                     </tbody>
709                 </table>
710             </div>
711         </div>
712         <div class="section">
713             <div class="section-head"><div class="section-title">Raise a New Request</div></div>
714             <div class="form-card">
715                 <div class="form-grid">
716                     <div class="form-group full">
717                         <label>Device</label>
718                         <select id="sr-device-select">
719                             <option value="">-- Select your device --</option>
720                         </select>
721                     </div>
722                     <div class="form-group">
723                         <label>Request Type</label>
724                         <select id="sr-type">
725                             <option value="REPAIR">Repair</option>
726                             <option value="REPLACEMENT">Replacement</option>
727                             <option value="UPGRADE">Upgrade</option>
728                             <option value="MAINTENANCE">Maintenance</option>
729                             <option value="RETURN">Return</option>
730                         </select>
731                     </div>
732                     <div class="form-group" style="display:flex;align-
items:center;gap:10px;padding-top:22px">
733                         <input type="checkbox" id="sr-urgent"
style="width:16px;height:16px;padding:0;flex-shrink:0">
734                         <label for="sr-urgent" style="text-transform:none;font-size:13px;font-
weight:500;color:var(--text);cursor:pointer;letter-spacing:0">Mark as Urgent</label>
735                     </div>
736                     <div class="form-group full">
737                         <label>Issue Description</label>
738                         <textarea id="sr-description" rows="3" placeholder="Describe the issue in
detail..."></textarea>
739                     </div>
740                 </div>
741                 <div class="form-actions">
742                     <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
743                     <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
744                 </div>
745             </div>
746         </div>
747     </div>
748
749     <!-- PROFILE -->
750     <div id="profile-view" class="view">
751         <div class="section">
752             <div class="section-head"><div class="section-title">My Profile</div></div>
753             <div class="profile-grid" id="profile-data">
754                 <div class="profile-field"><div class="field-label">Name</div><div class="field-
value" id="p-name"></div></div>
755                 <div class="profile-field"><div class="field-label">Email</div><div class="field-
value" id="p-email"></div></div>
756                 <div class="profile-field"><div class="field-label">Department</div><div

```



```

class="field-value" id="p-dept"></div></div>
757         <div class="profile-field"><div class="field-label">Role</div><div class="field-
value" id="p-role"></div></div>
758         <div class="profile-field"><div class="field-label">Status</div><div class="field-
value" id="p-status"></div></div>
759         <div class="profile-field"><div class="field-label">Timezone</div><div
class="field-value" id="p-tz"></div></div>
760         </div>
761     </div>
762 </div>
763
764 </div><!-- /content -->
765 </div><!-- /main -->
766 </div><!-- /app -->
767
768 <!-- Tooltip for calendar events -->
769 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
770
771 <script>
772     const API = 'http://localhost:8080';
773     let availableRestaurants = [];
774
775     /* =====
776     CALENDAR GLOBALS
777     ===== */
778     let employeeCalendar = null;    // FullCalendar instance
779     let calendarInitialized = false;
780
781     // Event color map
782     const STATUS_COLORS = {
783         APPROVED : { bg:'rgba(52,211,153,0.25)', border:'#34d399', text:'#34d399' },
784         PENDING  : { bg:'rgba(251,191,36,0.25)',   border:'#fbbf24', text:'#fbbf24' },
785         REJECTED : { bg:'rgba(248,113,113,0.25)',  border:'#f87171', text:'#f87171' },
786         HOLIDAY  : { bg:'rgba(56,189,248,0.25)',  border:'#38bdf8', text:'#38bdf8' },
787     };
788
789     const TYPE_LABEL = { SICK:' Sick', CASUAL:' Casual', MATERNITY:' Maternity', UNPAID:' Unpaid'
790     };
791
792     /* =====
793     AUTH / INIT
794     ===== */
795     function extractRoleFromToken(token) {
796         try {
797             const payload = JSON.parse(atob(token.split('.')[1]));
798             return payload.role || payload.roles || payload.authorities || '';
799         } catch(e) { return ''; }
800     }
801
802     (function(){
803         const token = localStorage.getItem('jwt_token');
804         const authType = localStorage.getItem('auth_type');
805         if(!token && authType !== 'session') { window.location.href = '/google.html'; return; }
806         if(token) {
807             try {
808                 const roles = String(extractRoleFromToken(token));
809                 if(roles.includes('ROLE_MANAGER')) { window.location.href='/manager-dashboard.html';
return; }
810                 if(roles.includes('ROLE_VENDOR')) { window.location.href='/vendor-dashboard.html';
return; }
811                 if(roles.includes('ROLE_ADMIN') || roles.includes('ADMIN')) {
window.location.href='/dashboard.html'; return; }
812             } catch(e) { console.warn('Could not decode JWT for role check'); }
813         }
814         initUser();
815     })();
816
817     function getAuthHeader(){
818         const token = localStorage.getItem('jwt_token');
819         return token ? {'Authorization':`Bearer ${token}`} : {};
820     }
821
822     async function apiFetch(path, opts={}){
823         const token = localStorage.getItem('jwt_token');
824         const authType = localStorage.getItem('auth_type');
825         const res = await fetch(`${API}${path}`, {

```

```

826         ...opts,
827         headers: {'Content-Type': 'application/json', ...getAuthHeader(), ...(opts.headers || {})},
828         credentials: (!token && authType === 'session') ? 'include' : 'omit'
829     });
830     if(res.status === 401){
831         localStorage.removeItem('jwt_token'); localStorage.removeItem('auth_type');
832         showAlert('Session expired. Please login again.', 'error');
833         setTimeout(() => window.location.href = '/google.html', 1500);
834         throw new Error('Unauthorized');
835     }
836     if(res.status === 403) throw new Error('ACCESS_DENIED');
837     if(!res.ok){
838         const errText = await res.text();
839         throw new Error(errText || 'Request failed');
840     }
841     const text = await res.text();
842     try { return text ? JSON.parse(text) : null; }
843     catch { return text; }
844 }
845
846 function asArray(value){ return Array.isArray(value) ? value : []; }
847
848 function daysBetween(start, end){
849     if(!start || !end) return '-';
850     const d = (new Date(end) - new Date(start)) / 86400000 + 1;
851     return d > 0 ? d : '-';
852 }
853
854 async function initUser(){
855     try {
856         let email = '', name = '';
857         const token = localStorage.getItem('jwt_token');
858         if(token){
859             try{ const p = JSON.parse(atob(token.split('.')[1])); email = p.sub || p.email || ''; }catch(e){}
860         }
861         try{
862             const me = await apiFetch('/employee/me');
863             if(me){
864                 name = me.name || email; email = me.email || email;
865                 document.getElementById('p-name').textContent = me.name || '-';
866                 document.getElementById('p-email').textContent = me.email || '-';
867                 document.getElementById('p-dept').textContent = me.dept || '-';
868                 document.getElementById('p-role').textContent = me.role || 'EMPLOYEE';
869                 document.getElementById('p-status').textContent = me.status || '-';
870                 document.getElementById('p-tz').textContent = me.timezone || '-';
871             }
872             }catch(e){ console.warn('Could not load profile'); }
873         const initials = (name || email || '?').charAt(0).toUpperCase();
874         document.getElementById('avatar-initials').textContent = initials;
875         document.getElementById('sidebar-name').textContent = name || email || 'Employee';
876         document.getElementById('sidebar-email').textContent = email;
877         document.getElementById('topbar-email').textContent = email;
878     }catch(e){ console.error(e); }
879     loadDashboard();
880 }
881
882 /* =====
883 VIEW SWITCHING
884 ===== */
885 function showView(name, el){
886     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
887     document.getElementById(`${name}-view`).classList.add('active');
888     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
889     if(el) el.classList.add('active');
890     document.getElementById('pageTitle').textContent =
891         name.split('-').map(w => w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
892
893     const loaders = {
894         dashboard: loadDashboard,
895         'my-leaves': loadMyLeaves,
896         'apply-leave': () => {},
897         'my-calendar': loadMyCalendar,
898         restaurants: loadRestaurants,
899         subscriptions: loadSubscriptions,
900         'add-subscription': () => {},
901         profile: () => {},

```

```

902         'my-devices':loadMyAssignedDevices,
903         'service-requests':loadServiceRequests
904     };
905     if(loaders[name]) loaders[name]();
906     if(name==='add-subscription') loadRestaurantChoices();
907 }
908
909 /* =====
910     DASHBOARD
911     ===== */
912     async function loadDashboard(){
913         const [leavesResult, dashboardResult] = await Promise.allSettled([
914             apiFetch('/leave_requests/employee'),
915             apiFetch('/api/employees/me/leave-dashboard')
916         ]);
917         const leavesPayload = leavesResult.status==='fulfilled' ? leavesResult.value : null;
918         const dashboard = dashboardResult.status==='fulfilled' ? dashboardResult.value : null;
919         const arr = asArray(leavesPayload?.requests || leavesPayload);
920         const pending = arr.filter(l=>l.status==='PENDING').length;
921         const approved = arr.filter(l=>l.status==='APPROVED').length;
922         const rejected = arr.filter(l=>l.status==='REJECTED').length;
923         document.getElementById('stat-pending').textContent=pending;
924         document.getElementById('stat-approved').textContent=approved;
925         document.getElementById('stat-rejected').textContent=rejected;
926         const balances = dashboard?.leaveBalances ? Object.values(dashboard.leaveBalances) : [];
927         const available = balances.reduce((t,b)=>t+Number(b.available||0),0);
928         document.getElementById('stat-balance').textContent=available;
929         const recent=arr.slice(-5).reverse();
930         document.getElementById('recent-leaves').innerHTML=recent.length
931             ? recent.map(l=>`<tr>
932                 <td><span class="chip ${l.leaveType||''}.toLowerCase()>${l.leaveType||'N/A'}</span></td>
933                 <td>${l.startDate||'-'}</td><td>${l.endDate||'-'}</td>
934                 <td>${l.reason||'-'}.substring(0,30}</td>
935                 <td><span class="chip ${l.status||''}.toLowerCase()>${l.status||'-'}</span></td>
936             </tr>`).join('')
937             : leavesResult.status==='rejected'
938             ? '<tr><td colspan="5" class="empty-state">Could not load leave data</td></tr>'
939             : '<tr><td colspan="5" class="empty-state">No leave requests yet</td></tr>';
940         try{
941             const subs = await apiFetch('/subscription/user');
942             const arr2 = subs || [];
943             document.getElementById('dashboard-sub').innerHTML = arr2.length
944                 ? arr2.filter(s=>s.status==='ACTIVE').slice(0,4).map(s=>`
945                 <div class="sub-item">
946                     <div class="meal-name">${s.restaurantName||'Restaurant'}</div>
947                     <div class="meal-meta">${s.mealSlot||''} · ${s.scheduleType||''}${s.dayOfWeek?' '
948                     <div class="meal-meta" style="margin-top:4px;color:var(--accent)">Next:
949                     ${s.nextDeliveryTime?new Date(s.nextDeliveryTime).toLocaleString():'N/A'}</div>
950                     </div>`).join('')
951                 : '<div style="padding:16px;color:var(--text3);font-style:italic">No active
952                 subscriptions</div>';
953             }catch(e){
954                 document.getElementById('dashboard-sub').innerHTML=`<div style="padding:16px;color:var(--
955                 text3)">Could not load subscriptions</div>`;
956             }
957         }
958
959 /* =====
960     MY LEAVES
961     ===== */
962     async function loadMyLeaves(){
963         try{
964             const data=await apiFetch('/leave_requests/employee');
965             const leaves=data?.requests||data||[];
966             document.getElementById('my-leaves-list').innerHTML=leaves.length
967                 ? leaves.map(l=>`<tr>
968                     <td><span class="chip
969                     ${l.leaveType||''}.toLowerCase()>${l.leaveType||'N/A'}</span></td>
970                     <td>${l.startDate||'-'}</td><td>${l.endDate||'-'}</td>
971                     <td>${l.numberOfDays||daysBetween(l.startDate,l.endDate)}</td>
972                     <td>${l.reason||'-'}.substring(0,30}</td>
973                     <td><span class="chip ${l.status||''}.toLowerCase()>${l.status}</span></td>
974                     <td>${l.status==='PENDING'?`<button class="btn btn-danger btn-sm"
975                     onclick="cancelLeave(${l.leaveRequestId})">Cancel</button>`:'-'}</td>
976                 </tr>`).join('')

```

```

972         : '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
973     }catch(e){
974         document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load leave requests</td></tr>';
975     }
976 }
977
978 async function applyLeave(){
979     const leaveType=document.getElementById('leave-type').value;
980     const startDate=document.getElementById('leave-start').value;
981     const endDate =document.getElementById('leave-end').value;
982     const reason =document.getElementById('leave-reason').value.trim();
983     if(!startDate||!endDate){showAlert('Please fill all required fields','error');return;}
984     try{
985         await
apiFetch('/leave_requests',{method:'POST',body:JSON.stringify({leaveType,startDate,endDate,reason})});
986         showAlert('Leave request submitted!','success');
987         clearLeaveForm();
988         loadDashboard();
989         loadMyLeaves();
990     }catch(e){showAlert('Failed to submit: '+e.message,'error');}
991 }
992 document.getElementById('leave-type').addEventListener('change', function(){
993     document.getElementById('sick-leave-hint').style.display =
994     this.value === 'SICK' ? 'block' : 'none';
995 });
996
997 async function cancelLeave(id){
998     if(!confirm('Cancel this leave request?')) return;
999     try{
1000         await apiFetch(`/leave_requests/cancel/${id}`, {method:'PUT'});
1001         showAlert('Request cancelled successfully','success');
1002         loadMyLeaves();
1003     }catch(e){
1004         if(e.message==='ACCESS_DENIED') showAlert('You do not have permission to cancel this
leave','error');
1005         else showAlert('Failed to cancel: '+e.message,'error');
1006     }
1007 }
1008
1009 function clearLeaveForm(){
1010     ['leave-start','leave-end','leave-reason'].forEach(id=>document.getElementById(id).value='');
1011 }
1012
1013 /* =====
1014 MY CALENDAR (NEW)
1015 ===== */
1016 async function loadMyCalendar(){
1017     // 1. Load balance cards
1018     await loadCalendarBalanceCards();
1019     // 2. Build or refresh FullCalendar
1020     if(!calendarInitialized){
1021         initFullCalendar();
1022     } else {
1023         await refreshCalendarEvents();
1024     }
1025 }
1026
1027 async function loadCalendarBalanceCards(){
1028     // Default state
1029     ['cal-sick-remaining','cal-casual-remaining','cal-mat-remaining','cal-total-used']
1030     .forEach(id=>{ document.getElementById(id).textContent='--'; });
1031
1032     try{
1033         const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1034         const balances = dashboard?.leaveBalances || {};
1035
1036         let totalUsed=0, totalAllowed=0;
1037
1038         // SICK
1039         const sick = balances['SICK'] || balances['sick'] || {};
1040         const sickRem = Number(sick.available || 0);
1041         const sickTotal = Number(sick.total || 0);
1042         const sickUsed = sickTotal - sickRem;
1043         document.getElementById('cal-sick-remaining').textContent = sickRem;
1044         document.getElementById('cal-sick-sub').textContent = `${sickUsed} used of ${sickTotal}

```

```

days`;
1045
1046      // CASUAL
1047      const cas = balances['CASUAL'] || balances['casual'] || {};
1048      const casRem = Number(cas.available || 0);
1049      const casTotal = Number(cas.total || 0);
1050      const casUsed = casTotal - casRem;
1051      document.getElementById('cal-casual-remaining').textContent = casRem;
1052      document.getElementById('cal-casual-sub').textContent = `${casUsed} used of ${casTotal}
days`;
1053
1054      // MATERNITY
1055      const mat = balances['MATERNITY'] || balances['maternity'] || {};
1056      const matRem = Number(mat.available || 0);
1057      const matTotal = Number(mat.total || 0);
1058      const matUsed = matTotal - matRem;
1059      document.getElementById('cal-mat-remaining').textContent = matRem;
1060      document.getElementById('cal-mat-sub').textContent = `${matUsed} used of ${matTotal}
days`;
1061
1062      // TOTAL
1063      Object.values(balances).forEach(b=>{
1064          totalUsed += (Number(b.total||0) - Number(b.available||0));
1065          totalAllowed += Number(b.total||0);
1066      });
1067      document.getElementById('cal-total-used').textContent = totalUsed;
1068      document.getElementById('cal-total-sub').textContent = `out of ${totalAllowed} days total`;
1069
1070      }catch(e){
1071          console.error('Could not load leave balance for calendar:', e);
1072          ['cal-sick-sub', 'cal-casual-sub', 'cal-mat-sub', 'cal-total-sub']
1073              .forEach(id=>{ document.getElementById(id).textContent='unavailable'; });
1074      }
1075  }
1076
1077  async function buildCalendarEvents(){
1078      const events = [];
1079      try{
1080          // - Employee leaves -
1081          const data = await apiFetch('/leave_requests/employee');
1082          const leaves = data?.requests || data || [];
1083          leaves.forEach(l=>{
1084              const s = (l.status||'PENDING').toUpperCase();
1085              const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
1086              const typeLabel = TYPE_LABEL[l.leaveType] || l.leaveType || 'Leave';
1087
1088              // FullCalendar end date is exclusive, so add 1 day for multi-day events
1089              let endDate = l.endDate;
1090              if(endDate){
1091                  const d = new Date(endDate);
1092                  d.setDate(d.getDate()+1);
1093                  endDate = d.toISOString().split('T')[0];
1094              }
1095
1096              events.push({
1097                  id : `leave-${l.leaveRequestId}`,
1098                  title : `${typeLabel} (${s.charAt(0)+s.slice(1).toLowerCase()})`,
1099                  start : l.startDate,
1100                  end : endDate,
1101                  allDay : true,
1102                  backgroundColor : c.bg,
1103                  borderColor : c.border,
1104                  textColor : c.text,
1105                  extendedProps : {
1106                      type : 'leave',
1107                      status : s,
1108                      reason : l.reason || '-',
1109                      leaveType: l.leaveType
1110                  }
1111              });
1112          });
1113      }catch(e){ console.warn('Could not load leaves for calendar:', e); }
1114
1115      try{
1116          // - Public holidays (admin-added) -
1117          const holidays = await apiFetch('/holidays');

```

```

1118         (holidays||[]).forEach(h=>{
1119             const c = STATUS_COLORS.HOLIDAY;
1120             events.push({
1121                 id      : `holiday-${h.id}|h.holidayId|Math.random()`,
1122                 title   : `${h.name}|h.holidayName|'Holiday'}`,
1123                 start   : h.date|h.holidayDate,
1124                 allDay   : true,
1125                 backgroundColor : c.bg,
1126                 borderColor : c.border,
1127                 textColor   : c.text,
1128                 extendedProps : { type:'holiday', description: h.description||'' }
1129             });
1130         });
1131     }catch(e){ console.warn('Could not load holidays for calendar:', e); }
1132
1133     return events;
1134 }
1135
1136 function initFullCalendar(){
1137     const calendarEl = document.getElementById('employee-calendar');
1138
1139     employeeCalendar = new FullCalendar.Calendar(calendarEl, {
1140         initialView: 'dayGridMonth',
1141         height      : 'auto',
1142         headerToolbar:{
1143             left : 'prev,next today',
1144             center: 'title',
1145             right : 'dayGridMonth,dayGridWeek,listMonth'
1146         },
1147         buttonText:{
1148             today : 'Today',
1149             month : 'Month',
1150             week  : 'Week',
1151             listMonth: 'List'
1152         },
1153         dayMaxEvents: 3,      // "+N more" link if overflowing
1154         navLinks      : true, // click day number day view
1155         events        : [],   // start empty, populate below
1156
1157         // Event hover tooltip
1158         eventMouseEnter: function(info){
1159             const ep = info.event.extendedProps;
1160             const tooltip = document.getElementById('cal-tooltip');
1161             let html = `<div class="tt-title">${info.event.title}</div>`;
1162             if(ep.type==='leave'){
1163                 html += `<div class="tt-row"> ${info.event.startStr}
1164 ${info.event.endStr}|info.event.startStr}</div>`;
1165                 if(ep.reason && ep.reason!=='-')
1166                     html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
1167                 html += `<div class="tt-row">Status: <strong
1168 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
1169             } else {
1170                 if(ep.description) html += `<div class="tt-row">${ep.description}</div>`;
1171             }
1172             tooltip.innerHTML = html;
1173             tooltip.style.display = 'block';
1174         },
1175         eventMouseLeave: function(){
1176             document.getElementById('cal-tooltip').style.display='none';
1177         },
1178         eventClick: function(info){
1179             // Click navigates to leave history
1180             const ep = info.event.extendedProps;
1181             if(ep.type==='leave'){
1182                 showView('my-leaves', document.querySelector('.nav-item:nth-child(4)'));
1183             }
1184         },
1185     });
1186
1187     employeeCalendar.render();
1188     calendarInitialized = true;
1189
1190     // Load events asynchronously after render
1191     refreshCalendarEvents();
1192 }

```

```

1192     async function refreshCalendarEvents(){
1193         if(!employeeCalendar) return;
1194         const events = await buildCalendarEvents();
1195         // Remove all existing events and re-add
1196         employeeCalendar.removeAllEvents();
1197         events.forEach(e=>employeeCalendar.addEvent(e));
1198     }
1199
1200     async function refreshCalendar(){
1201         await loadCalendarBalanceCards();
1202         await refreshCalendarEvents();
1203         showAlert('Calendar refreshed!', 'success');
1204     }
1205
1206     // Move tooltip with mouse
1207     document.addEventListener('mousemove', function(e){
1208         const tt = document.getElementById('cal-tooltip');
1209         if(tt.style.display==='block'){
1210             tt.style.left = (e.clientX+14)+'px';
1211             tt.style.top = (e.clientY+14)+'px';
1212             // Keep tooltip on screen
1213             const rect = tt.getBoundingClientRect();
1214             if(rect.right > window.innerWidth)
1215                 tt.style.left = (e.clientX - rect.width - 10)+'px';
1216             if(rect.bottom > window.innerHeight)
1217                 tt.style.top = (e.clientY - rect.height - 10)+'px';
1218         }
1219     });
1220
1221     /* =====
1222     RESTAURANTS
1223     ===== */
1224     async function loadRestaurants(){
1225         try{
1226             const data=await apiFetch('/restaurants');
1227             availableRestaurants=data||[];
1228             document.getElementById('restaurants-list').innerHTML=data?.length
1229                 ? data.map(r=>`<tr>
1230                     <td>${r.name||'-'}</td>
1231                     <td>${r.supportedMealSlots?.join(', ')}||'-'}</td>
1232                     <td><span class="chip
1233 ${r.active?'active':'rejected'}>${r.active?'Active':'Inactive'}</span></td>
1234                     <td><button class="btn btn-secondary btn-sm"
1235 onClick='openSubscriptionForRestaurant(${r.restaurantId||r.id},${JSON.stringify(r.supportedMealSlots||[])}>
1236                     Subscribe</button></td>
1237                     </tr>`).join('')
1238                 : '<tr><td colspan="4" class="empty-state">No restaurants available</td></tr>';
1239             }catch(e){
1240                 document.getElementById('restaurants-list').innerHTML=`<tr><td colspan="4" class="empty-
1241 state">Could not load restaurants</td></tr>`;
1242             }
1243         }
1244     }
1245
1246     async function loadRestaurantChoices(){
1247         const select=document.getElementById('sub-restaurant-id');
1248         try{
1249             const data=await apiFetch('/restaurants');
1250             availableRestaurants=data||[];
1251             select.innerHTML=availableRestaurants.length
1252                 ? '<option value="">Select a restaurant</option>'+
1253                 availableRestaurants.map(r=>`<option
1254 value="${r.restaurantId||r.id}">${r.name}</option>`).join('')
1255                 : '<option value="">No restaurants available</option>';
1256             }catch(e){ select.innerHTML=`<option value="">Could not load restaurants</option>`; }
1257         }
1258     }
1259
1260     function openSubscriptionForRestaurant(restaurantId, supportedMealSlots){
1261         showView('add-subscription', null);
1262         if(supportedMealSlots?.length) document.getElementById('sub-meal-
1263 slot').value=supportedMealSlots[0];
1264         loadRestaurantChoices().then(()=>{ document.getElementById('sub-restaurant-
1265 id').value=String(restaurantId); });
1266     }
1267
1268     /* =====

```

```

1262     SUBSCRIPTIONS
1263     ===== */
1264     async function loadSubscriptions(){
1265         try{
1266             const data=await apiFetch('/subscription/user');
1267             const arr=data||[];
1268             document.getElementById('subscriptions-list').innerHTML=arr.length
1269                 ? arr.map(s=><tr>
1270                     <td>${s.restaurantName||s.restaurantId||'-'}</td>
1271                     <td>${s.mealsSlot||'-'}</td>
1272                     <td>${s.scheduleType||'-'}${s.dayOfWeek?' . '+s.dayOfWeek:'}</td>
1273                     <td>${s.nextDeliveryTime?new Date(s.nextDeliveryTime).toLocaleString():'-'}</td>
1274                     <td><span class="chip
1275     ${s.status||''}.toLowerCase())">${s.status||'ACTIVE'}</span></td>
1276                     <td><button class="btn btn-danger btn-sm"
1277     onclick="cancelSub(${s.subscriptionId})">Cancel</button></td>
1278                 </tr>`).join('')
1279                 : '<tr><td colspan="6" class="empty-state">No subscriptions found</td></tr>';
1280             }catch(e){
1281                 document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6" class="empty-
1282     state">Could not load subscriptions</td></tr>';
1283             }
1284         }
1285     });
1286     document.getElementById('sub-meal-slot').addEventListener('change', loadRestaurantChoices);
1287
1288     async function addSubscription(){
1289         const restaurantId=parseInt(document.getElementById('sub-restaurant-id').value);
1290         const mealsSlot =document.getElementById('sub-meal-slot').value;
1291         const scheduleType=document.getElementById('sub-schedule-type').value;
1292         const dayOfWeek =document.getElementById('sub-day-of-week')?.value;
1293         if(!restaurantId||!mealsSlot){showAlert('Please fill all fields','error');return;}
1294         const body={scheduleType,mealsSlots:[mealsSlot],slotOrders:[{mealSlot,restaurantId}]};
1295         if(scheduleType==='WEEKLY'&&dayOfWeek) body.dayOfWeek=dayOfWeek;
1296         try{
1297             await apiFetch('/subscription',{method:'POST',body:JSON.stringify(body)});
1298             showAlert('Subscribed successfully!','success');
1299             loadSubscriptions(); loadDashboard();
1300         }catch(e){ showAlert('Subscription failed: '+e.message,'error'); }
1301     }
1302
1303     async function cancelSub(id){
1304         if(!confirm('Cancel this subscription?')) return;
1305         try{
1306             await apiFetch(`/subscription/${id}/expire`,{method:'PUT'});
1307             showAlert('Subscription cancelled','success');
1308             loadSubscriptions();
1309         }catch(e){ showAlert('Failed to cancel: '+e.message,'error'); }
1310     }
1311
1312     /* =====
1313     MY DEVICES
1314     ===== */
1315     async function loadMyAssignedDevices(){
1316         try{
1317             const devices=await apiFetch('/devices/my');
1318             const tbody=document.getElementById('my-devices-list');
1319             if(!devices||!devices.length){
1320                 tbody.innerHTML='<tr><td colspan="7" class="empty-state">No devices assigned to
1321     you</td></tr>';
1322                 return;
1323             }
1324             const today=new Date();
1325             tbody.innerHTML=devices.map(d=>{
1326                 const we=d.warrantyExpiryDate?new Date(d.warrantyExpiryDate):null;
1327                 const wd=we?(we>today
1328                     ?`<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
1329                     :`<span style="color:var(--red);font-size:11px"> Expired
1330     ${d.warrantyExpiryDate}</span>`)
1331                 :`<span style="color:var(--text3);font-size:11px">—</span>`;
1332                 return `<tr>
1333                     <td>#${d.id}</td>
1334                     <td><strong>${d.deviceName||'-'}</strong></td>

```



```

1333         <td>${d.brand||'-'}</td>
1334         <td><span class="chip">${(d.deviceType||'').toLowerCase()}</span></td>
1335         <td>${wd}</td>
1336         <td style="font-size:12px;color:var(--text2)">${d.assignedDate||'-'}</td>
1337         <td><span class="chip
${(d.deviceStatus||'').toLowerCase()}>${d.deviceStatus||'-'}</span></td>
1338     </tr>`;
1339     }).join('');
1340     }catch(e){ console.error(e); }
1341 }
1342
1343 /* =====
1344     SERVICE REQUESTS
1345     ===== */
1346     async function loadServiceRequests(){
1347         await Promise.all([loadServiceRequestsList(),loadMyDevicesForSR()]);
1348     }
1349
1350     async function loadServiceRequestsList(){
1351         try{
1352             const requests=await apiFetch('/service-requests/my');
1353             const tbody=document.getElementById('service-requests-list');
1354             if(!requests||!requests.length){
1355                 tbody.innerHTML=<tr><td colspan="7" class="empty-state">No service requests
yet</td></tr>`;
1356                 return;
1357             }
1358             tbody.innerHTML=requests.map(r=>`<tr>
1359                 <td style="color:var(--text3);font-family:var(--mono)">#${r.id}</td>
1360                 <td>${r.deviceName||'-'}</td>
1361                 <td><span class="chip pending">${r.requestType}</span></td>
1362                 <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap">${r.issueDescription||'-'}</td>
1363                 <td>${r.urgent?'<span class="chip rejected">YES</span>':'<span style="color:var(--
text3)">No</span>'}</td>
1364                 <td><span class="chip ${((r.status||'').toLowerCase())}">${r.status||'-'}</span></td>
1365                 <td style="font-size:12px;color:var(--text3)">${r.raisedAt?new
Date(r.raisedAt).toLocaleDateString():'-'}</td>
1366                 </tr>`).join('');
1367             }catch(e){ console.error(e); }
1368         }
1369
1370     async function loadMyDevicesForSR(){
1371         try{
1372             const devices=await apiFetch('/devices/my');
1373             const select=document.getElementById('sr-device-select');
1374             select.innerHTML=<option value="">-- Select your device --</option>`;
1375             (devices||[]).forEach(d=>{
1376                 const opt=document.createElement('option');
1377                 opt.value=d.id;
1378                 opt.textContent=`${d.deviceName} (${d.brand||d.deviceType||'Device'})`;
1379                 select.appendChild(opt);
1380             });
1381         }catch(e){ console.error('Failed to load devices for SR',e); }
1382     }
1383
1384     async function submitServiceRequest(){
1385         const deviceId      =parseInt(document.getElementById('sr-device-select').value);
1386         const requestType    =document.getElementById('sr-type').value;
1387         const issueDescription=document.getElementById('sr-description').value.trim();
1388         const urgent         =document.getElementById('sr-urgent').checked;
1389         if(!deviceId){showAlert('Please select a device','error');return;}
1390         if(!issueDescription){showAlert('Please describe the issue','error');return;}
1391         try{
1392             await apiFetch('/service-
requests',{method:'POST',body:JSON.stringify({deviceId,requestType,issueDescription,urgent})});
1393             showAlert('Service request submitted!','success');
1394             clearServiceRequestForm();
1395             loadServiceRequestsList();
1396         }catch(e){ showAlert('Failed: '+e.message,'error'); }
1397     }
1398
1399     function clearServiceRequestForm(){
1400         document.getElementById('sr-device-select').value='';
1401         document.getElementById('sr-type').value='REPAIR';
1402         document.getElementById('sr-description').value='';

```

```

1403     document.getElementById('sr-urgent').checked=false;
1404 }
1405
1406 /* =====
1407     LOGOUT + ALERT
1408     ===== */
1409     function logout(){
1410         localStorage.removeItem('jwt_token');
1411         localStorage.removeItem('refresh_token');
1412         localStorage.removeItem('auth_type');
1413         localStorage.removeItem('last_role');
1414         fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>{
1415             document.cookie='JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1416             window.location.href='/google.html';
1417         });
1418     }
1419     async function loadNotifCount() {
1420         try {
1421             const data = await apiFetch('/notifications/unread-count');
1422             const badge = document.getElementById('notif-badge');
1423             if (data && data.count > 0) {
1424                 badge.textContent = data.count;
1425                 badge.style.display = 'flex';
1426             } else {
1427                 badge.style.display = 'none';
1428             }
1429         } catch(e) {
1430             console.warn('Could not load notification count:', e);
1431         }
1432     }
1433
1434     async function toggleNotifPanel() {
1435         const panel = document.getElementById('notif-panel');
1436         if (panel.style.display === 'none') {
1437             panel.style.display = 'block';
1438             try {
1439                 await loadNotifications();
1440                 await apiFetch('/notifications/mark-read', { method: 'POST' });
1441                 document.getElementById('notif-badge').style.display = 'none';
1442             } catch(e) {
1443                 console.warn('Notification panel error:', e);
1444             }
1445         } else {
1446             panel.style.display = 'none';
1447         }
1448     }
1449
1450     async function loadNotifications() {
1451         try {
1452             const data = await apiFetch('/notifications');
1453             const list = document.getElementById('notif-list');
1454             if (!data || data.length === 0) {
1455                 list.innerHTML = '<div class="notif-item">No notifications yet.</div>';
1456                 return;
1457             }
1458             list.innerHTML = data.map(n => `
1459                 <div class="notif-item ${n.read ? '' : 'unread'}">
1460                     <div>${n.message}</div>
1461                     <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
1462                 </div>
1463             `).join('');
1464         } catch(e) {
1465             document.getElementById('notif-list').innerHTML =
1466                 '<div class="notif-item">Could not load notifications.</div>';
1467             console.warn('loadNotifications failed:', e);
1468         }
1469     }
1470
1471     // Load badge count on page load
1472     loadNotifCount();
1473     // Refresh count every 30 seconds
1474     setInterval(loadNotifCount, 30000);
1475
1476     // Close panel when clicking outside
1477     document.addEventListener('click', e => {
1478         const panel = document.getElementById('notif-panel');

```

```
1479     const wrapper = document.querySelector('.notif-wrapper');
1480     if (!panel.contains(e.target) && !wrapper.contains(e.target)) {
1481         panel.style.display = 'none';
1482     }
1483 });
1484
1485     function showAlert(msg,type){
1486         const el=document.createElement('div');
1487         el.className=`alert alert-${type}`;
1488         el.textContent=msg;
1489         document.body.appendChild(el);
1490         setTimeout(()=>el.remove(),3000);
1491     }
1492 </script>
1493 </body>
1494 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Sign In</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#070910;
12             --panel:#0e1118;
13             --panel2:#131821;
14             --border:rgba(255,255,255,0.07);
15             --border-lit:rgba(255,255,255,0.14);
16             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
17             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
18             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.12);
19             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
20             --orange:#f59e0b;--orange-dim:rgba(245,158,11,0.12);
21             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
22             --font:'Outfit', sans-serif;--mono:'JetBrains Mono', monospace;
23         }
24
25         body{
26             font-family:var(--font);
27             background:var(--bg);
28             color:var(--text);
29             min-height:100vh;
30             display:flex;
31             overflow-y:auto;
32         }
33
34         /* ---- BACKGROUND ---- */
35         .bg-layer{
36             position:fixed;inset:0;z-index:0;
37             background:
38                 radial-gradient(ellipse 60% 50% at 20% 50%, rgba(79,140,255,0.06) 0%, transparent 70%),
39                 radial-gradient(ellipse 40% 60% at 80% 20%, rgba(167,139,250,0.05) 0%, transparent 60%),
40                 radial-gradient(ellipse 50% 40% at 70% 80%, rgba(52,211,153,0.04) 0%, transparent 60%);
41         }
42         .grid-overlay{
43             position:fixed;inset:0;z-index:0;
44             background-image:
45                 linear-gradient(rgba(255,255,255,0.025) 1px, transparent 1px),
46                 linear-gradient(90deg, rgba(255,255,255,0.025) 1px, transparent 1px);
47             background-size:40px 40px;
48             mask-image:radial-gradient(ellipse 80% 80% at center, black 20%, transparent 80%);
49         }
50
51         #oauth-loading {
52             display: none;
53             position: fixed; inset: 0;
54             background: linear-gradient(135deg, #0d0f14 0%, #131720 100%);
55             z-index: 9999;
56             justify-content: center;
57             align-items: center;
58             flex-direction: column;
59             gap: 20px;
60         }
61         #oauth-loading.visible { display: flex; }
62         .spinner {
63             width: 48px; height: 48px;
64             border: 3px solid #2a2f3f;
65             border-top-color: #4f8cff;
66             border-radius: 50%;
67             animation: spin 0.9s linear infinite;
68         }
69         @keyframes spin { to { transform: rotate(360deg); } }
70

```

```

71      /* ---- SPLIT LAYOUT ---- */
72      .left-panel{
73          flex:1;display:flex;flex-direction:column;justify-content:center;
74          padding:60px;position:relative;z-index:1;
75          animation:fadeLeft 0.8s ease both;
76      }
77      @keyframes fadeLeft{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
78      .brand{margin-bottom:48px}
79      .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:20px}
80      .brand-icon{
81          width:44px;height:44px;background:linear-gradient(135deg,#4f8cff,#6366f1);
82          border-radius:12px;display:flex;align-items:center;justify-content:center;
83          font-weight:700;font-size:18px;color:#fff;
84          box-shadow:0 0 30px rgba(79,140,255,0.35);
85          flex-shrink:0;
86      }
87      .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
88      .brand-tagline{font-size:13px;color:var(--text2)}
89      .hero-text h1{
90          font-size:clamp(36px,4vw,52px);font-weight:700;
91          line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;
92      }
93      .hero-text h1 .hl-blue{color:var(--blue)}
94      .hero-text h1 .hl-violet{color:var(--violet)}
95      .hero-text h1 .hl-orange{color:var(--orange)}
96      .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:380px}
97      .feature-pills{display:flex;flex-wrap:wrap;gap:8px;margin-top:32px}
98      .pill{
99          display:inline-flex;align-items:center;gap:6px;
100         padding:6px 14px;border-radius:20px;font-size:12px;font-weight:500;
101         border:1px solid var(--border);background:var(--panel);color:var(--text2);
102     }
103     .pill .dot{width:6px;height:6px;border-radius:50%;flex-shrink:0}
104     .pill.blue .dot{background:var(--blue)} .pill.blue{border-
color:rgba(79,140,255,0.2);color:rgba(79,140,255,0.8)}
105     .pill.violet .dot{background:var(--violet)} .pill.violet{border-
color:rgba(167,139,250,0.2);color:rgba(167,139,250,0.8)}
106     .pill.orange .dot{background:var(--orange)} .pill.orange{border-
color:rgba(245,158,11,0.2);color:rgba(245,158,11,0.8)}
107
108     /* ---- RIGHT PANEL (FORM) ---- */
109     .right-panel{
110         width:460px;flex-shrink:0;
111         display:flex;align-items:flex-start;justify-content:center;
112         overflow-y:auto;
113         padding:32px 40px;position:relative;z-index:1;
114         border-left:1px solid var(--border);
115         background:rgba(14,17,24,0.8);
116         backdrop-filter:blur(20px);
117         animation:fadeRight 0.8s ease both;
118     }
119     @keyframes
fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
120     .form-box{width:100%}
121
122     /* ---- ROLE SELECTOR — 3 roles only: Employee | Manager | Admin ---- */
123     .role-label{font-size:10px;font-weight:600;color:var(--text3);letter-spacing:1.5px;text-
transform:uppercase;font-family:var(--mono);margin-bottom:10px}
124     .role-tabs{display:grid;grid-template-columns:repeat(3,1fr);gap:6px;margin-bottom:28px}
125     .role-tab{
126         padding:10px 8px;border-radius:10px;border:1.5px solid var(--border);
127         background:var(--bg);color:var(--text2);font-family:var(--font);
128         font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s;
129         display:flex;flex-direction:column;align-items:center;gap:4px;
130     }
131     .role-tab .role-icon{font-size:16px}
132     .role-tab:hover{border-color:var(--border-lit);color:var(--text)}
133     .role-tab.active-emp{border-color:var(--blue);background:var(--blue-dim);color:var(--blue)}
134     .role-tab.active-mgr{border-color:var(--violet);background:var(--violet-dim);color:var(--violet)}
135     .role-tab.active-adm{border-color:var(--orange);background:var(--orange-dim);color:var(--orange)}
136
137     /* ---- AUTH TABS ---- */
138     .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-
bottom:24px;border:1px solid var(--border)}
139     .auth-tab{
140         flex:1;padding:9px;border:none;background:transparent;

```

```

141         color:var(--text2);font-family:var(--font);font-weight:600;
142         font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;
143     }
144     .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}
145
146     /* ---- FORM ELEMENTS ---- */
147     .form-group{margin-bottom:16px}
148     .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-
transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
149     .form-input{
150         width:100%;padding:11px 14px;
151         background:var(--bg);border:1px solid var(--border);
152         border-radius:9px;color:var(--text);
153         font-family:var(--font);font-size:14px;
154         transition:all 0.2s;
155     }
156     .form-input:focus{outline:none;border-color:var(--blue);box-shadow:0 0 3px rgba(79,140,255,0.08)}
157     .form-input::placeholder{color:var(--text3)}
158     .form-input.focus-violet:focus{border-color:var(--violet);box-shadow:0 0 3px
159     rgba(167,139,250,0.08)}
160     .form-input.focus-orange:focus{border-color:var(--orange);box-shadow:0 0 3px
161     rgba(245,158,11,0.08)}
162
163     /* ---- BUTTONS ---- */
164     .btn-login{
165         width:100%;padding:12px;border:none;border-radius:9px;
166         font-family:var(--font);font-size:14px;font-weight:600;
167         cursor:pointer;transition:all 0.2s;margin-top:4px;
168         position:relative;overflow:hidden;
169     }
170     .btn-login.blue{background:var(--blue);color:#fff}
171     .btn-login.blue:hover{background:#6ba0ff;box-shadow:0 6px 20px
172     rgba(79,140,255,0.35);transform:translateY(-1px)}
173     .btn-login.violet{background:var(--violet);color:#fff}
174     .btn-login.violet:hover{background:#b8a0fc;box-shadow:0 6px 20px
175     rgba(167,139,250,0.35);transform:translateY(-1px)}
176     .btn-login.orange{background:var(--orange);color:#fff}
177     .btn-login.orange:hover{background:#f7b23b;box-shadow:0 6px 20px
178     rgba(245,158,11,0.35);transform:translateY(-1px)}
179     .btn-login.active{transform:translateY(0)}
180     .btn-login.loading{opacity:0.7;pointer-events:none}
181
182     .divider{display:flex;align-items:center;gap:12px;margin:20px 0;color:var(--text3);font-
183     size:11px;font-family:var(--mono)}
184     .divider::before,.divider::after{content:'';flex:1;height:1px;background:var(--border)}
185
186     .btn-google{
187         width:100%;padding:11px;background:transparent;
188         border:1px solid var(--border-lit);border-radius:9px;
189         color:var(--text2);font-family:var(--font);font-size:13px;font-weight:500;
190         cursor:pointer;transition:all 0.2s;
191         display:flex;align-items:center;justify-content:center;gap:10px;
192     }
193     .btn-google:hover{background:rgba(255,255,255,0.04);border-color:rgba(255,255,255,0.2);color:var(--
194     text)}
195
196     /* ---- GOOGLE ROLE SELECT ---- */
197     .google-role-row{display:flex;gap:8px;margin-bottom:10px;align-items:center}
198     .google-role-label{font-size:11px;color:var(--text3);font-family:var(--mono);white-space:nowrap}
199     .google-role-select{
200         flex:1;padding:8px 10px;background:var(--bg);
201         border:1px solid var(--border);border-radius:8px;
202         color:var(--text);font-family:var(--font);font-size:13px;cursor:pointer;
203     }
204     .google-role-select:focus{outline:none;border-color:var(--border-lit)}
205     .google-role-select option{background:var(--panel)}
206
207     /* ---- VENDOR LINK ---- */
208     .vendor-link-row{
209         margin-top:18px;padding-top:16px;
210         border-top:1px solid var(--border);
211         display:flex;align-items:center;justify-content:space-between;
212     }
213     .vendor-link-row p{font-size:12px;color:var(--text3);}
214     .btn-vendor-link{
215         padding:7px 16px;border-radius:8px;

```

```

209         border:1px solid rgba(248,113,113,0.3);
210         background:var(--red-dim);color:var(--red);
211         font-family:var(--font);font-size:12px;font-weight:600;
212         cursor:pointer;text-decoration:none;transition:all 0.2s;
213         white-space:nowrap;
214     }
215     .btn-vendor-link:hover{background:rgba(248,113,113,0.2);border-color:var(--red);}
216
217     /* ---- ERROR / STATUS ---- */
218     .status-box{
219         border-radius:9px;padding:10px 14px;margin-top:14px;
220         font-size:13px;font-weight:500;display:none;
221         animation:fadeIn 0.3s ease;
222     }
223     @keyframes fadeIn{from{opacity:0;transform:translateY(-4px)}to{opacity:1;transform:translateY(0)}}
224     .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid
225     rgba(248,113,113,0.2)}
226     .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid
227     rgba(52,211,153,0.2)}
228     .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid
229     rgba(79,140,255,0.2)}
230
231     /* ---- FOOTER ---- */
232     .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
233     .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
234     .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
235     .tz-select{
236         flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);
237         border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;
238     }
239     .tz-select:focus{outline:none;border-color:var(--border-lit)}
240     .tz-select option{background:var(--panel)}
241     .tz-time{font-size:10px;color:var(--blue);font-family:var(--mono)}
242
243     /* ---- RESPONSIVE ---- */
244     @media(max-width:900px){
245         .left-panel{display:none}
246         .right-panel{width:100%;border-left:none;background:var(--bg)}
247     }
248 </style>
249 </head>
250 <body>
251 <div id="oauth-loading">
252 <div class="spinner"></div>
253 <div style="text-align: center;">
254 <p style="color: #e0e0e0; font-size: 16px; font-weight: 600; margin: 0 0 6px; font-family:
255 'Outfit', sans-serif;">Signing you in...</p>
256 <p style="color: #8b92a8; font-size: 13px; margin: 0; font-family: 'Outfit', sans-serif;">Verifying
257 your Google account</p>
258 </div>
259 </div>
260 <div class="bg-layer"></div>
261 <div class="grid-overlay"></div>
262
263 <!-- LEFT HERO -->
264 <div class="left-panel">
265 <div class="brand">
266 <div class="brand-logo">
267 <div class="brand-icon">E</div>
268 <div>
269 <div class="brand-name">EMS Portal</div>
270 <div class="brand-tagline">Employee Management System</div>
271 </div>
272 </div>
273 </div>
274 <div class="hero-text">
275 <h1>Your workspace,<br><span class="hl-blue">one login</span><br>away.</h1>
276 <p>Employees, Managers, and Admins each get their own tailored experience. Sign in to access your
277 personalized dashboard.</p>
278 </div>
279 <div class="feature-pills">
280 <div class="pill blue"><span class="dot"></span>Employee Portal</div>
281 <div class="pill violet"><span class="dot"></span>Manager Console</div>
282 <div class="pill orange"><span class="dot"></span>Admin Control Center</div>
283 <div class="pill blue"><span class="dot"></span>Leave Management</div>
284 <div class="pill violet"><span class="dot"></span>Meal Subscriptions</div>

```

```

279     <div class="pill orange"><span class="dot"></span>Delivery Tracking</div>
280   </div>
281 </div>
282
283 <!-- RIGHT FORM -->
284 <div class="right-panel">
285   <div class="form-box">
286
287     <!-- ROLE SELECTOR: 3 roles only – Vendor removed -->
288     <div class="role-label">Sign in as</div>
289     <div class="role-tabs">
290       <button class="role-tab active-emp" id="role-emp" onclick="selectRole('EMPLOYEE')">
291         <span class="role-icon"></span>Employee
292       </button>
293       <button class="role-tab" id="role-mgr" onclick="selectRole('MANAGER')">
294         <span class="role-icon"></span>Manager
295       </button>
296       <button class="role-tab" id="role-adm" onclick="selectRole('ADMIN')">
297         <span class="role-icon"></span>Admin
298       </button>
299     </div>
300
301     <!-- AUTH METHOD TABS -->
302     <div class="auth-tabs">
303       <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
304       <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
305     </div>
306
307     <!-- JWT FORM -->
308     <div id="jwt-form">
309       <div class="form-group">
310         <label class="form-label">Enter your Email</label>
311         <input class="form-input" id="jwt-username" type="text"
placeholder="your.email@company.com" autocomplete="off">
312       </div>
313       <div class="form-group">
314         <label class="form-label">Password</label>
315         <input class="form-input" id="jwt-password" type="password" placeholder="....."
autocomplete="current-password">
316       </div>
317       <button class="btn-login blue" id="jwt-btn" onclick="jwtLogin()">Sign In</button>
318     </div>
319
320     <!-- SESSION FORM -->
321     <div id="session-form" style="display:none">
322       <div class="form-group">
323         <label class="form-label">Enter your Email</label>
324         <input class="form-input" id="session-username" type="text"
placeholder="your.email@company.com">
325       </div>
326       <div class="form-group">
327         <label class="form-label">Password</label>
328         <input class="form-input" id="session-password" type="password" placeholder=".....">
329       </div>
330       <button class="btn-login blue" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
331     </div>
332
333     <!-- OAUTH – Vendor removed from Google role select -->
334     <div class="divider">or continue with</div>
335     <div class="google-role-row">
336       <span class="google-role-label">Sign in as</span>
337       <select class="google-role-select" id="oauth-role">
338         <option value="EMPLOYEE">Employee</option>
339         <option value="MANAGER">Manager</option>
340         <option value="ADMIN">Admin</option>
341       </select>
342     </div>
343     <button class="btn-google" onclick="googleLogin()">
344       <svg width="18" height="18" viewBox="0 0 24 24">
345         <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04
2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
346         <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71
1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
347         <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-
2.09V7.07H2.18c1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>

```



```

348         <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1
7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
349     </svg>
350     Continue with Google
351 </button>
352
353 <div class="status-box" id="status-box"></div>
354
355 <!-- Vendor Login Link -->
356 <div class="vendor-link-row">
357     <p>Are you a vendor?</p>
358     <a class="btn-vendor-link" href="/vendor-login.html">Vendor Login </a>
359 </div>
360
361 <!-- FOOTER -->
362 <div class="form-footer">
363     <div class="tz-row">
364         <span class="tz-label">TZ</span>
365         <select class="tz-select" id="tz-select" onchange="updateTz()">
366             <option value="Asia/Kolkata">India – IST UTC+5:30</option>
367             <option value="UTC">UTC</option>
368             <option value="America/New_York">New York – EST UTC-5</option>
369             <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
370             <option value="Europe/London">London – GMT UTC+0</option>
371             <option value="Europe/Paris">Paris – CET UTC+1</option>
372             <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
373             <option value="Asia/Dubai">Dubai – GST UTC+4</option>
374             <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
375             <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
376         </select>
377         <span class="tz-time" id="tz-time">--:--:--</span>
378     </div>
379 </div>
380
381 </div>
382 <!-- TOTP MODAL OVERLAY -->
383 <div id="totp-overlay" style="display:none;position:fixed;inset:0;z-
index:999;background:rgba(7,9,16,0.85);backdrop-filter:blur(8px);display:none;align-items:center;justify-
content:center;">
384     <div style="background:var(--panel);border:1px solid var(--border-lit);border-
radius:16px;padding:36px;width:340px;text-align:center;animation:fadeIn 0.25s ease;">
385         <div style="font-size:28px;margin-bottom:8px;"></div>
386         <h3 style="font-size:17px;font-weight:700;color:var(--text);margin-bottom:6px;">Two-Factor
Verification</h3>
387         <p style="font-size:13px;color:var(--text2);margin-bottom:24px;">Enter the 6-digit code from
your authenticator app.</p>
388         <input id="totp-code" class="form-input" type="text" inputmode="numeric" maxlength="6"
placeholder="000000"
389             style="text-align:center;font-size:22px;font-family:var(--mono);letter-
spacing:8px;padding:14px;"
390             oninput="this.value=this.value.replace(/\D/g, '')">
391         <div id="totp-status" style="display:none;margin-top:10px;font-size:13px;color:var(--
red);"></div>
392         <button id="totp-verify-btn" class="btn-login blue" style="margin-top:16px;"
onclick="submitTotp()">Verify</button>
393         <button onclick="cancelTotp()" style="margin-
top:10px;width:100%;padding:10px;border:none;background:transparent;color:var(--text2);font-family:var(--
font);font-size:13px;cursor:pointer...
394     </div>
395 </div>
396 </div>
397 </div>
398
399 <script>
400     const API = 'http://localhost:8080';
401     const GOOGLE_CLIENT_ID = '525324511556-s9eugd1o761n6cjje2nr60cv8ho6joq2.apps.googleusercontent.com';
402     const REDIRECT_URI = 'http://localhost:8080/google.html';
403
404     // Role dashboard mapping – VENDOR removed from this page
405     const DASHBOARDS = {
406         EMPLOYEE: '/employee-dashboard.html',
407         MANAGER: '/manager-dashboard.html',
408         ADMIN: '/dashboard.html',
409         USER: '/pending.html'
410     };
411
412     let currentRole = 'EMPLOYEE';

```

```

413
414     function buildAuthOptions(token, authType) {
415         const headers = {};
416         if (token) headers.Authorization = `Bearer ${token}`;
417         return {
418             headers,
419             credentials: authType === 'session' ? 'include' : 'same-origin'
420         };
421     }
422
423     async function resolveRoleFromServer(token, authType) {
424         const authOptions = buildAuthOptions(token, authType);
425
426         // Check employee/manager/admin endpoint – vendors are not expected here
427         const employeeResponse = await fetch(`${API}/employee/me`, authOptions);
428         if (employeeResponse.ok) {
429             const employee = await employeeResponse.json();
430             return employee.role || 'EMPLOYEE';
431         }
432
433         throw new Error('Unable to determine authenticated role');
434     }
435
436     async function redirectByRole(token, authType = localStorage.getItem('auth_type')) {
437         let role;
438
439         if (token) {
440             role = getRoleFromJWT(token);
441         } else {
442             try {
443                 const res = await fetch(`${API}/employee/me`, { credentials: 'include' });
444                 if (res.ok) {
445                     const emp = await res.json();
446                     role = emp.role || 'USER';
447                 } else {
448                     role = 'USER';
449                 }
450             } catch(e) {
451                 role = localStorage.getItem('last_role') || 'USER';
452             }
453         }
454
455         console.log('Final role for redirect:', role);
456         localStorage.setItem('last_role', role);
457
458         // Block ALL vendor types from using the main login page
459         if (role === 'FOOD_VENDOR' || role === 'TECH_VENDOR') {
460             localStorage.removeItem('jwt_token');
461             localStorage.removeItem('auth_type');
462             localStorage.removeItem('last_role');
463             // Reset button state
464             const jwtBtn = document.getElementById('jwt-btn');
465             const sesBtn = document.getElementById('session-btn');
466             if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
467             if (sesBtn) { sesBtn.classList.remove('loading'); sesBtn.textContent = 'Sign In with Session'; }
468             showStatus('error', 'Vendors must use the Vendor Login page. ');
469             return;
470         }
471
472         const destination = DASHBOARDS[role];
473         if (!destination) {
474             // Unknown role – don't redirect anywhere, just show error
475             localStorage.removeItem('jwt_token');
476             localStorage.removeItem('auth_type');
477             showStatus('error', 'Unknown role: ' + role + '. Please contact your administrator. ');
478             const jwtBtn = document.getElementById('jwt-btn');
479             if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
480             return;
481         }
482
483         console.log('Redirecting to:', destination);
484         window.location.href = destination;
485     }
486
487     // ---- Role selector (3 tabs: Employee | Manager | Admin) ----
488     function selectRole(role) {

```

```

489     currentRole = role;
490     document.getElementById('role-emp').className = 'role-tab';
491     document.getElementById('role-mgr').className = 'role-tab';
492     document.getElementById('role-adm').className = 'role-tab';
493
494     const classMap = { EMPLOYEE: 'active-emp', MANAGER: 'active-mgr', ADMIN: 'active-adm' };
495     const idMap = { EMPLOYEE: 'role-emp', MANAGER: 'role-mgr', ADMIN: 'role-adm' };
496     document.getElementById(idMap[role]).classList.add(classMap[role]);
497
498     document.getElementById('oauth-role').value = role;
499
500     const colorMap = { EMPLOYEE: 'blue', MANAGER: 'violet', ADMIN: 'orange' };
501     const col = colorMap[role];
502     document.getElementById('jwt-btn').className = `btn-login ${col}`;
503     document.getElementById('session-btn').className = `btn-login ${col}`;
504
505     document.querySelectorAll('.form-input').forEach(inp => {
506         inp.className = (`form-input focus-${col === 'blue' ? '' : col}`).trim();
507     });
508 }
509
510 // ---- Auth tab switching ----
511 function switchTab(type) {
512     document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
513     document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
514     document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
515     document.getElementById('tab-session').classList.toggle('active', type === 'session');
516     hideStatus();
517 }
518
519 // ---- Status display ----
520 function showStatus(type, msg) {
521     const box = document.getElementById('status-box');
522     box.className = `status-box ${type}`;
523     box.textContent = msg;
524     box.style.display = 'block';
525 }
526 function hideStatus() {
527     document.getElementById('status-box').style.display = 'none';
528 }
529
530 // ---- JWT Login ----
531 let _preAuthToken = null; // holds token between step 1 and step 2
532
533 async function jwtLogin() {
534     const username = document.getElementById('jwt-username').value.trim();
535     const password = document.getElementById('jwt-password').value;
536     if (!username || !password) {
537         showStatus('error', 'Please enter your username and password');
538         return;
539     }
540
541     const btn = document.getElementById('jwt-btn');
542     btn.classList.add('loading'); btn.textContent = 'Signing in...';
543     showStatus('loading', 'Authenticating...');
544
545     try {
546         const res = await fetch(`${API}/Authenticate`, {
547             method: 'POST',
548             headers: { 'Content-Type': 'application/json' },
549             body: JSON.stringify({ username, password })
550         });
551
552         if (res.ok) {
553             const data = await res.json();
554
555             // --- TOTP required: show step-2 modal ---
556             if (data.requiresTotp) {
557                 _preAuthToken = data.preAuthToken;
558                 btn.classList.remove('loading'); btn.textContent = 'Sign In';
559                 hideStatus();
560                 openTotpModal();
561                 return;
562             }
563
564             // --- No TOTP: issue JWT directly ---

```

```

565         localStorage.setItem('jwt_token', data.accessToken);
566         localStorage.setItem('refresh_token', data.refreshToken || '');
567         localStorage.setItem('auth_type', 'jwt');
568         localStorage.setItem('last_role', currentRole);
569         localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
570         showStatus('success', 'Login successful! Redirecting...');
571         setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
572
573     } else {
574         const err = await res.text();
575         showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
576         btn.classList.remove('loading'); btn.textContent = 'Sign In';
577     }
578 } catch (e) {
579     showStatus('error', 'Connection error: ' + e.message);
580     btn.classList.remove('loading'); btn.textContent = 'Sign In';
581 }
582 }
583
584 function openTotpModal() {
585     const overlay = document.getElementById('totp-overlay');
586     overlay.style.display = 'flex';
587     document.getElementById('totp-code').value = '';
588     document.getElementById('totp-status').style.display = 'none';
589     setTimeout(() => document.getElementById('totp-code').focus(), 100);
590 }
591
592 function cancelTotp() {
593     _preAuthToken = null;
594     document.getElementById('totp-overlay').style.display = 'none';
595 }
596
597 async function submitTotp() {
598     const code = document.getElementById('totp-code').value.trim();
599     if (code.length !== 6) {
600         document.getElementById('totp-status').textContent = 'Please enter a 6-digit code.';
601         document.getElementById('totp-status').style.display = 'block';
602         return;
603     }
604
605     const verifyBtn = document.getElementById('totp-verify-btn');
606     verifyBtn.classList.add('loading'); verifyBtn.textContent = 'Verifying...';
607     document.getElementById('totp-status').style.display = 'none';
608
609     try {
610         const res = await fetch(`${API}/Authenticate/totp`, {
611             method: 'POST',
612             headers: { 'Content-Type': 'application/json' },
613             body: JSON.stringify({ preAuthToken: _preAuthToken, code })
614         });
615
616         if (res.ok) {
617             const data = await res.json();
618             _preAuthToken = null;
619             document.getElementById('totp-overlay').style.display = 'none';
620             localStorage.setItem('jwt_token', data.accessToken);
621             localStorage.setItem('refresh_token', data.refreshToken || '');
622             localStorage.setItem('auth_type', 'jwt');
623             localStorage.setItem('last_role', currentRole);
624             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
625             showStatus('success', 'Verified! Redirecting...');
626             setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
627         } else {
628             const err = await res.json().catch(() => ({}));
629             document.getElementById('totp-status').textContent = err.error || 'Invalid code. Try
again.';
630             document.getElementById('totp-status').style.display = 'block';
631             document.getElementById('totp-code').value = '';
632             document.getElementById('totp-code').focus();
633         }
634     } catch (e) {
635         document.getElementById('totp-status').textContent = 'Connection error: ' + e.message;
636         document.getElementById('totp-status').style.display = 'block';
637     } finally {
638         verifyBtn.classList.remove('loading'); verifyBtn.textContent = 'Verify';
639     }

```

```

640     }
641
642     // ---- Session Login ----
643     async function sessionLogin() {
644         const username = document.getElementById('session-username').value.trim();
645         const password = document.getElementById('session-password').value;
646         if (!username || !password) { showStatus('error', 'Please enter your username and password');
return; }
647
648         const btn = document.getElementById('session-btn');
649         btn.classList.add('loading'); btn.textContent = 'Signing in...';
650         showStatus('loading', 'Authenticating...');
651
652         try {
653             const res = await fetch(`${API}/session/login`, {
654                 method: 'POST',
655                 headers: { 'Content-Type': 'application/json' },
656                 body: JSON.stringify({ username, password }),
657                 credentials: 'include'
658             });
659             if (res.ok) {
660                 localStorage.setItem('auth_type', 'session');
661                 localStorage.setItem('last_role', currentRole);
662                 localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
663                 showStatus('success', 'Login successful! Redirecting...');
664                 setTimeout(() => redirectByRole(null, 'session'), 600);
665             } else {
666                 const err = await res.json().catch(() => ({}));
667                 showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
668                 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
669             }
670         } catch(e) {
671             showStatus('error', 'Connection error: ' + e.message);
672             btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
673         }
674     }
675
676     // ---- Google OAuth – goes through backend /auth/google/init ----
677     function googleLogin() {
678         const role = document.getElementById('oauth-role').value;
679         const timezone = document.getElementById('tz-select').value;
680         window.location.href =
681             `/auth/google/init` +
682             `?role=${encodeURIComponent(role)}` +
683             `&timezone=${encodeURIComponent(timezone)}`;
684     }
685
686     function getRoleFromJWT(token) {
687         try {
688             const payload = JSON.parse(atob(token.split('.')[1]));
689             console.log('JWT payload:', payload);
690             const role = payload.role || 'USER';
691             // Strip ROLE_ prefix if accidentally present
692             return typeof role === 'string' ? role.replace(/^ROLE_/, '') : 'USER';
693         } catch (e) {
694             console.error('Failed to parse JWT:', e);
695             return 'USER';
696         }
697     }
698
699     // ---- Spring redirects back here with tokens as query params ----
700     async function handleOAuthCallback() {
701         const params = new URLSearchParams(window.location.search);
702         const accessToken = params.get('accessToken');
703         const refreshToken = params.get('refreshToken');
704
705         if (!accessToken) return;
706
707         window.history.replaceState({}, document.title, window.location.pathname);
708
709         localStorage.setItem('jwt_token', accessToken);
710         localStorage.setItem('refresh_token', refreshToken || '');
711         localStorage.setItem('auth_type', 'jwt');
712
713         const role = getRoleFromJWT(accessToken);
714         console.log('Authenticated role:', role);

```

```

715     localStorage.setItem('last_role', role);
716
717     showStatus('success', 'Signed in! Redirecting...');
718     setTimeout(() => {
719         window.location.href = DASHBOARDS[role] || '/google.html';
720     }, 800);
721 }
722
723 // ---- Timezone ----
724 function updateTz() {
725     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
726     refreshTzClock();
727 }
728 function refreshTzClock() {
729     const tz = document.getElementById('tz-select').value;
730     document.getElementById('tz-time').textContent =
731         new Date().toLocaleTimeString('en-GB', { timeZone: tz, hour12: false });
732 }
733
734 // ---- Enter key support ----
735 document.addEventListener('keydown', e => {
736     if (e.key !== 'Enter') return;
737     // If TOTP modal is open, submit TOTP
738     if (document.getElementById('totp-overlay').style.display === 'flex') {
739         submitTotp();
740         return;
741     }
742     if (document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
743     else sessionLogin();
744 });
745 // ---- Init ----
746 window.onload = () => {
747     const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
748     document.getElementById('tz-select').value = savedTz;
749     refreshTzClock();
750     setInterval(refreshTzClock, 1000);
751
752     const params = new URLSearchParams(window.location.search);
753     const accessToken = params.get('accessToken');
754     if (accessToken) {
755         document.getElementById('oauth-loading').classList.add('visible');
756         document.querySelector('.right-panel').style.display = 'none';
757         document.querySelector('.left-panel').style.display = 'none';
758         handleOAuthCallback();
759     }
760 };
761 </script>
762 </body>
763 </html>
764

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Leave Management</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <style>
9          * { margin: 0; padding: 0; box-sizing: border-box; }
10         :root {
11             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
12             --panel: #1e2436; --panel2: #252c40;
13             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
14             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
15             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
16             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
17             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
18             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
19             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
20             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
21         }
22         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
23         .app { display: flex; height: 100vh; }
24
25         /* Sidebar */
26         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
27         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
28         .logo h2 { font-size: 20px; color: var(--text); }
29         .logo span { color: var(--accent); }
30         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
31         .nav-section { padding: 12px 20px 4px; font-size: 10px; font-weight: 600; color: var(--text2); text-
transform: uppercase; letter-spacing: 0.08em; }
32         .nav-item { padding: 10px 20px; margin: 2px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; font-size: 1...
33         .nav-item svg { opacity: 0.6; flex-shrink: 0; }
34         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
35         .nav-item:hover svg, .nav-item.active svg { opacity: 1; }
36         .nav-item .badge-count { margin-left: auto; background: var(--amber-dim); color: var(--amber); font-
size: 10px; padding: 2px 6px; border-radius: 10px; font-weight: 600; }
37
38         /* Main */
39         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
40         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
41         .page-title { font-size: 20px; font-weight: 600; }
42         .topbar-right { display: flex; align-items: center; gap: 16px; }
43         .user-pill { display: flex; align-items: center; gap: 8px; background: var(--bg3); border: 1px solid
var(--border2); border-radius: 20px; padding: 6px 12px; }
44         .user-avatar { width: 24px; height: 24px; border-radius: 50%; background: var(--accent-dim);
display: flex; align-items: center; justify-content: center; font-size: 10px; font-weight: 600; ...
45         .user-name { font-size: 12px; color: var(--text2); }
46         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; font-family: var(--font); font-
...
47         .logout-btn:hover { background: rgba(248,113,113,0.2); }
48         .content { flex: 1; overflow-y: auto; padding: 24px; }
49
50         /* Views */
51         .view { display: none; }
52         .view.active { display: block; }
53
54         /* Stats */
55         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
56         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); position: relative; overflow: hidden; }
57         .stat-card::before { content: ''; position: absolute; top: 0; left: 0; right: 0; height: 2px; }
58         .stat-card.blue::before { background: var(--accent); }

```

```

59     .stat-card.green::before { background: var(--green); }
60     .stat-card.amber::before { background: var(--amber); }
61     .stat-card.red::before { background: var(--red); }
62     .stat-label { font-size: 11px; color: var(--text2); text-transform: uppercase; letter-spacing:
0.05em; margin-bottom: 8px; }
63     .stat-value { font-size: 28px; font-weight: 600; }
64     .stat-value.blue { color: var(--accent); }
65     .stat-value.green { color: var(--green); }
66     .stat-value.amber { color: var(--amber); }
67     .stat-value.red { color: var(--red); }
68
69     /* Section header */
70     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; flex-wrap: wrap; gap: 10px; }
71     .section-title { font-size: 16px; font-weight: 600; }
72     .section-actions { display: flex; gap: 8px; align-items: center; }
73
74     /* Table */
75     .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow: hidden; }
76     table { width: 100%; border-collapse: collapse; }
77     th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text2); font-size:
11px; font-weight: 600; text-transform: uppercase; letter-spacing: 0.04em; border-bottom: ...
78     td { padding: 12px 16px; border-top: 1px solid var(--border); font-size: 13px; }
79     tr:hover td { background: rgba(255,255,255,0.02); }
80     .empty-state { text-align: center; padding: 48px; color: var(--text2); font-size: 13px; }
81
82     /* Chips */
83     .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-
block; }
84     .chip.pending, .chip.PENDING { background: var(--amber-dim); color: var(--amber); }
85     .chip.approved, .chip.APPROVED { background: var(--green-dim); color: var(--green); }
86     .chip.rejected, .chip.REJECTED { background: var(--red-dim); color: var(--red); }
87     .chip.cancelled, .chip.CANCELLED { background: rgba(139,146,168,0.12); color: var(--text2); }
88     .chip.sick, .chip.SICK { background: var(--red-dim); color: var(--red); }
89     .chip.casual, .chip.CASUAL { background: var(--accent-dim); color: var(--accent); }
90     .chip.maternity, .chip.MATERNITY { background: var(--violet-dim); color: var(--violet); }
91
92     /* Buttons */
93     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
94     .btn-primary { background: var(--accent); color: white; }
95     .btn-primary:hover { background: #6ba0ff; }
96     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
97     .btn-secondary:hover { color: var(--text); }
98     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
99     .btn-danger:hover { background: rgba(248,113,113,0.2); }
100    .btn-success { background: var(--green-dim); color: var(--green); border: 1px solid
rgba(54,211,153,0.3); }
101    .btn-success:hover { background: rgba(54,211,153,0.2); }
102    .btn-sm { padding: 4px 10px; font-size: 11px; }
103    .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
104
105    /* Form */
106    .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 640px; }
107    .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
108    .form-group { display: flex; flex-direction: column; gap: 6px; }
109    .form-group.full { grid-column: 1 / -1; }
110    label { font-size: 11px; font-weight: 600; color: var(--text2); text-transform: uppercase; letter-
spacing: 0.04em; }
111    input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; tr...
112    input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
113    input[readonly] { opacity: 0.5; cursor: not-allowed; }
114    select option { background: var(--bg3); }
115
116    /* Balance cards */
117    .balance-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap:
16px; margin-bottom: 24px; }
118    .balance-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 20px; }
119    .balance-name { font-size: 12px; color: var(--text2); text-transform: uppercase; letter-spacing:
0.05em; margin-bottom: 4px; }
120    .balance-row { display: flex; align-items: baseline; gap: 6px; margin-bottom: 10px; }

```



```

121     .balance-big { font-size: 28px; font-weight: 600; }
122     .balance-of { font-size: 13px; color: var(--text2); }
123     .balance-bar { height: 4px; background: var(--border2); border-radius: 2px; overflow: hidden; }
124     .balance-fill { height: 100%; border-radius: 2px; transition: width 0.4s ease; }
125
126     /* Tabs */
127     .tab-bar { display: flex; gap: 2px; background: var(--bg3); border-radius: 8px; padding: 4px;
margin-bottom: 20px; width: fit-content; }
128     .tab-btn { padding: 7px 16px; border-radius: 6px; font-size: 12px; font-weight: 500; cursor:
pointer; color: var(--text2); border: none; background: transparent; font-family: var(--font); t...
129     .tab-btn.active { background: var(--panel); color: var(--text); }
130     .tab-btn:hover:not(.active) { color: var(--text); }
131
132     /* Modal */
133     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
134     .modal-overlay.open { display: flex; }
135     .modal { background: var(--panel); border-radius: 12px; border: 1px solid var(--border2); padding:
24px; width: 460px; max-width: 90%; }
136     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
137     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
138
139     /* Alert */
140     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; font-size: 13px; font-weight: 500; }
141     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid
rgba(54,211,153,0.3); }
142     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
143     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
144
145     /* Pending count indicator */
146     .pending-dot { width: 7px; height: 7px; border-radius: 50%; background: var(--amber); display:
inline-block; margin-left: 4px; animation: pulse 2s infinite; }
147     @keyframes pulse { 0%,100% { opacity: 1; } 50% { opacity: 0.4; } }
148 </style>
149 </head>
150 <body>
151 <div class="app">
152
153     <!-- Sidebar -->
154     <div class="sidebar">
155         <div class="logo">
156             <h2>E<span>M</span>S</h2>
157             <p>Leave Management</p>
158         </div>
159         <div style="flex:1; padding: 8px 0;">
160             <div class="nav-section">Overview</div>
161             <div class="nav-item active" onclick="showView('dashboard',this)">
162                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><rect x="3" y="3" width="7" height="7"/><rect x="14" y="3" width="7" height="7"/><...
163                 Dashboard
164             </div>
165             <div class="nav-section">My Leave</div>
166             <div class="nav-item" onclick="showView('apply',this)">
167                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><circle cx="12" cy="12" r="10"/><line x1="12" y1="8" x2="12" y2="16"/><line x1="8"...
168                 Apply for Leave
169             </div>
170             <div class="nav-item" onclick="showView('my-leaves',this)">
171                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M14 2H6a2 2 0 0 0 2 2V16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2V8z"/><polyline po...
172                 My Requests
173             </div>
174             <div class="nav-item" onclick="showView('balance',this)">
175                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/><line...
176                 Leave Balance
177             </div>
178             <div class="nav-section">Admin</div>
179             <div class="nav-item" id="nav-pending" onclick="showView('pending',this)">
180                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="9 11 12 14 22 4"/><path d="M21 12v7a2 2 0 0 1 2 2H5a2 2 0 0 1 2-...
181                 Pending Approvals
182                 <span class="badge-count" id="pending-count" style="display:none">0</span>

```

```

183         </div>
184         <div class="nav-item" onclick="showView('all-leaves',this)">
185             <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/>...
186             All Leave Requests
187         </div>
188     </div>
189     <div style="padding: 16px; border-top: 1px solid var(--border);">
190         <a href="/dashboard.html" style="color:var(--text2);font-size:12px;text-
decoration:none;display:flex;align-items:center;gap:8px;">
191             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="15 18 9 12 15 6"/></svg>
192             Back to Admin Console
193         </a>
194     </div>
195 </div>
196
197 <!-- Main -->
198 <div class="main-content">
199     <div class="topbar">
200         <div class="page-title" id="pageTitle">Dashboard</div>
201         <div class="topbar-right">
202             <div class="user-pill">
203                 <div class="user-avatar" id="user-initials">--</div>
204                 <span class="user-name" id="username-display">Loading...</span>
205             </div>
206             <button class="logout-btn" onclick="logout()">Logout</button>
207         </div>
208     </div>
209
210     <div class="content">
211
212         <!-- DASHBOARD VIEW -->
213         <div id="dashboard-view" class="view active">
214             <div class="stats-grid">
215                 <div class="stat-card blue"><div class="stat-label">Total Requests</div><div
class="stat-value blue" id="s-total">--</div></div>
216                 <div class="stat-card amber"><div class="stat-label">Pending</div><div class="stat-
value amber" id="s-pending">--</div></div>
217                 <div class="stat-card green"><div class="stat-label">Approved</div><div class="stat-
value green" id="s-approved">--</div></div>
218                 <div class="stat-card red"><div class="stat-label">Rejected</div><div class="stat-value
red" id="s-rejected">--</div></div>
219             </div>
220             <div class="section-head">
221                 <div class="section-title">Recent Leave Requests</div>
222                 <button class="btn btn-secondary btn-sm" onclick="loadDashboard()"> Refresh</button>
223             </div>
224             <div class="table-wrap">
225                 <table>
226                     <thead><tr><th>Employee ID</th><th>Leave Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th></tr></thead>
227                     <tbody id="recent-leaves-body"></tbody>
228                 </table>
229             </div>
230         </div>
231
232         <!-- APPLY LEAVE VIEW -->
233         <div id="apply-view" class="view">
234             <div class="section-title" style="margin-bottom:20px;">Apply for Leave</div>
235             <div class="form-card">
236                 <div class="form-grid">
237                     <div class="form-group full">
238                         <label>Your Email</label>
239                         <input type="email" id="leave-email" readonly
placeholder="your.email@company.com">
240                     </div>
241                     <div class="form-group">
242                         <label>Leave Type</label>
243                         <select id="leave-type">
244                             <option value="SICK">Sick Leave</option>
245                             <option value="CASUAL">Casual Leave</option>
246                             <option value="MATERNITY">Maternity Leave</option>
247                         </select>
248                     </div>
249                 </div>

```

```

250             <label>Priority</label>
251             <select id="leave-priority" style="color:var(--text2);">
252                 <option value="">– Optional –</option>
253                 <option value="NORMAL">Normal</option>
254                 <option value="URGENT">Urgent</option>
255             </select>
256         </div>
257         <div class="form-group">
258             <label>Start Date</label>
259             <input type="date" id="leave-start">
260         </div>
261         <div class="form-group">
262             <label>End Date</label>
263             <input type="date" id="leave-end">
264         </div>
265         <div class="form-group full">
266             <label>Reason</label>
267             <textarea id="leave-reason" rows="3" placeholder="Please provide a brief reason
for your leave request..."></textarea>
268         </div>
269     </div>
270     <div style="display:flex;gap:12px;margin-top:20px;">
271         <button class="btn btn-primary" onclick="applyLeave()">Submit Leave
Request</button>
272         <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
273     </div>
274 </div>
275 </div>
276
277 <!-- MY LEAVES VIEW -->
278 <div id="my-leaves-view" class="view">
279     <div class="section-head">
280         <div class="section-title">My Leave Requests</div>
281         <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
282     </div>
283     <div class="table-wrap">
284         <table>
285             <thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead>
286             <tbody id="my-leaves-list"></tbody>
287         </table>
288     </div>
289 </div>
290
291 <!-- LEAVE BALANCE VIEW -->
292 <div id="balance-view" class="view">
293     <div class="section-head">
294         <div class="section-title">Leave Balance</div>
295         <button class="btn btn-secondary btn-sm" onclick="loadBalance()"> Refresh</button>
296     </div>
297     <div class="balance-grid" id="balance-grid">
298         <div class="balance-card" style="color:var(--text2);text-
align:center;padding:40px;">Loading balance...</div>
299     </div>
300     <div class="section-head" style="margin-top:8px;">
301         <div class="section-title">Leave History</div>
302     </div>
303     <div class="table-wrap">
304         <table>
305             <thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th><th>Remarks</th></tr></thead>
306             <tbody id="balance-history-list"></tbody>
307         </table>
308     </div>
309 </div>
310
311 <!-- PENDING APPROVALS VIEW -->
312 <div id="pending-view" class="view">
313     <div class="section-head">
314         <div class="section-title">Pending Approvals</div>
315         <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">
Refresh</button>
316     </div>
317     <div class="table-wrap">
318         <table>
319             <thead><tr><th>ID</th><th>Employee

```

```

ID</th><th>Type</th><th>Start</th><th>End</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
320         <tbody id="pending-leaves-list"></tbody>
321     </table>
322 </div>
323 </div>
324
325 <!-- ALL LEAVES VIEW -->
326 <div id="all-leaves-view" class="view">
327     <div class="section-head">
328         <div class="section-title">All Leave Requests</div>
329         <div class="section-actions">
330             <select id="status-filter" onchange="filterAllLeaves()" style="padding:6px
10px;font-size:12px;border-radius:6px;">
331                 <option value="">All Statuses</option>
332                 <option value="PENDING">Pending</option>
333                 <option value="APPROVED">Approved</option>
334                 <option value="REJECTED">Rejected</option>
335                 <option value="CANCELLED">Cancelled</option>
336             </select>
337             <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">
Refresh</button>
338         </div>
339     </div>
340     <div class="table-wrap">
341         <table>
342             <thead><tr><th>ID</th><th>Employee
ID</th><th>Type</th><th>Start</th><th>End</th><th>Days</th><th>Status</th><th>Actions</th></tr></thead>
343             <tbody id="all-leaves-list"></tbody>
344         </table>
345     </div>
346 </div>
347
348 </div><!-- /content -->
349 </div><!-- /main-content -->
350 </div><!-- /app -->
351
352 <!-- Approval Modal -->
353 <div class="modal-overlay" id="action-modal">
354     <div class="modal">
355         <div class="modal-title" id="action-modal-title">Review Leave Request</div>
356         <input type="hidden" id="action-leave-id">
357         <input type="hidden" id="action-type">
358         <div class="form-group" style="margin-bottom:12px;">
359             <label>Remarks (optional)</label>
360             <textarea id="action-remarks" rows="3" placeholder="Add remarks for the
employee..."></textarea>
361         </div>
362         <div class="modal-actions">
363             <button class="btn btn-secondary" onclick="closeModal('action-modal')">Cancel</button>
364             <button class="btn btn-primary" id="action-submit-btn"
onclick="submitApproval()">Approve</button>
365         </div>
366     </div>
367 </div>
368
369 <!-- Cancel Modal -->
370 <div class="modal-overlay" id="cancel-modal">
371     <div class="modal">
372         <div class="modal-title">Cancel Leave Request</div>
373         <p style="color:var(--text2);font-size:13px;margin-bottom:16px;">Are you sure you want to cancel
this leave request? This action cannot be undone.</p>
374         <input type="hidden" id="cancel-leave-id">
375         <div class="form-group" style="margin-bottom:12px;">
376             <label>Reason for cancellation (optional)</label>
377             <textarea id="cancel-remarks" rows="2" placeholder="Reason..."></textarea>
378         </div>
379         <div class="modal-actions">
380             <button class="btn btn-secondary" onclick="closeModal('cancel-modal')">Keep Request</button>
381             <button class="btn btn-danger" onclick="confirmCancel()">Yes, Cancel</button>
382         </div>
383     </div>
384 </div>
385
386 <script>
387     const API = '';
388     let currentUser = null;

```

```

389     let allLeavesCache = [];
390
391     // Auth
392     function getAuthHeader() {
393         const token = localStorage.getItem('jwt_token');
394         return token ? { 'Authorization': `Bearer ${token}` } : {};
395     }
396
397     async function apiFetch(path, opts = {}) {
398         const response = await fetch(`${API}${path}`, {
399             ...opts,
400             headers: { 'Content-Type': 'application/json', ...getAuthHeader(), ...opts.headers },
401             credentials: 'include'
402         });
403         if (response.status === 401) {
404             localStorage.removeItem('jwt_token');
405             localStorage.removeItem('auth_type');
406             showAlert('Session expired. Please login again.', 'error');
407             setTimeout(() => window.location.href = '/google.html', 1500);
408             throw new Error('Unauthorized');
409         }
410         if (!response.ok) {
411             const err = await response.json().catch(() => ({ message: response.statusText }));
412             throw new Error(err.message || `Error ${response.status}`);
413         }
414         const text = await response.text();
415         return text ? JSON.parse(text) : {};
416     }
417
418     function logout() {
419         localStorage.removeItem('jwt_token');
420         localStorage.removeItem('auth_type');
421         window.location.href = '/google.html';
422     }
423
424     (async function initAuth() {
425         const token = localStorage.getItem('jwt_token');
426         const authType = localStorage.getItem('auth_type');
427         if (!token && authType !== 'session') { window.location.href = '/google.html'; return; }
428         try {
429             const employee = await apiFetch('/employee/me');
430             currentUser = employee;
431             const initials = (employee.name || employee.email || 'U').split(' ').map(w =>
w[0]).join('').substring(0,2).toUpperCase();
432             document.getElementById('user-initials').textContent = initials;
433             document.getElementById('username-display').textContent = employee.name || employee.email ||
'User';
434             document.getElementById('leave-email').value = employee.email || '';
435             loadDashboard();
436             loadPendingCount();
437         } catch(e) {
438             if (e.message !== 'Unauthorized') window.location.href = '/google.html';
439         }
440     })();
441
442     // Navigation
443     const viewTitles = {
444         dashboard: 'Leave Dashboard',
445         apply: 'Apply for Leave',
446         'my-leaves': 'My Leave Requests',
447         balance: 'Leave Balance',
448         pending: 'Pending Approvals',
449         'all-leaves': 'All Leave Requests'
450     };
451     const viewLoaders = {
452         dashboard: loadDashboard,
453         apply: () => {},
454         'my-leaves': loadMyLeaves,
455         balance: loadBalance,
456         pending: loadPendingLeaves,
457         'all-leaves': loadAllLeaves
458     };
459
460     function showView(name, el) {
461         document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
462         document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));

```

```

463     document.getElementById(name + '-view').classList.add('active');
464     if (el) el.classList.add('active');
465     document.getElementById('pageTitle').textContent = viewTitles[name] || name;
466     if (viewLoaders[name]) viewLoaders[name]();
467 }
468
469 // Helpers
470 function daysBetween(start, end) {
471     if (!start || !end) return '-';
472     const d = (new Date(end) - new Date(start)) / 86400000 + 1;
473     return d > 0 ? d : '-';
474 }
475
476 function chip(val) {
477     if (!val) return '-';
478     const cls = val.toLowerCase();
479     return `

```

```

536     async function applyLeave() {
537         const leaveType = document.getElementById('leave-type').value;
538         const startDate = document.getElementById('leave-start').value;
539         const endDate = document.getElementById('leave-end').value;
540         const reason = document.getElementById('leave-reason').value.trim();
541         if (!startDate || !endDate) { showAlert('Please select both start and end dates.', 'error');
return; }
542         if (new Date(endDate) < new Date(startDate)) { showAlert('End date cannot be before start date.',
'error'); return; }
543         try {
544             await apiFetch('/leave_requests', {
545                 method: 'POST',
546                 body: JSON.stringify({ leaveType, startDate, endDate, reason })
547             });
548             showAlert('Leave request submitted successfully!', 'success');
549             clearLeaveForm();
550             loadPendingCount();
551         } catch(e) { showAlert('Failed to submit: ' + e.message, 'error'); }
552     }
553
554     function clearLeaveForm() {
555         document.getElementById('leave-start').value = '';
556         document.getElementById('leave-end').value = '';
557         document.getElementById('leave-reason').value = '';
558     }
559
560     // My Leaves
561     async function loadMyLeaves() {
562         try {
563             const data = await apiFetch('/leave_requests/employee');
564             const leaves = data.requests || (Array.isArray(data) ? data : []);
565             const tbody = document.getElementById('my-leaves-list');
566             tbody.innerHTML = leaves.length
567                 ? leaves.map(l => `<tr>
568 <td>${l.leaveRequestId}</td>
569 <td>${chip(l.leaveType)}</td>
570 <td>${l.startDate || '-'}</td>
571 <td>${l.endDate || '-'}</td>
572 <td>${daysBetween(l.startDate, l.endDate)}</td>
573 <td>${chip(l.status)}</td>
574 <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;"
title="${l.reason||''}">${l.reason || '-'}</td>
575 <td>
576     ${l.status === 'PENDING'
577         ? `<button class="btn btn-danger btn-sm"
onclick="openCancelModal('${l.leaveRequestId}')">Cancel</button>`
578         : '-'}
579         </td>
580 </tr>`).join('')
581             : `<tr><td colspan="8" class="empty-state">No leave requests found</td></tr>`;
582         } catch(e) { showAlert('Failed to load your leaves: ' + e.message, 'error'); }
583     }
584
585     function openCancelModal(id) {
586         document.getElementById('cancel-leave-id').value = id;
587         document.getElementById('cancel-remarks').value = '';
588         document.getElementById('cancel-modal').classList.add('open');
589     }
590
591     async function confirmCancel() {
592         const id = document.getElementById('cancel-leave-id').value;
593         try {
594             await apiFetch(`/leave_requests/cancel/${id}`, { method: 'PUT' });
595             showAlert('Leave request cancelled.', 'success');
596             closeModal('cancel-modal');
597             loadMyLeaves();
598             loadPendingCount();
599         } catch(e) { showAlert('Failed to cancel: ' + e.message, 'error'); }
600     }
601
602     // Leave Balance
603     const balanceColors = {
604         SICK: '#f87171', CASUAL: '#4f8cff', MATERNITY: '#a78bfa', UNPAID: '#8b92a8'
605     };
606
607     function normalizeDashboardBalances(data) {

```

```

608     if (!data) return [];
609     if (data.leaveBalances) {
610         return Object.values(data.leaveBalances).map(balance => ({
611             type: balance.leaveType,
612             opening: Number(balance.opening ?? 0),
613             accrued: Number(balance.accrued ?? 0),
614             used: Number(balance.used ?? 0),
615             available: Number(balance.available ?? 0)
616         }));
617     }
618     return ['sick', 'casual', 'maternity']
619         .map(key => data[key])
620         .filter(Boolean)
621         .map(balance => ({
622             type: balance.leaveType,
623             opening: Number(balance.opening ?? 0),
624             accrued: Number(balance.accrued ?? 0),
625             used: Number(balance.used ?? 0),
626             available: Number(balance.available ?? 0)
627         }));
628 }
629
630 async function loadBalance() {
631     try {
632         const data = await apiFetch('/api/employees/me/leave-dashboard');
633         const grid = document.getElementById('balance-grid');
634         const balances = normalizeDashboardBalances(data);
635
636         if (balances.length) {
637             grid.innerHTML = balances.map(info => {
638                 const type = info.type;
639                 const used = info.used;
640                 const total = info.opening + info.accrued;
641                 const left = info.available;
642                 const pct = total ? Math.min(100, Math.round((used / total) * 100)) : 0;
643                 const color = balanceColors[type] || '#4f8cff';
644                 return `<div class="balance-card">
645                     <div class="balance-name">${type.replace(/_/g, ' ')}</div>
646                     <div class="balance-row">
647                         <span class="balance-big" style="color:${color}">${left}</span>
648                         <span class="balance-of"> / ${total} days left</span>
649                     </div>
650                     <div class="balance-bar"><div class="balance-fill"
651 style="width:${pct}%;background:${color}"></div></div>
652                 </div>`;
653             }).join('');
654         } else {
655             grid.innerHTML = `<div class="balance-card" style="color:var(--text2);grid-column:1/-
1;text-align:center;padding:32px;">
656                 Leave balance data not available from the API. Check <code>/api/employees/me/leave-dashboard</code>
657                 endpoint.
658             </div>`;
659         }
660     } catch (e) {
661         showAlert('Failed to load balance: ' + e.message, 'error');
662     }
663 }
664
665 // Pending Approvals
666 async function loadPendingLeaves() {
667     try {
668         const leaves = await apiFetch('/leave_requests/pending');
669         const arr = Array.isArray(leaves) ? leaves : [];

```



```

681         const count = arr.filter(l => l.status === 'PENDING').length;
682         const el = document.getElementById('pending-count');
683         if (count > 0) { el.textContent = count; el.style.display = 'inline'; }
684         else el.style.display = 'none';
685         const tbody = document.getElementById('pending-leaves-list');
686         tbody.innerHTML = arr.length
687             ? arr.map(l => `<tr>
688                 <td>${l.leaveRequestId}</td>
689                 <td>${l.employeeId || '-'}</td>
690                 <td>${chip(l.leaveType)}</td>
691                 <td>${l.startDate || '-'}</td>
692                 <td>${l.endDate || '-'}</td>
693                 <td>${daysBetween(l.startDate, l.endDate)}</td>
694                 <td style="max-width:140px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;"
695                 title="${l.reason||''}">${l.reason || '-'}</td>
696                 <td class="action-row">
697                     <button class="btn btn-success btn-sm"
698                     onclick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
699                     <button class="btn btn-danger btn-sm"
700                     onclick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>
701                 </td>
702             `).join('')
703             : `<tr><td colspan="8" class="empty-state">No pending leave requests</td></tr>`;
704     } catch(e) { showAlert('Failed to load pending: ' + e.message, 'error'); }
705 }
706
707 function openActionModal(id, action) {
708     document.getElementById('action-leave-id').value = id;
709     document.getElementById('action-type').value = action;
710     document.getElementById('action-remarks').value = '';
711     document.getElementById('action-modal-title').textContent = `${action === 'APPROVED' ? 'Approve' :
712     'Reject'} Leave Request #${id}`;
713     const btn = document.getElementById('action-submit-btn');
714     btn.textContent = action === 'APPROVED' ? 'Approve' : 'Reject';
715     btn.className = action === 'APPROVED' ? 'btn btn-success' : 'btn btn-danger';
716     document.getElementById('action-modal').classList.add('open');
717 }
718
719 async function submitApproval() {
720     const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
721     const action = document.getElementById('action-type').value;
722     const remarks = document.getElementById('action-remarks').value;
723     try {
724         await apiFetch('/leave_requests/approval', {
725             method: 'PUT',
726             body: JSON.stringify({ leaveRequestId, action, remarks })
727         });
728         showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
729         closeModal('action-modal');
730         loadPendingLeaves();
731         loadDashboard();
732     } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
733 }
734
735 // All Leaves
736 async function loadAllLeaves() {
737     try {
738         const leaves = await apiFetch('/leave_requests');
739         allLeavesCache = Array.isArray(leaves) ? leaves : [];
740         renderAllLeaves(allLeavesCache);
741     } catch(e) { showAlert('Failed to load all leaves: ' + e.message, 'error'); }
742 }
743
744 function filterAllLeaves() {
745     const status = document.getElementById('status-filter').value;
746     const filtered = status ? allLeavesCache.filter(l => l.status === status) : allLeavesCache;
747     renderAllLeaves(filtered);
748 }
749
750 function renderAllLeaves(leaves) {
751     const tbody = document.getElementById('all-leaves-list');
752     tbody.innerHTML = leaves.length
753         ? leaves.map(l => `<tr>
754             <td>${l.leaveRequestId}</td>
755             <td>${l.employeeId || '-'}</td>
756             <td>${chip(l.leaveType)}</td>

```

```
753         <td>${l.startDate || '-'}</td>
754         <td>${l.endDate || '-'}</td>
755         <td>${daysBetween(l.startDate, l.endDate)}</td>
756         <td>${chip(l.status)}</td>
757         <td class="action-row">
758             ${l.status === 'PENDING'
759                 ? `<button class="btn btn-success btn-sm"
onclick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
760                 <button class="btn btn-danger btn-sm"
onclick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>`
761                 : '-'}
762         </td>
763     </tr>`.join('')
764     : `<tr><td colspan="8" class="empty-state">No leave requests found</td></tr>`;
765 }
766 </script>
767 </body>
768 </html>
769
```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Title</title>
6 </head>
7 <body>
8 <h1>hi</h1>
9 </body>
10 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Manager Console</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700&family=IBM+Plex+Mono:w
ght@400;500&display=swap" rel="stylesheet">
8      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
9      <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
10     <style>
11         *{margin:0;padding:0;box-sizing:border-box}
12         :root{
13             --bg:#0a0b0f;--bg2:#0f1117;--bg3:#14171f;
14             --panel:#171c28;--panel2:#1d2336;
15             --border:rgba(167,139,250,0.08);--border2:rgba(167,139,250,0.18);
16             --text:#e8e0f5;--text2:#8b82a8;--text3:#504d68;
17             --accent:#a78bfa;--accent-dim:rgba(167,139,250,0.1);
18             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
19             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
20             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
21             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
22             --font:'Plus Jakarta Sans',sans-serif;--mono:'IBM Plex Mono',monospace;
23             --sidebar:250px;
24         }
25         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
26         .app{display:flex;height:100vh}
27
28         /* SIDEBAR */
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative;overflow:hidden}
30         .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:200px;background:linear-
gradient(to top,rgba(167,139,250,0.04),transparent);pointer-events:none}
31         .logo{padding:22px 20px;border-bottom:1px solid var(--border);position:relative}
32         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
33         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,var(--accent),#7c3aed);border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
34         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
35         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono)}
36         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:linear-
gradient(90deg,rgba(167,139,250,0.15),rgba(124,58,237,0.08));border:1px solid rgba(167,139,250,...
37         .role-dot{width:6px;height:6px;background:var(--accent);border-radius:50%;animation:glow 2s
infinite}
38         @keyframes glow{0%,100%{box-shadow:0 0 4px var(--accent)}50%{box-shadow:0 0 10px var(--accent),0 0
20px rgba(167,139,250,0.4)}}
39         .nav{flex:1;padding:10px 0;overflow-y:auto}
40         .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase}
41         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
42         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
43         .nav-item.active{background:linear-gradient(90deg,var(--accent-
dim),rgba(124,58,237,0.06));color:var(--accent);border:1px solid rgba(167,139,250,0.2)}
44         .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:10px;font-
weight:700;padding:2px 7px;border-radius:10px;min-width:18px;text-align:center}
45         .nav-badge.hidden{display:none}
46         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
47         .user-card{display:flex;align-items:center;gap:10px}
48         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#7c3aed,var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
49         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
50         .user-info-side .role-text{font-size:10px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px}
51
52         /* MAIN */
53         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
54         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
55         .page-title{font-size:16px;font-weight:700;color:var(--text)}
56         .topbar-right{display:flex;align-items:center;gap:12px}

```

```

57     .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
58     .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
59     .content{flex:1;overflow-y:auto;padding:24px}
60     .view{display:none}
61     .view.active{display:block;animation:fadeIn 0.3s ease}
62     @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
63     .notif-wrapper { position: relative; cursor: pointer; }
64
65     .notif-badge {
66         position: absolute; top: -5px; right: -6px;
67         background: #e53e3e; color: white;
68         font-size: 10px; font-weight: 600;
69         min-width: 16px; height: 16px;
70         border-radius: 50%; display: flex;
71         align-items: center; justify-content: center;
72         padding: 0 3px;
73     }
74
75     .notif-panel {
76         position: absolute; top: 48px; right: 16px;
77         width: 320px; max-height: 400px; overflow-y: auto;
78         background: white; border: 1px solid #e2e8f0;
79         border-radius: 10px; box-shadow: 0 4px 16px rgba(0,0,0,0.1);
80         z-index: 999;
81     }
82
83     .notif-header {
84         padding: 12px 16px; font-weight: 600;
85         border-bottom: 1px solid #e2e8f0;
86     }
87
88     .notif-item {
89         padding: 12px 16px; font-size: 14px;
90         border-bottom: 1px solid #f1f5f9;
91     }
92
93     .notif-item.unread { background: #eff6ff; }
94     .notif-time { font-size: 11px; color: #94a3b8; margin-top: 4px; }
95
96     /* STATS */
97     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
98     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:14px;padding:20px;position:relative;overflow:hidden;transition:all 0.2s;cursor:default}
99     .stat-card:hover{border-color:var(--border2);transform:translateY(-1px)}
100     .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:3px}
101     .stat-card.purple::before{background:linear-gradient(90deg,#7c3aed,var(--accent))}
102     .stat-card.green::before{background:var(--green)}
103     .stat-card.amber::before{background:var(--amber)}
104     .stat-card.blue::before{background:var(--blue)}
105     .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-
bottom:10px;font-weight:600}
106     .stat-value{font-size:30px;font-weight:700;color:var(--text);font-family:var(--mono);line-height:1}
107     .stat-sub{font-size:11px;color:var(--text3);margin-top:6px}
108
109     /* SECTION */
110     .section{background:var(--panel);border:1px solid var(--border);border-radius:14px;margin-
bottom:20px;overflow:hidden}
111     .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
112     .section-title{font-size:14px;font-weight:700;color:var(--text)}
113     .section-sub{font-size:12px;color:var(--text3);margin-top:2px}
114
115     /* TABLE */
116     .tablewrap{overflow-x:auto}
117     table{width:100%;border-collapse:collapse}
118     th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-
spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border)}
119     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
120     tr:last-child td{border-bottom:none}
121     tr:hover td{background:rgba(167,139,250,0.03);color:var(--text)}
122
123     /* CHIPS */
124     .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600}

```

```

125     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
126     .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
127     .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
128     .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
129     .chip.open { background: var(--accent-dim); color: var(--accent); }
130     .chip.sick{background:var(--red-dim);color:var(--red)}
131     .chip.casual{background:var(--amber-dim);color:var(--amber)}
132     .chip.maternity{background:var(--accent-dim);color:var(--accent)}
133     .chip.employee{background:var(--blue-dim);color:var(--blue)}
134     .chip.manager{background:var(--accent-dim);color:var(--accent)}
135     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
136     .chip.unpaid { background: rgba(139,92,246,0.15); color: #a78bfa; border: 1px solid
rgba(139,92,246,0.2); }
137     .chip.assigned { background: var(--green-dim); color: var(--green);}
138     .chip.condemned { background: var(--red-dim); color: var(--red);}
139
140     /* BUTTONS */
141     .btn{padding:8px 16px;border-radius:8px;border:none;font-family:var(--font);font-size:12px;font-
weight:600;cursor:pointer;transition:all 0.2s}
142     .btn-primary{background:var(--accent);color:#fff}
143     .btn-primary:hover{background:#7c3aed;box-shadow:0 4px 12px rgba(167,139,250,0.3)}
144     .btn-approve{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
145     .btn-approve:hover{background:var(--green);color:#fff}
146     .btn-reject{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
147     .btn-reject:hover{background:var(--red);color:#fff}
148     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
149     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
150     .btn-sm{padding:5px 12px;font-size:11px}
151     .action-row{display:flex;gap:6px}
152
153     /* FORM */
154     .form-card{padding:24px}
155     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
156     .form-group{display:flex;flex-direction:column;gap:6px}
157     .form-group.full{grid-column:1/-1}
158     label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
159     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
160     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
161     input::placeholder{color:var(--text3)}
162     select option{background:var(--bg2)}
163     .form-actions{display:flex;gap:10px;margin-top:20px}
164
165     /* ALERT */
166     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
167     .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
168     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
169     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
170     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
171
172     /* GRID DASHBOARD */
173     .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px;margin-bottom:0}
174
175     /* SEARCH */
176     .search-bar{display:flex;align-items:center;gap:10px;margin-bottom:16px}
177     .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:9px 14px;color:var(--text);font-family:var(--font);font-size:13px}
178     .search-input:focus{outline:none;border-color:var(--accent)}
179     .search-input::placeholder{color:var(--text3)}
180
181     /* ===== */
182     /* CALENDAR VIEW STYLES */
183     /* ===== */
184
185     /* Leave balance summary cards above the calendar */
186     .leave-balance-grid {
187         display: grid;
188         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
189         gap: 14px;
190         padding: 20px;
191         border-bottom: 1px solid var(--border);
192     }

```

```

193     .leave-bal-card {
194         background: var(--bg3);
195         border: 1px solid var(--border);
196         border-radius: 10px;
197         padding: 16px 18px;
198         display: flex;
199         flex-direction: column;
200         gap: 6px;
201         position: relative;
202         overflow: hidden;
203         transition: border-color 0.2s;
204     }
205     .leave-bal-card:hover { border-color: var(--border2); }
206     .leave-bal-card::before {
207         content: '';
208         position: absolute;
209         left: 0; top: 0; bottom: 0;
210         width: 3px;
211         border-radius: 3px 0 0 3px;
212     }
213     .leave-bal-card.sick::before { background: var(--red); }
214     .leave-bal-card.casual::before { background: var(--amber); }
215     .leave-bal-card.mat::before { background: var(--accent); }
216     .leave-bal-card.total::before { background: var(--green); }
217     .lbc-icon { font-size: 18px; }
218     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }
219     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-
height: 1; }
220     .lbc-sub { font-size: 11px; color: var(--text3); }
221
222     /* Legend row */
223     .cal-legend {
224         display: flex;
225         align-items: center;
226         gap: 20px;
227         padding: 12px 20px;
228         border-bottom: 1px solid var(--border);
229         flex-wrap: wrap;
230     }
231     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase;
letter-spacing: 0.8px; margin-right: 4px; }
232     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2);
}
233     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
234
235     /* FullCalendar dark-theme overrides */
236     #employee-calendar {
237         padding: 16px 20px 20px;
238     }
239     /* Toolbar */
240     #employee-calendar .fc .fc-toolbar-title {
241         font-family: var(--font);
242         font-size: 15px;
243         font-weight: 600;
244         color: var(--text);
245     }
246     #employee-calendar .fc .fc-button {
247         background: var(--panel2);
248         border: 1px solid var(--border2);
249         color: var(--text2);
250         font-family: var(--font);
251         font-size: 12px;
252         font-weight: 600;
253         border-radius: 6px;
254         padding: 5px 12px;
255         transition: all 0.2s;
256         box-shadow: none;
257         text-transform: capitalize;
258     }
259     #employee-calendar .fc .fc-button:hover,
260     #employee-calendar .fc .fc-button-active,
261     #employee-calendar .fc .fc-button:not(:disabled):active {
262         background: var(--accent-dim);
263         border-color: var(--accent);
264         color: var(--accent);

```

```

265         box-shadow: none;
266     }
267     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
268
269     /* Grid */
270     #employee-calendar .fc-theme-standard td,
271     #employee-calendar .fc-theme-standard th,
272     #employee-calendar .fc-theme-standard .fc-scrollgrid {
273         border-color: var(--border);
274     }
275     #employee-calendar .fc .fc-col-header-cell-cushion {
276         font-family: var(--font);
277         font-size: 11px;
278         font-weight: 600;
279         color: var(--text3);
280         text-transform: uppercase;
281         letter-spacing: 0.8px;
282         padding: 8px 4px;
283     }
284     #employee-calendar .fc .fc-daygrid-day-number {
285         font-family: var(--mono);
286         font-size: 12px;
287         color: var(--text3);
288         padding: 6px 8px;
289     }
290     #employee-calendar .fc .fc-daygrid-day.fc-day-today {
291         background: var(--accent-dim) !important;
292     }
293     #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number {
294         color: var(--accent);
295         font-weight: 700;
296     }
297     #employee-calendar .fc .fc-daygrid-day:hover {
298         background: rgba(56,189,248,0.03);
299     }
300     #employee-calendar .fc-daygrid-day-frame {
301         background: var(--bg3);
302     }
303     #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame {
304         background: var(--bg2);
305     }
306
307     /* Events */
308     #employee-calendar .fc-event {
309         border: none !important;
310         border-radius: 4px;
311         font-family: var(--font);
312         font-size: 11px;
313         font-weight: 600;
314         padding: 2px 6px;
315         cursor: pointer;
316     }
317     #employee-calendar .fc-event-title { font-weight: 600; }
318
319     /* Event tooltip */
320     .cal-tooltip {
321         position: fixed;
322         background: var(--panel2);
323         border: 1px solid var(--border2);
324         border-radius: 8px;
325         padding: 10px 14px;
326         font-size: 12px;
327         color: var(--text);
328         z-index: 9999;
329         pointer-events: none;
330         max-width: 200px;
331         box-shadow: 0 8px 24px rgba(0,0,0,0.4);
332     }
333     .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
334     .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
335
336     /* balance loading state */
337     .lbc-count.loading { color: var(--text3); font-size: 16px; }
338 </style>
339 </head>
340 <body>

```



```
341 <div class="app">
342   <!-- SIDEBAR -->
343   <div class="sidebar">
344     <div class="logo">
345       <div class="logo-row">
346         <div class="logo-icon">M</div>
347         <h2>EMS Console</h2>
348       </div>
349       <p>Management Portal</p>
350       <div class="role-badge"><span class="role-dot"></span>Manager</div>
351     </div>
352     <nav class="nav">
353       <div class="nav-section">Overview</div>
354       <div class="nav-item active" onclick="showView('dashboard',this)">
355         <span></span> Dashboard
356       </div>
357       <div class="nav-section">Team</div>
358       <div class="nav-item" onclick="showView('employees',this)"><span></span> All Employees</div>
359       <div class="nav-item" onclick="showView('add-employee',this)"><span></span> Add Employee</div>
360       <div class="nav-item" onclick="showView('promote-users',this)"><span></span> Promote Users
361       <span class="nav-badge" id="users-badge">0</span></div>
362       <div class="nav-section">Leave Management</div>
363       <div class="nav-item" onclick="showView('pending-leaves',this)">
364         <span></span> Pending Approvals
365         <span class="nav-badge" id="pending-badge">0</span>
366       </div>
367       <div class="nav-item" onclick="showView('all-leaves',this)"><span></span> All Leave
368       Requests</div>
369       <div class="nav-section">Personal</div>
370       <div class="nav-item" onclick="showView('apply-leave',this)"><span></span> Apply Leave</div>
371       <div class="nav-item" onclick="showView('my-leaves',this)"><span></span> My Leaves</div>
372       <div class="nav-item" onclick="showView('my-calendar',this)"><span class="icon"></span> My
373       Calendar</div>
374       <div class="nav-item" onclick="showView('my-subscriptions',this)"><span></span> My
375       Subscriptions</div>
376       <div class="nav-item" onclick="showView('add-subscription',this)"><span></span> Subscribe to
377       Meal</div>
378       <div class="nav-item" onclick="showView('restaurants',this)"><span></span> Restaurants</div>
379       <div class="nav-item" onclick="showView('my-devices',this)"><span></span> My Devices</div>
380       <div class="nav-item" onclick="showView('service-requests', this)"><span class="icon"></span>
381       Service Requests</div>
382       <div class="nav-item" onclick="showView('search-employee',this)"><span></span> Search
383       Employee</div>
384     </nav>
385     <div class="sidebar-footer">
386       <div class="user-card">
387         <div class="avatar" id="avatar-initials">M</div>
388         <div class="user-info-side">
389           <div class="name" id="sidebar-name">Manager</div>
390           <div class="role-text">Manager</div>
391         </div>
392       </div>
393     </div>
394   </div>
395   <!-- MAIN -->
396   <div class="main">
397     <div class="topbar">
398       <div class="page-title" id="pageTitle">Dashboard</div>
399       <!-- Bell icon with badge -->
400       <div class="notif-wrapper" onclick="toggleNotifPanel()">
401         <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
402           stroke="currentColor" stroke-width="2" stroke-linecap="round">
403           <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
404           <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
405         </svg>
406         <span id="notif-badge" class="notif-badge" style="display:none"></span>
407       </div>
408       <!-- Dropdown panel -->
409       <div id="notif-panel" class="notif-panel" style="display:none">
410         <div class="notif-header">Notifications</div>
411         <div id="notif-list"></div>
412       </div>
413       <div class="topbar-right">
414         <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
```



```

476             <input class="search-input" id="emp-search" placeholder="Search by name, email
or department..." oninput="filterEmployees()">
477         </div>
478     </div>
479     <div class="tablewrap">
480         <table>
481 <thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Actions</th>
</tr></thead>
482             <tbody id="employees-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
483         </table>
484     </div>
485 </div>
486 </div>
487
488 <!-- ADD EMPLOYEE -->
489 <div id="add-employee-view" class="view">
490     <div class="section">
491         <div class="section-head"><div class="section-title">Add New Employee</div></div>
492         <div class="form-card">
493             <div class="form-grid">
494                 <div class="form-group"><label>Full Name</label><input id="emp-name"
placeholder="John Doe"></div>
495                 <div class="form-group"><label>Email</label><input type="email" id="emp-email"
placeholder="john@company.com"></div>
496                 <div class="form-group"><label>Password</label><input type="password" id="emp-
pass" placeholder="Temporary password"></div>
497                 <div class="form-group"><label>Gender</label>
498                     <select id="emp-gender">
499                         <option value="M">Male</option>
500                         <option value="F">Female</option>
501                         <option value="O">Other</option>
502                     </select>
503                 </div>
504                 <div class="form-group"><label>Timezone</label>
505                     <select id="emp-timezone">
506                         <option value="UTC">UTC</option>
507                         <option value="Asia/Kolkata">Asia/Kolkata (IST)</option>
508                         <option value="America/New_York">America/New_York (EST)</option>
509                         <option value="America/Los_Angeles">America/Los_Angeles (PST)</option>
510                         <option value="Europe/London">Europe/London (GMT)</option>
511                         <option value="Asia/Singapore">Asia/Singapore (SGT)</option>
512                     </select>
513                 </div>
514             </div>
515             <div class="form-actions">
516                 <button class="btn btn-primary" onclick="addEmployee()">Add Employee</button>
517                 <button class="btn btn-secondary" onclick="clearEmpForm()">Clear</button>
518             </div>
519         </div>
520     </div>
521 </div>
522
523 <!-- SEARCH EMPLOYEE -->
524 <div id="search-employee-view" class="view">
525     <div class="section">
526         <div class="section-head">
527             <div><div class="section-title"> Search Employee</div><div class="section-sub">Look
up any employee across the organisation</div></div>
528         </div>
529         <div style="padding:20px;border-bottom:1px solid var(--border)">
530             <div style="display:flex;gap:10px">
531                 <input class="search-input" id="org-search-input" placeholder="Enter exact
employee name..."
532                     style="flex:1" onkeydown="if(event.key==='Enter')searchEmployee()">
533                 <button class="btn btn-primary" onclick="searchEmployee()">Search</button>
534             </div>
535             <div id="org-search-error" style="font-size:12px;color:var(--red);margin-
top:8px;display:none"></div>
536         </div>
537         <div id="org-search-results" style="padding:20px;display:none">
538
539     <!-- Employee Info Card -->
540     <div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-
bottom:20px">

```

```

541         <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px">
542             <div style="font-size:10px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:12px">Employee Info</div>
543             <div style="display:flex;flex-direction:column;gap:8px">
544                 <div style="display:flex;justify-content:space-between">
545                     <span style="font-size:12px;color:var(--text3)">Name</span>
546                     <span style="font-size:13px;font-weight:600;color:var(--text)"
id="sr-name"></span>
547                 </div>
548                 <div style="display:flex;justify-content:space-between">
549                     <span style="font-size:12px;color:var(--text3)">ID</span>
550                     <span style="font-size:12px;font-family:var(--mono);color:var(--
accent)" id="sr-id"></span>
551                 </div>
552                 <div style="display:flex;justify-content:space-between">
553                     <span style="font-size:12px;color:var(--text3)">Email</span>
554                     <span style="font-size:12px;font-family:var(--mono);color:var(--
text2)" id="sr-email"></span>
555                 </div>
556                 <div style="display:flex;justify-content:space-between">
557                     <span style="font-size:12px;color:var(--text3)">Department</span>
558                     <span style="font-size:13px;color:var(--text)" id="sr-
dept"></span>
559                 </div>
560             </div>
561         </div>
562         <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px">
563             <div style="font-size:10px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:12px">Status</div>
564             <div style="display:flex;flex-direction:column;gap:8px">
565                 <div style="display:flex;justify-content:space-between;align-
items:center">
566                     <span style="font-size:12px;color:var(--text3)">Work Status</span>
567                     <span id="sr-status"></span>
568                 </div>
569                 <div style="display:flex;justify-content:space-between;align-
items:center">
570                     <span style="font-size:12px;color:var(--text3)">Attendance</span>
571                     <span style="font-size:13px;font-weight:600;color:var(--text)"
id="sr-attendance"></span>
572                 </div>
573             </div>
574         </div>
575     </div>
576
577     <!-- Devices -->
578     <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;overflow:hidden">
579         <div style="padding:12px 16px;border-bottom:1px solid var(--border)">
580             <span style="font-size:12px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px">Assigned Devices</span>
581         </div>
582         <table style="width:100%;border-collapse:collapse">
583             <thead>
584                 <tr>
585                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
586                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
587                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
588                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
589                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
590                     <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
591                 </tr>
592             </thead>
593             <tbody id="sr-devices"></tbody>
594         </table>
595     </div>
596 </div>
597

```

```

598             <!-- Empty / initial state -->
599             <div id="org-search-empty" style="padding:40px;text-align:center;color:var(--
text3);font-style:italic">
600                 Enter a name above to look up an employee
601             </div>
602         </div>
603     </div>
604
605     <!-- PROMOTE USERS -->
606     <div id="promote-users-view" class="view">
607         <div class="section">
608             <div class="section-head">
609                 <div><div class="section-title"> Promote Users</div><div class="section-
sub">Onboard pending user registrations to your department</div></div>
610                 <button class="btn btn-secondary btn-sm" onclick="loadUsers()"> Refresh</button>
611             </div>
612             <div class="tablewrap">
613                 <table>
614                     <thead><tr><th>Name</th><th>Email</th><th>Timezone</th><th>Temp
Password</th><th>Action</th></tr></thead>
615                     <tbody id="users-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
616                 </table>
617             </div>
618         </div>
619     </div>
620
621     <!-- PENDING LEAVE APPROVALS -->
622     <div id="pending-leaves-view" class="view">
623         <div class="section">
624             <div class="section-head">
625                 <div><div class="section-title"> Pending Leave Approvals</div><div class="section-
sub">Review and take action</div></div>
626                 <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">
Refresh</button>
627             </div>
628             <div class="tablewrap">
629                 <table>
630
631                     <thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
632                     <tbody id="pending-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
633                 </table>
634             </div>
635         </div>
636
637
638
639     <!-- ALL LEAVES -->
640     <div id="all-leaves-view" class="view">
641         <div class="section">
642             <div class="section-head">
643                 <div><div class="section-title">All Leave Requests</div></div>
644                 <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">
Refresh</button>
645             </div>
646             <div class="tablewrap">
647                 <table>
648
649                     <thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
650                     <tbody id="all-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
651                 </table>
652             </div>
653         </div>
654
655     <!-- APPLY LEAVE -->
656     <div id="apply-leave-view" class="view">
657         <div class="section">
658             <div class="section-head">
659                 <div>
660                     <div class="section-title"> Apply for Leave</div>
661                     <div class="section-sub">Submit a new leave request</div>

```

```

662         </div>
663     </div>
664     <div class="form-card">
665         <div class="form-grid">
666             <div class="form-group">
667                 <label>Leave Type</label>
668                 <select id="leave-type">
669                     <option value="SICK">Sick Leave</option>
670                     <option value="CASUAL">Casual Leave</option>
671                     <option value="MATERNITY">Maternity Leave</option>
672                     <option value="UNPAID">Unpaid Leave</option>
673                 </select>
674                 <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(251,191,36,0.2);bord...
675                 Sick leave is limited to <strong>1 day per month</strong>. If already
used this month, please select Unpaid Leave.
676             </div>
677         </div>
678         <div class="form-group">
679             <label>Reason</label>
680             <input id="leave-reason" placeholder="Brief reason for leave">
681         </div>
682         <div class="form-group">
683             <label>Start Date</label>
684             <input type="date" id="leave-start">
685         </div>
686         <div class="form-group">
687             <label>End Date</label>
688             <input type="date" id="leave-end">
689         </div>
690     </div>
691     <div class="form-actions">
692         <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
693         <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
694     </div>
695 </div>
696 </div>
697 </div>
698
699 <!-- MY LEAVES -->
700 <div id="my-leaves-view" class="view">
701     <div class="section">
702         <div class="section-head">
703             <div class="section-title">My Leave Requests</div>
704             <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
705         </div>
706         <div class="tablewrap">
707             <table>
708
709 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Status</th></tr></thead>
710 <tbody id="my-leaves-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
711             </table>
712         </div>
713     </div>
714
715 <!-- ===== -->
716 <!-- MY CALENDAR VIEW (NEW) -->
717 <!-- ===== -->
718 <div id="my-calendar-view" class="view">
719     <div class="section">
720         <div class="section-head">
721             <div>
722                 <div class="section-title">My Leave Calendar</div>
723                 <div class="section-sub">All approved leaves and public holidays</div>
724             </div>
725             <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
726         </div>
727
728 <!-- Leave Balance Summary Cards -->
729 <div class="leave-balance-grid">
730     <!-- Sick Leave Card -->
731     <div class="leave-bal-card sick">
732         <span class="lbc-icon"></span>

```

```
733             <span class="lbc-label">Sick Leave</span>
734             <span class="lbc-count" id="cal-sick-remaining">--</span>
735             <span class="lbc-sub" id="cal-sick-sub">loading...</span>
736         </div>
737         <!-- Casual Leave Card -->
738         <div class="leave-bal-card casual">
739             <span class="lbc-icon"></span>
740             <span class="lbc-label">Casual Leave</span>
741             <span class="lbc-count" id="cal-casual-remaining">--</span>
742             <span class="lbc-sub" id="cal-casual-sub">loading...</span>
743         </div>
744         <!-- Maternity Leave Card -->
745         <div class="leave-bal-card mat">
746             <span class="lbc-icon"></span>
747             <span class="lbc-label">Maternity Leave</span>
748             <span class="lbc-count" id="cal-mat-remaining">--</span>
749             <span class="lbc-sub" id="cal-mat-sub">loading...</span>
750         </div>
751         <!-- Total Used Card -->
752         <div class="leave-bal-card total">
753             <span class="lbc-icon"></span>
754             <span class="lbc-label">Total Used</span>
755             <span class="lbc-count" id="cal-total-used">--</span>
756             <span class="lbc-sub" id="cal-total-sub">loading...</span>
757         </div>
758     </div>
759
760     <!-- Legend -->
761     <div class="cal-legend">
762         <span class="legend-label">Legend:</span>
763         <div class="legend-item">
764             <div class="legend-dot" style="background:#34d399"></div>
765             Approved Leave
766         </div>
767         <div class="legend-item">
768             <div class="legend-dot" style="background:#fbbf24"></div>
769             Pending Leave
770         </div>
771         <div class="legend-item">
772             <div class="legend-dot" style="background:#f87171"></div>
773             Rejected Leave
774         </div>
775         <div class="legend-item">
776             <div class="legend-dot" style="background:#38bdf8"></div>
777             Public Holiday
778         </div>
779     </div>
780
781     <!-- FullCalendar Mount Point -->
782     <div id="employee-calendar"></div>
783 </div>
784 </div>
785 <!-- END CALENDAR VIEW -->
786
787 <!-- MY SUBSCRIPTIONS -->
788 <div id="my-subscriptions-view" class="view">
789     <div class="section">
790         <div class="section-head">
791             <div><div class="section-title"> My Subscriptions</div><div class="section-
792 sub">Your active meal subscriptions</div></div>
793             <button class="btn btn-secondary btn-sm" onclick="loadMySubscriptions()">
Refresh</button>
794         </div>
795         <div class="tablewrap">
796             <table>
797 <thead><tr><th>Slot</th><th>Restaurant</th><th>Address</th><th>Schedule</th><th>Day</th><th>Status</th><th>Next
Delivery</th></tr></thead>
798 <tbody id="my-subscriptions-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
799             </table>
800         </div>
801     </div>
802
803     <!-- ADD SUBSCRIPTION -->
```

```

804         <div id="add-subscription-view" class="view">
805             <div class="section">
806                 <div class="section-head">
807                     <div><div class="section-title"> Subscribe to Meal</div><div class="section-
808 sub">Add a new meal slot subscription</div></div>
809                 </div>
810                 <div class="form-card">
811                     <div class="form-grid">
812                         <div class="form-group">
813                             <label>Schedule Type</label>
814                             <select id="sub-schedule" onchange="toggleDayOfWeek()">
815                                 <option value="DAILY">Daily</option>
816                                 <option value="WEEKLY">Weekly</option>
817                             </select>
818                         </div>
819                         <div class="form-group" id="day-of-week-group" style="display:none">
820                             <label>Day of Week</label>
821                             <select id="sub-day">
822                                 <option value="MONDAY">Monday</option>
823                                 <option value="TUESDAY">Tuesday</option>
824                                 <option value="WEDNESDAY">Wednesday</option>
825                                 <option value="THURSDAY">Thursday</option>
826                                 <option value="FRIDAY">Friday</option>
827                                 <option value="SATURDAY">Saturday</option>
828                                 <option value="SUNDAY">Sunday</option>
829                             </select>
830                         </div>
831                     </div>
832                 </div>
833             <!-- Slot Orders -->
834             <div style="margin-top:20px">
835                 <div style="display:flex;justify-content:space-between;align-
836 items:center;margin-bottom:12px">
837                     <label style="font-size:11px;font-weight:700;color:var(--text3);letter-
838 spacing:0.8px;text-transform:uppercase">Meal Slots & Restaurants</label>
839                     <button class="btn btn-secondary btn-sm" onclick="addSlotRow()"> Add
840 Slot</button>
841                 </div>
842                 <div id="slot-rows"></div>
843             </div>
844             <div class="form-actions">
845                 <button class="btn btn-primary"
846 onclick="submitSubscription()">Subscribe</button>
847                 <button class="btn btn-secondary"
848 onclick="clearSubscriptionForm()">Clear</button>
849             </div>
850         </div>
851         <!-- RESTAURANTS -->
852         <div id="restaurants-view" class="view">
853             <div class="section">
854                 <div class="section-head">
855                     <div><div class="section-title"> Restaurants</div><div class="section-sub">All
856 available restaurants</div></div>
857                     <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
858 Refresh</button>
859                 </div>
860                 <div class="tablewrap">
861                     <table>
862                         <thead><tr><th>ID</th><th>Name</th><th>Address</th><th>Status</th></tr></thead>
863                         <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
864 state">Loading...</td></tr></tbody>
865                     </table>
866                 </div>
867             </div>
868         </div>
869         <!-- MY DEVICES -->
870         <div id="my-devices-view" class="view">
871             <div class="section">
872                 <div class="section-head">
873                     <div>
874                         <div class="section-title"> My Assigned Devices</div>

```



```

871         <div class="section-sub">Devices currently assigned to you</div>
872     </div>
873     <button class="btn btn-secondary btn-sm" onclick="loadMyAssignedDevices()">
Refresh</button>
874 </div>
875 <div class="tablewrap">
876     <table>
877         <thead>
878             <tr>
879                 <th>ID</th>
880                 <th>Device Name</th>
881                 <th>Brand</th>
882                 <th>Type</th>
883                 <th>Warranty</th>
884                 <th>Assigned On</th>
885                 <th>Status</th>
886             </tr>
887         </thead>
888         <tbody id="my-devices-list">
889             <tr><td colspan="7" class="empty-state">Loading...</td></tr>
890         </tbody>
891     </table>
892 </div>
893 </div>
894 </div>
895
896 <!-- SERVICE REQUESTS -->
897 <div id="service-requests-view" class="view">
898
899     <!-- List -->
900     <div class="section" style="margin-bottom:20px">
901         <div class="section-head">
902             <div>
903                 <div class="section-title">My Service Requests</div>
904                 <div class="section-sub">All requests you have raised</div>
905             </div>
906             <button class="btn btn-secondary btn-sm" onclick="loadServiceRequestsList()">
Refresh</button>
907         </div>
908         <div class="tablewrap">
909             <table>
910                 <thead>
911                     <tr>
912                         <th>ID</th>
913                         <th>Device</th>
914                         <th>Type</th>
915                         <th>Description</th>
916                         <th>Urgent</th>
917                         <th>Status</th>
918                         <th>Raised At</th>
919                     </tr>
920                 </thead>
921                 <tbody id="service-requests-list">
922                     <tr><td colspan="7" class="empty-state">Loading...</td></tr>
923                 </tbody>
924             </table>
925         </div>
926     </div>
927
928     <!-- Raise form -->
929     <div class="section">
930         <div class="section-head"><div class="section-title">Raise a New Request</div></div>
931         <div class="form-card">
932             <div class="form-grid">
933                 <div class="form-group full">
934                     <label>Device</label>
935                     <select id="sr-device-select">
936                         <option value="">-- Select your device --</option>
937                     </select>
938                 </div>
939                 <div class="form-group">
940                     <label>Request Type</label>
941                     <select id="sr-type">
942                         <option value="REPAIR">Repair</option>
943                         <option value="REPLACEMENT">Replacement</option>
944                         <option value="UPGRADE">Upgrade</option>

```

```

945             <option value="MAINTENANCE">Maintenance</option>
946             <option value="RETURN">Return</option>
947         </select>
948     </div>
949     <div class="form-group" style="display:flex;align-
950 items:center;gap:10px;padding-top:22px">
951         <input type="checkbox" id="sr-urgent"
952 style="width:16px;height:16px;padding:0;flex-shrink:0">
953         <label for="sr-urgent" style="text-transform:none;font-size:13px;font-
954 weight:500;color:var(--text);cursor:pointer;letter-spacing:0">
955             Mark as Urgent
956         </label>
957     </div>
958     <div class="form-group full">
959         <label>Issue Description</label>
960         <textarea id="sr-description" rows="3" placeholder="Describe the issue in
961 detail..."></textarea>
962     </div>
963 </div>
964 <div class="form-actions">
965     <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
966 Request</button>
967     <button class="btn btn-secondary"
968 onclick="clearServiceRequestForm()">Clear</button>
969 </div>
970 </div>
971 </div>
972 <!-- Tooltip for calendar events -->
973 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
974 <script>
975     const API='http://localhost:8080';
976     let allEmployeesCache=[];
977     // Add these near the top of your <script>, right after: let allEmployeesCache=[];
978
979     let employeeCalendar = null;
980     let calendarInitialized = false;
981
982     const STATUS_COLORS = {
983         APPROVED : { bg:'rgba(52,211,153,0.25)', border:'#34d399', text:'#34d399' },
984         PENDING  : { bg:'rgba(251,191,36,0.25)', border:'#fbbf24', text:'#fbbf24' },
985         REJECTED  : { bg:'rgba(248,113,113,0.25)', border:'#f87171', text:'#f87171' },
986         HOLIDAY   : { bg:'rgba(167,139,250,0.25)', border:'#a78bfa', text:'#a78bfa' },
987     };
988
989     const TYPE_LABEL = { SICK:' Sick', CASUAL:' Casual', MATERNITY:' Maternity', UNPAID:' Unpaid'
990     };
991
992     (function(){
993         const token=localStorage.getItem('jwt_token');
994         const authType=localStorage.getItem('auth_type');
995         if(!token&&authType!=='session'){window.location.href='/login.html';return;}
996         if(token){
997             try{
998                 const p=JSON.parse(atob(token.split('.')[1]));
999                 const rawRole = p.role || p.roles || p.authorities || '';
1000                 const normalizedRoles = Array.isArray(rawRole)
1001                     ? rawRole.map(v => String(v).replace(/^ROLE_/, ''))
1002                     : [String(rawRole).replace(/^ROLE_/, '')];
1003                 if(normalizedRoles.includes('MANAGER')) { initUser(); return; }
1004                 if(normalizedRoles.includes('ADMIN')){window.location.href='/dashboard.html';return;}
1005                 if(normalizedRoles.includes('EMPLOYEE')){window.location.href='/employee-
1006 dashboard.html';return;}
1007                 if(normalizedRoles.includes('VENDOR')){window.location.href='/vendor-
1008 dashboard.html';return;}
1009             }catch(e){}
1010             initUser();
1011         }
1012     })();
1013
1014     function getAuthHeader(){

```

```

1013     const t=localStorage.getItem('jwt_token');
1014     return t?{'Authorization':`Bearer ${t}`}:{};
1015 }
1016 async function apiFetch(path, opts={}){
1017     const res = await fetch(`${API}${path}`, {
1018         ...opts,
1019         headers: {'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
1020         credentials: 'include'
1021     });
1022     if(res.status===401){
1023         localStorage.removeItem('jwt_token');
1024         localStorage.removeItem('auth_type');
1025         showAlert('Session expired','error');
1026         setTimeout(()=>window.location.href='/google.html',1500);
1027         throw new Error('Unauthorized');
1028     }
1029     const text = await res.text();
1030     if(!res.ok){
1031         let message = res.statusText || `Error ${res.status}`;
1032         try {
1033             const body = text ? JSON.parse(text) : null;
1034             message = body?.message || body?.error || message;
1035         } catch {
1036             message = text || message;
1037         }
1038         throw new Error(message);
1039     }
1040     if(!text) return null;
1041     try { return JSON.parse(text); } // try JSON first
1042     catch { return text; } // fall back to plain string
1043 }
1044
1045 function asArray(value){
1046     return Array.isArray(value) ? value : [];
1047 }
1048
1049 async function initUser(){
1050     const token=localStorage.getItem('jwt_token');
1051     let email='';
1052     if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
1053     document.getElementById('avatar-initials').textContent=(email||'M').charAt(0).toUpperCase();
1054     document.getElementById('sidebar-name').textContent=email||'Manager';
1055     document.getElementById('topbar-email').textContent=email;
1056     loadDashboard();
1057     loadUsers();
1058 }
1059
1060 function showView(name,el){
1061     document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
1062     document.getElementById(`${name}-view`).classList.add('active');
1063     document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
1064     if(el)el.classList.add('active');
1065     document.getElementById('pageTitle').textContent=name.split('-')
1066     .map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
1067     const loaders={dashboard:loadDashboard,employees:loadEmployees,'pending-
leaves':loadPendingLeaves,'all-leaves':loadAllLeaves,'my-leaves':loadMyLeaves,'my-calendar':loadMyCalendar,'add-
employ...
1067         if(loaders[name])loaders[name]();
1068     }
1069
1070     async function loadDashboard(){
1071         const [empsResult, leavesResult] = await Promise.allSettled([
1072             apiFetch('/manager/employees'),
1073             apiFetch('/manager/pending')
1074         ]);
1075
1076         const emps = asArray(empsResult.status === 'fulfilled' ? empsResult.value : []);
1077         const leaves = asArray(leavesResult.status === 'fulfilled' ? leavesResult.value : []);
1078
1079         if (empsResult.status === 'rejected') {
1080             console.error('Failed to load manager employees:', empsResult.reason);
1081             document.getElementById('dash-employees').innerHTML =
1082                 '<tr><td colspan="3" class="empty-state">Could not load employees</td></tr>';
1083         }
1084         if (leavesResult.status === 'rejected') {
1085             console.error('Failed to load pending leaves:', leavesResult.reason);

```

```

1086     document.getElementById('dash-pending').innerHTML =
1087         '<tr><td colspan="4" class="empty-state">Could not load pending requests</td></tr>';
1088     }
1089
1090     // Stats
1091     document.getElementById('stat-total').textContent = emps.length;
1092     document.getElementById('stat-active').textContent = emps.filter(e => e.status ===
1093 'ACTIVE').length;
1094     document.getElementById('stat-on-leave').textContent = emps.filter(e => e.status ===
1095 'ON_LEAVE').length;
1096     const pending = leaves.filter(l => l.status === 'PENDING');
1097     document.getElementById('stat-pending').textContent = pending.length;
1098     document.getElementById('pending-badge').textContent = pending.length;
1099     document.getElementById('pending-badge').classList.toggle('hidden', pending.length === 0);
1100
1101     // Build employeeId name lookup from employees list
1102     const empMap = {};
1103     emps.forEach(e => { empMap[e.employeeId] = e.name || '-'; });
1104
1105     // Dash pending
1106     if (leavesResult.status === 'fulfilled') {
1107         document.getElementById('dash-pending').innerHTML = pending.length
1108         ? pending.slice(0, 5).map(l => {
1109             const leaveId = l.leaveRequestId || 0;
1110             const empName = empMap[l.employeeId] || `#${l.employeeId}`;
1111             return `<tr>
1112                 <td>${empName}</td>
1113                 <td><span class="chip ${l.leaveType || ''}.toLowerCase()>${l.leaveType}</span></td>
1114                 <td>${l.startDate} ${l.endDate}</td>
1115                 <td><div class="action-row">
1116                     <button class="btn btn-approve btn-sm"
1117 onclick="approveLeave(${leaveId})"></button>
1118                     <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})"></button>
1119                 </div></td>
1120             </tr>`;
1121         }).join('')
1122         : '<tr><td colspan="4" class="empty-state">No pending requests </td></tr>';
1123     }
1124
1125     // Dash employees
1126     if (empsResult.status === 'fulfilled') {
1127         document.getElementById('dash-employees').innerHTML = emps.length
1128         ? emps.slice(0, 6).map(e => `<tr>
1129             <td>${e.name || '-'}</td>
1130             <td>${e.dept || '-'}</td>
1131             <td><span class="chip ${(e.status || '').toLowerCase()}>${e.status || '-'}</span></td>
1132             </tr>`).join('')
1133         : '<tr><td colspan="3" class="empty-state">No employees</td></tr>';
1134     }
1135 }
1136
1137 async function loadEmployees(){
1138     try{
1139         const emps=await apiFetch('/manager/employees');
1140         allEmployeesCache=emps||[];
1141         renderEmployees(allEmployeesCache);
1142     }catch(e){document.getElementById('employees-list').innerHTML='<tr><td colspan="7" class="empty-
1143 state">Could not load employees</td></tr>';}
1144 }
1145
1146 function renderEmployees(emps){
1147     document.getElementById('employees-list').innerHTML=emps.length
1148     ? emps.map(e=>`<tr><td style="font-family:var(--mono);font-
1149 size:12px">#${e.employeeId}</td><td>${e.name||'-'}</td><td style="font-family:var(--mono);font-
1150 size:12px">${e.email||'-'}</td>...
1151         : '<tr><td colspan="7" class="empty-state">No employees found</td></tr>';
1152     }
1153
1154 function filterEmployees(){
1155     const q=document.getElementById('emp-search').value.toLowerCase();
1156     renderEmployees(allEmployeesCache.filter(e=>(e.name||'').toLowerCase().includes(q)||
1157 (e.email||'').toLowerCase().includes(q)||
1158 (e.dept||'').toLowerCase().includes(q)));
1159 }
1160
1161 async function addEmployee(){

```

```

1154     const name      = document.getElementById('emp-name').value.trim();
1155     const email      = document.getElementById('emp-email').value.trim();
1156     const password    = document.getElementById('emp-pass').value;
1157     const gender      = document.getElementById('emp-gender').value;
1158     const timezone    = document.getElementById('emp-timezone').value;
1159     if(!name || !email || !password){
1160         showAlert('Please fill all required fields','error');
1161         return;
1162     }
1163     try{
1164         await apiFetch('/manager/employees', {
1165             method: 'POST',
1166             body: JSON.stringify({ name, email, password, timezone, gender })
1167         });
1168         showAlert('Employee added successfully!','success');
1169         clearEmpForm();
1170         loadEmployees();
1171     }catch(e){ showAlert('Failed: '+e.message,'error'); }
1172     }
1173
1174     function clearEmpForm(){
1175         ['emp-name','emp-email','emp-pass'].forEach(id=>document.getElementById(id).value='');
1176         document.getElementById('emp-timezone').value = 'UTC';
1177         document.getElementById('emp-gender').selectedIndex = 0;
1178     }
1179     async function deleteEmployee(id){
1180         if(!confirm('Remove this employee?'))return;
1181         try{await apiFetch(`/employee/${id}`, {method: 'DELETE'});showAlert('Employee
removed','success');loadEmployees();}
1182         catch(e){
1183             console.log(e);
1184             showAlert('Failed to remove employee','error');
1185         }
1186     }
1187
1188     async function loadPendingLeaves(){
1189     try{
1190         const leaves = await apiFetch('/manager/pending');
1191         console.log('Leave sample:', leaves?.[0]); // remove after confirming
1192         const pending=(leaves||[]).filter(l=>l.status==='PENDING');
1193         document.getElementById('pending-leaves-list').innerHTML=pending.length
1194             ? pending.map(l=>{
1195                 // Try all possible ID field names your DTO might use
1196                 const leaveId = l.leaveRequestId || l.leaveId || l.id || 0;
1197                 return `<tr>
1198                     <td>${l.employeeName||l.employeeId||'-'}</td>
1199                     <td><span class="chip ${l.leaveType||''}.toLowerCase()">${l.leaveType}</span></td>
1200                     <td>${l.startDate}</td>
1201                     <td>${l.endDate}</td>
1202                     <td>${l.numberOfDays||'-'}</td>
1203                     <td>${(l.reason||'').substring(0,40)}</td>
1204                     <td><div class="action-row">
1205                         <button class="btn btn-approve btn-sm" onclick="approveLeave(${leaveId})">
Approve</button>
1206                         <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})">
Reject</button>
1207                     </div></td>
1208                 </tr>`;
1209             }).join('')
1210             : '<tr><td colspan="7" class="empty-state">No pending approvals </td></tr>';
1211     }catch(e){
1212         document.getElementById('pending-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';
1213     }
1214 }
1215
1216     async function approveLeave(id) {
1217         console.log('Approving leaveRequestId:', id); // verify this is not 0
1218         try {
1219             const res = await fetch(`${API}/leave_requests/approval`, {
1220                 method: 'PUT',
1221                 headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1222                 credentials: 'include',
1223                 body: JSON.stringify({
1224                     leaveRequestId: id, // matches ActionDTO exactly
1225                     action: 'APPROVED', // check if your service expects uppercase or lowercase
1226                     remarks: ''

```

```

1226     })
1227   });
1228   const text = await res.text();
1229   if (res.ok) {
1230     showAlert('Leave approved ', 'success');
1231     loadPendingLeaves(); loadDashboard(); loadAllLeaves();
1232   } else {
1233     showAlert('Error: ' + text, 'error');
1234   }
1235 } catch(e) { showAlert(e.message, 'error'); }
1236 }
1237
1238 async function rejectLeave(id) {
1239   console.log('Rejecting leaveRequestId:', id); // verify this is not 0
1240   try {
1241     const res = await fetch(`${API}/leave_requests/approval`, {
1242       method: 'PUT',
1243       headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1244       credentials: 'include',
1245       body: JSON.stringify({
1246         leaveRequestId: id, // matches ActionDTO exactly
1247         action: 'REJECTED', // check if your service expects uppercase or lowercase
1248         remarks: ''
1249       })
1250     });
1251     const text = await res.text();
1252     if (res.ok) {
1253       showAlert('Leave rejected', 'success');
1254       loadPendingLeaves(); loadDashboard(); loadAllLeaves();
1255     } else {
1256       showAlert('Error: ' + text, 'error');
1257     }
1258   } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1259 }
1260
1261 async function loadAllLeaves(){
1262   try{
1263     const leaves=await apiFetch('/manager/requests');
1264     document.getElementById('all-leaves-list').innerHTML=(leaves||[]).length
1265       ? (leaves).map(l=><tr>
1266         <td>${l.employeeName||'-'}</td>
1267         <td><span class="chip ${l.leaveType||''}.toLowerCase()>${l.leaveType||'-'}</span></td>
1268         <td>${l.startDate||'-'}</td>
1269         <td>${l.endDate||'-'}</td>
1270         <td>${(l.reason||'-').substring(0,50)}</td>
1271         <td><span class="chip ${l.status||''}.toLowerCase()>${l.status||'-'}</span></td>
1272         </tr>`
1273       : `<tr><td colspan="7" class="empty-state">No leave requests</td></tr>`;
1274     }catch(e){document.getElementById('all-leaves-list').innerHTML=`<tr><td colspan="7" class="empty-state">Could not load</td></tr>`;}
1275   }
1276
1277   async function applyLeave() {
1278     const leaveType = document.getElementById('leave-type').value;
1279     const reason = document.getElementById('leave-reason').value.trim();
1280     const startDate = document.getElementById('leave-start').value;
1281     const endDate = document.getElementById('leave-end').value;
1282
1283     if (!reason || !startDate || !endDate) {
1284       showAlert('Please fill all fields', 'error');
1285       return;
1286     }
1287     if (endDate < startDate) {
1288       showAlert('End date cannot be before start date', 'error');
1289       return;
1290     }
1291
1292     try {
1293       const res = await fetch(`${API}/leave_requests`, {
1294         method: 'POST',
1295         headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1296         credentials: 'include',
1297         body: JSON.stringify({ leaveType, reason, startDate, endDate })
1298       });
1299
1300       if (res.ok) {

```

```

1301         showAlert('Leave request submitted successfully!', 'success');
1302         clearLeaveForm();
1303         loadMyLeaves();
1304     } else {
1305         const text = await res.text();
1306         showAlert('Failed: ' + text, 'error');
1307     }
1308 } catch(e) {
1309     showAlert('Error: ' + e.message, 'error');
1310 }
1311 }
1312 document.getElementById('leave-type').addEventListener('change', function(){
1313     document.getElementById('sick-leave-hint').style.display =
1314         this.value === 'SICK' ? 'block' : 'none';
1315 });
1316
1317 function clearLeaveForm() {
1318     document.getElementById('leave-type').value = 'SICK';
1319     document.getElementById('leave-reason').value = '';
1320     document.getElementById('leave-start').value = '';
1321     document.getElementById('leave-end').value = '';
1322 }
1323
1324 async function loadMyLeaves(){
1325     try{
1326         const data=await apiFetch('/leave_requests/employee');
1327         const leaves=data?.requests||data||[];
1328         document.getElementById('my-leaves-list').innerHTML=leaves.length
1329             ? leaves.map(l=><tr><td><span class="chip
1330 ${{l.leaveType||''}.toLowerCase()}>${l.leaveType}</span></td><td>${l.startDate}</td><td>${l.endDate}</td><td>${
1331 l.numberOfDays||'-'}</td><...
1332         : '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
1333     }catch(e){document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="5" class="empty-
1334 state">Could not load</td></tr>';}
1335 }
1336
1337 async function loadUsers(){
1338     try{
1339         const users = await apiFetch('/manager/Users');
1340         const badge = document.getElementById('users-badge');
1341         if(badge){ badge.textContent = (users||[]).length;
1342 badge.classList.toggle('hidden',(users||[]).length===0); }
1343         document.getElementById('users-list').innerHTML = (users||[]).length
1344             ? users.map(u=><tr>
1345                 <td>${u.name||'-'}</td>
1346                 <td style="font-family:var(--mono);font-size:12px">${u.email||'-'}</td>
1347                 <td style="font-family:var(--mono);font-size:12px">${u.timezone||'-'}</td>
1348                 <td><input type="password" placeholder="Set password" id="pwd-${CSS.escape(u.email)}"
1349                 style="padding:6px 10px;font-size:12px;width:160px"></td>
1350                 <td><button class="btn btn-approve btn-sm" onclick="promoteUser('${u.email}')">
1351 Promote</button></td>
1352             </tr>`).join('')
1353             : '<tr><td colspan="5" class="empty-state">No pending user registrations</td></tr>';
1354     }catch(e){
1355         document.getElementById('users-list').innerHTML='<tr><td colspan="5" class="empty-state">Could not
1356 load</td></tr>';
1357     }
1358 }
1359
1360 async function promoteUser(email){
1361     const pwdInput = document.getElementById('pwd-'+CSS.escape(email));
1362     const password = pwdInput?.value?.trim();
1363     if(!password){ showAlert('Please set a password before promoting','error'); return; }
1364     try{
1365         const res = await fetch(`${API}/manager/promote`,{
1366             method:'PUT',
1367             headers:{'Content-Type':'application/json',...getAuthHeader()},
1368             credentials:'include',
1369             body: JSON.stringify({ email, password })
1370         });
1371         const text = await res.text();
1372         if(res.ok){
1373             showAlert(`${email} promoted to Employee `, 'success');
1374             loadUsers();
1375         } else {
1376             showAlert('Error: '+text, 'error');
1377         }
1378     }

```

```

1371     }
1372   }catch(e){ showAlert('Failed: '+e.message,'error'); }
1373 }
1374   async function loadRestaurants(){
1375     try{
1376       const data = await apiFetch('/restaurants');
1377       document.getElementById('restaurants-list').innerHTML = (data||[]).length
1378         ? data.map(r=>`<tr>
1379           <td style="font-family:var(--mono);font-size:12px">#${r.restaurantId||r.id}</td>
1380           <td>${r.name||'-'}</td>
1381           <td style="font-size:12px;color:var(--text3)">${r.address||'-'}</td>
1382           <td><span class="chip active">${r.status||'ACTIVE'}</span></td>
1383         </tr>`).join('')
1384       : '<tr><td colspan="4" class="empty-state">No restaurants found</td></tr>';
1385     }catch(e){
1386       document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-
state">Could not load</td></tr>';
1387     }
1388   }
1389
1390 // SUBSCRIPTIONS
1391
1392   async function loadMySubscriptions(){
1393     try{
1394       const subs = await apiFetch('/subscription/user');
1395       document.getElementById('my-subscriptions-list').innerHTML = (subs||[]).length
1396         ? subs.map(s=>`<tr>
1397           <td><span class="chip pending">${s.mealSlot||'-'}</span></td>
1398           <td>${s.restaurantName||'-'}</td>
1399           <td style="font-size:12px;color:var(--text3)">${s.restaurantAddress||'-'}</td>
1400           <td>${s.scheduleType||'-'}</td>
1401           <td>${s.dayOfWeek||'-'}</td>
1402           <td><span class="chip ${((s.status||'')).toLowerCase()}>${s.status||'-'}</span></td>
1403           <td style="font-family:var(--mono);font-size:11px">${s.nextDeliveryTime ? new
Date(s.nextDeliveryTime).toLocaleString() : '-'}</td>
1404         </tr>`).join('')
1405       : '<tr><td colspan="7" class="empty-state">No subscriptions yet</td></tr>';
1406     }catch(e){
1407       document.getElementById('my-subscriptions-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';
1408     }
1409   }
1410
1411   let slotRowCount = 0;
1412   async function addSlotRow(){
1413     slotRowCount++;
1414     const id = slotRowCount;
1415     const div = document.createElement('div');
1416     div.id = `slot-row-${id}`;
1417     div.style.cssText = 'display:grid;grid-template-columns:1fr 1fr auto;gap:12px;margin-
bottom:10px;align-items:end';
1418
1419     // Fetch restaurants for the dropdown
1420     let options = '<option value="">Loading...</option>';
1421     try{
1422       const data = await apiFetch('/restaurants');
1423       options = data.map(r => `<option
value="${r.restaurantId||r.id}">${r.name}</option>`).join('');
1424     }catch(e){ options = '<option value="">Could not load</option>'; }
1425
1426     div.innerHTML = `
1427     <div class="form-group" style="margin:0">
1428       <label>Meal Slot</label>
1429       <select id="slot-meal-${id}">
1430         <option value="BREAKFAST">Breakfast</option>
1431         <option value="LUNCH">Lunch</option>
1432         <option value="DINNER">Dinner</option>
1433       </select>
1434     </div>
1435     <div class="form-group" style="margin:0">
1436       <label>Restaurant</label>
1437       <select id="slot-restaurant-${id}">${options}</select>
1438     </div>
1439     <button class="btn btn-reject btn-sm" onclick="removeSlotRow(${id})" style="margin-
bottom:1px"></button>
1440   `;

```



```

1441     document.getElementById('slot-rows').appendChild(div);
1442 }
1443
1444 function removeSlotRow(id){
1445     document.getElementById(`slot-row-${id}`)?.remove();
1446 }
1447
1448 function toggleDayOfWeek(){
1449     const isWeekly = document.getElementById('sub-schedule').value === 'WEEKLY';
1450     document.getElementById('day-of-week-group').style.display = isWeekly ? 'flex' : 'none';
1451 }
1452
1453 async function submitSubscription(){
1454     const scheduleType = document.getElementById('sub-schedule').value;
1455     const dayOfWeek    = scheduleType === 'WEEKLY' ? document.getElementById('sub-day').value : null;
1456
1457     const slotOrders = [];
1458     let valid = true;
1459
1460     document.querySelectorAll('[id^="slot-row-"]').forEach(row => {
1461         const idNum      = row.id.split('-')[2];
1462         const mealSlot    = document.getElementById(`slot-meal-${idNum}`)?.value;
1463         const restaurantVal = document.getElementById(`slot-restaurant-${idNum}`)?.value;
1464
1465         if(!mealSlot || !restaurantVal){ valid = false; return; }
1466
1467         const restaurantId = parseInt(restaurantVal);
1468         if(isNaN(restaurantId)){ valid = false; return; }
1469
1470         slotOrders.push({ mealSlot, restaurantId });
1471     });
1472
1473     if(!valid || slotOrders.length === 0){
1474         showAlert('Please add at least one valid slot','error');
1475         return;
1476     }
1477
1478     // Derive mealSlots from slotOrders to satisfy @NotEmpty
1479     const mealSlots = slotOrders.map(s => s.mealSlot);
1480
1481     try{
1482         const res = await fetch(`${API}/subscription`, {
1483             method: 'POST',
1484             headers: {'Content-Type':'application/json', ...getAuthHeader()},
1485             credentials: 'include',
1486             body: JSON.stringify({ scheduleType, dayOfWeek, mealSlots, slotOrders })
1487         });
1488         const text = await res.text();
1489         if(res.ok){
1490             showAlert('Subscription added successfully! ','success');
1491             clearSubscriptionForm();
1492             loadMySubscriptions();
1493         } else {
1494             showAlert('Error: '+text,'error');
1495         }
1496     }catch(e){ showAlert('Failed: '+e.message,'error'); }
1497 }
1498
1499 function clearSubscriptionForm(){
1500     document.getElementById('sub-schedule').value = 'DAILY';
1501     document.getElementById('day-of-week-group').style.display = 'none';
1502     document.getElementById('slot-rows').innerHTML = '';
1503     slotRowCount = 0;
1504 }
1505
1506 /* =====
1507 MY CALENDAR (NEW)
1508 ===== */
1509 async function loadMyCalendar(){
1510     // 1. Load balance cards
1511     await loadCalendarBalanceCards();
1512     // 2. Build or refresh FullCalendar
1513     if(!calendarInitialized){
1514         initFullCalendar();
1515     } else {
1516         await refreshCalendarEvents();

```

```

1517     }
1518 }
1519
1520 async function loadCalendarBalanceCards(){
1521     // Default state
1522     ['cal-sick-remaining', 'cal-casual-remaining', 'cal-mat-remaining', 'cal-total-used']
1523     .forEach(id=>{ document.getElementById(id).textContent='--'; });
1524
1525     try{
1526         const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1527         const balances = dashboard?.leaveBalances || {};
1528
1529         let totalUsed=0, totalAllowed=0;
1530
1531         // SICK
1532         const sick = balances['SICK'] || balances['sick'] || {};
1533         const sickRem = Number(sick.available || 0);
1534         const sickTotal = Number(sick.total || 0);
1535         const sickUsed = sickTotal - sickRem;
1536         document.getElementById('cal-sick-remaining').textContent = sickRem;
1537         document.getElementById('cal-sick-sub').textContent = `${sickUsed} used of ${sickTotal}
days`;
1538
1539         // CASUAL
1540         const cas = balances['CASUAL'] || balances['casual'] || {};
1541         const casRem = Number(cas.available || 0);
1542         const casTotal = Number(cas.total || 0);
1543         const casUsed = casTotal - casRem;
1544         document.getElementById('cal-casual-remaining').textContent = casRem;
1545         document.getElementById('cal-casual-sub').textContent = `${casUsed} used of ${casTotal}
days`;
1546
1547         // MATERNITY
1548         const mat = balances['MATERNITY'] || balances['maternity'] || {};
1549         const matRem = Number(mat.available || 0);
1550         const matTotal = Number(mat.total || 0);
1551         const matUsed = matTotal - matRem;
1552         document.getElementById('cal-mat-remaining').textContent = matRem;
1553         document.getElementById('cal-mat-sub').textContent = `${matUsed} used of ${matTotal}
days`;
1554
1555         // TOTAL
1556         Object.values(balances).forEach(b=>{
1557             totalUsed += (Number(b.total||0) - Number(b.available||0));
1558             totalAllowed += Number(b.total||0);
1559         });
1560         document.getElementById('cal-total-used').textContent = totalUsed;
1561         document.getElementById('cal-total-sub').textContent = `out of ${totalAllowed} days total`;
1562
1563     }catch(e){
1564         console.error('Could not load leave balance for calendar:', e);
1565         ['cal-sick-sub', 'cal-casual-sub', 'cal-mat-sub', 'cal-total-sub']
1566         .forEach(id=>{ document.getElementById(id).textContent='unavailable'; });
1567     }
1568 }
1569
1570 async function buildCalendarEvents(){
1571     const events = [];
1572     try{
1573         // - Employee leaves -
1574         const data = await apiFetch('/leave_requests/employee');
1575         const leaves = data?.requests || data || [];
1576         leaves.forEach(l=>{
1577             const s = (l.status||'PENDING').toUpperCase();
1578             const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
1579             const typeLabel = TYPE_LABEL[l.leaveType] || l.leaveType || 'Leave';
1580
1581             // FullCalendar end date is exclusive, so add 1 day for multi-day events
1582             let endDate = l.endDate;
1583             if(endDate){
1584                 const d = new Date(endDate);
1585                 d.setDate(d.getDate()+1);
1586                 endDate = d.toISOString().split('T')[0];
1587             }
1588
1589             events.push({

```

```

1590         id      : `leave-${l.leaveRequestId}`,
1591         title    : `${typeLabel} (${s.charAt(0)+s.slice(1).toLowerCase()}`,
1592         start    : l.startDate,
1593         end      : endDate,
1594         allDay   : true,
1595         backgroundColor : c.bg,
1596         borderColor : c.border,
1597         textColor : c.text,
1598         extendedProps : {
1599             type      : 'leave',
1600             status    : s,
1601             reason    : l.reason || '-',
1602             leaveType: l.leaveType
1603         }
1604     });
1605 });
1606 }catch(e){ console.warn('Could not load leaves for calendar:', e); }
1607
1608     try{
1609         // - Public holidays (admin-added) -
1610         const holidays = await apiFetch('/holidays');
1611         (holidays||[]).forEach(h=>{
1612             const c = STATUS_COLORS.HOLIDAY;
1613             events.push({
1614                 id      : `holiday-${h.id||h.holidayId||Math.random()}`,
1615                 title    : `${h.name||h.holidayName||'Holiday'}`,
1616                 start    : h.date||h.holidayDate,
1617                 allDay   : true,
1618                 backgroundColor : c.bg,
1619                 borderColor : c.border,
1620                 textColor : c.text,
1621                 extendedProps : { type:'holiday', description: h.description||'' }
1622             });
1623         });
1624     }catch(e){ console.warn('Could not load holidays for calendar:', e); }
1625
1626     return events;
1627 }
1628
1629 function initFullCalendar(){
1630     const calendarEl = document.getElementById('employee-calendar');
1631
1632     employeeCalendar = new FullCalendar.Calendar(calendarEl, {
1633         initialView: 'dayGridMonth',
1634         height      : 'auto',
1635         headerToolbar:{
1636             left : 'prev,next today',
1637             center: 'title',
1638             right : 'dayGridMonth,dayGridWeek,listMonth'
1639         },
1640         buttonText:{
1641             today      : 'Today',
1642             month      : 'Month',
1643             week       : 'Week',
1644             listMonth  : 'List'
1645         },
1646         dayMaxEvents: 3,      // "+N more" link if overflowing
1647         navLinks    : true,  // click day number day view
1648         events      : [],    // start empty, populate below
1649
1650         // Event hover tooltip
1651         eventMouseEnter: function(info){
1652             const ep = info.event.extendedProps;
1653             const tooltip = document.getElementById('cal-tooltip');
1654             let html = `<div class="tt-title">${info.event.title}</div>`;
1655             if(ep.type==='leave'){
1656                 html += `<div class="tt-row"> ${info.event.startStr}
1657 ${info.event.endStr||info.event.startStr}</div>`;
1658                 if(ep.reason && ep.reason!=='-'){
1659                     html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
1660                     html += `<div class="tt-row">Status: <strong
1661 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
1662                 } else {
1663                     if(ep.description) html += `<div class="tt-row">${ep.description}</div>`;
1664                 }
1665                 tooltip.innerHTML = html;

```

```

1664         tooltip.style.display = 'block';
1665     },
1666     eventMouseLeave: function(){
1667         document.getElementById('cal-tooltip').style.display='none';
1668     },
1669     eventClick: function(info){
1670         // Click navigates to leave history
1671         const ep = info.event.extendedProps;
1672         if(ep.type==='leave'){
1673             showView('my-leaves', document.querySelector('.nav-item:nth-child(4)'));
1674         }
1675     },
1676 });
1677
1678 employeeCalendar.render();
1679 calendarInitialized = true;
1680
1681 // Load events asynchronously after render
1682 refreshCalendarEvents();
1683 }
1684
1685 async function refreshCalendarEvents(){
1686     if(!employeeCalendar) return;
1687     const events = await buildCalendarEvents();
1688     // Remove all existing events and re-add
1689     employeeCalendar.removeAllEvents();
1690     events.forEach(e=>employeeCalendar.addEvent(e));
1691 }
1692
1693 async function refreshCalendar(){
1694     await loadCalendarBalanceCards();
1695     await refreshCalendarEvents();
1696     showAlert('Calendar refreshed!', 'success');
1697 }
1698
1699 // Move tooltip with mouse
1700 document.addEventListener('mousemove', function(e){
1701     const tt = document.getElementById('cal-tooltip');
1702     if(tt.style.display==='block'){
1703         tt.style.left = (e.clientX+14)+'px';
1704         tt.style.top = (e.clientY+14)+'px';
1705         // Keep tooltip on screen
1706         const rect = tt.getBoundingClientRect();
1707         if(rect.right > window.innerWidth)
1708             tt.style.left = (e.clientX - rect.width - 10)+'px';
1709         if(rect.bottom > window.innerHeight)
1710             tt.style.top = (e.clientY - rect.height - 10)+'px';
1711     }
1712 });
1713
1714 // ===== MY DEVICES =====
1715 async function loadMyAssignedDevices() {
1716     try {
1717         const devices = await apiFetch('/devices/my');
1718         const tbody = document.getElementById('my-devices-list');
1719         if (!devices || !devices.length) {
1720             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
1721             return;
1722         }
1723         const today = new Date();
1724         tbody.innerHTML = devices.map(d => {
1725             const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1726             const warrantyDisplay = warrantyExpiry
1727                 ? (warrantyExpiry > today
1728                     ? `<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
1729                     : `<span style="color:var(--red);font-size:11px"> Expired
${d.warrantyExpiryDate}</span>`)
1730                 : `<span style="color:var(--text3);font-size:11px">--</span>`;
1731             return `<tr>
1732 <td>#${d.id}</td>
1733 <td><strong>${d.deviceName || '-'}</strong></td>
1734 <td>${d.brand || '-'}</td>
1735 <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
1736 <td>${warrantyDisplay}</td>
1737 <td style="font-size:12px;color:var(--text2)">${d.assignedDate || '-'}</td>

```

```

1738 <td><span class="chip ${d.deviceStatus || ''}.toLowerCase()">${d.deviceStatus || '-'}</span></td>
1739 </tr>`;
1740     }).join('');
1741     } catch (e) {
1742         document.getElementById('my-devices-list').innerHTML =
1743             '<tr><td colspan="7" class="empty-state">Could not load devices</td></tr>';
1744     }
1745 }
1746
1747 // ===== SERVICE REQUESTS =====
1748 async function loadServiceRequests() {
1749     await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
1750 }
1751
1752 async function loadServiceRequestsList() {
1753     try {
1754         const requests = await apiFetch('/service-requests/my');
1755         const tbody = document.getElementById('service-requests-list');
1756         if (!requests || !requests.length) {
1757             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No service requests yet</td></tr>';
1758             return;
1759         }
1760         tbody.innerHTML = requests.map(r => `<tr>
1761             <td style="color:var(--text3);font-family:var(--mono)">#${r.id}</td>
1762             <td>${r.deviceName || '-'}</td>
1763             <td><span class="chip pending">${r.requestType}</span></td>
1764             <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap">
1765                 ${r.issueDescription || '-'}
1766             </td>
1767             <td>${r.urgent
1768                 ? '<span class="chip rejected">YES</span>'
1769                 : '<span style="color:var(--text3)">No</span>'}
1770             </td>
1771             <td><span class="chip ${r.status || ''}.toLowerCase()">${r.status || '-'}</span></td>
1772             <td style="font-size:12px;color:var(--text3)">
1773                 ${r.raisedAt ? new Date(r.raisedAt).toLocaleDateString() : '-'}
1774             </td>
1775         </tr>`).join('');
1776     } catch (e) { console.error(e); }
1777 }
1778
1779 async function loadMyDevicesForSR() {
1780     try {
1781         const devices = await apiFetch('/devices/my');
1782         const select = document.getElementById('sr-device-select');
1783         select.innerHTML = '<option value="">-- Select your device --</option>';
1784         (devices || []).forEach(d => {
1785             const opt = document.createElement('option');
1786             opt.value = d.id;
1787             opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
1788             select.appendChild(opt);
1789         });
1790     } catch (e) { console.error('Failed to load devices for SR', e); }
1791 }
1792
1793 async function submitServiceRequest() {
1794     const deviceId = parseInt(document.getElementById('sr-device-select').value);
1795     const requestType = document.getElementById('sr-type').value;
1796     const issueDescription = document.getElementById('sr-description').value.trim();
1797     const urgent = document.getElementById('sr-urgent').checked;
1798
1799     if (!deviceId) { showAlert('Please select a device', 'error'); return; }
1800     if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
1801
1802     try {
1803         await apiFetch('/service-requests', {
1804             method: 'POST',
1805             body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
1806         });
1807         showAlert('Service request submitted!', 'success');
1808         clearServiceRequestForm();
1809         loadServiceRequestsList();
1810     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1811 }
1812
1813 function clearServiceRequestForm() {

```

```

1814     document.getElementById('sr-device-select').value = '';
1815     document.getElementById('sr-type').value = 'REPAIR';
1816     document.getElementById('sr-description').value = '';
1817     document.getElementById('sr-urgent').checked = false;
1818 }
1819
1820 async function searchEmployee() {
1821     const name = document.getElementById('org-search-input').value.trim();
1822     const errorEl = document.getElementById('org-search-error');
1823     const resultsEl = document.getElementById('org-search-results');
1824     const emptyEl = document.getElementById('org-search-empty');
1825
1826     errorEl.style.display = 'none';
1827     resultsEl.style.display = 'none';
1828     emptyEl.style.display = 'block';
1829
1830     if (!name) {
1831         errorEl.textContent = 'Please enter a name to search.';
1832         errorEl.style.display = 'block';
1833         return;
1834     }
1835
1836     try {
1837         const emp = await apiFetch(`/manager/employees/search?name=${encodeURIComponent(name)}`);
1838
1839         // Populate info card
1840         document.getElementById('sr-name').textContent = emp.name || '-';
1841         document.getElementById('sr-id').textContent = `#${emp.employeeId}`;
1842         document.getElementById('sr-email').textContent = emp.email || '-';
1843         document.getElementById('sr-dept').textContent = emp.dept || '-';
1844         document.getElementById('sr-attendance').textContent = emp.attendance || '-';
1845
1846         // Status chip
1847         const statusMap = { ACTIVE: 'approved', ON_LEAVE: 'pending', INACTIVE: 'rejected' };
1848         const statusKey = (emp.status || '').toString();
1849         document.getElementById('sr-status').innerHTML =
1850             `

```

```

1882         : 'Search failed: ' + e.message;
1883         errorEl.style.display = 'block';
1884         emptyEl.style.display = 'block';
1885     }
1886 }
1887
1888 function logout(){
1889     // Clear all localStorage
1890     localStorage.removeItem('jwt_token');
1891     localStorage.removeItem('refresh_token');
1892     localStorage.removeItem('auth_type');
1893     localStorage.removeItem('last_role');
1894
1895     // Call server logout to invalidate session
1896     fetch(`${API}/session/logout`, {
1897         method: 'POST',
1898         credentials: 'include'
1899     }).finally(() => {
1900         // Force expire the session cookie on client side
1901         document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1902         window.location.href = '/google.html';
1903     });
1904 }
1905 async function loadNotifCount() {
1906     try {
1907         const data = await apiFetch('/notifications/unread-count');
1908         const badge = document.getElementById('notif-badge');
1909         if (data.count > 0) {
1910             badge.textContent = data.count;
1911             badge.style.display = 'flex';
1912         } else {
1913             badge.style.display = 'none';
1914         }
1915     } catch(e) { console.warn('Could not load notif count', e); }
1916 }
1917
1918 async function toggleNotifPanel() {
1919     const panel = document.getElementById('notif-panel');
1920     if (panel.style.display === 'none') {
1921         panel.style.display = 'block';
1922         await loadNotifications();
1923         try { await apiFetch('/notifications/mark-read', { method: 'POST' }); } catch(e) {}
1924         document.getElementById('notif-badge').style.display = 'none';
1925     } else {
1926         panel.style.display = 'none';
1927     }
1928 }
1929
1930 async function loadNotifications() {
1931     try {
1932         const data = await apiFetch('/notifications');
1933         const list = document.getElementById('notif-list');
1934         list.innerHTML = data.length
1935             ? data.map(n => `
1936                 <div class="notif-item ${n.read ? '' : 'unread'}">
1937                     <div>${n.message}</div>
1938                     <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
1939                 </div>`).join('')
1940             : '<div class="notif-item">No notifications yet.</div>';
1941     } catch(e) { console.warn('Could not load notifications', e); }
1942 }
1943
1944 loadNotifCount();
1945 setInterval(loadNotifCount, 30000);
1946
1947 document.addEventListener('click', e => {
1948     const panel = document.getElementById('notif-panel');
1949     const wrapper = document.querySelector('.notif-wrapper');
1950     if (panel && wrapper && !panel.contains(e.target) && !wrapper.contains(e.target)) {
1951         panel.style.display = 'none';
1952     }
1953 });
1954 function showAlert(msg,type){
1955     const el=document.createElement('div');
1956     el.className=`alert alert-${type}`;el.textContent=msg;
1957     document.body.appendChild(el);setTimeout(()=>el.remove(),3000);

```

```
1958     }
1959 </script>
1960 </body>
1961 </html>
1962
```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Account Pending</title>
6      <style>
7          * { margin: 0; padding: 0; box-sizing: border-box; }
8          body {
9              font-family: 'Outfit', sans-serif;
10             background: #070910;
11             color: #dde4f0;
12             min-height: 100vh;
13             display: flex;
14             align-items: center;
15             justify-content: center;
16         }
17         .card {
18             background: #0e1118;
19             border: 1px solid rgba(255,255,255,0.07);
20             border-radius: 16px;
21             padding: 48px;
22             max-width: 420px;
23             text-align: center;
24         }
25         .icon { font-size: 48px; margin-bottom: 20px; }
26         h1 { font-size: 22px; font-weight: 700; margin-bottom: 12px; }
27         p { font-size: 14px; color: #6b7591; line-height: 1.6; margin-bottom: 24px; }
28         .email {
29             background: #131821;
30             border: 1px solid rgba(79,140,255,0.2);
31             border-radius: 8px;
32             padding: 10px 16px;
33             font-size: 13px;
34             color: #4f8cff;
35             margin-bottom: 28px;
36             font-family: monospace;
37         }
38         .btn {
39             padding: 10px 24px;
40             background: transparent;
41             border: 1px solid rgba(255,255,255,0.1);
42             border-radius: 8px;
43             color: #6b7591;
44             font-size: 13px;
45             cursor: pointer;
46         }
47         .btn:hover { border-color: rgba(255,255,255,0.2); color: #dde4f0; }
48     </style>
49 </head>
50 <body>
51 <div class="card">
52     <div class="icon"></div>
53     <h1>Account Pending Approval</h1>
54     <p>You've successfully signed in with Google. Your account is currently awaiting approval from an admin
or manager.</p>
55     <div class="email" id="user-email">Loading...</div>
56     <p>Once approved, you'll have full access to the portal. Please contact your manager or admin.</p>
57     <button class="btn" onclick="logout()">Sign out</button>
58 </div>
59
60 <script>
61     // Show the user their email from JWT
62     const token = localStorage.getItem('jwt_token');
63     if (token) {
64         try {
65             const payload = JSON.parse(atob(token.split('.')[1]));
66             document.getElementById('user-email').textContent = payload.sub || 'Unknown';
67         } catch(e) {}
68     }
69
70     function logout() {
71         localStorage.clear();

```

```
72         window.location.href = '/google.html';
73     }
74 </script>
75 </body>
76 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>TechVendor Hub</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;500;600;700;800&family=DM+Mono:wght@400;500&display
=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#080b10;--bg2:#0c1018;--bg3:#111620;--bg4:#161d2a;
12             --panel:#131a26;--panel2:#1a2235;
13             --border:rgba(0,210,255,0.08);--border2:rgba(0,210,255,0.18);--border3:rgba(0,210,255,0.35);
14             --text:#d4e8f5;--text2:#6b8aaa;--text3:#3a5270;
15             --cyan:#00d2ff;--cyan-dim:rgba(0,210,255,0.08);--cyan-glow:rgba(0,210,255,0.15);
16             --amber:#f59e0b;--amber-dim:rgba(245,158,11,0.1);
17             --green:#10d48e;--green-dim:rgba(16,212,142,0.1);
18             --red:#f05454;--red-dim:rgba(240,84,84,0.1);
19             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.1);
20             --orange:#fb923c;--orange-dim:rgba(251,146,60,0.1);
21             --font:'Syne',sans-serif;--mono:'DM Mono',monospace;
22             --sidebar:260px;
23         }
24         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
25         .app{display:flex;height:100vh}
26
27         body::before{content:'';position:fixed;inset:0;background:repeating-linear-
gradient(0deg,transparent,transparent 2px,rgba(0,0,0,0.03) 2px,rgba(0,0,0,0.03) 4px);pointer-events:none;z-
index:9...
28
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;overflow:hidden;position:relative}
30         .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:300px;background:linear-
gradient(to top,rgba(0,210,255,0.03),transparent);pointer-events:none}
31
32         .logo{padding:24px 20px 20px;border-bottom:1px solid var(--border)}
33         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
34         .logo-hex{width:36px;height:36px;background:linear-gradient(135deg,#00d2ff,#0066ff);clip-
path:polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0% 25%);display:flex;align-items:center;justifyf...
35         .logo-name{font-size:17px;font-weight:800;color:var(--text);letter-spacing:-0.5px}
36         .logo-sub{font-size:10px;color:var(--text3);font-family:var(--mono);letter-spacing:1px;text-
transform:uppercase}
37         .vendor-badge{display:inline-flex;align-items:center;gap:6px;margin-top:10px;padding:5px
12px;background:var(--cyan-dim);border:1px solid var(--border2);border-radius:4px;font-size:10px;font...
38         .badge-dot{width:5px;height:5px;background:var(--cyan);border-radius:50%;animation:pulse 2s
infinite}
39         @keyframes pulse{0%,100%{opacity:1;box-shadow:0 0 0 0 var(--cyan-glow)}50%{opacity:0.7;box-shadow:0
0 0 4px transparent}}
40
41         .nav{flex:1;padding:12px 0;overflow-y:auto}
42         .nav::-webkit-scrollbar{width:3px}
43         .nav::-webkit-scrollbar-thumb{background:var(--border2);border-radius:2px}
44         .nav-section{padding:14px 20px 5px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2.5px;text-transform:uppercase;font-family:var(--mono)}
45         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-
radius:6px;cursor:pointer;transition:all 0.15s;color:var(--text2);font-size:13px;font-weight:500;p...
46         .nav-item:hover{background:var(--cyan-dim);color:var(--cyan)}
47         .nav-item.active{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
48         .nav-item.active::before{content:'';position:absolute;left:-8px;top:50%;transform:translateY(-
50%);width:3px;height:60%;background:var(--cyan);border-radius:0 2px 2px 0}
49         .nav-icon{width:16px;height:16px;opacity:0.7;flex-shrink:0}
50         .nav-item.active .nav-icon,.nav-item:hover .nav-icon{opacity:1}
51         .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:9px;font-
weight:700;padding:2px 6px;border-radius:3px;font-family:var(--mono)}
52         .nav-badge.amber{background:var(--amber)}
53
54         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
55         .user-row{display:flex;align-items:center;gap:10px}
56         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#00d2ff22,#0066ff22);border:1px

```

```

solid var(--border2);border-radius:6px;display:flex;align-items:center;justify-content:cente...
57 .user-name{font-size:12px;font-weight:600;color:var(--text)}
58 .user-role{font-size:10px;color:var(--cyan);font-family:var(--mono);text-transform:uppercase;letter-
spacing:0.8px}
59
60 .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
61 .topbar{background:var(--bg2);padding:0 24px;height:56px;display:flex;justify-content:space-
between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
62 .page-title{font-size:15px;font-weight:700;color:var(--text);display:flex;align-
items:center;gap:8px}
63 .page-title-prefix{color:var(--cyan);font-family:var(--mono);font-size:12px;font-weight:400}
64 .topbar-right{display:flex;align-items:center;gap:10px}
65 .topbar-email{font-size:11px;color:var(--text3);font-family:var(--mono)}
66 .signout-btn{padding:6px 14px;background:transparent;border:1px solid var(--border2);border-
radius:5px;color:var(--text2);font-family:var(--font);font-size:11px;font-weight:600;cursor:point...
67 .signout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
68
69 .content{flex:1;overflow-y:auto;padding:24px}
70 .content::-webkit-scrollbar{width:4px}
71 .content::-webkit-scrollbar-thumb{background:var(--border2)}
72 .view{display:none}
73 .view.active{display:block;animation:viewIn 0.25s ease}
74 @keyframes viewIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}
75
76 .stats-row{display:grid;grid-template-columns:repeat(4,1fr);gap:14px;margin-bottom:22px}
77 .stat{background:var(--panel);border:1px solid var(--border);border-radius:8px;padding:18px
20px;position:relative;overflow:hidden;cursor:default;transition:border-color 0.2s}
78 .stat:hover{border-color:var(--border2)}
79 .stat::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
80 .stat.c{--c:var(--cyan)} .stat.g{--c:var(--green)} .stat.a{--c:var(--amber)} .stat.r{--c:var(--red)}
.stat.v{--c:var(--violet)} .stat.o{--c:var(--orange)}
81 .stat::after{background:var(--c)}
82 .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;font-
weight:600;margin-bottom:8px;font-family:var(--mono)}
83 .stat-val{font-size:28px;font-weight:800;color:var(--text);line-height:1;font-family:var(--mono)}
84 .stat-sub{font-size:11px;color:var(--text3);margin-top:5px}
85 .stat-icon{position:absolute;right:16px;top:50%;transform:translateY(-50%);font-
size:24px;opacity:0.12}
86
87 .card{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-
bottom:18px;overflow:hidden}
88 .card-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
89 .card-title{font-size:13px;font-weight:700;color:var(--text);letter-spacing:0.2px}
90 .card-sub{font-size:11px;color:var(--text3);margin-top:2px;font-family:var(--mono)}
91 .card-body{padding:20px}
92 .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:18px}
93
94 .tablewrap{overflow-x:auto}
95 table{width:100%;border-collapse:collapse}
96 th{padding:9px 16px;text-align:left;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
97 td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:12px;color:var(--text2);font-
family:var(--mono)}
98 tr:last-child td{border-bottom:none}
99 tr:hover td{background:rgba(0,210,255,0.02);color:var(--text)}
100 td.main-cell{font-family:var(--font);font-size:13px;color:var(--text);font-weight:500}
101
102 .chip{display:inline-flex;align-items:center;gap:5px;padding:3px 9px;border-radius:3px;font-
size:10px;font-weight:700;letter-spacing:0.8px;text-transform:uppercase;font-family:var(--mono)}
103 .chip::before{content:'';width:4px;height:4px;border-radius:50%;flex-shrink:0}
104 .chip.available{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)} .chip.available::before{background:var(--green)}
105 .chip.assigned{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.assigned::before{background:var(--cyan)}
106 .chip.under_repair{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(245,158,11,0.2)} .chip.under_repair::before{background:var(--amber)}
107 .chip.condemned{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
.chip.condemned::before{background:var(--red)}
108 .chip.returned_to_vendor{background:var(--violet-dim);color:var(--violet);border:1px solid
rgba(167,139,250,0.2)} .chip.returned_to_vendor::before{background:var(--violet)}
109 .chip.open{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
.chip.open::before{background:var(--amber)}
110 .chip.acknowledged{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.acknowledged::before{background:var(--cyan)}
111 .chip.sent_for_repair{background:var(--orange-dim);color:var(--orange);border:1px solid

```

```

111 rgba(251,146,60,0.2)} .chip.sent_for_repair::before{background:var(--orange)}
112 .chip.repair_done{background:var(--green-dim);color:var(--green);border:1px solid
113 rgba(16,212,142,0.2)} .chip.repair_done::before{background:var(--green)}
114 .chip.resolved{background:var(--green-dim);color:var(--green);border:1px solid
115 rgba(16,212,142,0.2)} .chip.resolved::before{background:var(--green)}
116 .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
117 .chip.rejected::before{background:var(--red)}
118 .chip.sent{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
119 .chip.sent::before{background:var(--amber)}
120 .chip.in_progress{background:var(--orange-dim);color:var(--orange);border:1px solid
121 rgba(251,146,60,0.2)} .chip.in_progress::before{background:var(--orange)}
122 .chip.fixed{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
123 .chip.fixed::before{background:var(--green)}
124 .chip.irreparable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
125 .chip.irreparable::before{background:var(--red)}
126 .chip.laptop{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
127 .chip.laptop::before{background:var(--cyan)}
128 .chip.phone{background:var(--violet-dim);color:var(--violet);border:1px solid
129 rgba(167,139,250,0.2)} .chip.phone::before{background:var(--violet)}
130 .chip.tablet{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
131 .chip.tablet::before{background:var(--green)}
132 .chip.monitor,.chip.other,.chip.desktop,.chip.keyboard,.chip.mouse,.chip.headset{background:var(--
133 amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)} .chip.monitor::before,...
134 .chip.active_assign{background:var(--green-dim);color:var(--green);border:1px solid
135 rgba(16,212,142,0.2)} .chip.active_assign::before{background:var(--green)}
136 .chip.returned{background:var(--text3);color:var(--text2);border:1px solid var(--border)}
137 .chip.returned::before{background:var(--text2)}
138 .chip.not_fixable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
139 .chip.not_fixable::before{background:var(--red)}
140
141
142 .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--font);font-size:12px;font-
143 weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.3px}
144 .btn-primary{background:var(--cyan);color:#080b10}
145 .btn-primary:hover{background:#33dbff;box-shadow:0 0 20px var(--cyan-glow)}
146 .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border2)}
147 .btn-ghost:hover{border-color:var(--border3);color:var(--text)}
148 .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
149 .btn-danger:hover{background:var(--red);color:#fff}
150 .btn-warn{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
151 .btn-warn:hover{background:var(--amber);color:#080b10}
152 .btn-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
153 .btn-success:hover{background:var(--green);color:#080b10}
154 .btn-sm{padding:5px 10px;font-size:11px}
155 .btn-xs{padding:3px 8px;font-size:10px}
156 .action-row{display:flex;gap:6px;flex-wrap:wrap}
157
158
159 .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
160 .form-group{display:flex;flex-direction:column;gap:6px}
161 .form-group.full{grid-column:1/-1}
162 .form-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
163 spacing:1.5px;font-family:var(--mono)}
164 .form-input{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:10px
165 13px;color:var(--text);font-family:var(--mono);font-size:12px;transition:border-color 0.2s;wi...
166 .form-input:focus{outline:none;border-color:var(--cyan);box-shadow:0 0 3px var(--cyan-dim)}
167 .form-input:placeholder{color:var(--text3)}
168 select.form-input option{background:var(--bg2)}
169 .form-actions{display:flex;gap:10px;margin-top:18px}
170
171
172 .empty{text-align:center;padding:40px 20px;color:var(--text3);font-family:var(--mono);font-
173 size:12px}
174
175 .empty-icon{font-size:32px;margin-bottom:10px;opacity:0.4}
176
177 .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:6px;z-
178 index:2000;animation:alertIn 0.25s ease;font-size:12px;font-weight:600;font-family:var(--mono);display:flex;a...
179 .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
180 rgba(16,212,142,0.3)}
181 .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.3)}
182 @keyframes alertIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
183
184
185 .search-row{display:flex;gap:10px;padding:14px 16px;border-bottom:1px solid var(--border)}
186 .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
187 radius:6px;padding:8px 13px;color:var(--text);font-family:var(--mono);font-size:12px}
188 .search-input:focus{outline:none;border-color:var(--cyan)}
189 .search-input:placeholder{color:var(--text3)}
190
191
192 .timeline{padding:20px;display:flex;flex-direction:column;gap:0}

```

```

166     .tl-item{display:flex;gap:14px;padding-bottom:20px;position:relative}
167     .tl-item:last-child{padding-bottom:0}
168     .tl-item:not(:last-
child)::after{content:'';position:absolute;left:14px;top:28px;bottom:0;width:1px;background:var(--border)}
169     .tl-dot{width:28px;height:28px;border-radius:50%;border:2px solid var(--border2);background:var(--
bg3);display:flex;align-items:center;justify-content:center;flex-shrink:0;font-size:11px;z-...
170     .tl-dot.cyan{border-color:var(--cyan);background:var(--cyan-dim)}
171     .tl-dot.amber{border-color:var(--amber);background:var(--amber-dim)}
172     .tl-dot.green{border-color:var(--green);background:var(--green-dim)}
173     .tl-dot.red{border-color:var(--red);background:var(--red-dim)}
174     .tl-content{flex:1;padding-top:4px}
175     .tl-title{font-size:12px;font-weight:600;color:var(--text);margin-bottom:2px}
176     .tl-meta{font-size:10px;color:var(--text3);font-family:var(--mono)}
177
178     /* Outcome hint box */
179     .outcome-hint{margin:12px 0 0;padding:10px 14px;border-radius:6px;font-size:11px;font-family:var(--
mono);transition:all 0.2s}
180     .outcome-hint.success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)}
181     .outcome-hint.danger{background:var(--red-dim);color:var(--red);border:1px solid
rgba(240,84,84,0.2)}
182
183     /* Inline repair panel */
184     .repair-inline{background:var(--bg3);border:1px solid var(--border2);border-radius:8px;padding:18px
20px;margin:0 16px 16px;animation:viewIn 0.2s ease}
185
186     .modal-overlay{display:none;position:fixed;inset:0;background:rgba(8,11,16,0.85);z-
index:1500;align-items:center;justify-content:center;backdrop-filter:blur(4px)}
187     .modal-overlay.open{display:flex}
188     .modal{background:var(--panel2);border:1px solid var(--border2);border-radius:12px;width:560px;max-
width:95%;max-height:85vh;overflow-y:auto;animation:modalIn 0.2s ease}
189     @keyframes modalIn{from{opacity:0;transform:scale(0.96)}to{opacity:1;transform:scale(1)}}
190     .modal-head{padding:20px 24px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
191     .modal-title{font-size:16px;font-weight:700;color:var(--text)}
192     .modal-close{background:none;border:none;color:var(--text3);font-size:20px;cursor:pointer;padding:0
4px;transition:color 0.2s}
193     .modal-close:hover{color:var(--text)}
194     .modal-body{padding:24px}
195     .detail-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-bottom:20px}
196     .detail-item{display:flex;flex-direction:column;gap:4px}
197     .detail-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:1.5px;font-family:var(--mono)}
198     .detail-value{font-size:13px;color:var(--text);font-family:var(--mono)}
199 </style>
200 </head>
201 <body>
202 <div class="app">
203
204     <!-- SIDEBAR -->
205     <div class="sidebar">
206         <div class="logo">
207             <div class="logo-mark">
208                 <div class="logo-hex">TV</div>
209                 <div>
210                     <div class="logo-name">TechVendor</div>
211                     <div class="logo-sub">Asset Hub</div>
212                 </div>
213             </div>
214             <div class="vendor-badge"><span class="badge-dot"></span>Tech Vendor</div>
215         </div>
216
217         <nav class="nav">
218             <div class="nav-section">Overview</div>
219             <div class="nav-item active" onclick="showView('dashboard',this)">
220                 <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="1" width="6"
height="6" rx="1" stroke="currentColor" stroke-width="1.5"/><rect x="9" y="1" width="6" height="6" ...
221                 Dashboard
222             </div>
223
224             <div class="nav-section">Devices</div>
225             <div class="nav-item" onclick="showView('my-devices',this)">
226                 <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="3" width="14"
height="10" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M5 13v2M11 13v2M3 15h10" str...
227                 My Devices
228             </div>

```

```
229         <div class="nav-item" onclick="showView('add-device',this)">
230             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="8" r="7"
stroke="currentColor" stroke-width="1.5"/><path d="M8 5v6M5 8h6" stroke="currentColor" stroke-width...
231             Add Device
232         </div>
233
234         <div class="nav-section">Service</div>
235         <div class="nav-item" onclick="showView('service-requests',this)">
236             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><path d="M8 11l1.5 4.5H14l-3.5 2.5 1.5 1.5
4.5L8 10l-4 2.5 1.5-4.5L2 5.5h4.5L8 1z" stroke="currentColor" stroke-width="1.5" stroke-l...
237             Service Requests
238             <span class="nav-badge amber" id="sr-badge" style="display:none">0</span>
239         </div>
240
241         <div class="nav-section">Assignments</div>
242         <div class="nav-item" onclick="showView('assignments',this)">
243             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="6" cy="5" r="3"
stroke="currentColor" stroke-width="1.5"/><path d="M1 14c0-3 2-5 5-5" stroke="currentColor" stroke-...
244             Assignments
245         </div>
246
247         <div class="nav-section">Account</div>
248         <div class="nav-item" onclick="showView('profile',this)">
249             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="5" r="3"
stroke="currentColor" stroke-width="1.5"/><path d="M2 14c0-3.3 2.7-6 6-6s6 2.7 6 6" stroke="current...
250             My Profile
251         </div>
252     </nav>
253
254     <div class="sidebar-footer">
255         <div class="user-row">
256             <div class="avatar" id="sidebar-avatar">TV</div>
257             <div>
258                 <div class="user-name" id="sidebar-name">TechVendor</div>
259                 <div class="user-role">Tech Vendor</div>
260             </div>
261         </div>
262     </div>
263 </div>
264
265 <!-- MAIN -->
266 <div class="main">
267     <div class="topbar">
268         <div class="page-title">
269             <span class="page-title-prefix">// </span>
270             <span id="pageTitle">dashboard</span>
271         </div>
272         <div class="topbar-right">
273             <span class="topbar-email" id="topbar-email"></span>
274             <button class="signout-btn" onclick="logout()">Sign Out</button>
275         </div>
276     </div>
277
278     <div class="content">
279
280         <!-- DASHBOARD -->
281         <div id="dashboard-view" class="view active">
282             <div class="stats-row">
283                 <div class="stat c">
284                     <div class="stat-label">Total Devices</div>
285                     <div class="stat-val" id="stat-total">--</div>
286                     <div class="stat-sub">in catalog</div>
287                     <div class="stat-icon"></div>
288                 </div>
289                 <div class="stat g">
290                     <div class="stat-label">Available</div>
291                     <div class="stat-val" id="stat-avail">--</div>
292                     <div class="stat-sub">ready to assign</div>
293                     <div class="stat-icon"></div>
294                 </div>
295                 <div class="stat a">
296                     <div class="stat-label">Pending Repairs</div>
297                     <div class="stat-val" id="stat-repair">--</div>
298                     <div class="stat-sub">awaiting your update</div>
299                     <div class="stat-icon"></div>
300             </div>
```

```

301         <div class="stat r">
302             <div class="stat-label">Open Requests</div>
303             <div class="stat-val" id="stat-open">--</div>
304             <div class="stat-sub">sent for repair</div>
305             <div class="stat-icon">!</div>
306         </div>
307     </div>
308
309     <div class="dash-grid">
310         <div class="card">
311             <div class="card-head">
312                 <div>
313                     <div class="card-title">Repair Queue</div>
314                     <div class="card-sub">devices sent to you for repair</div>
315                 </div>
316                 <button class="btn btn-ghost btn-sm" onclick="showView('service-
requests',null)">View all </button>
317             </div>
318             <div class="tablewrap">
319                 <table>
320                     <thead><tr><th>Device</th><th>Issue</th><th>Status</th></tr></thead>
321                     <tbody id="dash-sr"></tbody>
322                 </table>
323             </div>
324         </div>
325
326         <div class="card">
327             <div class="card-head">
328                 <div>
329                     <div class="card-title">Active Repairs</div>
330                     <div class="card-sub">currently being serviced</div>
331                 </div>
332                 <button class="btn btn-ghost btn-sm" onclick="showView('repair-
logs',null)">View all </button>
333             </div>
334             <div class="tablewrap">
335                 <table>
336                     <thead><tr><th>Device</th><th>Sent</th><th>Status</th></tr></thead>
337                     <tbody id="dash-repairs"></tbody>
338                 </table>
339             </div>
340         </div>
341     </div>
342
343     <div class="card">
344         <div class="card-head">
345             <div class="card-title">Device Status Overview</div>
346         </div>
347         <div class="tablewrap">
348             <table>
349                 <thead><tr><th>Device</th><th>Type</th><th>Status</th><th>Assigned
To</th></tr></thead>
350                 <tbody id="dash-devices"></tbody>
351             </table>
352         </div>
353     </div>
354 </div>
355
356 <!-- MY DEVICES -->
357 <div id="my-devices-view" class="view">
358     <div class="card">
359         <div class="card-head">
360             <div>
361                 <div class="card-title">My Device Catalog</div>
362                 <div class="card-sub">all devices supplied by you</div>
363             </div>
364             <div class="action-row">
365                 <select class="form-input btn-sm" id="device-filter" onchange="filterDevices()"
style="width:140px;padding:5px 10px;font-size:11px">
366                     <option value="">All Status</option>
367                     <option value="AVAILABLE">Available</option>
368                     <option value="ASSIGNED">Assigned</option>
369                     <option value="UNDER_REPAIR">Under Repair</option>
370                     <option value="CONDEMNED">Condemned</option>
371                     <option value="RETURNED_TO_VENDOR">Returned</option>
372                 </select>

```



```

373         <button class="btn btn-ghost btn-sm" onclick="loadMyDevices()">
Refresh</button>
374     </div>
375 </div>
376 <div class="search-row">
377     <input class="search-input" id="device-search" placeholder="Search by name, serial,
brand..." oninput="filterDevices()">
378 </div>
379 <div class="tablewrap">
380     <table>
381         <thead><tr><th>ID</th><th>Device
Name</th><th>Brand</th><th>Type</th><th>Warranty</th><th>Status</th><th>Actions</th></tr></thead>
382         <tbody id="devices-list"></tbody>
383     </table>
384 </div>
385 </div>
386 </div>
387
388 <!-- ADD DEVICE -->
389 <div id="add-device-view" class="view">
390     <div class="card" style="max-width:700px">
391         <div class="card-head">
392             <div>
393                 <div class="card-title">Add New Device</div>
394                 <div class="card-sub">register a device to your catalog</div>
395             </div>
396         </div>
397         <div class="card-body">
398             <div class="form-grid">
399                 <div class="form-group">
400                     <label class="form-label">Device Name</label>
401                     <input class="form-input" id="d-name" placeholder="Dell XPS 15 9520">
402                 </div>
403                 <div class="form-group">
404                     <label class="form-label">Brand</label>
405                     <input class="form-input" id="d-brand" placeholder="Dell">
406                 </div>
407                 <div class="form-group">
408                     <label class="form-label">Device Type</label>
409                     <select class="form-input" id="d-type">
410                         <option value="LAPTOP">Laptop</option>
411                         <option value="DESKTOP">Desktop</option>
412                         <option value="PHONE">Phone</option>
413                         <option value="TABLET">Tablet</option>
414                         <option value="MONITOR">Monitor</option>
415                         <option value="KEYBOARD">Keyboard</option>
416                         <option value="MOUSE">Mouse</option>
417                         <option value="HEADSET">Headset</option>
418                         <option value="OTHER">Other</option>
419                     </select>
420                 </div>
421                 <div class="form-group">
422                     <label class="form-label">Warranty Expiry</label>
423                     <input class="form-input" type="date" id="d-warranty">
424                 </div>
425             </div>
426             <div class="form-actions">
427                 <button class="btn btn-primary" onclick="addDevice()">Register Device</button>
428                 <button class="btn btn-ghost" onclick="clearDeviceForm()">Clear</button>
429             </div>
430         </div>
431     </div>
432 </div>
433
434 <!--
435     SERVICE REQUESTS - vendor repair queue
436     GET /service-requests/vendor/me     SENT_FOR_REPAIR list
437     PUT /service-requests/vendor       RepairDTO { requestId, resolution, status }
438     status=REPAIR_DONE   device becomes ASSIGNED
439     status=anything else device becomes CONDEMNED
440 -->
441 <div id="service-requests-view" class="view">
442     <div class="card">
443         <div class="card-head">
444             <div>
445                 <div class="card-title">Repair Queue</div>

```

```

446             <div class="card-sub">devices sent to you – mark as repaired or condemned</div>
447         </div>
448         <button class="btn btn-ghost btn-sm" onclick="loadServiceRequests()">
Refresh</button>
449     </div>
450     <div class="tablewrap">
451         <table>
452             <thead>
453                 <tr>
454                     <th>ID</th>
455                     <th>Device</th>
456                     <th>Issue Type</th>
457                     <th>Description</th>
458                     <th>Urgent</th>
459                     <th>Raised</th>
460                     <th>Actions</th>
461                 </tr>
462             </thead>
463             <tbody id="sr-list"></tbody>
464         </table>
465     </div>
466 </div>
467
468 <!-- INLINE REPAIR UPDATE PANEL -->
469 <div class="card" id="vendor-repair-panel" style="max-width:640px;display:none">
470     <div class="card-head">
471         <div>
472             <div class="card-title">Update Repair Result</div>
473             <div class="card-sub" id="vr-panel-subtitle">Request #</div>
474         </div>
475         <button class="btn btn-ghost btn-sm" onclick="closeRepairPanel()"> Cancel</button>
476     </div>
477     <div class="card-body">
478         <input type="hidden" id="vr-request-id">
479
480         <div class="form-grid">
481             <!-- Outcome selector -->
482             <div class="form-group full">
483                 <label class="form-label">Repair Outcome</label>
484                 <select class="form-input" id="vr-status" onchange="onOutcomeChange()">
485                     <option value="REPAIR_DONE"> Repair Done – device returns to
employee</option>
486                     <option value="NOT_FIXABLE"> Could Not Repair – device will be
condemned</option>
487                 </select>
488             </div>
489
490             <!-- Resolution notes -->
491             <div class="form-group full">
492                 <label class="form-label">Resolution Notes</label>
493                 <input class="form-input" id="vr-resolution"
placeholder="Describe what was repaired, parts replaced, or why it
could not be fixed...">
494             </div>
495         </div>
496
497             <!-- Dynamic outcome hint -->
498             <div id="vr-outcome-hint" class="outcome-hint success">
Device status <strong>ASSIGNED</strong> – returned to the employee
500             </div>
501
502             <div class="form-actions">
503                 <button class="btn btn-primary" onclick="submitRepairUpdate()">Submit
Update</button>
504                 <button class="btn btn-ghost" onclick="closeRepairPanel()">Cancel</button>
505             </div>
506         </div>
507     </div>
508 </div>
509 </div>
510
511 <!-- ASSIGNMENTS -->
512 <div id="assignments-view" class="view">
513     <div class="card">
514         <div class="card-head">
515             <div>
516                 <div class="card-title">Device Assignments</div>

```

```

517         <div class="card-sub">who has your devices</div>
518     </div>
519     <button class="btn btn-ghost btn-sm" onclick="loadAssignments()"> Refresh</button>
520 </div>
521 <div class="tablewrap">
522     <table>
523         <thead>
524             <tr>
525                 <th>Device</th>
526                 <th>Type</th>
527                 <th>Employee</th>
528                 <th>Assigned Date</th>
529                 <th>Status</th>
530             </tr>
531         </thead>
532         <tbody id="assignments-list"></tbody>
533     </table>
534 </div>
535 </div>
536 </div>
537
538 <!-- PROFILE -->
539 <div id="profile-view" class="view">
540     <div class="card" style="max-width:600px">
541         <div class="card-head">
542             <div class="card-title">My Profile</div>
543         </div>
544         <div class="card-body">
545             <div style="display:flex;align-items:center;gap:16px;margin-bottom:24px;padding-
bottom:20px;border-bottom:1px solid var(--border)">
546                 <div style="width:56px;height:56px;background:var(--cyan-dim);border:2px solid
var(--border2);border-radius:10px;display:flex;align-items:center;justify-content:center;f...
547                 <div>
548                     <div style="font-size:18px;font-weight:700;color:var(--text)" id="profile-
name">--</div>
549                     <div style="font-size:11px;color:var(--cyan);font-family:var(--
mono);margin-top:3px">TECH_VENDOR</div>
550                 </div>
551             </div>
552             <div class="detail-grid">
553                 <div class="detail-item"><div class="detail-label">Email</div><div
class="detail-value" id="profile-email">--</div></div>
554                 <div class="detail-item"><div class="detail-label">Phone</div><div
class="detail-value" id="profile-phone">--</div></div>
555                 <div class="detail-item"><div class="detail-label">Registered</div><div
class="detail-value" id="profile-reg">--</div></div>
556                 <div class="detail-item"><div class="detail-label">Status</div><div
class="detail-value"><span class="chip active_assign">ACTIVE</span></div></div>
557             </div>
558         </div>
559     </div>
560 </div>
561
562 </div>
563 </div>
564 </div>
565
566 <!-- DEVICE DETAIL MODAL -->
567 <div class="modal-overlay" id="device-modal">
568     <div class="modal">
569         <div class="modal-head">
570             <div class="modal-title" id="modal-device-name">Device Details</div>
571             <button class="modal-close" onclick="closeModal()"></button>
572         </div>
573         <div class="modal-body">
574             <div class="detail-grid" id="modal-detail-grid"></div>
575             <div style="margin-top:16px">
576                 <div class="card-title" style="margin-bottom:12px;font-size:12px;color:var(--text3);font-
family:var(--mono);text-transform:uppercase;letter-spacing:1px">Repair History</div>
577                 <div id="modal-repair-history"></div>
578             </div>
579         </div>
580     </div>
581 </div>
582
583 <script>

```

```

584     const API = 'http://localhost:8080';
585     let devicesCache = [], srCache = [], repairsCache = [], assignmentsCache = [];
586
587     // AUTH
588     (function(){
589         const token = localStorage.getItem('jwt_token');
590         if(!token){ window.location.href='/login.html'; return; }
591         try{
592             const p = JSON.parse(atob(token.split('.')[1]));
593             const role = (p.role||'').replace(/^ROLE_/, '');
594             if(role !== 'TECH_VENDOR'){ window.location.href='/google.html'; return; }
595             const email = p.sub||p.email||'TechVendor';
596             document.getElementById('topbar-email').textContent = email;
597             document.getElementById('sidebar-name').textContent = email.split('@')[0];
598             document.getElementById('sidebar-avatar').textContent = email.charAt(0).toUpperCase();
599             document.getElementById('profile-avatar').textContent = email.charAt(0).toUpperCase();
600             document.getElementById('profile-email').textContent = email;
601         }catch(e){ window.location.href='/login.html'; }
602         initDashboard();
603     })();
604
605     function getAuthHeader(){
606         const t = localStorage.getItem('jwt_token');
607         return t ? {'Authorization': `Bearer ${t}`} : {};
608     }
609
610     async function apiFetch(path, opts={}){
611         const res = await fetch(`${API}${path}`, {
612             ...opts,
613             headers: {'Content-Type': 'application/json', ...getAuthHeader(), ...(opts.headers||{})},
614             credentials: 'include'
615         });
616         if(res.status===401){
617             localStorage.removeItem('jwt_token');
618             window.location.href='/login.html';
619             throw new Error('Unauthorized');
620         }
621         const text = await res.text();
622         if(!res.ok) throw new Error(text || 'Request failed');
623         return text ? JSON.parse(text) : null;
624     }
625
626     // NAV
627     function showView(name, el){
628         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
629         document.getElementById(`${name}-view`).classList.add('active');
630         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
631         if(el) el.classList.add('active');
632         document.getElementById('pageTitle').textContent = name.replace(/-/g, ' ');
633         const loaders = {
634             'dashboard': loadDashboard,
635             'my-devices': loadMyDevices,
636             'service-requests': loadServiceRequests,
637             'repair-logs': loadRepairLogs,
638             'assignments': loadAssignments,
639             'profile': loadProfile,
640             'add-device': ()=>{}
641         };
642         if(loaders[name]) loaders[name]();
643     }
644
645     // INIT
646     async function initDashboard(){
647         await Promise.all([loadMyDevices(), loadServiceRequests(), loadRepairLogs(), loadAssignments()]);
648         loadDashboard();
649     }
650
651     // DASHBOARD
652     function loadDashboard(){
653         document.getElementById('stat-total').textContent = devicesCache.length || 0;
654         document.getElementById('stat-avail').textContent =
655         devicesCache.filter(d=>d.deviceStatus==='AVAILABLE').length || 0;
656         document.getElementById('stat-repair').textContent = srCache.length || 0;
657         document.getElementById('stat-open').textContent = srCache.length || 0;
658
659         const badge = document.getElementById('sr-badge');

```

```

659     if(srCache.length){ badge.textContent=srCache.length; badge.style.display=''; }
660     else badge.style.display='none';
661
662     document.getElementById('dash-sr').innerHTML = srCache.slice(0,5).map(s=>`
663         <tr>
664             <td class="main-cell">${s.deviceName||'Device #' +s.deviceId}</td>
665             <td>${(s.requestType||'').toLowerCase().replace('_',' ')}</td>
666             <td>${chip('SENT_FOR_REPAIR')}</td>
667         </tr>`).join('') ||
668         `<tr><td colspan="3" class="empty">No pending repairs</td></tr>`;
669
670     document.getElementById('dash-repairs').innerHTML =
repairsCache.filter(r=>r.repairStatus!=='FIXED'&&r.repairStatus!=='IRREPARABLE').slice(0,5).map(r=>`
671         <tr>
672             <td class="main-cell">${r.deviceName||'Device #' +r.deviceId}</td>
673             <td>${r.sentDate||'-'}</td>
674             <td>${chip(r.repairStatus)}</td>
675         </tr>`).join('') ||
676         `<tr><td colspan="3" class="empty">No active repairs</td></tr>`;
677
678     document.getElementById('dash-devices').innerHTML = devicesCache.slice(0,8).map(d=>`
679         <tr>
680             <td class="main-cell">${d.deviceName||'-'}</td>
681             <td>${chip(d.deviceType)}</td>
682             <td>${chip(d.deviceStatus)}</td>
683             <td>${d.assignedEmployeeName||'-'}</td>
684         </tr>`).join('') ||
685         `<tr><td colspan="4" class="empty">No devices</td></tr>`;
686     }
687
688     // DEVICES
689     async function loadMyDevices(){
690         try{
691             const data = await apiFetch('/devices/vendor/me');
692             devicesCache = data || [];
693             renderDevices(devicesCache);
694         }catch(e){ renderDevices([]); }
695     }
696
697     function renderDevices(list){
698         document.getElementById('devices-list').innerHTML = list.length
699         ? list.map(d=>`<tr>
700             <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${d.id}</td>
701             <td class="main-cell">${d.deviceName||'-'}</td>
702             <td>${d.brand||'-'}</td>
703             <td>${chip(d.deviceType)}</td>
704             <td style="font-size:11px;font-family:var(--mono)">
705                 ${d.warrantyExpiryDate
706                 ? (new Date(d.warrantyExpiryDate) > new Date()
707                 ? `<span style="color:var(--green)"> ${d.warrantyExpiryDate}</span>`
708                 : `<span style="color:var(--red)"> ${d.warrantyExpiryDate}</span>`)
709                 : `<span style="color:var(--text3)">—</span>`}
710             </td>
711             <td>${chip(d.deviceStatus)}</td>
712             <td><div class="action-row">
713                 <button class="btn btn-ghost btn-xs" onclick="viewDevice(${d.id})">Details</button>
714             </div></td>
715         </tr>`).join('')
716         : `<tr><td colspan="7"><div class="empty"><div class="empty-icon"></div>No devices registered
yet</div></tr>`;
717     }
718
719     function filterDevices(){
720         const q = document.getElementById('device-search').value.toLowerCase();
721         const f = document.getElementById('device-filter').value;
722         renderDevices(devicesCache.filter(d=>{
723             const matchQ = !q ||
(d.deviceName||'').toLowerCase().includes(q)||d.serialNumber||'').toLowerCase().includes(q)||d.brand||'').toLo
werCase().includes(q);
724             const matchF = !f || d.deviceStatus === f;
725             return matchQ && matchF;
726         }));
727     }
728
729     async function addDevice(){
730         const body = {

```

```

731     deviceName: document.getElementById('d-name').value.trim(),
732     brand: document.getElementById('d-brand').value.trim(),
733     deviceType: document.getElementById('d-type').value,
734     warrantyExpiryDate: document.getElementById('d-warranty').value || null,
735 };
736 if(!body.deviceName || !body.brand || !body.deviceType){
737     showAlert('Device name, brand and type are required','error'); return;
738 }
739 try{
740     await apiFetch('/devices', {method:'POST', body:JSON.stringify(body)});
741     showAlert('Device registered successfully!','success');
742     clearDeviceForm();
743     loadMyDevices();
744 }catch(e){ showAlert('Failed: '+e.message,'error'); }
745 }
746
747 function clearDeviceForm(){
748     ['d-name','d-brand','d-warranty'].forEach(id=>document.getElementById(id).value='');
749     document.getElementById('d-type').value='LAPTOP';
750 }
751
752 // SERVICE REQUESTS (vendor repair queue)
753 // GET /service-requests/vendor/me list of SENT_FOR_REPAIR requests
754 // PUT /service-requests/vendor { requestId, resolution, status }
755 // REPAIR_DONE device becomes ASSIGNED (back to employee)
756 // anything else device becomes CONDEMNED
757 //
758 async function loadServiceRequests(){
759     try{
760         const data = await apiFetch('/service-requests/vendor/me');
761         srCache = data || [];
762         renderServiceRequests(srCache);
763         // refresh badge
764         const badge = document.getElementById('sr-badge');
765         if(srCache.length){ badge.textContent=srCache.length; badge.style.display=''; }
766         else badge.style.display='none';
767     }catch(e){ renderServiceRequests([]); }
768 }
769
770 function renderServiceRequests(list){
771     document.getElementById('sr-list').innerHTML = list.length
772     ? list.map(s=><tr>
773         <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${s.id}</td>
774         <td class="main-cell">${s.deviceName}||'Device #'+s.deviceId</td>
775         <td>${(s.requestType||'').toLowerCase().replace(/_/g,' ')}</td>
776         <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap;color:var(--text2)">${s.issueDescription}||'-'</td>
777         <td>${s.urgent
778             ? '<span style="color:var(--red);font-family:var(--mono);font-size:10px;font-weight:700">
URGENT</span>'
779             : '<span style="color:var(--text3);font-family:var(--mono);font-size:10px"></span>'}</td>
780         <td style="font-size:10px;font-family:var(--mono);color:var(--text3)">${s.raisedAt} ?
s.raisedAt.split('T')[0] : '-'</td>
781         <td>
782             <button class="btn btn-success btn-xs" onclick="openRepairPanel(${s.id},
'${(s.deviceName}||'Device #'+s.deviceId).replace(/'/g,'\\')}')">
783                 Update Repair
784             </button>
785         </td>
786     </tr>`).join('')
787     : `<tr><td colspan="7"><div class="empty"><div class="empty-icon"></div>No devices in repair
queue</div></td></tr>`;
788 }
789
790 // REPAIR PANEL
791 function openRepairPanel(requestId, deviceName){
792     document.getElementById('vr-request-id').value = requestId;
793     document.getElementById('vr-panel-subtitle').textContent = `Request #${requestId} - ${deviceName}`;
794     document.getElementById('vr-status').value = 'REPAIR_DONE';
795     document.getElementById('vr-resolution').value = '';
796     onOutcomeChange(); // set correct hint
797     const panel = document.getElementById('vendor-repair-panel');
798     panel.style.display = 'block';
799     panel.scrollIntoView({behavior:'smooth', block:'start'});
800 }
801

```

```

802     function closeRepairPanel(){
803         document.getElementById('vendor-repair-panel').style.display = 'none';
804     }
805
806     function onOutcomeChange(){
807         const status = document.getElementById('vr-status').value;
808         const hint = document.getElementById('vr-outcome-hint');
809         if(status === 'REPAIR_DONE'){
810             hint.className = 'outcome-hint success';
811             hint.innerHTML = ' Device status <strong>ASSIGNED</strong> – returned to the employee';
812         } else {
813             hint.className = 'outcome-hint danger';
814             hint.innerHTML = ' Device status <strong>CONDEMNED</strong> – device will be marked unusable';
815         }
816     }
817
818     async function submitRepairUpdate(){
819         const requestId = document.getElementById('vr-request-id').value;
820         const status = document.getElementById('vr-status').value;
821         const resolution = document.getElementById('vr-resolution').value.trim();
822
823         if(!resolution){
824             showAlert('Please enter resolution notes','error'); return;
825         }
826
827         // RepairDTO: { requestId, resolution, status }
828         const body = {
829             requestId: parseInt(requestId),
830             resolution,
831             status // "REPAIR_DONE" ASSIGNED | anything else CONDEMNED
832         };
833
834         try{
835             await apiFetch('/service-requests/vendor', {method:'PUT', body:JSON.stringify(body)});
836             const label = status === 'REPAIR_DONE' ? 'Repair marked done – device returned to employee' :
837 'Device condemned';
838             showAlert(label, 'success');
839             closeRepairPanel();
840             // Refresh everything that could have changed
841             await Promise.all([loadServiceRequests(), loadMyDevices(), loadRepairLogs()]);
842             loadDashboard();
843         }catch(e){ showAlert('Failed: '+e.message,'error'); }
844
845         // REPAIR LOGS
846         async function loadRepairLogs(){
847             try{
848                 const data = await apiFetch('/tech-vendor/repair-logs');
849                 repairsCache = data || [];
850                 renderRepairLogs(repairsCache);
851             }catch(e){ renderRepairLogs([]); }
852         }
853
854         function renderRepairLogs(list){
855             document.getElementById('repair-list').innerHTML = list.length
856             ? list.map(r=><tr>
857                 <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${r.id}</td>
858                 <td class="main-cell">${r.deviceName}||Device #'+r.deviceId}</td>
859                 <td style="max-width:140px;overflow:hidden;text-overflow:ellipsis;white-
860 space:nowrap">${r.issueReported}||'-'}</td>
861                 <td>${r.technicianName}||'-'}</td>
862                 <td style="font-size:11px;font-family:var(--mono)">${r.sentDate}||'-'}</td>
863                 <td style="font-size:11px;font-family:var(--mono)">${r.expectedReturnDate}||'-'}</td>
864                 <td style="font-family:var(--mono)">${r.repairCost ? ''+r.repairCost : '-'}</td>
865                 <td><chip(r.repairStatus)</td>
866                 <td><button class="btn btn-ghost btn-xs"
867 onclick="openRepairUpdate(${r.id})">Update</button></td>
868             </tr>` ).join('')
869             : ``<tr><td colspan="9"><div class="empty"><div class="empty-icon"></div>No repair
870 logs</div></td></tr>`;
871         }
872
873         function openRepairUpdate(id){
874             document.getElementById('repair-id').value = id;
875             const form = document.getElementById('repair-update-form');
876             form.style.display = 'block';

```

```

874     form.scrollIntoView({behavior: 'smooth'});
875 }
876
877 async function updateRepair(){
878     const id = document.getElementById('repair-id').value;
879     const body = {
880         repairStatus: document.getElementById('repair-status-input').value,
881         technicianName: document.getElementById('repair-tech').value.trim(),
882         repairCost: parseFloat(document.getElementById('repair-cost').value)||null,
883         invoiceNumber: document.getElementById('repair-invoice').value.trim(),
884         actualReturnDate: document.getElementById('repair-return-date').value||null,
885         fixedSuccessfully: document.getElementById('repair-fixed').value==='true',
886         repairNotes: document.getElementById('repair-notes').value.trim(),
887         postRepairAction: document.getElementById('repair-action').value
888     };
889     try{
890         await apiFetch(`/tech-vendor/repair-logs/${id}`, {method: 'PUT', body: JSON.stringify(body)});
891         showAlert('Repair log updated!', 'success');
892         document.getElementById('repair-update-form').style.display='none';
893         loadRepairLogs(); loadMyDevices(); loadDashboard();
894     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
895 }
896
897 // ASSIGNMENTS
898 async function loadAssignments(){
899     try{
900         const data = await apiFetch('/devices/assignments');
901         assignmentsCache = data || [];
902         document.getElementById('assignments-list').innerHTML = assignmentsCache.length
903             ? assignmentsCache.map(a=><tr>
904                 <td class="main-cell">${a.deviceName||'-'}</td>
905                 <td>${chip(a.deviceType)}</td>
906                 <td>${a.employeeName||'-'}</td>
907                 <td style="font-size:11px;font-family:var(--mono)">${a.assignedDate||'-'}</td>
908                 <td>${chip(a.status==='ACTIVE'? 'active_assign':a.status?.toLowerCase())}</td>
909             </tr>`).join('')
910             : `<tr><td colspan="5"><div class="empty"><div class="empty-icon"></div>No assignments
found</div></td></tr>`;
911     }catch(e){}
912 }
913
914 // PROFILE
915 async function loadProfile(){
916     try{
917         const data = await apiFetch('/vendors/me');
918         document.getElementById('profile-name').textContent = data.name || '--';
919         document.getElementById('profile-email').textContent = data.email || '--';
920         document.getElementById('profile-phone').textContent = data.phone || '--';
921         document.getElementById('profile-reg').textContent = data.registeredAt || '--';
922         document.getElementById('profile-avatar').textContent = (data.name||'T').charAt(0).toUpperCase();
923         document.getElementById('sidebar-name').textContent = data.name || 'TechVendor';
924     }catch(e){}
925 }
926
927 // DEVICE DETAIL MODAL
928 async function viewDevice(id){
929     const d = devicesCache.find(x=>x.id===id);
930     if(!d) return;
931     document.getElementById('modal-device-name').textContent = d.deviceName;
932
933     const today = new Date();
934     const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
935     const warrantyDisplay = warrantyExpiry
936         ? (warrantyExpiry > today
937             ? `<span style="color:var(--green);font-family:var(--mono)"> Valid until
938 ${d.warrantyExpiryDate}</span>`
939             : `<span style="color:var(--red);font-family:var(--mono)"> Expired
940 ${d.warrantyExpiryDate}</span>`)
941         : `<span style="color:var(--text3);font-family:var(--mono)">></span>`;
942
943     document.getElementById('modal-detail-grid').innerHTML = `
944         <div class="detail-item"><div class="detail-label">Brand</div><div class="detail-
value">${d.brand||'-'}</div></div>
945         <div class="detail-item"><div class="detail-label">Type</div><div class="detail-
value">${chip(d.deviceType)}</div></div>
946         <div class="detail-item"><div class="detail-label">Status</div><div class="detail-

```



```

value">${chip(d.deviceStatus)}</div></div>
945     <div class="detail-item"><div class="detail-label">Warranty</div><div class="detail-
value">${warrantyDisplay}</div></div>
946     <div class="detail-item"><div class="detail-label">Assigned To</div><div class="detail-
value">${d.assignedEmployeeName||'<span style="color:var(--text3)">Unassigned</span>'}</div></div>
947     <div class="detail-item"><div class="detail-label">Added On</div><div class="detail-
value">${d.createdAt ? d.createdAt.split('T')[0] : '-'}</div></div>
948 `;
949
950     const repairs = repairsCache.filter(r=>r.deviceId===id);
951     document.getElementById('modal-repair-history').innerHTML = repairs.length
952     ? `<div class="timeline">${repairs.map(r=>`
953         <div class="tl-item">
954             <div class="tl-dot
955 ${r.repairStatus=== 'FIXED'? 'green': r.repairStatus=== 'IRREPARABLE'? 'red': r.repairStatus=== 'IN_PROGRESS'? 'amber': '
cyan'}"></div>
956                 <div class="tl-content">
957                     <div class="tl-title">${r.issueReported||'Repair'}</div>
958                     <div class="tl-meta">${chip(r.repairStatus)} · ${r.sentDate||'-'}
959 ${r.actualReturnDate||'pending'} · ${r.repairCost ? ''+r.repairCost : 'cost pending'}</div>
960                     </div>
961                 </div>`}).join('')}</div>`
962     : `<div class="empty" style="padding:16px">No repair history</div>`;
963
964     document.getElementById('device-modal').classList.add('open');
965 }
966
967 function closeModal(){
968     document.getElementById('device-modal').classList.remove('open');
969 }
970
971 // HELPERS
972 function chip(val){
973     if(!val) return '-';
974     const cls = val.toLowerCase().replace(/ /g, '_');
975     return `<span class="chip ${cls}">${val.replace(/_/g, ' ')}</span>`;
976 }
977
978 function showAlert(msg, type){
979     const el = document.createElement('div');
980     el.className = `alert alert-${type}`;
981     el.innerHTML = `<span>${type==='success'? '' : ''}</span>${msg}`;
982     document.body.appendChild(el);
983     setTimeout(()=>el.remove(), 3500);
984 }
985
986 function logout(){
987     localStorage.removeItem('jwt_token');
988     localStorage.removeItem('auth_type');
989     fetch(`${API}/session/logout`, {method: 'POST'}).finally(()=>window.location.href='/vendor-
login.html');
990 }
991 </script>
992 </body>
993 </html>

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Two-Factor Authentication Setup</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <!-- qrcode.js for QR rendering -->
9      <script src="https://cdnjs.cloudflare.com/ajax/libs/qrcodejs/1.0.0/qrcode.min.js"></script>
10     <style>
11         *{margin:0;padding:0;box-sizing:border-box}
12         :root{
13             --bg:#070910;--panel:#0e1118;--panel2:#131821;
14             --border:rgba(255,255,255,0.07);--border-lit:rgba(255,255,255,0.16);
15             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
16             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.13);
17             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
18             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
19             --amber:#f9c74f;
20             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
21         }
22         body{font-family:var(--font);background:var(--bg);color:var(--text);min-height:100vh;}
23
24         .bg-layer{position:fixed;inset:0;z-index:0;
25             background:
26                 radial-gradient(ellipse 50% 50% at 20% 30%, rgba(79,140,255,0.06) 0%,transparent 70%),
27                 radial-gradient(ellipse 40% 40% at 80% 70%, rgba(52,211,153,0.04) 0%,transparent 60%);
28         }
29         .grid-overlay{position:fixed;inset:0;z-index:0;
30             background-image:linear-gradient(rgba(255,255,255,0.015) 1px,transparent 1px),linear-
gradient(90deg,rgba(255,255,255,0.015) 1px,transparent 1px);
31             background-size:40px 40px;
32         }
33
34         /* Nav */
35         .nav{position:relative;z-index:10;display:flex;align-items:center;justify-content:space-
between;padding:18px 40px;border-bottom:1px solid var(--border);}
36         .nav-brand{display:flex;align-items:center;gap:10px;}
37         .nav-icon{width:36px;height:36px;background:linear-gradient(135deg,#4f8cff,#818cf8);border-
radius:9px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:14px;c...
38         .nav-name{font-size:16px;font-weight:700;}
39         .nav-back{font-size:13px;color:var(--text2);text-decoration:none;display:flex;align-
items:center;gap:6px;transition:color .2s;}
40         .nav-back:hover{color:var(--blue);}
41
42         /* Page */
43         .page{position:relative;z-index:1;max-width:680px;margin:0 auto;padding:48px 24px 80px;}
44
45         .page-header{margin-bottom:36px;}
46         .page-header h1{font-size:28px;font-weight:700;letter-spacing:-.4px;margin-bottom:6px;}
47         .page-header p{font-size:14px;color:var(--text2);line-height:1.6;}
48
49         /* Status badge */
50         .status-bar{display:flex;align-items:center;justify-content:space-between;background:var(--
panel);border:1px solid var(--border);border-radius:14px;padding:18px 22px;margin-bottom:28px;}
51         .status-left{display:flex;align-items:center;gap:14px;}
52         .status-icon{width:44px;height:44px;border-radius:12px;display:flex;align-items:center;justify-
content:center;font-size:22px;flex-shrink:0;}
53         .status-icon.on{background:var(--green-dim);}
54         .status-icon.off{background:var(--border);}
55         .status-title{font-size:15px;font-weight:600;margin-bottom:2px;}
56         .status-desc{font-size:12px;color:var(--text2);}
57         .badge{font-size:11px;font-weight:700;padding:4px 10px;border-radius:20px;letter-spacing:.04em;text-
transform:uppercase;}
58         .badge-on{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.25);}
59         .badge-off{background:var(--border);color:var(--text2);border:1px solid var(--border-lit);}
60
61         /* Cards */
62         .card{background:var(--panel);border:1px solid var(--border);border-radius:14px;padding:24px;margin-
bottom:20px;}

```

```

63     .card-title{font-size:15px;font-weight:600;margin-bottom:4px;display:flex;align-
items:center;gap:8px;}
64     .card-subtitle{font-size:13px;color:var(--text2);margin-bottom:20px;}
65
66     /* Steps list */
67     .steps-list{display:flex;flex-direction:column;gap:16px;margin-bottom:24px;}
68     .step-row{display:flex;gap:14px;align-items:flex-start;}
69     .step-num{width:28px;height:28px;border-radius:50%;background:var(--blue-dim);border:1px solid
rgba(79,140,255,0.3);color:var(--blue);font-size:13px;font-weight:700;display:flex;align-items...
70     .step-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px;}
71     .step-text span{font-size:12px;color:var(--text2);line-height:1.5;}
72
73     /* QR area */
74     .qr-area{display:none;flex-direction:column;align-
items:center;gap:20px;padding:24px;background:var(--panel2);border:1px solid var(--border);border-
radius:12px;margin-bottom:20px;}
75     .qr-area.show{display:flex;}
76     #qr-container{background:#fff;padding:14px;border-radius:10px;display:inline-block;}
77     .qr-container canvas, .qr-container img{display:block;}
78     .secret-box{width:100%;background:var(--bg);border:1px solid var(--border);border-
radius:9px;padding:12px 16px;text-align:center;}
79     .secret-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-
spacing:.06em;margin-bottom:6px;}
80     .secret-val{font-family:var(--mono);font-size:14px;color:var(--amber);letter-spacing:.12em;word-
break:break-all;}
81     .copy-btn{font-size:11px;color:var(--blue);background:none;border:none;cursor:pointer;font-
family:var(--font);padding:0;margin-top:4px;}
82     .copy-btn:hover{text-decoration:underline;}
83
84     /* OTP row */
85     .otp-row{display:flex;gap:10px;justify-content:center;margin:0 0 8px;}
86     .otp-digit{width:48px;height:58px;text-align:center;font-size:24px;font-family:var(--mono);font-
weight:500;background:var(--panel2);border:1px solid var(--border);border-radius:10px;color:v...
87     .otp-digit:focus{border-color:var(--blue);box-shadow:0 0 3px var(--blue-dim);}
88     .otp-digit.filled{border-color:rgba(79,140,255,0.4);}
89
90     /* Alerts */
91     .alert{border-radius:9px;padding:11px 14px;font-size:13px;margin-bottom:14px;display:none;align-
items:flex-start;gap:9px;}
92     .alert.show{display:flex;}
93     .alert-err{background:var(--red-dim);border:1px solid rgba(248,113,113,0.25);color:var(--red);}
94     .alert-ok{background:var(--green-dim);border:1px solid rgba(52,211,153,0.25);color:var(--green);}
95     .alert-icon{font-size:15px;flex-shrink:0;}
96
97     /* Buttons */
98     .btn{padding:12px 22px;border-radius:10px;border:none;font-family:var(--font);font-size:14px;font-
weight:600;cursor:pointer;transition:all .2s;}
99     .btn-primary{background:var(--blue);color:#fff;}
100    .btn-primary:hover{background:#6ba1ff;box-shadow:0 4px 16px rgba(79,140,255,0.3);}
101    .btn-primary.disabled{opacity:.5;cursor:not-allowed;box-shadow:none;}
102    .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.25);}
103    .btn-danger:hover{background:rgba(248,113,113,0.2);}
104    .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border);}
105    .btn-ghost:hover{border-color:var(--border-lit);color:var(--text);}
106
107    .btn-row{display:flex;gap:10px;flex-wrap:wrap;}
108
109    .spinner{display:inline-block;width:14px;height:14px;border:2px solid rgba(255,255,255,0.2);border-
top-color:#fff;border-radius:50%;animation:spin .6s linear infinite;vertical-align:middle;...
110    @keyframes spin{to{transform:rotate(360deg)}}
111
112    /* Danger zone */
113    .danger-card{background:var(--red-dim);border:1px solid rgba(248,113,113,0.2);border-
radius:14px;padding:20px 24px;display:flex;align-items:center;justify-content:space-between;gap:16px;}
114    .danger-text strong{font-size:14px;color:var(--red);display:block;margin-bottom:3px;}
115    .danger-text span{font-size:12px;color:rgba(248,113,113,0.7);}
116
117    .loading{text-align:center;padding:60px 0;color:var(--text2);}
118    .loading .spinner{border-top-color:var(--blue);border-color:var(--border-
lit);width:24px;height:24px;}
119    </style>
120 </head>
121 <body>
122 <div class="bg-layer"></div>
123 <div class="grid-overlay"></div>
124

```

```

125 <nav class="nav">
126     <div class="nav-brand">
127         <div class="nav-icon">EMS</div>
128         <div class="nav-name">EmployeeMS</div>
129     </div>
130     <a class="nav-back" href="javascript:history.back()"> Back to dashboard</a>
131 </nav>
132
133 <div class="page">
134     <div class="page-header">
135         <h1>Two-Factor Authentication</h1>
136         <p>Add an extra layer of security to your account. Once enabled, you'll need to provide a 6-digit
code from your authenticator app every time you sign in.</p>
137     </div>
138
139     <!-- Loading state -->
140     <div class="loading" id="loading">
141         <span class="spinner"></span>
142     </div>
143
144     <!-- Main content (shown after status loads) -->
145     <div id="main-content" style="display:none;">
146
147         <!-- Status bar -->
148         <div class="status-bar">
149             <div class="status-left">
150                 <div class="status-icon" id="status-icon"></div>
151                 <div>
152                     <div class="status-title">Two-Factor Authentication</div>
153                     <div class="status-desc" id="status-desc">Loading...</div>
154                 </div>
155             </div>
156             <span class="badge" id="status-badge">—</span>
157         </div>
158
159         <!-- ===== SETUP SECTION (shown when disabled) ===== -->
160         <div id="setup-section">
161             <div class="card">
162                 <div class="card-title"> Enable 2FA</div>
163                 <div class="card-subtitle">Follow these steps to add two-factor authentication to your
account.</div>
164
165                 <div class="steps-list">
166                     <div class="step-row">
167                         <div class="step-num">1</div>
168                         <div class="step-text">
169                             <strong>Install an authenticator app</strong>
170                             <span>Download <a href="https://support.google.com/accounts/answer/1066447"
target="_blank" style="color:var(--blue)">Google Authenticator</a> or <a href="https://authy...
171                         </div>
172                     </div>
173                     <div class="step-row">
174                         <div class="step-num">2</div>
175                         <div class="step-text">
176                             <strong>Scan the QR code</strong>
177                             <span>Click "Generate QR Code" below, then scan it with your authenticator app.
You can also enter the secret key manually.</span>
178                         </div>
179                     </div>
180                     <div class="step-row">
181                         <div class="step-num">3</div>
182                         <div class="step-text">
183                             <strong>Verify the setup</strong>
184                             <span>Enter the 6-digit code shown in your app to confirm everything is working
correctly.</span>
185                         </div>
186                     </div>
187                 </div>
188
189                 <!-- QR area -->
190                 <div class="qr-area" id="qr-area">
191                     <div id="qr-container"></div>
192                     <div class="secret-box">
193                         <div class="secret-label">Manual entry key (backup)</div>
194                         <div class="secret-val" id="secret-display"></div>
195                         <button class="copy-btn" onclick="copySecret()">Copy key</button>

```

```

196         </div>
197     </div>
198
199     <!-- Alerts -->
200     <div class="alert alert-err" id="setup-err"><span class="alert-icon"></span><span
id="setup-err-msg"></span></div>
201     <div class="alert alert-ok" id="setup-ok"><span class="alert-icon"></span><span id="setup-
ok-msg"></span></div>
202
203     <!-- Verify code input (shown after QR generated) -->
204     <div id="verify-area" style="display:none;">
205         <p style="font-size:13px;color:var(--text2);margin-bottom:16px;">Enter the code shown
in your authenticator app to activate 2FA:</p>
206         <div class="otp-row" id="setup-otp-row">
207             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s0">
208             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s1">
209             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s2">
210             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s3">
211             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s4">
212             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s5">
213         </div>
214     </div>
215
216     <div class="btn-row" style="margin-top:8px;">
217         <button class="btn btn-ghost" id="btn-generate" onclick="generateQr()"> Generate QR
Code</button>
218         <button class="btn btn-primary" id="btn-verify" style="display:none;" disabled
onclick="verifySetup()">Activate 2FA</button>
219     </div>
220 </div>
221 </div>
222
223     <!-- ===== ACTIVE SECTION (shown when enabled) ===== -->
224     <div id="active-section" style="display:none;">
225         <div class="card">
226             <div class="card-title"> 2FA is active</div>
227             <div class="card-subtitle">Your account is protected. Every sign-in requires your
authenticator app code.</div>
228             <p style="font-size:13px;color:var(--text2);line-height:1.6;">
229                 If you lose access to your authenticator app, contact your admin to reset 2FA. We
recommend also saving backup codes.
230             </p>
231         </div>
232
233         <div class="danger-card">
234             <div class="danger-text">
235                 <strong>Disable Two-Factor Authentication</strong>
236                 <span>This will remove the extra protection from your account.</span>
237             </div>
238             <button class="btn btn-danger" onclick="disableTotp()">Disable 2FA</button>
239         </div>
240     </div>
241
242 </div>
243 </div>
244
245 <script>
246     const token = () => localStorage.getItem('jwt_token');
247     const authHeader = () => ({ 'Authorization': 'Bearer ' + token(), 'Content-Type': 'application/json'
});
248
249     // ---- Load status on page open ----
250     async function loadStatus() {
251         try {
252             const res = await fetch('/totp/status', { headers: authHeader() });
253             if (res.status === 401) { window.location.href = '/google.html'; return; }
254             const data = await res.json();
255             renderStatus(data.totpEnabled);
256         } catch (e) {
257             console.error(e);
258         } finally {
259             document.getElementById('loading').style.display = 'none';
260             document.getElementById('main-content').style.display = 'block';
261         }
262     }
263

```

```

264     function renderStatus(enabled) {
265         const icon = document.getElementById('status-icon');
266         const desc = document.getElementById('status-desc');
267         const badge = document.getElementById('status-badge');
268         const setup = document.getElementById('setup-section');
269         const active = document.getElementById('active-section');
270
271         if (enabled) {
272             icon.textContent = '';
273             icon.className = 'status-icon on';
274             desc.textContent = 'Your account requires a 6-digit code on every sign-in.';
275             badge.textContent = 'Enabled';
276             badge.className = 'badge badge-on';
277             setup.style.display = 'none';
278             active.style.display = 'block';
279         } else {
280             icon.textContent = '';
281             icon.className = 'status-icon off';
282             desc.textContent = 'Not enabled – only your password protects your account.';
283             badge.textContent = 'Disabled';
284             badge.className = 'badge badge-off';
285             setup.style.display = 'block';
286             active.style.display = 'none';
287         }
288     }
289
290     // ---- Generate QR ----
291     let currentSecret = null;
292
293     async function generateQr() {
294         const btn = document.getElementById('btn-generate');
295         btn.disabled = true;
296         btn.innerHTML = '<span class="spinner"></span>Generating...';
297         clearAlerts();
298
299         try {
300             const res = await fetch('/totp/setup', { method: 'POST', headers: authHeader() });
301             const data = await res.json();
302             if (!res.ok) { showAlert('setup-err', data.error || 'Failed to generate QR code'); return; }
303
304             currentSecret = data.secret;
305
306             // Render QR code
307             const qrContainer = document.getElementById('qr-container');
308             qrContainer.innerHTML = '';
309             new QRCode(qrContainer, {
310                 text: data.qrCodeUri,
311                 width: 180,
312                 height: 180,
313                 colorDark: '#000000',
314                 colorLight: '#ffffff',
315                 correctLevel: QRCode.CorrectLevel.M
316             });
317
318             document.getElementById('secret-display').textContent = formatSecret(data.secret);
319             document.getElementById('qr-area').classList.add('show');
320             document.getElementById('verify-area').style.display = 'block';
321             document.getElementById('btn-verify').style.display = 'inline-block';
322             btn.textContent = 'Regenerate';
323
324             // Focus first digit
325             setTimeout(() => document.getElementById('s0').focus(), 200);
326         } catch (e) {
327             showAlert('setup-err', 'Network error. Please try again.');
```

```

340     navigator.clipboard.writeText(currentSecret || '');
341     const btn = document.querySelector('.copy-btn');
342     btn.textContent = 'Copied!';
343     setTimeout(() => btn.textContent = 'Copy key', 1500);
344 }
345
346 // ---- OTP inputs for setup ----
347 const setupDigits = Array.from({ length: 6 }, (_, i) => document.getElementById('s' + i));
348 const btnVerify = document.getElementById('btn-verify');
349
350 setupDigits.forEach((d, i) => {
351     d.addEventListener('input', () => {
352         const val = d.value.replace(/\D/g, '');
353         d.value = val;
354         d.classList.toggle('filled', val.length > 0);
355         if (val && i < 5) setupDigits[i + 1].focus();
356         btnVerify.disabled = setupDigits.map(x => x.value).join('').length < 6;
357     });
358     d.addEventListener('keydown', e => {
359         if (e.key === 'Backspace' && !d.value && i > 0) {
360             setupDigits[i - 1].focus();
361             setupDigits[i - 1].value = '';
362             setupDigits[i - 1].classList.remove('filled');
363             btnVerify.disabled = true;
364         }
365         if (e.key === 'Enter' && !btnVerify.disabled) verifySetup();
366     });
367     d.addEventListener('paste', e => {
368         const pasted = (e.clipboardData || window.clipboardData).getData('text').replace(/\D/g,
369             '').slice(0, 6);
370         if (pasted.length === 6) {
371             pasted.split('').forEach((ch, idx) => { setupDigits[idx].value = ch;
372                 setupDigits[idx].classList.add('filled'); });
373             setupDigits[5].focus();
374             btnVerify.disabled = false;
375             e.preventDefault();
376         }
377     });
378 });
379
380 // ---- Verify setup ----
381 async function verifySetup() {
382     clearAlerts();
383     const code = setupDigits.map(d => d.value).join('');
384     if (code.length < 6) return;
385
386     btnVerify.disabled = true;
387     btnVerify.innerHTML = '<span class="spinner"></span>Activating...';
388
389     try {
390         const res = await fetch('/totp/verify-setup', {
391             method: 'POST',
392             headers: authHeader(),
393             body: JSON.stringify({ code })
394         });
395         const data = await res.json();
396         if (!res.ok) {
397             showAlert('setup-err', data.error || 'Invalid code. Try again.');
```

```
account less secure.')) return;
414
415     try {
416         const res = await fetch('/totp/disable', { method: 'DELETE', headers: authHeader() });
417         if (!res.ok) { alert('Failed to disable 2FA.');
```

return; }

```
418         renderStatus(false);
419     } catch (e) {
420         alert('Network error.');
```

}

```
421     }
422 }
423
424 // ---- Alerts ----
425 function showAlert(id, msg) {
426     const el = document.getElementById(id);
427     el.classList.add('show');
```

document.getElementById(id + '-msg').textContent = msg;

```
429 }
430 function clearAlerts() {
431     ['setup-err', 'setup-ok'].forEach(id => document.getElementById(id).classList.remove('show'));
432 }
433
434 // ---- Boot ----
435 if (!localStorage.getItem('jwt_token')) {
436     window.location.href = '/google.html';
437 } else {
438     loadStatus();
439 }
440 </script>
441 </body>
442 </html>
```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Vendor Hub</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&family=Space+Mono:wght@400;700&
isplay=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#08100a;--bg2:#0c160e;--bg3:#101c12;
12             --panel:#131f15;--panel2:#182219;
13             --border:rgba(52,211,153,0.08);--border2:rgba(52,211,153,0.18);
14             --text:#d4edd9;--text2:#6b9474;--text3:#3d5e44;
15             --accent:#34d399;--accent-dim:rgba(52,211,153,0.1);
16             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
17             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
18             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
19             --font:'DM Sans',sans-serif;--mono:'Space Mono',monospace;
20             --sidebar:245px;
21         }
22         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
23         .app{display:flex;height:100vh}
24
25         /* SIDEBAR */
26         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative}
27         .sidebar::before{content:'';position:absolute;left:0;top:0;bottom:0;width:2px;background:linear-
gradient(to bottom,transparent,var(--accent),transparent)}
28         .logo{padding:22px 20px;border-bottom:1px solid var(--border)}
29         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
30         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
31         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
32         .logo p{font-size:10px;color:var(--text3);font-family:var(--mono)}
33         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:var(--accent-
dim);border:1px solid rgba(52,211,153,0.2);border-radius:4px;font-size:10px;font-weight:7...
34         .nav{flex:1;padding:10px 0;overflow-y:auto}
35         .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase;font-family:var(--mono)}
36         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:6px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
37         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
38         .nav-item.active{background:var(--accent-dim);color:var(--accent);border-left:2px solid var(--
accent);margin-left:6px}
39         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
40         .user-card{display:flex;align-items:center;gap:10px}
41         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:6px;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
42         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
43         .user-info-side .role-text{font-size:9px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:1px}
44
45         /* MAIN */
46         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
47         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
48         .page-title{font-size:16px;font-weight:700;color:var(--text);font-family:var(--mono)}
49         .topbar-right{display:flex;align-items:center;gap:12px}
50         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:4px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;transition:all...
51         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
52         .content{flex:1;overflow-y:auto;padding:24px}
53         .view{display:none}
54         .view.active{display:block;animation:fadeIn 0.25s ease}
55         @keyframes fadeIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}
56
57         /* STATS */
58         .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:14px;margin-

```

```

bottom:24px}
59     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:10px;padding:18px;position:relative;transition:border-color 0.2s}
60     .stat-card:hover{border-color:var(--border2)}
61     .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:2px;border-radius:10px
10px 0 0}
62     .stat-card.green::before{background:var(--accent)}
63     .stat-card.amber::before{background:var(--amber)}
64     .stat-card.blue::before{background:var(--blue)}
65     .stat-card.red::before{background:var(--red)}
66     .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1.2px;margin-
bottom:10px;font-weight:600;font-family:var(--mono)}
67     .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
68     .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
69
70     /* SECTION */
71     .section{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-
bottom:18px;overflow:hidden}
72     .section-head{padding:14px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
73     .section-title{font-size:13px;font-weight:700;color:var(--text);font-family:var(--mono)}
74     .section-sub{font-size:11px;color:var(--text3);margin-top:2px}
75
76     /* TABLE */
77     .tablewrap{overflow-x:auto}
78     table{width:100%;border-collapse:collapse}
79     th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-
spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
80     td{padding:11px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
81     tr:last-child td{border-bottom:none}
82     tr:hover td{background:rgba(52,211,153,0.03);color:var(--text)}
83
84     /* CHIPS */
85     .chip{display:inline-flex;align-items:center;padding:3px 8px;border-radius:4px;font-size:10px;font-
weight:700;font-family:var(--mono);letter-spacing:0.5px}
86     .chip.active{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(52,211,153,0.2)}
87     .chip.inactive{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
88     .chip.pending{background:var(--amber-dim);color:var(--amber)}
89     .chip.delivered{background:var(--accent-dim);color:var(--accent)}
90     .chip.processing{background:var(--blue-dim);color:var(--blue)}
91     .chip.breakfast{background:var(--amber-dim);color:var(--amber)}
92     .chip.lunch{background:var(--accent-dim);color:var(--accent)}
93     .chip.dinner{background:var(--blue-dim);color:var(--blue)}
94
95     /* BUTTONS */
96     .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--mono);font-size:11px;font-
weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.5px;text-transform:uppercase}
97     .btn-primary{background:var(--accent);color:#08100a}
98     .btn-primary:hover{background:#22c77a;box-shadow:0 4px 12px rgba(52,211,153,0.3)}
99     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
100    .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
101    .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
102    .btn-danger:hover{background:var(--red);color:#fff}
103    .btn-warning{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
104    .btn-warning:hover{background:var(--amber);color:#000}
105    .btn-sm{padding:5px 10px;font-size:10px}
106    .action-row{display:flex;gap:6px}
107
108    /* FORM */
109    .form-card{padding:22px}
110    .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
111    .form-group{display:flex;flex-direction:column;gap:5px}
112    .form-group.full{grid-column:1/-1}
113    label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-family:var(--mono)}
114    input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:6px;padding:9px 12px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-colo...
115    input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
116    input::placeholder,textarea::placeholder{color:var(--text3)}
117    select option{background:var(--bg2)}
118    .form-actions{display:flex;gap:10px;margin-top:18px}
119
120    /* ALERT */
121    .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:8px;z-
index:1100;animation:slideIn 0.3s ease;font-size:12px;font-weight:700;font-family:var(--mono)}
122    .alert-success{background:var(--accent-dim);color:var(--accent);border:1px solid

```

```

rgba(52,211,153,0.3)}
123     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
124     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
125     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:28px;font-family:var(--
mono);font-size:12px}
126
127     /* RESTAURANT CARDS */
128     .restaurant-grid{display:grid;grid-template-columns:repeat(auto-
fill,minmax(240px,1fr));gap:14px;padding:20px}
129     .restaurant-card{background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:18px;transition:all 0.2s;position:relative;overflow:hidden}
130     .restaurant-
card::before{content:'';position:absolute;top:0;left:0;bottom:0;width:3px;background:var(--accent)}
131     .restaurant-card:hover{border-color:var(--border2);transform:translateX(2px)}
132     .restaurant-card.inactive::before{background:var(--red)}
133     .rest-name{font-weight:700;color:var(--text);margin-bottom:6px;font-size:14px}
134     .rest-meta{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:12px}
135     .rest-actions{display:flex;gap:8px}
136 </style>
137 </head>
138 <body>
139 <div class="app">
140     <!-- SIDEBAR -->
141     <div class="sidebar">
142         <div class="logo">
143             <div class="logo-row">
144                 <div class="logo-icon">V</div>
145                 <h2>Vendor Hub</h2>
146             </div>
147             <p>EMS Vendor Portal</p>
148             <div class="role-badge">// FOOD VENDOR</div>
149         </div>
150         <nav class="nav">
151             <div class="nav-section">// overview</div>
152             <div class="nav-item active" onclick="showView('dashboard',this)"><span></span> Dashboard</div>
153             <div class="nav-section">// restaurants</div>
154             <div class="nav-item" onclick="showView('my-restaurants',this)"><span></span> My
Restaurants</div>
155             <div class="nav-item" onclick="showView('add-restaurant',this)"><span></span> Add
Restaurant</div>
156             <div class="nav-section">// operations</div>
157             <div class="nav-item" onclick="showView('subscriptions',this)"><span></span>
Subscriptions</div>
158             <div class="nav-item" onclick="showView('deliveries',this)"><span></span> Deliveries</div>
159             <div class="nav-section">// account</div>
160             <div class="nav-item" onclick="showView('profile',this)"><span></span> My Profile</div>
161         </nav>
162         <div class="sidebar-footer">
163             <div class="user-card">
164                 <div class="avatar" id="avatar-initials">V</div>
165                 <div class="user-info-side">
166                     <div class="name" id="sidebar-name">Vendor</div>
167                     <div class="role-text">// vendor</div>
168                 </div>
169             </div>
170         </div>
171     </div>
172
173     <!-- MAIN -->
174     <div class="main">
175         <div class="topbar">
176             <div class="page-title" id="pageTitle">// dashboard</div>
177             <div class="topbar-right">
178                 <span style="font-size:11px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
179                 <button class="logout-btn" onclick="logout()">SIGN OUT</button>
180             </div>
181         </div>
182         <div class="content">
183
184             <!-- DASHBOARD -->
185             <div id="dashboard-view" class="view active">
186                 <div class="stats-grid">
187                     <div class="stat-card green">
188                         <div class="stat-label">Restaurants</div>
189                         <div class="stat-value" id="stat-restaurants">--</div>

```

```

190         <div class="stat-sub">active listings</div>
191     </div>
192     <div class="stat-card amber">
193         <div class="stat-label">Subscriptions</div>
194         <div class="stat-value" id="stat-subs">--</div>
195         <div class="stat-sub">active customers</div>
196     </div>
197     <div class="stat-card blue">
198         <div class="stat-label">Deliveries</div>
199         <div class="stat-value" id="stat-deliveries">--</div>
200         <div class="stat-sub">total today</div>
201     </div>
202     <div class="stat-card red">
203         <div class="stat-label">Pending</div>
204         <div class="stat-value" id="stat-pending">--</div>
205         <div class="stat-sub">deliveries pending</div>
206     </div>
207 </div>
208 <div style="display:grid;grid-template-columns:1fr 1fr;gap:18px">
209     <div class="section">
210         <div class="section-head">
211             <div><div class="section-title">// my_restaurants</div></div>
212             <button class="btn btn-secondary btn-sm" onclick="showView('my-
restaurants',null)">view </button>
213         </div>
214         <div id="dash-restaurants" style="padding:4px 0"></div>
215     </div>
216     <div class="section">
217         <div class="section-head">
218             <div><div class="section-title">// recent_deliveries</div></div>
219             <button class="btn btn-secondary btn-sm"
onclick="showView('deliveries',null)">view </button>
220         </div>
221         <div class="tablewrap">
222             <table>
223                 <thead><tr><th>Meal</th><th>Slot</th><th>Status</th></tr></thead>
224                 <tbody id="dash-deliveries"><tr><td colspan="3" class="empty-
state">loading...</td></tr></tbody>
225             </table>
226         </div>
227     </div>
228 </div>
229 </div>
230
231 <!-- MY RESTAURANTS -->
232 <div id="my-restaurants-view" class="view">
233     <div class="section">
234         <div class="section-head">
235             <div><div class="section-title">// my_restaurants</div><div class="section-
sub">Manage your listings</div></div>
236             <button class="btn btn-secondary btn-sm" onclick="loadMyRestaurants()">
refresh</button>
237         </div>
238         <div class="restaurant-grid" id="restaurants-grid">
239             <div class="empty-state">loading...</div>
240         </div>
241     </div>
242 </div>
243
244 <!-- ADD RESTAURANT -->
245 <div id="add-restaurant-view" class="view">
246     <div class="section">
247         <div class="section-head"><div class="section-title">// add_restaurant</div></div>
248         <div class="form-card">
249             <div class="form-grid">
250                 <div class="form-group full">
251                     <label>Restaurant Name</label>
252                     <input id="rest-name" placeholder="My Restaurant Name">
253                 </div>
254
255                 <div class="form-group full">
256                     <label>Address</label>
257                     <input id="rest-address" placeholder="Street, area, city">
258                 </div>
259
260                 <div class="form-group full">

```

```

261             <label>Meal Slots</label>
262             <div style="display:flex;gap:14px;flex-wrap:wrap;padding:10px
12px;background:var(--bg3);border:1px solid var(--border);border-radius:6px">
263                 <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
264                     <input type="checkbox" class="rest-slot-check" value="BREAKFAST"
style="width:auto"> Breakfast
265                 </label>
266                 <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
267                     <input type="checkbox" class="rest-slot-check" value="LUNCH"
style="width:auto"> Lunch
268                 </label>
269                 <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
270                     <input type="checkbox" class="rest-slot-check" value="DINNER"
style="width:auto"> Dinner
271                 </label>
272             </div>
273         </div>
274
275         <div class="form-group full">
276             <label>Description (optional)</label>
277             <textarea id="rest-desc" rows="2" placeholder="Brief description of your
restaurant..."></textarea>
278         </div>
279     </div>
280     <div class="form-actions">
281         <button class="btn btn-primary" onclick="addRestaurant()">CREATE
RESTAURANT</button>
282         <button class="btn btn-secondary" onclick="clearRestForm()">CLEAR</button>
283     </div>
284 </div>
285 </div>
286 </div>
287
288 <!-- SUBSCRIPTIONS -->
289 <div id="subscriptions-view" class="view">
290     <div class="section">
291         <div class="section-head">
292             <div><div class="section-title">// subscriptions</div><div class="section-
sub">Customers subscribed to your restaurants</div></div>
293             <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
refresh</button>
294         </div>
295         <div class="tablewrap">
296             <table>
297                 <thead><tr><th>Employee</th><th>Restaurant</th><th>Meal
Slot</th><th>From</th><th>To</th><th>Status</th></tr></thead>
298                 <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
299             </table>
300         </div>
301     </div>
302 </div>
303
304 <!-- DELIVERIES -->
305 <div id="deliveries-view" class="view">
306     <div class="section">
307         <div class="section-head">
308             <div><div class="section-title">// deliveries</div></div>
309             <button class="btn btn-secondary btn-sm" onclick="loadDeliveries()">
refresh</button>
310         </div>
311         <div class="tablewrap">
312             <table>
313                 <thead><tr><th>ID</th><th>Restaurant</th><th>Meal
Slot</th><th>Date</th><th>Status</th><th>Action</th></tr></thead>
314                 <tbody id="deliveries-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
315             </table>
316         </div>
317     </div>
318 </div>
319
320 <!-- PROFILE -->

```

```

321         <div id="profile-view" class="view">
322             <div class="section">
323                 <div class="section-head"><div class="section-title">// my_profile</div></div>
324                 <div style="padding:24px;display:grid;grid-template-columns:1fr 1fr;gap:16px"
id="vendor-profile">
325                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">NAME</div><div style="font-family...
326                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">EMAIL</div><div style="font-famil...
327                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">VENDOR ID</div><div style="font-f...
328                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">ROLE</div><div style="font-family...
329                     </div>
330                 </div>
331             </div>
332
333         </div>
334     </div>
335 </div>
336
337 <script>
338     const API='http://localhost:8080';
339     let vendorId=null;
340
341     (function(){
342         const token=localStorage.getItem('jwt_token');
343         const authType=localStorage.getItem('auth_type');
344         if(!token&&authType!=='session'){window.location.href='/login.html';return;}
345         if(token){
346             try{
347                 const p=JSON.parse(atob(token.split('.')[1]));
348                 const roles=p.roles||p.authorities||'';
349                 if(roles.includes('ROLE_VENDOR')) { initUser(); return; }
350                 if(roles.includes('ROLE_EMPLOYEE')){window.location.href='/employee-
dashboard.html';return;}
351                 if(roles.includes('ROLE_MANAGER')){window.location.href='/manager-dashboard.html';return;}
352                 if(roles.includes('ROLE_ADMIN')){window.location.href='/dashboard.html';return;}
353             }catch(e){}
354         }
355         initUser();
356     })();
357
358     function getAuthHeader(){const t=localStorage.getItem('jwt_token');return t?{'Authorization':`Bearer
${t}`}:{};};
359     async function apiFetch(path,opts={}){
360         const res=await fetch(`${API}${path}`,{...opts,headers:{'Content-
Type':'application/json',...getAuthHeader(),...(opts.headers||{})},credentials:'include'});
361         if(res.status===401){localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');showAlert('SESSIO
N EXPIRED','error');setTimeout(()=>window.location.href='/login.html',1500);t...
362         const text=await res.text();return text?JSON.parse(text):null;
363     }
364
365     async function initUser(){
366         const token=localStorage.getItem('jwt_token');
367         let email='';
368         if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
369         document.getElementById('avatar-initials').textContent=(email||'V').charAt(0).toUpperCase();
370         document.getElementById('sidebar-name').textContent=(email.split('@')[0]||'Vendor');
371         document.getElementById('topbar-email').textContent=email;
372         document.getElementById('p-email').textContent=email;
373
374         try{
375             const me = await apiFetch('/vendors/me');
376             if(me){
377                 vendorId = me.id || me.vendorId || null;
378                 document.getElementById('p-name').textContent = me.name || '-';
379                 document.getElementById('p-id').textContent = vendorId || '-';
380             }
381         }catch(e){
382             console.error('Could not load vendor profile', e);
383         }
384
385         loadDashboard();
386     }

```

```

387
388     function showView(name,el){
389         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
390         document.getElementById(`${name}-view`).classList.add('active');
391         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
392         if(el)el.classList.add('active');
393         document.getElementById('pageTitle').textContent='// '+name.replace(/-/g, '_');
394         const loaders={dashboard:loadDashboard,'my-restaurants':loadMyRestaurants,'add-
restaurant':()=>{},'subscriptions':loadSubscriptions,'deliveries':loadDeliveries,'profile':()=>{}};
395         if(loaders[name])loaders[name]();
396     }
397
398     async function loadDashboard(){
399         try{
400             const [rests,subs,deliveries]=await Promise.all([
401                 apiFetch('/restaurants/vendor').catch(()=>[]),
402                 apiFetch('/subscriptions').catch(()=>[]),
403                 apiFetch('/deliveries').catch(()=>[])
404             ]);
405
406             const restArr=rests||[];
407             const subArr=subs||[];
408             const delArr=deliveries||[];
409
410             document.getElementById('stat-
restaurants').textContent=restArr.filter(r=>r.active!==false).length;
411             document.getElementById('stat-subs').textContent=subArr.length;
412             document.getElementById('stat-deliveries').textContent=delArr.length;
413             document.getElementById('stat-
pending').textContent=delArr.filter(d=>d.status==='PENDING').length;
414
415             document.getElementById('dash-restaurants').innerHTML=restArr.length
416                 ? restArr.slice(0,4).map(r=><div style="padding:12px 20px;border-bottom:1px solid var(--
border);display:flex;justify-content:space-between;align-items:center"><div><div style="font...
417                 : '<div class="empty-state">no restaurants yet</div>';
418
419             document.getElementById('dash-deliveries').innerHTML=delArr.length
420                 ? delArr.slice(0,5).map(d=><tr><td>${d.restaurantName||d.restaurantId||'-'}</td><td><span
class="chip ${((d.mealSlot||'')).toLowerCase()}">${d.mealSlot||'-'}</span></td><td><span cla...
421                 : '<tr><td colspan="3" class="empty-state">no deliveries</td></tr>';
422             }catch(e){console.error(e);}
423         }
424
425
426     async function loadMyRestaurants(){
427         try{
428             const rests=await apiFetch('/restaurants/vendor');
429             const arr=rests||[];
430             document.getElementById('restaurants-grid').innerHTML=arr.length
431                 ? arr.map(r=><div class="restaurant-card ${r.active===false?'inactive':''}>
432                 <div class="rest-name">${r.name||'-'}</div>
433                 <div class="rest-meta">${r.supportedMealSlots?.join(', ')}||'-'} · ID:
${r.id||r.restaurantId||'-'}</div>
434                 <div style="font-size:12px;color:var(--text2);margin-bottom:8px">${r.address||'-'}</div>
435                 <div style="font-size:11px;color:var(--text3);margin-bottom:12px">${r.description||''}</div>
436                 <div class="rest-actions">
437                     ${r.active!==false
438                     ? <button class="btn btn-warning btn-sm"
onclick="deactivateRestaurant(${r.id}|r.restaurantId)">DEACTIVATE</button>`
439                     : `<span style="font-size:10px;color:var(--red);font-family:var(--
mono)">INACTIVE</span>`}
440                 </div>
441             </div>`).join('')
442                 : '<div class="empty-state">no restaurants found. create one!</div>';
443             }catch(e){
444                 document.getElementById('restaurants-grid').innerHTML='<div class="empty-state">could not load
restaurants</div>';
445             }
446         }
447
448
449     async function addRestaurant(){
450         const name=document.getElementById('rest-name').value.trim();
451         const address=document.getElementById('rest-address').value.trim();
452         const description=document.getElementById('rest-desc').value.trim();
453         const supportedMealSlots=Array.from(document.querySelectorAll('.rest-slot-

```

```

check:checked')).map(el=>el.value);
454
455     if(!name || !address || supportedMealSlots.length===0){
456         showAlert('NAME, ADDRESS AND AT LEAST ONE MEAL SLOT REQUIRED','error');
457         return;
458     }
459
460     try{
461         await apiFetch('/restaurants',{
462             method:'POST',
463             body:JSON.stringify({name,address,description,supportedMealSlots})
464         });
465         showAlert('RESTAURANT CREATED!','success');
466         clearRestForm();
467         loadMyRestaurants();
468         loadDashboard();
469     }catch(e){
470         showAlert('FAILED: '+e.message,'error');
471     }
472 }
473
474
475 function clearRestForm(){
476     ['rest-name','rest-address','rest-desc'].forEach(id=>document.getElementById(id).value='');
477     document.querySelectorAll('.rest-slot-check').forEach(el=>el.checked=false);
478 }
479
480 async function loadSubscriptions(){
481     try{
482         const data=await apiFetch('/subscriptions');
483         document.getElementById('subscriptions-list').innerHTML=(data||[]).length
484             ? data.map(s=><tr><td style="font-family:var(--mono);font-size:12px">${s.employeeEmail}|${s.employeeId}|-</td><td>${s.restaurantName}|${s.restaurantId}|-</td><td><span class="chip...
485             : '<tr><td colspan="6" class="empty-state">no subscriptions found</td></tr>';
486     }catch(e){document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6" class="empty-state">could not load subscriptions</td></tr>';}
487 }
488
489 async function loadDeliveries(){
490     try{
491         const data=await apiFetch('/deliveries');
492         document.getElementById('deliveries-list').innerHTML=(data||[]).length
493             ? data.map(d=><tr><td style="font-family:var(--mono);font-size:11px">#${d.id}|-</td><td>${d.restaurantName}|${d.restaurantId}|-</td><td><span class="chip
494             : '<tr><td colspan="6" class="empty-state">no deliveries found</td></tr>';
495     }catch(e){document.getElementById('deliveries-list').innerHTML='<tr><td colspan="6" class="empty-state">could not load deliveries</td></tr>';}
496 }
497
498 async function markDelivered(id){
499     try{await apiFetch(`/deliveries/${id}/delivered`,{method:'PUT'});showAlert('MARKED AS
DELIVERED','success');loadDeliveries();}
500     catch(e){showAlert('FAILED','error');}
501 }
502
503 function logout(){
504     // Clear all localStorage
505     localStorage.removeItem('jwt_token');
506     localStorage.removeItem('refresh_token');
507     localStorage.removeItem('auth_type');
508     localStorage.removeItem('last_role');
509
510     // Call server logout to invalidate session
511     fetch(`${API}/session/logout`, {
512         method: 'POST',
513         credentials: 'include'
514     }).finally(() => {
515         // Force expire the session cookie on client side
516         document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
517         window.location.href = '/google.html';
518     });
519 }
520
521 function showAlert(msg,type){

```



```
522     const el=document.createElement('div');
523     el.className=`alert alert-${type}`;el.textContent=msg;
524     document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
525 }
526 </script>
527 </body>
528 </html>
529
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Vendor Login</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#070910;
12             --panel:#0e1118;
13             --panel2:#131821;
14             --border:rgba(255,255,255,0.07);
15             --border-lit:rgba(255,255,255,0.14);
16             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
17             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
18             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
20             --amber:#f9c74f;--amber-dim:rgba(249,199,79,0.12);
21             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
22         }
23         body{font-family:var(--font);background:var(--bg);color:var(--text);min-
height:100vh;display:flex;overflow:auto;}
24
25         .bg-layer{position:fixed;inset:0;z-index:0;
26             background:
27                 radial-gradient(ellipse 60% 50% at 20% 50%, rgba(248,113,113,0.06) 0%,transparent 70%),
28                 radial-gradient(ellipse 40% 60% at 80% 20%, rgba(249,199,79,0.05) 0%,transparent 60%),
29                 radial-gradient(ellipse 50% 40% at 70% 80%, rgba(248,113,113,0.04) 0%,transparent 60%);
30         }
31         .grid-overlay{position:fixed;inset:0;z-index:0;background-image:linear-
gradient(rgba(255,255,255,0.02) 1px,transparent 1px),linear-gradient(90deg,rgba(255,255,255,0.02)
1px,transparent 1px)...
32
33         /* LEFT */
34         .left-panel{flex:1;display:flex;flex-direction:column;justify-
content:center;padding:60px;position:relative;z-index:1;animation:fadeLeft 0.8s ease both;}
35         @keyframes fadeLeft{from{opacity:0;transform:translateX(-
24px)}to{opacity:1;transform:translateX(0)}}
36         .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:32px}
37         .brand-icon{width:44px;height:44px;background:linear-gradient(135deg,#f87171,#fb923c);border-
radius:12px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:18p...
38         .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
39         .brand-tagline{font-size:13px;color:var(--text2)}
40         .hero-text h1{font-size:clamp(36px,4vw,50px);font-weight:700;line-height:1.1;letter-spacing:-
1px;color:var(--text);margin-bottom:16px;}
41         .hl-red{color:var(--red)}
42         .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:360px}
43
44         .info-cards{display:flex;flex-direction:column;gap:12px;margin-top:40px;max-width:360px;}
45         .info-card{background:var(--panel);border:1px solid var(--border);border-radius:10px;padding:14px
18px;display:flex;align-items:flex-start;gap:12px;}
46         .info-card-icon{font-size:18px;flex-shrink:0;margin-top:1px}
47         .info-card-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px}
48         .info-card-text span{font-size:12px;color:var(--text2)}
49
50         .back-link{display:inline-flex;align-items:center;gap:6px;font-size:12px;color:var(--text3);text-
decoration:none;margin-top:36px;transition:color 0.2s;}
51         .back-link:hover{color:var(--text2)}
52
53         /* RIGHT */
54         .right-panel{width:460px;flex-shrink:0;display:flex;align-items:flex-start;overflow-y:auto;justify-
content:center;padding:32px 40px;position:relative;z-index:1;border-left:1px solid var(--b...
55         @keyframes
fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
56         .form-box{width:100%}
57
58         .vendor-badge{display:inline-flex;align-items:center;gap:8px;background:var(--red-dim);border:1px
solid rgba(248,113,113,0.25);border-radius:20px;padding:6px 14px;font-size:12px;font-weight...

```

```

59     .vendor-badge .dot{width:7px;height:7px;border-radius:50%;background:var(--red);animation:pulse 1.4s
infinite;}}
60     @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
61
62     .form-title{font-size:22px;font-weight:700;color:var(--text);margin-bottom:6px;}
63     .form-subtitle{font-size:13px;color:var(--text2);margin-bottom:28px;}
64
65     /* AUTH TABS */
66     .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-
bottom:24px;border:1px solid var(--border)}
67     .auth-tab{flex:1;padding:9px;border:none;background:transparent;color:var(--text2);font-family:var(-
font);font-weight:600;font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;}
68     .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}
69
70     /* FORM ELEMENTS */
71     .form-group{margin-bottom:16px}
72     .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-
transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
73     .form-input{width:100%;padding:11px 14px;background:var(--bg);border:1px solid var(--border);border-
radius:9px;color:var(--text);font-family:var(--font);font-size:14px;transition:all 0.2s;}
74     .form-input:focus{outline:none;border-color:var(--red);box-shadow:0 0 3px rgba(248,113,113,0.08)}
75     .form-input::placeholder{color:var(--text3)}
76
77     /* BUTTONS */
78     .btn-login{width:100%;padding:12px;border:none;border-radius:9px;font-family:var(--font);font-
size:14px;font-weight:600;cursor:pointer;transition:all 0.2s;margin-top:4px;background:var(--re...
79     .btn-login:hover{background:#f98a8a;box-shadow:0 6px 20px
rgba(248,113,113,0.35);transform:translateY(-1px)}
80     .btn-login.active{transform:translateY(0)}
81     .btn-login.loading{opacity:0.7;pointer-events:none}
82
83     /* STATUS */
84     .status-box{border-radius:9px;padding:10px 14px;margin-top:14px;font-size:13px;font-
weight:500;display:none;animation:fadeIn 0.3s ease;}
85     @keyframes fadeIn{from{opacity:0;transform:translateY(4px)}to{opacity:1;transform:translateY(0)}}
86     .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
87     .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
88     .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid
rgba(79,140,255,0.2)}
89
90     .notice{background:rgba(249,199,79,0.06);border:1px solid rgba(249,199,79,0.15);border-
radius:9px;padding:12px 14px;margin-top:20px;font-size:12px;color:var(--text2);line-height:1.5;}
91     .notice strong{color:var(--amber)}
92
93     /* FOOTER */
94     .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
95     .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
96     .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
97     .tz-select{flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);border-
radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;}
98     .tz-select:focus{outline:none;border-color:var(--border-lit)}
99     .tz-time{font-size:10px;color:var(--red);font-family:var(--mono)}
100
101     @media(max-width:900px){
102         .left-panel{display:none}
103         .right-panel{width:100%;border-left:none;background:var(--bg)}
104     }
105 </style>
106 </head>
107 <body>
108 <div class="bg-layer"></div>
109 <div class="grid-overlay"></div>
110
111 <!-- LEFT -->
112 <div class="left-panel">
113     <div class="brand-logo">
114         <div class="brand-icon">V</div>
115     </div>
116     <div class="brand-name">Vendor Hub</div>
117     <div class="brand-tagline">Employee Management System</div>
118 </div>
119 </div>
120 <div class="hero-text">
121     <h1>Manage your<br><span class="hl-red">vendor account</span><br>securely.</h1>
122     <p>Access your deliveries, restaurants, and subscriptions. Your account is created and managed by

```

```
the system admin.</p>
123     </div>
124     <div class="info-cards">
125         <div class="info-card">
126             <span class="info-card-icon"></span>
127             <div class="info-card-text">
128                 <strong>Delivery Tracking</strong>
129                 <span>Monitor your active and scheduled meal deliveries in real-time.</span>
130             </div>
131         </div>
132         <div class="info-card">
133             <span class="info-card-icon"></span>
134             <div class="info-card-text">
135                 <strong>Restaurant Management</strong>
136                 <span>View and update restaurant details and meal offerings.</span>
137             </div>
138         </div>
139         <div class="info-card">
140             <span class="info-card-icon"></span>
141             <div class="info-card-text">
142                 <strong>Account by Admin Only</strong>
143                 <span>Vendor accounts are created by the system admin. Contact your admin if you don't have
credentials.</span>
144             </div>
145         </div>
146     </div>
147     <a class="back-link" href="/google.html"> Back to main login</a>
148 </div>
149
150 <!-- RIGHT FORM -->
151 <div class="right-panel">
152     <div class="form-box">
153
154         <div class="vendor-badge"><span class="dot"></span>Vendor Portal</div>
155         <div class="form-title">Vendor Sign In</div>
156         <div class="form-subtitle">Enter the credentials provided by your system admin.</div>
157
158         <!-- AUTH TABS -->
159         <div class="auth-tabs">
160             <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
161             <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
162         </div>
163
164         <!-- JWT FORM -->
165         <div id="jwt-form">
166             <div class="form-group">
167                 <label class="form-label">Email / Username</label>
168                 <input class="form-input" id="jwt-username" type="text" placeholder="vendor@company.com"
autocomplete="off">
169             </div>
170             <div class="form-group">
171                 <label class="form-label">Password</label>
172                 <input class="form-input" id="jwt-password" type="password" placeholder="....."
autocomplete="current-password">
173             </div>
174             <button class="btn-login" id="jwt-btn" onclick="jwtLogin()">Sign In as Vendor</button>
175         </div>
176
177         <!-- SESSION FORM -->
178         <div id="session-form" style="display:none">
179             <div class="form-group">
180                 <label class="form-label">Email / Username</label>
181                 <input class="form-input" id="session-username" type="text"
placeholder="vendor@company.com">
182             </div>
183             <div class="form-group">
184                 <label class="form-label">Password</label>
185                 <input class="form-input" id="session-password" type="password" placeholder=".....">
186             </div>
187             <button class="btn-login" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
188         </div>
189
190     <div class="status-box" id="status-box"></div>
191
192     <div class="notice">
```

```

193         <strong>Don't have an account?</strong> Vendor accounts are created exclusively by the system
admin. Please contact your administrator to set up access.
194     </div>
195
196     <!-- FOOTER -->
197     <div class="form-footer">
198         <div class="tz-row">
199             <span class="tz-label">TZ</span>
200             <select class="tz-select" id="tz-select" onchange="updateTz()">
201                 <option value="Asia/Kolkata">India – IST UTC+5:30</option>
202                 <option value="UTC">UTC</option>
203                 <option value="America/New_York">New York – EST UTC-5</option>
204                 <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
205                 <option value="Europe/London">London – GMT UTC+0</option>
206                 <option value="Europe/Paris">Paris – CET UTC+1</option>
207                 <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
208                 <option value="Asia/Dubai">Dubai – GST UTC+4</option>
209                 <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
210                 <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
211             </select>
212             <span class="tz-time" id="tz-time">--:--:--</span>
213         </div>
214     </div>
215
216 </div>
217 </div>
218
219 <script>
220     const API = 'http://localhost:8080';
221
222     const DASHBOARDS = {
223         FOOD_VENDOR: '/vendor-dashboard.html',
224         TECH_VENDOR: '/tech-vendor-dashboard.html'
225     };
226
227     // ALREADY LOGGED IN?
228     (function(){
229         const token = localStorage.getItem('jwt_token');
230         if(!token) return;
231         try{
232             const role = getRoleFromJWT(token);
233             if(DASHBOARDS[role]) window.location.href = DASHBOARDS[role];
234         }catch(e){}
235     })();
236
237     // HELPERS
238     function getRoleFromJWT(token){
239         const payload = JSON.parse(atob(token.split('.')[1]));
240         const raw = payload.role || payload.roles || payload.authorities || payload.scope || '';
241         const role = Array.isArray(raw) ? raw[0] : raw;
242         return String(role).replace(/^ROLE_/, '').toUpperCase().trim();
243     }
244
245     function showStatus(type, msg){
246         const box = document.getElementById('status-box');
247         box.className = `status-box ${type}`;
248         box.textContent = msg;
249         box.style.display = 'block';
250     }
251     function hideStatus(){
252         document.getElementById('status-box').style.display = 'none';
253     }
254
255     function switchTab(type){
256         document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
257         document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
258         document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
259         document.getElementById('tab-session').classList.toggle('active', type === 'session');
260         hideStatus();
261     }
262
263     function redirectByRole(role){
264         const dest = DASHBOARDS[role];
265         if(!dest){
266             showStatus('error', 'This portal is for vendors only. Please use the main login page.');
```

```

268     }
269     showStatus('success', `Login successful! Redirecting to ${role === 'TECH_VENDOR' ? 'Tech Vendor' :
'Food Vendor'} dashboard...`);
270     setTimeout(() => window.location.href = dest, 700);
271     return true;
272 }
273
274 // JWT LOGIN
275 async function jwtLogin(){
276     const username = document.getElementById('jwt-username').value.trim();
277     const password = document.getElementById('jwt-password').value;
278     if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }
279
280     const btn = document.getElementById('jwt-btn');
281     btn.classList.add('loading'); btn.textContent = 'Signing in...';
282     showStatus('loading', 'Authenticating...');
283
284     try{
285         const res = await fetch(`${API}/Authenticate`, {
286             method: 'POST',
287             headers: {'Content-Type': 'application/json'},
288             body: JSON.stringify({username, password})
289         });
290
291         if(res.ok){
292             const data = await res.json();
293             const token = data.accessToken || data.token;
294             const role = getRoleFromJWT(token);
295
296             if(!DASHBOARDS[role]){
297                 showStatus('error', 'This portal is for vendors only. Please use the main login page.');
```

btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';

```

299                 return;
300             }
301
302             localStorage.setItem('jwt_token', token);
303             localStorage.setItem('refresh_token', data.refreshToken || '');
304             localStorage.setItem('auth_type', 'jwt');
305             localStorage.setItem('last_role', role);
306             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
307
308             btn.textContent = 'Signed In ';
309             redirectByRole(role);
310
311         }else{
312             const err = await res.text();
313             showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
314             btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
315         }
316     }catch(e){
317         showStatus('error', 'Connection error: ' + e.message);
318         btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
319     }
320 }
321
322 // SESSION LOGIN
323 async function sessionLogin(){
324     const username = document.getElementById('session-username').value.trim();
325     const password = document.getElementById('session-password').value;
326     if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }
327
328     const btn = document.getElementById('session-btn');
329     btn.classList.add('loading'); btn.textContent = 'Signing in...';
330     showStatus('loading', 'Authenticating...');
331
332     try{
333         const res = await fetch(`${API}/session/login`, {
334             method: 'POST',
335             headers: {'Content-Type': 'application/json'},
336             body: JSON.stringify({username, password}),
337             credentials: 'include'
338         });
339
340         if(res.ok){
341             // Session login – fetch /employee/me or /vendor/me to get role
342             try{

```

```

343         const meRes = await fetch(`${API}/vendor/me`, {credentials: 'include'});
344         if(meRes.ok){
345             const me = await meRes.json();
346             const role = String(me.role || '').replace(/^ROLE_/, '').toUpperCase();
347             localStorage.setItem('auth_type', 'session');
348             localStorage.setItem('last_role', role);
349             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
350             btn.textContent = 'Signed In ';
351             redirectByRole(role);
352             return;
353         }
354     }catch(e){}
355
356     // Fallback if /vendor/me not available
357     showStatus('error', 'Could not determine vendor type. Please use JWT login.');
```

358 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';

359

```

360     }else{
361         const err = await res.json().catch(()=>({}));
362         showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
363         btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
364     }
365     }catch(e){
366         showStatus('error', 'Connection error: ' + e.message);
367         btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
368     }
369 }
370
371 // TIMEZONE
372 function updateTz(){
373     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
374     refreshTzClock();
375 }
376 function refreshTzClock(){
377     const tz = document.getElementById('tz-select').value;
378     document.getElementById('tz-time').textContent =
379         new Date().toLocaleTimeString('en-GB', {timeZone: tz, hour12: false});
380 }
381
382 document.addEventListener('keydown', e => {
383     if(e.key !== 'Enter') return;
384     if(document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
385     else sessionLogin();
386 });
387
388 window.onload = () => {
389     const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
390     document.getElementById('tz-select').value = savedTz;
391     refreshTzClock();
392     setInterval(refreshTzClock, 1000);
393 };
394 </script>
395 </body>
396 </html>

```

167 Current:

src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
1 package com.example.EmployeeManagementSystem;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class EmployeeManagementSystemApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```



```

1 package com.example.EmployeeManagementSystem.service;
2
3
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6 import com.example.EmployeeManagementSystem.Entity.LeaveType;
7 import com.example.EmployeeManagementSystem.Enum.Gender;
8 import com.example.EmployeeManagementSystem.Enum.Status;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15 import org.mockito.*;
16
17 import java.math.BigDecimal;
18 import java.time.LocalDate;
19 import java.util.List;
20 import java.util.Optional;
21
22 import static org.assertj.core.api.Assertions.assertThat;
23 import static org.mockito.ArgumentMatchers.any;
24 import static org.mockito.Mockito.*;
25
26 class LeaveAccrualServiceTest {
27
28     @Mock
29     EmployeeRepo employeeRepo;
30     @Mock
31     LeaveTypeRepository leaveTypeRepo;
32     @Mock
33     LeaveEntitlementRepository entitlementRepo;
34
35     @InjectMocks
36     LeaveAccrualService service;
37
38     @BeforeEach void setup() { MockitoAnnotations.openMocks(this); }
39
40     @Test
41     void maleEmployee_doesNotAccrueMaternity() {
42         Employee emp = employee(Gender.M);
43         LeaveType maternity = leaveType("MATERNITY", true, Gender.F);
44
45         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
46         when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
47
48         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
49
50         verify(entitlementRepo, never()).save(any());
51     }
52
53     @Test
54     void femaleEmployee_accruesMaternity() {
55         Employee emp = employee(Gender.F);
56         LeaveType maternity = leaveType("MATERNITY", true, Gender.F);
57
58         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
59         when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
60         when(entitlementRepo.findByEmployeeIdAndLeaveTypeIdAndYear(any(), any(), any()))
61             .thenReturn(Optional.empty());
62
63         // Simulate new entitlement creation
64         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
67
68         ArgumentCaptor<LeaveEntitlement> captor = ArgumentCaptor.forClass(LeaveEntitlement.class);
69         verify(entitlementRepo, atLeastOnce()).save(captor.capture());
70
71         LeaveEntitlement saved = captor.getAllValues().stream()

```

```

72         .filter(e -> e.getAccruedThisYear().compareTo(BigDecimal.ONE) == 0)
73         .findFirst().orElseThrow();
74
75         assertThat(saved.getAccruedThisYear()).isEqualToComparingTo("1");
76     }
77
78     @Test
79     void employeeJoiningAfterAccrualDate_isSkipped() {
80         Employee emp = employee(Gender.M);
81         emp.setJoined_at(LocalDate.of(2025, 6, 15)); // joins mid-month
82         LeaveType casual = leaveType("CASUAL", false, null);
83
84         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
85         when(leaveTypeRepo.findAll()).thenReturn(List.of(casual));
86
87         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
88
89         verify(entitlementRepo, never()).save(any());
90     }
91
92     // --- helpers ---
93     private Employee employee(Gender gender) {
94         Employee e = new Employee();
95         e.setEmployeeId(1L);
96         e.setGender(gender);
97         e.setStatus(Status.ACTIVE);
98         e.setJoined_at(LocalDate.of(2023, 1, 1));
99         return e;
100     }
101
102     private LeaveType leaveType(String name, boolean genderRestricted, Gender restriction) {
103         LeaveType lt = new LeaveType();
104         lt.setId(1);
105         lt.setName(name);
106         lt.setGenderRestricted(genderRestricted);
107         lt.setGenderRestriction(restriction);
108         lt.setCarriesForward(!name.equals("SICK"));
109         lt.setMonthlyReset(name.equals("SICK"));
110         return lt;
111     }
112 }

```

```

1  package com.example.EmployeeManagementSystem.service;
2
3  import com.example.EmployeeManagementSystem.Entity.Employee;
4  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
6  import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
7  import com.example.EmployeeManagementSystem.Enum.LeaveType;
8  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9  import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
10 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15 import org.mockito.InjectMocks;
16 import org.mockito.Mock;
17 import org.mockito.MockitoAnnotations;
18
19 import java.math.BigDecimal;
20 import java.time.LocalDate;
21 import java.util.Optional;
22
23 import static org.assertj.core.api.Assertions.assertThat;
24 import static org.mockito.ArgumentMatchers.any;
25 import static org.mockito.Mockito.when;
26
27 class LeaveRequestServiceTest {
28
29     @Mock
30     LeaveRequestRepo leaveRequestRepo;
31     @Mock
32     EmployeeRepo employeeRepo;
33     @Mock
34     LeaveTypeRepository leaveTypeRepository;
35     @Mock
36     LeaveEntitlementRepository leaveEntitlementRepository;
37
38     @InjectMocks
39     LeaveRequestService service;
40
41     @BeforeEach
42     void setup() {
43         MockitoAnnotations.openMocks(this);
44     }
45
46     @Test
47     void approveLeave_updatesUsedEntitlementDays() {
48         Employee employee = new Employee();
49         employee.setEmployeeId(1L);
50
51         LeaveRequest request = new LeaveRequest();
52         request.setId(10L);
53         request.setEmployee(employee);
54         request.setLeaveType(LeaveType.CASUAL);
55         request.setStartDate(LocalDate.of(2026, 5, 4));
56         request.setEndDate(LocalDate.of(2026, 5, 6));
57         request.setStatus(LeaveStatus.PENDING);
58
59         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
60             new com.example.EmployeeManagementSystem.Entity.LeaveType();
61         casual.setId(2);
62         casual.setName("CASUAL");
63         casual.setCarriesForward(true);
64
65         LeaveEntitlement entitlement = new LeaveEntitlement();
66         entitlement.setEmployee(employee);
67         entitlement.setLeaveType(casual);
68         entitlement.setYear(2026);
69         entitlement.setOpeningBalance(BigDecimal.ZERO);
70         entitlement.setAccruedThisYear(BigDecimal.TEN);
71         entitlement.setUsedThisYear(BigDecimal.ONE);

```

```
72     entitlement.setClosingBalance(BigDecimal.ZERO);
73
74     when(leaveRequestRepo.findById(10L)).thenReturn(Optional.of(request));
75     when(leaveRequestRepo.save(any())).thenReturn(inv -> inv.getArgument(0));
76     when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
77     when(leaveEntitlementRepository.findByIdAndLeaveTypeIdAndYear(1L, 2, 2026))
78         .thenReturn(Optional.of(entitlement));
79     when(leaveEntitlementRepository.save(any())).thenReturn(inv -> inv.getArgument(0));
80
81     service.approveLeave(10L);
82
83     assertThat(entitlement.getUsedThisYear()).isEqualByComparingTo("4");
84 }
85 }
86
```

```

1  package com.example.EmployeeManagementSystem.service;
2
3  import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
4  import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
5  import org.junit.jupiter.api.BeforeEach;
6  import org.junit.jupiter.api.Test;
7  import org.mockito.*;
8  import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
9  import com.example.EmployeeManagementSystem.Entity.LeaveType;
10 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
11 import com.example.EmployeeManagementSystem.Entity.Employee;
12 import java.math.BigDecimal;
13 import java.time.LocalDate;
14 import java.util.List;
15
16 import static org.assertj.core.api.Assertions.assertThat;
17 import static org.mockito.Mockito.*;
18
19 class SickLeaveResetServiceTest {
20
21     @Mock
22     LeaveTypeRepository leaveTypeRepo;
23     @Mock
24     LeaveEntitlementRepository entitlementRepo;
25     @InjectMocks SickLeaveResetService service;
26
27     @BeforeEach void setup() { MockitoAnnotations.openMocks(this); }
28
29     @Test
30     void unusedSickDays_areZeroedAtMonthEnd() {
31         LeaveType sick = sickLeaveType();
32         LeaveEntitlement entitlement = entitlement(sick, BigDecimal.valueOf(3), BigDecimal.ONE);
33         // accrued=3, used=1 available=2 (should be wiped)
34
35         when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
36         when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
37             .thenReturn(List.of(entitlement));
38         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
39
40         service.runMonthlyReset(LocalDate.of(2025, 6, 30));
41
42         // After reset: accrued should equal used available = 0
43         assertThat(entitlement.getAccruedThisYear())
44             .isEqualToByComparingTo(entitlement.getUsedThisYear());
45         assertThat(entitlement.getAvailableBalance())
46             .isEqualToByComparingTo(BigDecimal.ZERO);
47     }
48
49     @Test
50     void fullyUsedSickLeave_noChangeNeeded() {
51         LeaveType sick = sickLeaveType();
52         LeaveEntitlement entitlement = entitlement(sick, BigDecimal.ONE, BigDecimal.ONE);
53         // accrued=1, used=1 available=0, nothing to wipe
54
55         when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
56         when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
57             .thenReturn(List.of(entitlement));
58         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
59
60         service.runMonthlyReset(LocalDate.of(2025, 6, 30));
61
62         // accrued unchanged
63         assertThat(entitlement.getAccruedThisYear()).isEqualToByComparingTo("1");
64     }
65
66     // --- helpers ---
67     private LeaveType sickLeaveType() {
68         LeaveType lt = new LeaveType();
69         lt.setId(1);
70         lt.setName("SICK");
71         lt.setMonthlyReset(true);

```

```
72         lt.setCarriesForward(false);
73         return lt;
74     }
75
76     private LeaveEntitlement entitlement(LeaveType type,
77                                         BigDecimal accrued,
78                                         BigDecimal used) {
79         Employee emp = new Employee();
80         emp.setEmployeeId(1L);
81
82         LeaveEntitlement e = new LeaveEntitlement();
83         e.setEmployee(emp);
84         e.setLeaveType(type);
85         e.setYear(2025);
86         e.setOpeningBalance(BigDecimal.ZERO);
87         e.setAccruedThisYear(accrued);
88         e.setUsedThisYear(used);
89         e.setClosingBalance(BigDecimal.ZERO);
90         return e;
91     }
92 }
93
```

```

1  package com.example.EmployeeManagementSystem.service;
2
3  import com.example.EmployeeManagementSystem.Entity.Employee;
4  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5  import com.example.EmployeeManagementSystem.Entity.LeaveType;
6  import com.example.EmployeeManagementSystem.Enum.Status;
7  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8  import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
9  import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
10 import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
11 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
12
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15
16 import org.mockito.InjectMocks;
17 import org.mockito.Mock;
18 import org.mockito.MockitoAnnotations;
19
20 import java.math.BigDecimal;
21 import java.util.List;
22 import java.util.Optional;
23
24 import static org.mockito.ArgumentMatchers.any;
25 import static org.mockito.ArgumentMatchers.eq;
26
27 import static org.mockito.Mockito.when;
28 import static org.mockito.Mockito.verify;
29 import static org.mockito.Mockito.never;
30 import static org.mockito.Mockito.atLeastOnce;
31
32 import static org.assertj.core.api.Assertions.assertThat;
33 class YearEndCarryForwardServiceTest {
34
35     @Mock
36     EmployeeRepo employeeRepo;
37     @Mock
38     LeaveEntitlementRepository entitlementRepo;
39     @Mock
40     YearEndCarryForwardLogRepository logRepo;
41     @Mock
42     LeaveWarningRepository warningRepo;
43     @InjectMocks
44     YearEndCarryForwardService service;
45
46     @BeforeEach
47     void setup() { MockitoAnnotations.openMocks(this); }
48
49     @Test
50     void whenTotalUnderCap_noCapApplied() {
51         Employee emp = employee();
52         LeaveType casual = carryForwardType("CASUAL");
53
54         // casual: opening=0, accrued=8, used=2 closing=6
55         LeaveEntitlement ent = entitlement(emp, casual, 0, 8, 2);
56
57         when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
58         when(employeeRepo.findById(Status.ACTIVE)).thenReturn(List.of(emp));
59         when(entitlementRepo.findByIdByEmployeeIdAndYear(emp.getId(), 2025))
60             .thenReturn(List.of(ent));
61         when(entitlementRepo.findByIdByEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
62             .thenReturn(Optional.empty());
63         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
64         when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runYearEnd(2025);
67
68         // closing balance = 6, no cap needed, nothing wasted
69         assertThat(ent.getClosingBalance()).isEqualToByComparingTo("6");
70         verify(warningRepo, never()).save(any());
71     }

```

72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137

```
@Test
void whenTotalExceedsCap_capAppliedProportionally() {
    Employee emp = employee();
    LeaveType casual = carryForwardType("CASUAL");
    LeaveType maternity = carryForwardType("MATERNITY");

    // casual: closing=20, maternity: closing=16 total=36, over cap by 6
    LeaveEntitlement casualEnt = entitlement(emp, casual, 0, 20, 0);
    LeaveEntitlement maternityEnt = entitlement(emp, maternity, 0, 16, 0);

    when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
    when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
    when(entitlementRepo.findByEmployeeEmployeeIdAndYear(emp.getEmployeeId(), 2025))
        .thenReturn(List.of(casualEnt, maternityEnt));
    when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
        .thenReturn(Optional.empty());
    when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
    when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

    service.runYearEnd(2025);

    // casual: 20/36 * 30 = 16.67, maternity: 16/36 * 30 = 13.33, total = 30
    BigDecimal casualCapped = casualEnt.getClosingBalance();
    BigDecimal maternityCapped = maternityEnt.getClosingBalance();
    BigDecimal total = casualCapped.add(maternityCapped);

    assertThat(total).isEqualByComparingTo("30.00");
    verify(warningRepo, atLeastOnce()).save(any()); // wasted warning issued
}

@Test
void idempotencyGuard_preventsDoubleRun() {
    when(logRepo.existsByProcessedYear(2025)).thenReturn(true);

    service.runYearEnd(2025);

    verify(employeeRepo, never()).findByStatus(any());
}

// --- helpers ---
private Employee employee() {
    Employee e = new Employee(); e.setEmployeeId(1L); return e;
}

private LeaveType carryForwardType(String name) {
    LeaveType lt = new LeaveType();
    lt.setId(name.equals("CASUAL") ? 2 : 3);
    lt.setName(name);
    lt.setCarriesForward(true);
    lt.setMonthlyReset(false);
    return lt;
}

private LeaveEntitlement entitlement(Employee emp, LeaveType type,
    int opening, int accrued, int used) {
    LeaveEntitlement e = new LeaveEntitlement();
    e.setEmployee(emp); e.setLeaveType(type); e.setYear(2025);
    e.setOpeningBalance(BigDecimal.valueOf(opening));
    e.setAccruedThisYear(BigDecimal.valueOf(accrued));
    e.setUsedThisYear(BigDecimal.valueOf(used));
    e.setClosingBalance(BigDecimal.ZERO);
    return e;
}
}
```